Abstract

A General Framework for Hindley/Milner Type Systems with Constraints

Martin Sulzmann

Yale University

2000

In my dissertation, I introduce a general framework for Hindley/Milner systems with constraints. The basic idea is to factor out the common core of previous extensions of the Hindley/Milner system. I present a Hindley/Milner system where the constraint part is a parameter. Specific applications can be obtained by providing specific constraint systems which capture the application in mind. For instance, the Hindley/Milner system can be recovered by instantiating the constraint part to the standard Herbrand constraint system. Type system instances of the general framework are sound if the underlying constraint system is sound. Furthermore, I give a generic type inference algorithm for the general framework, under sufficient conditions on the specific constraint system type inference yields principal types.

A General Framework for Hindley/Milner Type Systems with Constraints

A Dissertation
Presented to the Faculty of the Graduate School
of
Yale University
in Candidacy for the Degree of
Doctor of Philosophy

by
Martin Sulzmann

Dissertation Director: Paul Hudak

May 2000

Copyright © 2000 by Martin Sulzmann

All Rights Reserved

Acknowledgments

Many people have provided help during my time at Yale University and at various places all over the world. First and foremost, I would like to thank my supervisor Paul Hudak, who provided the financial support to make this dissertation possible. It was not only a challenge being in his research group but also a challenge being with him on the ski slopes. I also have to thank my external advisor Martin Odersky. From the early days at Karlsruhe as my Diploma supervisor, he was always an inspiring resource. I would never have gotten this far without him. Thanks to all the participants of the SPAM meeting group at Yale University. In particular, I would like to thank the following people: Alastair Reid for challenging discussions, Valery Trifonov, Stefan Monnier, Kevin Glynn and Sandra Alves for their careful remarks and proof–reading parts of my dissertation. Several research visits also have allowed me to meet other people. They have helped me to make significant improvements to my dissertation. Among these, I would like to thank Martin Müller for his helpful comments and discussions; Christoph Zenger for being so particular about proving things, proof–reading parts of my dissertation and his helpful comments; John Maraist for making my second research visit to Australia possible. I would also like to thank Martin Wehr for challenging discussions about type theory back in Karlsruhe and introducing me to category theory.

In addition, thanks goes to the University of Saarbrücken, the University of South Australia and Fachhochschule Furtwangen for their hospitality during various research visits.

This work would not have been possible without the support of my family and friends — you know who are you — thanks to all of you.

This work was supported in part by a Yale University Fellowship and DARPA Grant F30602-96-2-0232.

Contents

1 Introduction **1**

1.1 Dissertation Outline 2

1.2 Contributions 4

2 Background **7**

2.1 Type Systems 7

2.2 The Hindley/Milner System 9

2.2.1 The Logical System 9

2.2.2 The Semantic Model 11

2.2.3 The Inference System 13

2.2.4 Summary 16

3 The HM(X) framework **17**

3.1 Constraints 19

3.1.1 Summary 27

3.2 Logical System 27

3.2.1 Summary 32

3.3 Type Soundness 33

3.3.1 A Denotational Approach 33

3.3.2 A Syntactic Approach 38

3.3.3 Summary . . . 40
3.4 Properties . . . 41
3.4.1 Instance Relation . . . 41
3.4.2 Basic Definitions . . . 43
3.4.3 Derived Typing Rules . . . 44
3.4.4 Normalized Derivations . . . 45
3.4.5 Nested Forall Quantifiers . . . 47
3.4.6 Summary . . . 49
3.5 Constrained Polymorphism . . . 49
3.5.1 A Comparison between Different Quantifier Introduction Rules 50
3.5.2 Open versus Closed World Systems . . . 56
3.5.3 Odersky/Wadler/Wehr Overloading . . . 58
3.5.4 Ohori Records . . . 62
3.5.5 Projection in Solved Form . . . 67
3.5.6 Summary . . . 69
3.6 Terms versus Constraints . . . 70
3.6.1 A Motivating Example . . . 70
3.6.2 Formalizing a Connection between Terms and Constraints . . 72
3.6.3 Summary . . . 75
3.7 Logical System in Constraint Form . . . 75
3.7.1 Summary . . . 79

4 Type Inference in HM(X) 80

4.1 A Problem when Inferring Principal Types . . . 81
4.1.1 Summary . . . 84
4.1.2 Outline . . . 85

4.2 Normalization 86
4.2.1 Lifting of Normalization 92
4.3 Inference in Term Form 94
4.3.1 Soundness and Completeness Results 97
4.3.2 Principal Types 100
4.4 Inference in Constraint Form 101
4.4.1 Constraint Inference 101
4.4.2 Term Inference 104
4.4.3 Discussion 110
4.5 Summary 111

5 Applications **114**

5.1 Ohori Records 115
5.1.1 Extensible Records 119
5.1.2 Record Concatenation 123
5.1.3 Polymorphic Field Labels 127
5.1.4 Records with Overloaded Field Labels 133
5.1.5 Summary 133
5.2 Odersky/Wadler/Wehr Overloading 135
5.2.1 Overloaded Curried Functions 139
5.2.2 Overloaded Identifiers as First–Class Values 143
5.2.3 Summary 144
5.3 Program checking 145
5.3.1 Problems 145
5.3.2 Array–bounds checking 146

6 Conclusion **160**

6.1 Achievements 160

6.2 Implementation 161

6.3 Future Work 162

Bibliography **163**

A Extensions **173**

A.1 Extensible Records and Polymorphic Field Labels 173

A.2 Symmetric Record Concatenation 174

A.3 Polymorphic Recursion 175

A.4 Case 175

B Miscellaneous **179**

B.1 Satisfiability in REC^l is NP–complete 179

B.2 A Motivating Example II 180

C Proofs **182**

C.1 Proofs of Section 3.3.1 182

C.2 Proofs of Section 3.4.1 185

C.3 Proofs of Section 3.4.4 187

C.4 Proofs of Section 3.6 194

C.5 Proofs of Section 3.7 198

C.6 Proofs of Section 4.2 200

C.7 Proofs of Section 4.3 206

C.8 Proofs of Section 4.4 223

C.9 Proofs of Chapter 5 227

D HM(X) in Haskell **230**

E A Catalogue of Notation **252**

List of Figures

1.1 Outline of my dissertation 3

2.1 Hindley/Milner type system 11
2.2 Hindley/Milner type inference 15
2.3 The Hindley/Milner system 16

3.1 Logical system 29
3.2 Normalized typing rules 46
3.3 Versions of the quantifier introduction rule 50
3.4 Logical type system in constraint form 77

4.1 Type inference in term form 95
4.2 By need type inference in term form 97
4.3 Type inference in constraint form 102
4.4 Revised type inference in term form 106
4.5 Revised type inference in term form II 109
4.6 Type inference in term form 112
4.7 Type inference in constraint form 113

5.1 Closure for extensible records 121
5.2 Closure for record concatenation 125

C.1 Measure of expression size 183

Chapter 1

Introduction

The Hindley/Milner system has proven to be one of the most popular and widely studied type systems. The Hindley/Milner system is not only of theoretical interest but has also provided the basis of the type system for various programming languages such as Haskell [PH97] and ML [MTHM97].

Traditionally, the Hindley/Milner [Mil78] system is split into three parts: the logical system, the semantic model and the inference system. The logical system describes the valid typing judgments, whereas the semantic model gives a semantic meaning to typing judgments. The relationship between both is usually captured by the phrase "well–typed programs do not go wrong". This means that any well–typed program can safely be executed and will not cause any run–time errors. In this context, I consider programs which do not contain type annotations. The inference system on the other hand, allows to infer a type for a given untyped program. In the case of the standard Hindley/Milner type system, the connection between the inference and logical system is expressed by the principal types theorem [DM82]. A principal type represents the most general type for a given program. All other types which can be given to the program are derivable from the principal type. The

inference system infers principal types if the program is typable at all.

In recent years, there have been quite a big number of extensions of the Hindley/Milner system. Many of these extensions deal with some form of constraints. In order to establish the standard results almost the same steps/proofs have to be repeated. What is missing is a general framework which makes it easier to develop extensions of the Hindley/Milner system in a concise way.

In my dissertation, I introduce a general framework for Hindley/Milner systems with constraints, henceforth referred to as HM(X) parameterized in the constraint domain. Specific type system instances of HM(X) are obtained by providing a constraint system X, which captures the properties in mind. The standard Hindley/Milner systems falls out by instantiating X with the standard Herbrand constraint system. HM(X) stays in the tradition of the standard Hindley/Milner system. Type system instances of HM(X) are sound under a standard compositional semantics. I present a type inference algorithm for HM(X) that is generic in the constraint domain X. Under sufficient conditions on X, type inference yields principal types.

1.1 Dissertation Outline

Chapter 2 provides basic background about type systems and a general overview of the Hindley/Milner system. In Chapter 3, I formally introduce the HM(X) framework. First, I define the constraint language used throughout my dissertation. Based on the constraint language, I introduce the language of types. Once the constraint and type language are defined, type system instances of HM(X) can be described. I describe two possible presentations of HM(X) type system instances, one in term form and the other in constraint form and compare their relationship. In addition, I highlight the advantages of the novel formulation of the quantifier introduction found in

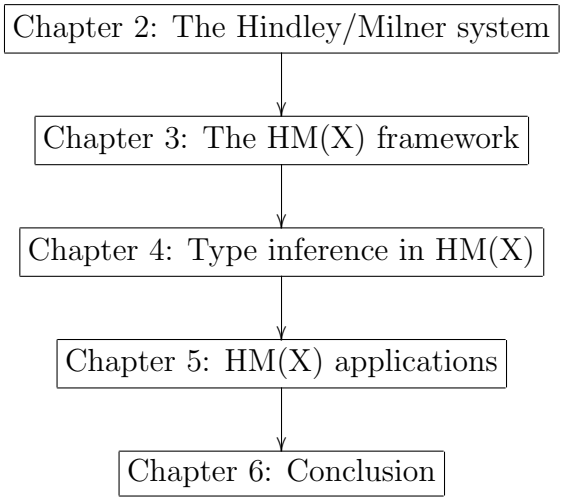


Figure 1.1: Outline of my dissertation

HM(X). Then, I undertake a discussion of two approaches to establish a type soundness theorem. The first approach stays in the tradition of Milner's type soundness result. I give a standard untyped compositional semantic based on the ideal model. The second approach is based on a subject reduction property. Finally, I present the inference system in Chapter 4. I introduce the new concept of normalization which can be seen as an extension of the well–known concept of unification. Under the assumption that most general normalizers exist, type inference yields principal types. Chapter 5 presents applications of the HM(X) framework. I describe Ohori's record calculus [Oho95] and overloading in the style of Odersky/Wadler/Wehr [OWW95] as instances of the HM(X) system. In addition, I give several useful extensions of these systems. Chapter 6 concludes my dissertation.

Figure 1.1 highlights the general structure of my dissertation. Readers familiar with type systems and the Hindley/Milner system in particular, can skip Chapter 2 and immediately move on to Chapter 3. Each chapter contains an overview of the content of the chapter at the beginning. Each section contains a summary section

which allows for quick access to the content and overview of results. A significant part of my dissertation deals with proofs. Most proofs can be found in the appendix unless the proof is rather short. Some proofs are omitted if the proof is straightforward. Appendix E contains a catalogue of notation.

1.2 Contributions

My dissertation evolved out of work started with Martin Odersky and Martin Wehr. Preliminary results can be found in [SOW97, OSW99]. The main contributions my dissertation makes to the field are listed below:

1. HM(X) claims to be a general framework for Hindley/Milner systems. To support this claim, I show how previous approaches can be seen as instances of the HM(X) system. I consider specifically systems which deal with records [Oho95, Rém89, Gas98] overloading [OWW95, NP95, Jon92] and subtyping [Kae92, AW93, TS96]. Further instances are systems for program analysis such as dimension analysis [Ken96]. Details can be found in in Section 3.2 and Chapter 5.

2. The HM(X) systems comes with a novel formulation of the quantifier introduction rule due to Martin Odersky which improves on previous formulations (see Section 3.5).

3. HM(X) type system instances are sound under a standard compositional semantics. The catch–phrase"well–typed programs do not go wrong" can be extended to the HM(X) system provided that the constraint system X is sound. See Section 3.3 for details. The adoption of Milner's type soundness approach is an idea started by Martin Odersky.

4. Type descriptions usually consist of a term algebra which describes the valid type terms. For instance, recursive types such as $\mu\alpha.\alpha \to \beta$ are represented explicitly through the μ notation in the type language. Types could also be represented in the constraint language as solutions for some variables. The constraint $(\alpha = \alpha \to \beta)$ represents the above recursive type in the constraint language. I give two presentations of the formal HM(X) system. One of the presentations is in term form where complex terms might possibly appear explicitly in the syntactic domain of types. The other presentation is in constraint form where all typing information is represented in the constraint domain. The term representation is the more common one, but the constraint representation allows for an abstraction from any syntactic considerations and even an achievement of higher expressiveness. Section 3.6 discusses this topic in more detail. This part of my dissertation resulted from discussions with Martin Müller and Christoph Zenger.

5. I introduce a generic inference system for HM(X). Typing problems are commonly transformed into constraint problems that require solutions. Constraints solving consists usually of a form of unification or only a satisfiability check in the simplest case. I give a characterization of constraint solutions which I call normalization. If the constraint system enjoys most general normalizers, type inference always yields principal types. I propose various styles of type inference algorithms as well as soundness and completeness results. I specifically solve the open problem of type inference in non–regular equational theories [Ken96]. See Chapter 4 for details. The characterization of normalization results from work started with Martin Odersky and Martin Wehr.

6. In Appendix D, I give a specification of the HM(X) inference system in terms

of the functional language Haskell. The goal is to provide a rigorous specification of the essential inference components rather than to provide an optimized implementation.

7. Type inference has already been widely studied for several variations [Rém92a, EST95b, Smi91] of the Hindley/Milner system. Therefore, quite a number of constraint solvers [BS94] have already been developed. Previous constraint solvers operate mostly on a more restrictive constraint language as compared to the constraint language found in HM(X). I show how these constraint solvers fit into HM(X) and can even be lifted to the more general constraint domain found in HM(X). Details can be found in Section 4.2.

   In summary, soundness and completeness of type inference are the main technical contributions of my dissertation, see Section 4.5 for an overview of the achieved results. Furthermore, the HM(X) framework comes with a clean semantic definition of constraints. This allows for a formulation of the quantifier introduction rule which is a significant improvement over previous formulations.

Chapter 2

Background

This chapter provides some basic background material. I start with an introduction to the area of type systems. Specifically, I introduce the Hindley/Milner system. This section owes much to [Car97].

2.1 Type Systems

Type systems are formal systems which ensure certain properties such as the prevention of run–time errors when executing a program. In typed languages like Pascal [HW73], ML [MTHM97] and Haskell [PH97], program variables can be given types. Type systems are used to distinguish between well–typed programs and ill–typed ones. Typed languages enforce *static checking*. Whether a program is well–typed or not is checked at compile–time. This is in contrast to untyped languages like Lisp. In untyped languages, *dynamic* checking is found. Whether a program is well–typed or not is checked at run–time. Static checking allows to omit run–time tests if the language is type sound. Typed languages often enjoy a type soundness property. A language is type *sound* if any given well–typed program does not case a run–time error. Hence, type sound languages do not need run–time checks. Stat-

ically typed languages are also easier to debug. Program errors can be detected at compile–time which would have been otherwise deferred to run–time. A disadvantage of statically typed languages is that they are more restrictive than dynamically typed languages. Of course, there must be a trade–off between statically and dynamically typed languages.

In general, one finds often a mix between static and dynamic checking. Languages like Haskell or ML are considered to be statically typed, but some dynamic checks still remain, for example array–bounds check or division by zero check. It has been noted that the type systems in Haskell and ML are simply not expressive enough to capture properties such as array–bounds checking.

Type systems enforce certain properties such as the prevention of run–time errors. In essence, a type system can be used in two orthogonal directions. It can either extend the expressiveness of the language itself, or enforce stronger properties. For instance, consider a system which allows for dimension checking. This sort of system is able to distinguish between values representing different dimensions such as mass and time. Adding a value of dimension time to a value of dimension mass is then prohibited. In such a dimension system, more errors can already be detected at compile–time. I refer to Section 3.2 for the technical presentation of this.

On the other hand, a more expressive type system allows the set of well–typed programs to be extended. For instance, a polymorphic type system allows more programs to be typed than a monomorphic type system. Strachey [Str67] was the first to describe the concept of *parametric* polymorphism. [1] The identity function is a standard example. Given any input value x the identity function returns x as an output value. What would be a possible type for the identity function? Assuming we have a monomorphic type system, then programs such as the identity function are

[1] Later in my dissertation, I introduce the concept of *constrained* polymorphism.

therefore only allowed to take values belonging to one specific type such as Int or Bool. But then, it is no longer possible to apply integer or boolean values to the identity function in the same context. In fact, the identity function behaves independent of any specific type. In a polymorphic type system, the identity function can be given a polymorphic type. A polymorphic type represents a set of monomorphic types. This allows integer and boolean values to be applied to the identity function in the same context. Hence, a polymorphic type system allows more programs to be typed than a monomorphic type system.

2.2 The Hindley/Milner System

I give a description of the Hindley/Milner type system following [Dam85, Mil78].

The Hindley/Milner system is divided into three parts. The logical system describes the valid type judgments, the semantic model gives a meaning to valid judgments, and finally the inference system which allows to infer a type for a given untyped program. It should be noted that standard Hindley/Milner programs do not contain type annotations, although extensions [OL96] with type annotations have been explored. In the case of the standard Hindley/Milner type system, the connection between the inference and logical system is expressed by the principal types theorem [DM82]. A principal type represents the most general type for a given program. All other types which can be given to the program are derivable from the principal type. The inference system infers principal types if the program is typable at all.

2.2.1 The Logical System

Hindley/Milner programs are of the following form.

$$\begin{array}{lrcl} \textbf{Values} & v & ::= & x \mid \lambda x.e \\ \textbf{Expressions} & e & ::= & v \mid e\,e \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \end{array}$$

There are variables, lambda–abstraction and function application. Furthermore, there are let definitions. The Hindley/Milner approach has a polymorphic type system where polymorphism is introduced through the let–statement. This kind of polymorphism is also often referred to as let–polymorphism. The type language is as follows.

$$\begin{array}{lrcl} \textbf{Types} & \tau & ::= & \alpha \mid \tau \to \tau \\ \textbf{Type schemes} & \sigma & ::= & \tau \mid \forall\alpha.C \Rightarrow \sigma \end{array}$$

The type language is divided into types (also referred to as *monomorphic* types) and type schemes (also referred to as *polymorphic* types). Typical monomorphic types are type variables and function types. Types such as Int or Bool are also monomorphic types. Type schemes are universally quantified types where the type variables are only allowed to range over monomorphic types. I refer to monomorphic and polymorphic types simply as types if there are no ambiguities. It is a standard convention to let τ range monomorphic types and σ range over polymorphic types.

The set of well–typed Hindley/Milner programs is described by typing judgments. A *typing judgment* is of the form $\Gamma \vdash e : \sigma$ where Γ contains the free variables associated with its type, e is an expression and σ is a type scheme. A typing judgment is *valid* if it can be derived by application of the typing rules. The typing rules are given in Figure 2.1. Note, in rule (Abs) and (Let) the type environment Γ_x denotes the type environment $\{y : \sigma \mid y : \sigma \in \Gamma \quad y \neq x\}$. Extension of a type environment is written as $\Gamma_x.x : \sigma$.

$$\text{(Var)} \qquad \Gamma \vdash x : \sigma \quad (x : \sigma \in \Gamma)$$

$$\text{(Abs)} \qquad \frac{\Gamma_x.x : \tau \vdash e : \tau'}{\Gamma_x \vdash \lambda x.e : \tau \to \tau'}$$

$$\text{(App)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 e_2 : \tau_2}$$

$$\text{(Let)} \qquad \frac{\Gamma_x \vdash e : \sigma \qquad \Gamma_x.x : \sigma \vdash e' : \tau'}{\Gamma_x \vdash \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

$$\text{(}\forall\text{ Intro)} \qquad \frac{\Gamma \vdash e : \tau \qquad \bar{\alpha} \notin \mathit{fv}(\Gamma)}{\Gamma \vdash e : \forall \bar{\alpha}.\tau}$$

$$\text{(}\forall\text{ Elim)} \qquad \frac{\Gamma \vdash e : \forall \bar{\alpha}.\tau'}{\Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'}$$

Figure 2.1: Hindley/Milner type system

2.2.2 The Semantic Model

The semantic model given in [Mil78] is based on an ideal semantics [MPS86].

The meaning of a term is a value in the CPO $\mathcal{V}$, where $\mathcal{V}$ contains all continuous functions from $\mathcal{V}$ to $\mathcal{V}$ and an error element $\mathbf{W}$, usually pronounced "wrong". Depending on the concrete type system used, $\mathcal{V}$ might contain other elements as well. I require that the values of additional type constructors are representable in the CPO $\mathcal{V}$. Then $\mathcal{V}$ is the least solution of the equation

$$\mathcal{V} \;=\; \mathbf{W}_\perp + \mathcal{V} \to \mathcal{V}.$$

The meaning function on terms is as follows:

$$
\begin{array}{lcl}
[\![x]\!]\eta & = & \eta(x) \\
[\![\lambda u.e]\!]\eta & = & \lambda v.[\![e]\!]\eta[u := v] \\
[\![e\, e']\!]\eta & = & \mathsf{if}\ [\![e]\!]\eta \in \mathcal{V} \to \mathcal{V} \wedge [\![e']\!]\eta \neq \mathbf{W} \\
 & & \mathsf{then}\ ([\![e]\!]\eta)\,([\![e']\!]\eta) \\
 & & \mathsf{else}\ \mathbf{W} \\
 & & \\
[\![\mathsf{let}\ x = e\ \mathsf{in}\ e']\!]\eta & = & \mathsf{if}\ [\![e]\!]\eta \neq \mathbf{W} \\
 & & \mathsf{then}\ [\![e']\!]\eta[x := [\![e]\!]\eta] \\
 & & \mathsf{else}\ \mathbf{W}
\end{array}
$$

Note that I employ a call–by value semantics. It can be shown that the meaning of a well-typed program is always different from "wrong".

A *monotype* is a type τ with $fv(\tau) = \emptyset$. I let μ range over monotypes. Types denote ideals, i.e. non–empty, downward-closed and limit-closed subsets of $\mathcal{V}$. The meaning function $[\![\cdot]\!]$ maps closed types and type schemes to ideals.

$$
\begin{array}{l}
[\![\mu_1 \to \mu_2]\!] = \\
\quad \{f \in \mathcal{V} \to \mathcal{V} \mid v \in [\![\mu_1]\!] \Rightarrow f\, v \in [\![\mu_2]\!]\} \\
[\![T\, \mu_1 \ldots \mu_m]\!] = \\
\quad \{\bot\} \cup \\
\quad \bigcup\{k\, [\![\mu'_1]\!] \ldots [\![\mu'_n]\!] \mid \\
\qquad \Gamma_0 \vdash k : \mu'_1 \to \ldots \to \mu'_n \to T\, \mu_1 \ldots \mu_m\} \\
[\![\forall \bar{\alpha}.\tau]\!] = \\
\quad \bigcap\{[\![[\bar{\mu}/\bar{\alpha}]\tau]\!]\}
\end{array}
$$

The following statements hold.

Lemma 1 *Let σ be a closed type scheme. Then $[\![\sigma]\!]$ is an ideal and $\mathbf{W} \notin [\![\sigma]\!]$.*

Definition 1 *A variable environment η* models *a closed typing environment Γ, written $\eta \models \Gamma$, if for all $x : \sigma \in \Gamma$, $\eta(x) \in [\![\sigma]\!]$.*

Theorem 1 (Type Soundness) *Let $\Gamma \vdash e : \sigma$ be a valid typing judgment. Let ϕ be a substitution such that $\phi\Gamma$ and $\phi\sigma$ are closed. Let η be a variable environment such that $\eta \models \phi\Gamma$. Then*

$$(1) \quad \mathbf{W} \notin [\![\phi\sigma]\!]$$

$$(2) \quad [\![e]\!]\eta \in [\![\phi\sigma]\!]$$

The type soundness result is commonly summarized through the catch–phrase "well types programs do not go wrong".

2.2.3 The Inference System

It is common knowledge that the type inference problem in the Hindley/Milner system can be reduced to a unification problem. Robinson [Rob65] gives an algorithm which computes the most general unifier of two types or otherwise reports failure. I refer to Robinson's algorithm as a function *unify*. The interested reader should refer to [LMM87, Rob65] for a more detailed exposition.

Given two types τ_1 and τ_2. Then τ_1 and τ_2 are *unifiable* iff there exists a substitution ϕ such that $\phi\tau_1 = \phi\tau_2$. In such a case, the substitution ϕ is called a *unifier* of types τ_1 and τ_2. A substitution ψ is a *most general unifier* (commonly abbreviated by mgu) of τ_1 and τ_2 iff $\psi\tau_1 = \psi\tau_2$ and given another unifier ϕ of τ_1 and τ_2, then there exists a substitution ρ such that $\phi = \rho \circ \psi$.

Robinson could state the following result.

Theorem 2 (Robinson) *Given two types τ_1 and τ_2. Then τ_1 and τ_2 are unifiable and a most general unifier exists (and this most general unifier is unique up to variable renaming), or τ_1 and τ_2 do not have any unifiers.*

This allows to define the following well–defined *unify* function:

$$\begin{aligned} & \mathit{unify}(\tau_1, \tau_2) \\ = \;& \psi \quad \text{if } \psi \text{ is a mgu of } \tau_1 \text{ and } \tau_2 \\ = \;& \mathit{fail} \quad \text{otherwise} \end{aligned}$$

The inference system can be found in Figure 2.2. In contrast to the original presentation [DM82], I present the inference system as a deduction system with clauses of the form $\psi, \Gamma \vdash^W e : \tau$ with type environment Γ, expression e as input values and substitution ψ, type τ as output values. For each syntactic construct of expressions e there is one clause. The deduction rules can be interpreted operationally, as a logic program that constructs a bottom–up derivation of $\vdash^W$ clauses. The operator $\sqcup$ in rules (App) and (Let) builds the least upper bound of substitutions.

In the (Var) rule, an unqualified type τ is represented as $\forall\emptyset.\tau$. This avoids a separate case of this rule for unqualified types. In rule (App), the typing problem is translated into a unification problem. Then, the three resulting substitutions are combined with the $\sqcup$ operator which computes the least upper bound by constructing the mgu.

In [DM82] the following results were stated. Any judgment derivable by the inference system is also derivable in the logical system.

Theorem 3 (Soundness of Inference) *Given a term e and a type environment Γ. If $\psi, \Gamma \vdash^W e : \tau$ then $\psi\Gamma \vdash e : \tau$ and $\psi\tau = \tau$.*

Before I state the completeness result, I restate the definition of a generic instance.

$$\text{(Var)} \qquad \frac{x : (\forall \bar{\alpha}.\tau) \in \Gamma \qquad \bar{\beta} \text{ new}}{id, \Gamma \vdash^W x : [\bar{\beta}/\bar{\alpha}]\tau}$$

$$\text{(Abs)} \qquad \frac{\psi, \Gamma_x.x : \alpha \vdash^W e : \tau \qquad \alpha \text{ new}}{\psi_{\setminus\{\alpha\}}, \Gamma_x \vdash^W \lambda x.e : \psi(\alpha) \to \tau}$$

$$\text{(App)} \qquad \frac{\begin{array}{c}\psi_1, \Gamma \vdash^W e_1 : \tau_1 \qquad \psi_2, \Gamma \vdash^W e_2 : \tau_2 \\ \psi_3 = \mathit{unify}(\tau_1, \tau_2 \to \alpha) \qquad \alpha \text{ new} \\ \psi = \psi_1 \sqcup \psi_2 \sqcup \psi_3\end{array}}{\psi_{|fv(\Gamma)}, \Gamma \vdash^W e_1 e_2 : \psi \circ \psi'(\alpha)}$$

$$\text{(Let)} \qquad \frac{\begin{array}{c}\psi_1, \Gamma_x \vdash^W e : \tau \qquad \sigma = \forall \bar{\alpha}.\tau \qquad \bar{\alpha} = fv(\tau) \backslash fv(\Gamma_x) \\ \psi_2, \Gamma_x.x : \sigma \vdash^W e' : \tau' \\ \psi' = \psi_1 \sqcup \psi_2\end{array}}{\psi_{|fv(\Gamma_x)}, \Gamma_x \vdash^W \mathsf{let}\ x = e\ \mathsf{in}\ e' : \psi\tau'}$$

Figure 2.2: Hindley/Milner type inference

Given a type scheme $\sigma = \forall \alpha_1, \ldots \alpha_n.\tau$. Then $\sigma' = \forall \beta_1, \ldots, \beta_m.\tau'$ is a *generic instance* of σ if $\tau' = [\tau_i/\beta_i]\tau$ for some types $\tau_1, \ldots, \tau_n$ and β_j are not free in σ. In this case, I write $\sigma < \sigma'$.

Furthermore, I define a generalization function *gen*. Given a type environment Γ and a type τ. Then $\mathit{gen}(\Gamma, \tau) = \forall \bar{\alpha}.\tau$ where $\bar{\alpha} = fv(\tau) \backslash fv(\Gamma)$.

Theorem 4 (Completeness of Inference) *Let $\phi\Gamma \vdash e : \sigma$ be a typing judgment. Then $\psi, \Gamma \vdash^W e : \tau$ for some substitutions ψ, ρ and type τ such that $\rho\mathit{gen}(\psi\Gamma, \tau) < \sigma$.*

The completeness theorem can be simplified for top–level programs to the following corollary, which states that the type inference algorithm computes principal types. Given a type environment Γ and an expression σ, then σ is a *principal type* of e under Γ iff $\Gamma \vdash e : \sigma$ and if $\Gamma \vdash e : \sigma'$ for any other σ' then $\sigma < \sigma'$.

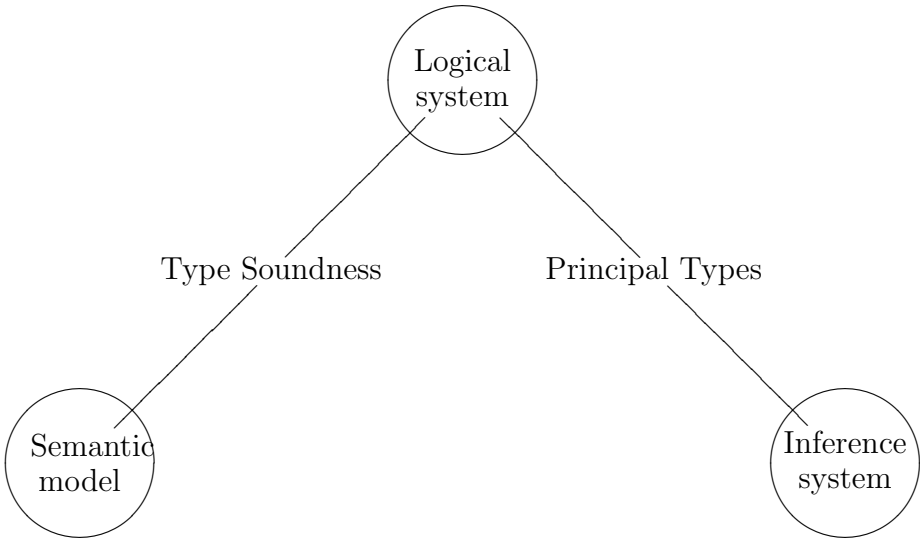


Figure 2.3: The Hindley/Milner system

Corollary 1 (Principal Types) *Let* $\Gamma \vdash e : \sigma$ *be a closed typing judgment. Then* $id, \Gamma \vdash^W e : \tau$ *such that* $gen(\Gamma, \tau) < \sigma$.

2.2.4 Summary

The Hindley/Milner system consists of three components: the logical system which describes the well–typed programs, the inference system which infers principal types and the semantic model which ensures that well–typed programs do not go wrong. Figure 2.3 exhibits the relationships among the three components. In the remaining part of my dissertation, I consider an extension of the Hindley/Milner system with constraints which retains the two elementary properties of the standard Hindley/Milner system: the type soundness result and the principal types property.

Chapter 3

The HM(X) framework

This chapter starts with some basic terminology and results about constraints. It follows the presentation of the logical part of the HM(X) system. Type system instances of HM(X) are defined by a constraint system X (which describes the constraint language) and a term algebra $\mathcal{T}$ (which describes the type language). The constraint language might be richer than the constraints which are allowed to actually appear in typing judgments. I refer to the set of constraints which are allowed to appear in typing judgments as solved forms, denoted by $\mathcal{S}$. For instance, the standard Hindley/Milner system is obtained by taking X to be the Herbrand constraint system and $\mathcal{S}$ to consist only of $\{\}$ which represents the true constraint.

One of the novel aspects in HM(X) is the use of a semantic notion of constraints. This is reflected in the logical system which allows a formulation of the following quantifier introduction rule:

$$(\forall \text{ Intro}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

where C and D are constraints, Γ is a type environment, e is a program and σ is a

type scheme. The novelty is the existential quantifier $\exists\alpha$ which allows to bind free type variables also in the constraint part.

I present two alternative approaches to achieve type soundness for HM(X) instances. The first approach stays in the tradition of Milner's original semantics. I give a standard untyped compositional semantic based on the ideal model [MPS86]. I also provide a second approach which is along the lines of [WF91]. The key idea is based upon an operational formulation of a language's semantics by rewriting.

Section 3.4 discusses properties of HM(X) type systems and introduces some basic definitions used throughout my dissertation.

Section 3.5 relates HM(X) to previous approaches. I discuss specifically issues regarding the different formulations of the quantifier introduction rule found in previous approaches [Jon92, AW93, Smi91, TS96]. I show how the HM(X) quantifier introduction rule relates to the open or closed world assumption found in overloading systems [OWW95, Smi91]. In addition, I consider Ohori's record calculus [Oho95] and the message passing system of Nishimura [Nis98]. I also point out the problems in their systems which are related to their formulation of the quantifier introduction rule.

In fact, I give two presentations of the logical system: one in term form, the other in constraint form. Type descriptions are usually represented in term form. For instance, recursive types such as $\mu\alpha.\alpha \to \beta$ are represented explicitly through the μ notation in the type language. The following is a valid judgment in a variation of the Hindley/Milner system with recursive types:

$$\{x : \mu\alpha.\alpha \to \beta\} \vdash xx : \beta$$

Variable x is applied to itself, therefore it needs to be of recursive type. Recursive

types are represented by the μ notation in the syntactic domain of types. But recursive types can also be represented in the constraint language by equality constraints. The above judgment can also be represented as

$$(\alpha = \alpha \to \beta), \{x : \alpha\} \vdash xx : \beta$$

where the constraint $(\alpha = \alpha \to \beta)$ contains all essential typing information. I refer to the former presentation of judgments as the *term* presentation and to the latter as the *constraint* presentation. Section 3.6 studies the two possible representation forms. The idea of expressing all typing information in the constraint language is further developed in Section 3.7.

3.1 Constraints

I present a characterization of constraint systems along the lines of Henkin [HMT71] and Saraswat [Sar93]. Building on the standard notions of *simple* and *cylindric constraint systems*, I introduce *term constraint systems* as constraint systems which have a well-behaved notion of substitution. It must be noted that the notion of constraints found here differs from the standard notion where constraint systems are regarded as a kind of first–order theory. Conceptually, there would not be any problems using the standard formulation of constraint systems, I prefer to follow the notion of constraints found in [HMT71, Sar93]. The notion of cylindric algebras captures the minimal requirements needed to describe the constraint part of the HM(X) framework. The constraint system X will be the parameter which allows the HM(X) framework to be customized to different application domains.

Definition 2 *A* simple constraint system *is a structure* $(\Omega, \vdash^e)$ *where* Ω *is a non–*

empty set of tokens *or* (primitive) constraints*. I also refer to such constraints as* predicates*. The relation* $\vdash^e \subseteq p\Omega \times \Omega$ *is an* entailment relation *where* $p\Omega$ *is the set of finite subsets of* Ω*. I call* $C \in p\Omega$ *a constraint set or simply a constraint.*

A constraint system $(\Omega, \vdash^e)$ *must satisfy for all constraints* $C, D \in p\Omega$*:*

C1 $C \vdash^e P$ *whenever* $P \in C$ *and*

C2 $C \vdash^e Q$ *whenever*

$C \vdash^e P$ *for all* $P \in D$ *and* $D \vdash^e Q$

I extend $\vdash^e$ to be a relation on $p\Omega \times p\Omega$ by: $C \vdash^e D$ iff $C \vdash^e P$ for every $P \in D$. Furthermore, I define $C =^e D$ iff $C \vdash^e D$ and $D \vdash^e C$. The term $\vdash^e C$ is an abbreviation for $\emptyset \vdash^e C$ and $\mathsf{true} = \{ P \mid \emptyset \vdash^e P\}$ represents the true element.

I give an example how to generate a simple constraint system based on a first–order language $\mathcal{L}$.

Example 1 *For any first–order language* $\mathcal{L}$*, and countably infinite set of variables Var, take* Ω *to be an arbitrary subset of open* ($\mathcal{L}$*, Var)–formulas, and* $\vdash^e$ *to be the entailment relation with respect to some class* Δ *of* $\mathcal{L}$*–structures. That is,* $\{P_1, \ldots, P_n\} \vdash^e Q$ *iff for every structure* $M \in \Delta$*, an* M*–valuation realizes* Q *whenever it realizes each of* $P_1, \ldots, P_n$*. Such a* $(\Omega, \vdash^e)$ *is a simple constraint system.*

The extension of a simple constraint system with a *projection operator* $\exists\bar{\alpha}$ leads to a cylindric constraint system.

Definition 3 *A* cylindric constraint system *is a structure* $\mathcal{CCS} = (\Omega, \vdash^e, Var, \{\exists\alpha \mid \alpha \in$

Var$\}$) *such that:*

- $(\Omega, \vdash^e)$ *is a simple constraint system,*
- *Var is an infinite set of variables,*
- *For each variable* $\alpha \in Var$, $\exists\alpha : p\Omega \to p\Omega$ *is an operation satisfying:*

  E1 $C \vdash^e \exists\alpha.C$

  E2 $C \vdash^e D$ *implies* $\exists\alpha.C \vdash^e \exists\alpha.D$

  E3 $\exists\alpha.(C \wedge \exists\alpha.D) =^e (\exists\alpha.C) \wedge (\exists\alpha.D)$

  E4 $\exists\alpha.\exists\beta.C =^e \exists\beta.\exists\alpha.C$

Remark 1 *For simplicity, I omit set notation for constraints, and connect constraints by* $\wedge$ *instead of the union operator* $\cup$ *and generally do not enclose simple constraints* P *in opening and closing braces. For instance,* $P \wedge Q$ *is an abbreviation for* $\{P\} \cup \{Q\}$*. I assume that* $\wedge$ *binds tighter than* $\exists\bar{\alpha}$*. For instance,* $\exists\bar{\alpha}.C \wedge D$ *stands for* $\exists\bar{\alpha}.(C \wedge D)$*. I write* $C =^e D$ *iff* $C \vdash^e D$ *and* $D \vdash^e C$*.*

Example 2 *Let the token set* Ω *consist of some subclass of* $(\mathcal{L}, Var)$ *formulas closed under existential quantification of finite conjunctions. Each operator* $\exists\bar{\alpha}$ *is then interpreted by the function which maps each finite set* $\{P_1, \ldots, P_n\}$ *of tokens to the set of tokens* $\{\exists\bar{\alpha}.P_1 \wedge \ldots \wedge P_n\}$*. This way the four conditions above are satisfied.*

The projection operator $\exists\bar{\alpha}$ allows us to bind variables $\bar{\alpha}$ in a constraint. If the constraint system models a boolean algebra, projection corresponds to existential quantification. Based on the projection operator, I define the free variables $fv(C)$ and satisfiability of a constraint C.

Definition 4 *Let* C *be a constraint. Then* $fv(C) = \{\alpha \mid \exists\alpha.C \neq^e C\}$*.*

Definition 5 *Let C be a constraint. Then C is* satisfiable *iff* $\vdash^e \exists fv(C).C$.

Lemma 2 *Let C be a constraint. Then C is satisfiable iff $\exists\alpha.C$ is satisfiable.*

Proof: I only consider the case where $\alpha \in fv(C)$. C is satisfiable iff $\vdash^e \exists fv(C).C$ iff $\vdash^e \exists fv(C).\exists\alpha.C \vdash^e \exists fv(\exists\alpha.C).\exists\alpha.C$ iff $\exists\alpha.C$ is satisfiable. ■

The final step in the modelling of constraint systems is the extension from cylindric constraint systems to term constraint systems. I assume a term algebra $\mathcal{T}$ with signature $\Sigma = (Var, Cons)$ as given. *Var* is a set of variables and *Cons* is a set of type constructors containing at least the function constructor $\rightarrow$ of arity 2. In examples below I will sometimes use a multi-sorted algebra, in which terms and constructors are partitioned into sorts. Always present will be the sort of *types* which is ranged over by τ.

Definition 6 *A* substitution *ϕ is an idempotent mapping from the set of variables Var to the term algebra Term(Σ) which is the identity everywhere except on a finite set of variables. Substitutions are extended homomorphically to types.*

Definition 7 *A* term constraint system *$\mathcal{TCS}_{\mathcal{T}} = (\Omega, \vdash^e, Var, \{\exists\alpha \mid \alpha \in Var\})$ over a term algebra $\mathcal{T}$ is a cylindric constraint system with predicates of the form*

$$p(\tau_1, \ldots, \tau_n) \qquad (\tau_i \in \mathcal{T})$$

such that the following holds:

- *For each pair of types τ, τ' there is an equality predicate $(\tau = \tau')$ in $\mathcal{TCS}_{\mathcal{T}}$, which satisfies:*

  D1 $\vdash^e (\alpha = \alpha)$

  D2 $(\alpha = \beta) \vdash^e (\beta = \alpha)$

  D3 $(\alpha = \beta) \wedge (\beta = \gamma) \vdash^e (\alpha = \gamma)$

  D4 $(\alpha = \beta) \wedge \exists\alpha.(C \wedge (\alpha = \beta)) \vdash^e C$

  D5 $(\tau = \tau') \vdash^e (T[\tau] = T[\tau'])$

  where $T[]$ is an arbitrary term context

- *For each predicate $p(\tau_1, \ldots, \tau_n)$,*

  D6 $[\tau/\alpha]p(\tau_1, \ldots, \tau_n) =^e \exists\alpha.(p(\tau_1, \ldots, \tau_n) \wedge (\alpha = \tau))$

  where $\alpha \notin fv(\tau)$

Remark 2 *Conditions* **D1** – **D4** *are the conditions imposed on a cylindric constraint system with diagonal elements, which is usually taken as the foundation of constraint programming languages.* **D4** *says that equals can be substituted for equals; it is in effect the Leibniz principle.* **D5** *states that (=) is a congruence.* **D6** *connects the syntactic operation of a substitution over predicates* [1] *with the semantic concepts of projection and equality. Substitution is extended to arbitrary constraints in the canonical way:*

$$[\tau/\alpha](P_1 \wedge \ldots \wedge P_n) = [\tau/\alpha]P_1 \wedge \ldots \wedge [\tau/\alpha]P_n.$$

Lemma 3 (Renaming) *Let C be a constraint and β a new type variable. Then $\exists\alpha.C =^e \exists\beta.[\beta/\alpha]C$.*

[1] Substitutions is only defined on types but is extended naturally to predicates; $[\tau/\alpha]p(\tau_1, \ldots, \tau_n) = p([\tau/\alpha]\tau_1, \ldots, [\tau/\alpha]\tau_n)$.

Proof: $\exists\alpha.C =^e \exists\alpha.\exists\beta.((\alpha = \beta) \wedge C) =^e \exists\beta.\exists\alpha.((\alpha = \beta) \wedge C) =^e \exists\beta.[\beta/\alpha]C.$

■

Lemma 4 (Normal Form) *Let C be a constraint and $\phi = [\bar{\tau}/\bar{\alpha}]$ be a substitution. Then $\phi C =^e \exists\bar{\alpha}.((\alpha_1 = \tau_1) \wedge \ldots \wedge (\alpha_n = \tau_n) \wedge C)$.*

Proof: I assume $\phi = [\tau_1/\alpha_1, \tau_2/\alpha_2]$. Remember that substitutions are idempotent mappings, hence $\alpha_1 \notin fv(\tau_2)$ and $\alpha_2 \notin fv(\tau_1)$. Then $\phi C =^e \exists\alpha_1.((\alpha_1 = \tau_1) \wedge (\exists\alpha_2.(\alpha_2 = \tau_2) \wedge C)) =^e \exists\alpha_1.\exists\alpha_2.((\alpha_1 = \tau_1) \wedge (\alpha_2 = \tau_2) \wedge C)$. Note that in the last proof step it was essential that $\alpha_2 \notin (\tau_1)$.

■

Lemma 5 (Substitution) *Let C, D be constraints such that $C \vdash^e D$ and ϕ be a substitution. Then $\phi C \vdash^e \phi D$.*

Proof: I assume $\phi = [\bar{\tau}/\bar{\alpha}]$. Given $C \vdash^e D$. Then $(C \wedge (\bar{\alpha} = \bar{\tau})) \vdash^e (D \wedge (\bar{\alpha} = \bar{\tau}))$. The entailment relation is closed under projection. Then $\exists\bar{\alpha}.(C \wedge (\bar{\alpha} = \bar{\tau})) \vdash^e \exists\bar{\alpha}.(D \wedge (\bar{\alpha} = \bar{\tau}))$.

■

Example 3 *For any term algebra $\mathcal{T}$ let* HERBRAND $= (\Omega, \vdash^e, Var, \{\exists\alpha \mid \alpha \in Var\})$ *be the minimal term constraint system where Ω contains only primitive constraints of the form $(\tau = \tau')$ where τ and τ' are types from $\mathcal{T}$. Equality in* HERBRAND *is syntactic,* i.e. *$\mathcal{T}$ is a free algebra. Satisfiability can be checked by (first–order) unification.*

A more refined example of a term constraint system deals with physical dimension types in the style of Kennedy [Ken96]. The idea is to distinguish between values representing different dimensions such as mass and time. To this purpose, the type language is now divided into dimension types and the common types.

Example 4 *Let $\mathcal{T}$ be the two-sorted term algebra consisting of* dimensions *and* types.

$$\begin{array}{lrcl} \textbf{Dimensions} & d & ::\supseteq & \alpha \mid i(d) \mid \mathit{prod}(d,d) \mid 1 \mid \mathbf{m} \mid \mathbf{s} \\ \textbf{Types} & \tau & ::= & \alpha \mid \dim(d) \mid \tau \to \tau \end{array}$$

The dimension constructor $i(\cdot)$ corresponds to the inverse of a dimension and $\mathit{prod}(\cdot,\cdot)$ to the product of two dimensions. Dimension constants are 1 *for the unit measure,* $\mathbf{m}$ *for meters and* $\mathbf{s}$ *for seconds. There might be other dimension constructors besides the mentioned ones, therefore the language of dimension types is left open. A type is either a type variable, or a dimension, or a function type.* DIM *is then the term constraint system which obeys the following additional conditions, which specify that dimension types form an Abelian group.*

$$\begin{array}{ll} \text{DIM}\mathbf{1} & \vdash^e (\mathit{prod}(\alpha,\beta) = \mathit{prod}(\beta,\alpha)) \\ \text{DIM}\mathbf{2} & \vdash^e (\mathit{prod}(\alpha,\mathit{prod}(\beta,\gamma)) = \mathit{prod}(\mathit{prod}(\alpha,\beta),\gamma)) \\ \text{DIM}\mathbf{3} & \vdash^e (\mathit{prod}(\alpha,1) = \alpha) \\ \text{DIM}\mathbf{4} & \vdash^e (\mathit{prod}(\alpha,i(\alpha)) = 1) \end{array}$$

A similar idea can be employed to encode a simple form of arithmetic.

Example 5 *Let $\mathcal{T}$ be a two–sorted algebra consisting of* indices *and* types.

$$\begin{array}{lrcl} \textbf{Indices} & i & ::= & \alpha \mid +(i,i) \mid 0 \mid 1 \\ \textbf{Types} & \tau & ::= & \alpha \mid \mathsf{index}(i) \mid \tau \to \tau \end{array}$$

The index constants 0 *and* 1 *are constant type constructors and represent the corresponding values. Constructor $+(\cdot,\cdot)$ has the obvious meaning. The constructors obey the usual laws of arithmetic. This can be specified by constraint rules of the form:*

$$\textbf{INDEX1} \quad \vdash^e (+(1,0) = 1)$$

$$\textbf{INDEX2} \quad \vdash^e (+(i_1, i_2) = +(i_2, i_1))$$

$$\textbf{INDEX3} \quad \vdash^e (+(+(i_1, i_2), i_3) = +(i_1, +(i_2, i_3)))$$

In the final example, I consider an extension of a term constraint system with subtyping.

Example 6 *A* subtype constraint system *over a term algebra $\mathcal{T}$ is a term constraint system with a subtype predicate $(\tau <: \tau')$ for each pair of types τ and τ' which satisfies the following conditions.*

$$\textbf{SUB1} \quad (\alpha = \alpha') =^e (\alpha <: \alpha') \wedge (\alpha' <: \alpha)$$

$$\textbf{SUB2} \quad \frac{D \vdash^e (\alpha_1' <: \alpha_1) \quad D \vdash^e (\alpha_2 <: \alpha_2')}{D \vdash^e (\alpha_1 \to \alpha_2 <: \alpha_1' \to \alpha_2')}$$

$$\textbf{SUB3} \quad \frac{D \vdash^e (\alpha_1 <: \alpha_2) \quad D \vdash^e (\alpha_2 <: \alpha_3)}{D \vdash^e (\alpha_1 <: \alpha_3)}$$

Let $\mathcal{SC}$ be a subtype constraint system with primitive types Int *and* Float *and record types of the form $\{l_1 : \tau_1, \ldots, l_n : \tau_n\}$. Records are modelled by admitting constructors of the form*

$$l_{1-} \ldots l_{n-} : \tau_1 \to \ldots \to \tau_n \to \{l_1 : \tau_1, \ldots, l_n : \tau_n\}$$

in the term algebra. I assume that record fields are ordered with respect to a given ordering relation on field labels. The additional types obey the following rules.

SUB4 $\vdash^e (\mathsf{Int} <: \mathsf{Float})$

SUB5 $\vdash^e (\{l_1 : \tau_1, \ldots, l_n : \tau_n, \ldots\} <: \{l_1 : \tau_1, \ldots, l_n : \tau_n\})$

SUB6 $$\frac{D \vdash^e (\tau_1 <: \tau_1') \ldots D \vdash^e (\tau_n <: \tau_n')}{D \vdash^e (\{l_1 : \tau_1, \ldots, l_n : \tau_n\} <: \{l_1 : \tau_1', \ldots, l_n : \tau_n'\})}$$

3.1.1 Summary

Constraints are semantically described by adopting the notion of cylindric algebras found in the constraint logic programming framework [JM94]. Cylindric algebras come with a projection operator $\exists\bar{\alpha}$ that binds some subset of variables $\bar{\alpha}$ in the constraint. In the usual case where constraints are boolean algebras, projection corresponds to existential quantification. The syntactic concept of substitutions can be described semantically by projection and equality, i.e. $[\tau/\alpha]C = \exists\alpha.((\alpha = \tau) \wedge C)$ where C is a constraint. I introduced term constraint systems which have a well–behaved notion of substitution. Term constraint systems will be the parameter which allows the HM(X) framework to be customized to different application domains. Commonly, I refer to term constraint systems as constraint systems.

3.2 Logical System

I assume given a term constraint system X over a term algebra $\mathcal{T}$ as the parameter. The HM(X) system consists of the following syntactic domains:

$$
\begin{array}{lrcl}
\textbf{Values} & v & ::= & x \,|\, c \,|\, \lambda x.e \\
\textbf{Expressions} & e & ::= & v \,|\, e\, e \,|\, \mathsf{let}\ x = e\ \mathsf{in}\ e \\
\textbf{Types} & \tau & ::\supseteq & \alpha \,|\ \tau \to \tau \\
\textbf{Type schemes} & \sigma & ::= & \tau \,|\, \forall \alpha.C \Rightarrow \sigma
\end{array}
$$

Compared to the standard Hindley/Milner system, types in HM(X) are now members of an arbitrary term algebra which contains at least type variables and function types. Hence, the type language is left open expressed by the symbol $\supseteq$. Section 3.1 described more complex type languages such as dimension types. Type schemes $\forall \alpha.C \Rightarrow \sigma$ now include a constraint component C, which restricts the types that can be substituted for the type variable α. Constraint components appearing in type schemes must always be satisfiable.

The language of terms is exactly as in [DM82], that is, I assume that any language constructs that make use of type constraints are expressible as predefined values, whose names and types are recorded in the initial type environment Γ_0.

The typing rules can be found in Figure 3.1. The most interesting rules in Figure 3.1 are the ($\forall$ Intro) rule and the ($\forall$ Elim) rule. By rule ($\forall$ Intro) I quantify some type variables. I often use vector notation for type variables in type schemes. The term $\forall \bar{\alpha}.D \Rightarrow \tau$ is an abbreviation for $\forall \alpha_1.\mathsf{true} \Rightarrow \ldots \forall \alpha_n.D \Rightarrow \tau$ and $\exists \bar{\alpha}.D$ is an abbreviation for $\exists \alpha_1. \ldots \exists \alpha_n.D$. Constraints C appearing on the left–hand side of the turnstile and in type schemes should be seen as the equivalence class $\bar{C} = \{D \,|\, D =^e C\}$ of all constraints which are semantically equivalent to C. It must be noted that the set of free variables is independent of a specific representative of the equivalence class. Given $C =^e D$ then $fv(C) = fv(D)$ which is easy to verify. In general, I omit the equivalence class notation.

$$\text{(Var)} \qquad C, \Gamma \vdash x : \sigma \quad (x : \sigma \in \Gamma)$$

$$\text{(Sub)} \qquad \frac{C, \Gamma \vdash e : \tau \quad C \vdash^e (\tau \preceq \tau')}{C, \Gamma \vdash e : \tau'}$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x.x : \tau \vdash e : \tau'}{C, \Gamma_x \vdash \lambda x.e : \tau \to \tau'}$$

$$\text{(App)} \qquad \frac{C, \Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad C, \Gamma \vdash e_2 : \tau_1}{C, \Gamma \vdash e_1 e_2 : \tau_2}$$

$$\text{(Let)} \qquad \frac{C, \Gamma_x \vdash e : \sigma \qquad C, \Gamma_x.x : \sigma \vdash e' : \tau'}{C, \Gamma_x \vdash \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

$$\text{(}\forall\text{ Intro)} \qquad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

$$\text{(}\forall\text{ Elim)} \qquad \frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash^e [\bar{\tau}/\bar{\alpha}]D}{C, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'}$$

Figure 3.1: Logical system

Typing judgments are of the form $C, \Gamma \vdash e : \sigma$ where C is a constraint in X, Γ a type environment and σ a type scheme. A typing judgment is *valid* if it can be derived by application of the typing rules and its constraint component is satisfiable.

In some typing situations, it might be useful to further restrict the set of constraints allowed to appear in typing judgments and type schemes. I assume that the set of constraints appearing in type schemes and on the left hand side of the turnstile is restricted to so–called *solved forms*. The set of solved forms, denoted by $\mathcal{S}$, is always a subset of the satisfiable constraints in X. Given a constraint C appearing on the left–hand side of the turnstile or in a type scheme, then there must be a constraint $D \in \mathcal{S}$ such that $C =^e D$ which states C, D are semantically equivalent.

The set $\mathcal{S}$ clearly affects the number of valid typing judgments for a given HM(X) instance. For instance, take $\mathcal{S}$ to be the empty set. Then there would not be any valid typing judgments at all. I elaborate more on such issues in Section 3.6.

Unlike in standard treatments of Hindley/Milner style systems, there is also a subsumption rule (Sub), which allows to derive term e with type τ' if one can derive term e with type τ and type τ subsumes type τ'. The subsumption relation $\preceq$ is determined by the constraint system X, and is assumed to satisfy the standard axioms for a partial ordering plus the contra-variance rule:

REFL $$(\alpha = \alpha') \vdash^e (\alpha \preceq \alpha') \wedge (\alpha' \preceq \alpha)$$

ASYM $$(\alpha \preceq \alpha') \wedge (\alpha' \preceq \alpha) \vdash^e (\alpha = \alpha')$$

TRANS $$\frac{D \vdash^e (\alpha_1 \preceq \alpha_2) \quad D \vdash^e (\alpha_2 \preceq \alpha_3)}{D \vdash^e (\alpha_1 \preceq \alpha_3)}$$

CONTRA $$\frac{D \vdash^e (\alpha_1' \preceq \alpha_1) \quad D \vdash^e (\alpha_2 \preceq \alpha_2')}{D \vdash^e (\alpha_1 \to \alpha_2 \preceq \alpha_1' \to \alpha_2')}$$

Except for these conditions, the choice of $\preceq$ is arbitrary. Please note, that the subsumption relation $\preceq$ not necessarily equals the subtyping relation in subtype systems such as the one considered in Example 6 in Section 3.1.

I maintain that the definition of an instance of the HM(X) system starts with the definition of an appropriate term algebra $\mathcal{T}$ and a constraint system X, which captures all desired properties and also defines the subsumption relation $\preceq$. The definition of the set $\mathcal{S}$ of solved forms describes the constraints, which are allowed to

appear in type schemes and typing judgments. It remains to define the set of primitive functions and record them in the initial type environment Γ_0. An instance of the HM(X) system is defined by the five–tuple $(\mathrm{X}, \preceq, \mathcal{T}, \mathcal{S}, \Gamma_0)$ to which I commonly refer as HM(X) or only X for short.

Example 7 *The Hindley/Milner system is an instance of the HM(X) type system framework. Take* X *to be the Herbrand constraint system over the algebra of types* τ*. Take the set of solved forms to be the set consisting only of* true*, which is represented by the empty token set. Take* $\preceq$ *to be syntactic type equality. Then the only type schemes arising in proof trees of valid typing judgments are of the form* $\forall\alpha.\{\} \Rightarrow \sigma$*, which corresponds to Hindley/Milner type schemes* $\forall\alpha.\sigma$*. The subsumption rule becomes the trivial tautology which states that a judgment can be derived if it can be derived. It is easy to convince oneself that a judgment* $\Gamma \vdash e : \sigma$ *is derivable in Hindley/Milner if and only if* $\{\}, \Gamma \vdash e : \sigma$ *is derivable in* HM(HERBRAND)*. It must be noted that the judgment* [2]

$$(\alpha = \beta), \{x : \alpha, y : \beta\} \vdash x = y : Bool$$

is not derivable in HM(HERBRAND) *because the constraint* $(\alpha = \beta)$ *is not contained in the set of solved forms.*

Example 8 *Let* X *be the constraint system* DIM*, let the set of solved forms be the set consisting only of* true*, and let subsumption* $\preceq$ *be the equality relation* $=$ *in* DIM*. Then Kennedy's system can be recovered simply by adding primitives to the initial type environment* Γ_0 *that deal with dimensions.* E.g. *I assume that*

$$div : \forall d_1, d_2.\, \dim(d_1) \to \dim(d_2) \to \dim(prod(d_1, i(d_2)))$$

[2] Given the primitive $=: \forall\alpha.\alpha \to \alpha \to \mathsf{Bool}$ in the initial type environment Γ_0.

is contained in Γ_0*. Other basic connectives are treated analogously.*

Example 9 *Let* X *be the subtype constraint system* $\mathcal{SC}$ *and let the subsumption relation* $\preceq$ *be equal to the subtyping relation* $<:$*. Let the set of solved forms* $\mathcal{S}$ *be all satisfiable constraints in* $\mathcal{SC}$*. For every record* $\{l_1 : \tau_1, \ldots, l_n : \tau_n\}$ *in a program, I add a datatype constructor*

$$l_1_\ldots_l_n : \tau_1 \to \ldots \to \tau_n \to \{l_1 : \tau_1, \ldots, l_n : \tau_n\}$$

and for every field label l *I add a function*

$$(_.l) : \{l : \tau\} \to \tau$$

to the initial type environment Γ_0*. The first corresponds to record creation, the second to record selection. Other basic primitive functions are defined analogously.*

The resulting system is related to the subtyping approach of the Hopkins Object Group [EST95b]. The main difference is that I use logical rules for quantifier introduction and elimination, whereas the Hopkins Object Group use a syntactic approach where quantifier introduction is coupled with let*, and quantifier elimination is coupled with variable use. They also consider updatable references and variants. If considering only programs involving record operations, then HM(*$\mathcal{SC}$*) and the subtyping approach of the Hopkins Object Group type the same set of programs.*

3.2.1 Summary

HM(X) instances are described by the five–tuple $(\mathrm{X}, \preceq, \mathcal{T}, \mathcal{S}, \Gamma_0)$. The term algebra $\mathcal{T}$ describes the language of types and the term constraint X the language of constraints. X furthermore defines the subsumption predicate $\preceq$. The set of solved forms $\mathcal{S}$

describes the constraints which are allowed to appear in type schemes and typing judgments. Constraints in $\mathcal{S}$ have to be at least satisfiable. Primitive functions are recorded in the initial type environment Γ_0.

3.3 Type Soundness

3.3.1 A Denotational Approach

I give a type soundness theorem based on an ideal semantics [MPS86] for HM(X) type systems. The result holds only for a restricted domain of types, e.g. recursive types are excluded. A more general result based on a syntactic approach can be found in the next section.

I show that HM(X) type system instances are sound, provided the underlying constraint system is sound and the subsumption predicate ($\preceq$) satisfies a coherence property. I state a constraint system is *sound* if every satisfiable constraint has a monotype solution. *Coherence* of a constraint system means that if a type τ subsumes a type τ', then the denotation of τ in the ideal model is a subset of the denotation of τ'.

Definition 8 *A* monotype *is a type τ with $fv(\tau) = \emptyset$.*

I let μ range over monotypes.

Definition 9 *A constraint system* X *is* sound *if for all type variables $\bar{\alpha}$ and constraints $C \in \mathcal{S}$, if $\vdash^e \exists\bar{\alpha}.C$ then there are monotypes $\bar{\mu}$ such that $\vdash^e \exists\bar{\alpha}.((\bar{\alpha} = \bar{\mu}) \wedge C)$.*

The soundness proof is based on an ideal semantics of types which is a direct extension of the semantics in [Mil78].

The meaning of a term is a value in the CPO $\mathcal{V}$, where $\mathcal{V}$ contains all continuous functions from $\mathcal{V}$ to $\mathcal{V}$ and an error element $\mathbf{W}$, usually pronounced "wrong". Depending on the concrete type system used, $\mathcal{V}$ might contain other elements as well. I require that the values of additional type constructors are representable in the CPO $\mathcal{V}$. Then $\mathcal{V}$ is the least solution of the equation

$$\mathcal{V} \;=\; \mathbf{W}_\perp \;+\; \mathcal{V} \to \mathcal{V} \;+\; \textstyle\sum_{k \in \mathcal{K}} \left(k\, \mathcal{V}_1 \ldots \mathcal{V}_{\mathrm{arity}(k)}\right)_\perp$$

where $\mathcal{K}$ is the set of values of an additional type constructor T.

The meaning function on terms is the same as in the original semantics of Hindley/Milner terms. I assume that any language constructs that make use of type constraints are expressible as predefined values, whose names and types are recorded in the initial type environment.

$$\begin{aligned}
[\![x]\!]\eta &= \eta(x) \\
[\![\lambda u.e]\!]\eta &= \lambda v.[\![e]\!]\eta[u := v] \\
[\![e\, e']\!]\eta &= \mathsf{if}\ [\![e]\!]\eta \in \mathcal{V} \to \mathcal{V} \wedge [\![e']\!]\eta \neq \mathbf{W} \\
&\quad\ \mathsf{then}\ ([\![e]\!]\eta)\,([\![e']\!]\eta) \\
&\quad\ \mathsf{else}\ \mathbf{W} \\
[\![\mathsf{let}\ x = e\ \mathsf{in}\ e']\!]\eta &= \mathsf{if}\ [\![e]\!]\eta \neq \mathbf{W} \\
&\quad\ \mathsf{then}\ [\![e']\!]\eta[x := [\![e]\!]\eta] \\
&\quad\ \mathsf{else}\ \mathbf{W}
\end{aligned}$$

I will show in the following that the meaning of a well-typed program is always different from "wrong".

As a first step, I give a meaning to types. Following [Mil78], I let types denote ideals, i.e. non–empty, downward-closed and limit-closed subsets of $\mathcal{V}$. The meaning function $[\![\cdot]\!]$ maps closed types and type schemes to ideals. On function types and type schemes it is defined as follows:

$$
\begin{aligned}
&[\![\mu_1 \to \mu_2]\!] = \\
&\quad \{f \in \mathcal{V} \to \mathcal{V} \mid v \in [\![\mu_1]\!] \Rightarrow f\, v \in [\![\mu_2]\!]\} \\
&[\![T\, \mu_1 \ldots \mu_m]\!] = \\
&\quad \{\bot\} \cup \\
&\quad \bigcup\{k\, [\![\mu'_1]\!] \ldots [\![\mu'_n]\!] \mid \\
&\qquad \mathsf{true}, \Gamma_0 \vdash k : \mu'_1 \to \ldots \to \mu'_n \to T\, \mu_1 \ldots \mu_m\} \\
&[\![\forall \bar{\alpha}.C \Rightarrow \tau]\!] = \\
&\quad \bigcap\{[\![[\bar{\mu}/\bar{\alpha}]\tau]\!] \mid \vdash^e [\bar{\mu}/\bar{\alpha}]C\}
\end{aligned}
$$

The following lemma can be proven by a straightforward induction on the structure of σ.

Lemma 6 *Let σ be a closed type scheme. Then $[\![\sigma]\!]$ is an ideal.*

Furthermore, I conclude that in a sound constraint system the error element is not contained in a closed type scheme.

Lemma 7 *Given a sound constraint system* X *and a closed type scheme σ. Then* $\mathbf{W} \notin [\![\sigma]\!]$.

Definition 10 *A variable environment η* models *a closed typing environment Γ, written $\eta \models \Gamma$, if for all $x : \sigma \in \Gamma$, $\eta(x) \in [\![\sigma]\!]$.*

Definition 11 *The constraint system* X *is* coherent *if for all monotypes μ and μ', if $\vdash^e (\mu \preceq \mu')$ then $[\![\mu]\!] \subseteq [\![\mu']\!]$.*

Furthermore, I need to restrict the set of substitutions which are allowed to be applied to typing judgments. I repeat the definition of *consistency* which can be found in Section 3.4.2. A substitution ϕ is consistent with respect to a type scheme $\sigma = \forall\bar{\alpha}.D \Rightarrow \tau$ if $\phi D \in \mathcal{S}$ where I assume there are no name clashes between $\bar{\alpha}$ and ϕ. This extends naturally to type environments. Furthermore, a substitution ϕ is *consistent* with respect to a constraint C if $\phi C \in \mathcal{S}$.

Theorem 5 (Type Soundness) *Let* $C, \Gamma \vdash e : \sigma$ *be a valid typing judgment in* HM(X)*, where* X *is a sound and coherent constraint system. Let* ϕ *be a substitution such that* $\phi\Gamma$ *and* $\phi\sigma$ *are closed and such that* $\vdash^e \phi C$ *and* ϕ *is consistent in* Γ*. Let* η *be a variable environment such that* $\eta \models \phi\Gamma$*. Then*

$$(1) \quad \mathbf{W} \notin [\![\phi\sigma]\!]$$
$$(2) \quad [\![e]\!]\eta \in [\![\phi\sigma]\!]$$

The type soundness theorem can be simplified to top–level programs. As a corollary, I find Milner's slogan "well types programs do not go wrong" carries over to sound constraint extensions.

Corollary 2 *Let* X *be a sound and coherent constraint system. Let* $\mathsf{true}, \Gamma \vdash e : \sigma$ *be a valid closed typing judgment in* HM(X)*. If* $\eta \models \Gamma$ *then* $[\![e]\!]\eta \neq \mathbf{W}$*.*

Proof: Immediate from (1) and (2) of Theorem 5. ∎

I observe that HM(HERBRAND), HM(DIM) satisfy the requirements. Hence, these applications are sound. HM($\mathcal{SC}$) does not satisfy the requirements because there one finds recursive types. I left out to give an ideal semantics for recursive types. In the next section, I present a syntactic type soundness result which includes the treatment of recursive types.

The type soundness result does not seem to apply to systems which are in constraint form, i.e. the set $\mathcal{S}$ contains now all satisfiable constraints. As an example consider a variation of HM(HERBRAND) where the set $\mathcal{S}$ contains all satisfiable constraints. Then, the constraint $\exists\alpha.(\alpha = \beta)$ is in solved form and $\vdash^e \exists\alpha.(\alpha = \beta)$, but there is no monotype μ such that $\vdash^e (\mu = \beta)$. Hence, soundness of the constraint system does not hold anymore. I thank Christoph Zenger for highlighting this point to me. The problem was that the semantic definition of free variables in a constraint does not reflect the set of syntactically free type variables. For instance, $fv(\exists\alpha.(\alpha = \beta)) = \emptyset$ but the type variable β appears syntactically free. But this did not seem to be an actual problem in case of type soundness. The assumptions of the type soundness result require it that there must be a substitution ϕ such that $\vdash^e \phi C$ and $\phi\Gamma$ and $\phi\sigma$ are closed. I conjecture that these invariants enforce that syntactically free type variables in constraint, such as β in $\exists\alpha.(\alpha = \beta)$, become always instantiated to monotypes.

To prevent such problems, I put stronger conditions on the constraint system. I can restate the type soundness result for strongly sound constraint systems. It must be noted that HM(HERBRAND) where the set $\mathcal{S}$ equals all satisfiable constraints in HERBRAND satisfies the conditions put on a strongly sound constraint system.

Definition 12 *A constraint system* X *is* strongly sound *iff for all constraints* $C \in \mathcal{S}$ *where* $C = \exists\bar{\alpha}.D$ *where* D *is projection–free,* $\vdash^e \exists fv(D).D$ *then there exists type variables* $\bar{\alpha}$ *and monotypes* $\bar{\mu}$ *such that* $\vdash^e [\bar{\mu}/\bar{\alpha}]D$.

A constraint D is projection–free if D is a set of primitive predicates.

In fact, I need one further assumption. Projection is not allowed to appear in syntactic form, i.e. the projection operator does not appear in syntactic form in constraints which are in solved form. This assumption is rather technical and does

not restrict the result. As stated in Theorem 6 in Section 3.5.2, judgments can always be transformed into projection-free judgments. Hence, the projection operator is not necessary to appear in syntactic form to establish the type soundness result. Please note, projection becomes important in case of type inference.

3.3.2 A Syntactic Approach

The previously described semantics is based on the ideal model [MPS86]. I only gave a meaning for basic types but excluded for example recursive types. Here, I give a syntactic type soundness result following the lines of [WF91] which includes recursive types. Syntactic type soundness for a given instance of the HM(X) is achieved if every satisfiable constraint has a monotype solution.

As mentioned earlier, the subtyping application HM($\mathcal{SC}$) relates to the subtyping approach of the Hopkins Object Group [EST95b]. In [TS96], it is shown that the regular tree model can be used to express solutions to subtype constraints. Hence, syntactic type soundness of HM($\mathcal{SC}$) falls out by adopting the techniques mentioned in [TS96]. Technically, the type language of HM($\mathcal{SC}$) needs to be extended with the top, bottom type and the recursive type constructor $\mu\alpha$ to express all constraint solutions within the type language.

The syntactic soundness result is straightforward along the type soundness result of functional ML described in detail in [WF91]. Hence, most of the ground–work is already done. Therefore, I will only give a brief overview how to achieve syntactic type soundness for HM(X).

The expressions of HM(X) are as follows:

$$\begin{array}{lrcl} \textbf{Values} & v & ::= & x \mid c \mid \lambda x.e \\ \textbf{Expressions} & e & ::= & v \mid e\, e \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \end{array}$$

Reduction is defined in the standard way:

$$
\begin{array}{lll}
(\delta) & c\,v \longrightarrow \delta(c,v) & \text{if } \delta(c,v) \text{ is defined} \\
(\beta_v) & (\lambda x.e)v \longrightarrow e[v/x] & \\
(\text{Let}) & \mathsf{let}\ x = v\ \mathsf{in}\ e \longrightarrow e[v/x] &
\end{array}
$$

I define $\longrightarrow^*$ as the transitive closure of $\longrightarrow$. The partial function δ allows to abstract from the precise set of constants. I assume that a constant and a closed value and yields a closed value. This is along the lines of [WF91].

Definition 13 *A constraint system* X *is* strongly sound *if for all constraints* $C \in \mathcal{S}$ *where* $C = \exists\bar{\alpha}.D$ *where* D *is projection–free,* $\vdash^e \exists fv(D).D$ *then there exists type variables* $\bar{\alpha}$ *and monotypes* $\bar{\mu}$ *such that* $\vdash^e [\bar{\mu}/\bar{\alpha}]D$.

Lemma 8 (Subject reduction) *Let* $C, \Gamma \vdash e : \sigma$ *be a valid typing judgment in* HM(X)*, where* X *is a strongly sound constraint system. Let* ϕ *be a substitution such that* $\phi\Gamma$ *and* $\phi\sigma$ *are closed and such that* $\vdash^e \phi C$ *and* ϕ *is consistent* [3] *in* Γ. *Let be an expression* e' *such that* $e \longrightarrow^* e'$. *Then* $\mathsf{true}, \phi\Gamma \vdash e' : \phi\sigma$.

One important observation is that the typing rules preserve satisfiability. This means the constraints in the conclusion of a typing rule are satisfiable iff the constraints in the premise are satisfiable. This allows to adopt the inductive proof of the subject reduction stated in [WF91]. Other proofs can be adopted in a similar way.

In order to state the syntactic type soundness result, I restate the definitions of faulty expressions and the evaluation function on expressions as stated in [WF91].

[3] A definition can be found in Section 3.4.2.

An expression e is *faulty* if there exists a subexpression (c, v) such that $\delta(c, v)$ is undefined. The evaluation function is defined as follows:

$$\begin{aligned} & \mathit{eval}(e) \\ = \; & \mathbf{W} \quad \text{if } e \longrightarrow^* e' \text{ and } e' \text{ is faulty} \\ = \; & v \quad \text{if } e \longrightarrow^* v \end{aligned}$$

Conjecture 1 (Syntactic Type Soundness) *Let $C, \Gamma \vdash e : \sigma$ be a valid typing judgment in* HM(X)*, where* X *is a sound constraint system. Let ϕ be a substitution such that $\phi\Gamma$ and $\phi\sigma$ are closed and such that $\vdash^e \phi C$ and ϕ is consistent* [4] *in Γ. Then $\mathit{eval}(e) \neq \mathbf{W}$.*

The above result is only stated as a conjecture. I have not formally proven all steps to establish the syntactic type soundness result. This seems a rather tedious exercise. I believe that all results found in [WF91] can straightforwardly be adopted.

3.3.3 Summary

The syntactic approach improves over the denotational approach, that is, it gives a semantic meaning to systems dealing with recursive types. In contrast, the denotational approach is more abstract. As an example, consider the common problem with division by zero. In the denotational approach, I can simply denote division by zero by the bottom element $\perp$ which is not possible to model in the syntactic approach. None of two approaches provides an efficient compilation method. Efficient compilation methods have already been studied in the context of records [Oho97, Oho95, Gas98]. I believe it is rather difficult to develop an efficient compilation method for general applications.

[4] A definition can be found right before the semantic type soundness result in Section 3.3.1.

3.4 Properties

In the first part, I introduce a comparison relation between type schemes. Then, I define basic definitions. I give some derived typing rules of the logical system in Figure 3.1 and give normalized derivations. Finally, I talk about nested forall quantifiers.

3.4.1 Instance Relation

I extend the subsumption predicate $\preceq$ to type schemes. Subsumption on type schemes is defined by a deduction system with clauses of the form $C \vdash^i \sigma \preceq \sigma'$, which state that the type scheme σ is more general than the type scheme σ' under the constraint C. The deduction system is defined as follows:

$$\text{(Sub)} \qquad \frac{C \vdash^e (\tau \preceq \tau')}{C \vdash^i \tau \preceq \tau'}$$

$$(\preceq \forall) \qquad \frac{C \wedge D \vdash^i \sigma \preceq \sigma' \qquad \bar{\alpha} \notin tv(\sigma) \cup tv(C)}{C \wedge \exists \bar{\alpha}.D \vdash^i \sigma \preceq (\forall \bar{\alpha}.D \Rightarrow \sigma')}$$

$$(\forall \preceq) \qquad \frac{C \vdash^i [\bar{\tau}/\bar{\alpha}]\sigma \preceq \sigma' \qquad C \vdash^e [\bar{\tau}/\bar{\alpha}]D \qquad [\bar{\tau}/\bar{\alpha}] \text{ consistent in } \sigma}{C \vdash^i (\forall \bar{\alpha}.D \Rightarrow \sigma) \preceq \sigma'}$$

The side condition $[\bar{\tau}/\bar{\alpha}]$ *consistent in* σ ensures that the constraints in the resulting type scheme $[\bar{\tau}/\bar{\alpha}]\sigma$ are still in solved form. See the next section for a formal definition.

I write $C \vdash^i \sigma \equiv \sigma'$ if $C \vdash^i \sigma \preceq \sigma'$ and $C \vdash^i \sigma' \preceq \sigma$. I refer to the relation $\vdash^i$

as the *instance* or also as the *comparison* relation between type schemes.

Based on the ideal semantics described in Section 3.3.1, it is straightforward to show that the instance relation is sound.

Lemma 9 *Given a sound and coherent constraint system* X. *Given a constraint* C, *type schemes* σ, σ' *and a substitution* ϕ *such that* $\phi\sigma, \phi\sigma'$ *closed,* ϕ *consistent in* σ, σ', $C \vdash^i \sigma \preceq \sigma'$ *and* $\vdash^e \phi C$. *Then* $[\![\phi\sigma]\!] \subseteq [\![\phi\sigma']\!]$.

Lemma 10 *The following inference rules are derivable.*

$$\textit{(Ref-}\sigma\textit{)} \qquad C \vdash^i \sigma \preceq \sigma$$

$$\textit{(Weaken)} \qquad \frac{D \vdash^e C \qquad C \vdash^i \sigma \preceq \sigma'}{D \vdash^i \sigma \preceq \sigma'}$$

$$\textit{(Trans)} \qquad \frac{C \vdash^i \sigma_1 \preceq \sigma_2 \qquad C \vdash^i \sigma_2 \preceq \sigma_3}{C \vdash^i \sigma_1 \preceq \sigma_3}$$

Furthermore it is possible to normalize derivations $C \vdash^i \sigma \preceq \sigma'$ such that all $(\forall \preceq)$ steps are performed before $(\preceq \forall)$ steps. I introduce the following new relation $\vdash^{i_2}$ which enforces to perform first $(\forall \preceq)$ steps and then $(\preceq \forall)$ steps. All rules are the same except for the $(\forall \preceq)$ rule:

$$(\forall \preceq_2) \qquad \frac{C \vdash^{i_2} [\bar{\tau}/\bar{\alpha}]\sigma \preceq \tau' \qquad C \vdash^e [\bar{\tau}/\bar{\alpha}]D \qquad [\bar{\tau}/\bar{\alpha}] \text{ consistent in } \sigma}{C \vdash^{i_2} (\forall \bar{\alpha}.D \Rightarrow \sigma) \preceq \tau'}.$$

The relations $\vdash^i$ and $\vdash^{i_2}$ are equivalent.

Lemma 11 (Equivalence of $\vdash^i$ and $\vdash^{i_2}$) *Let* C *be a constraint and* σ, σ' *be type schemes. Then* $C \vdash^i \sigma \preceq \sigma'$ *iff* $C \vdash^{i_2} \sigma \preceq \sigma'$.

I extend the relation $\vdash^i$ to type environments.

Definition 14 (Type Environment) *Let C be a constraint and Γ and Γ' be type contexts such that $\Gamma = \{e_1 : \sigma_1, \ldots, e_n : \sigma_n\}$ and $\Gamma' = \{e_1 : \sigma'_1, \ldots, e_n : \sigma'_n\}$. Then $C \vdash^i \Gamma' \preceq \Gamma$ iff $C \vdash^i \sigma'_i \preceq \sigma_i \quad \forall i \in \{1, \ldots, n\}$.*

I introduce an ordering on pairs (C, Γ) where C is a constraint and Γ is a type environment.

Definition 15 (Constrained Type Environment) *Given constraints C and C' and type environments Γ and Γ'. I refer to tuples (C, Γ) as a* constrained type environment. *Furthermore, $(C, \Gamma) \leq (C', \Gamma')$ iff $C \vdash^i \Gamma \preceq \Gamma'$ and $C \vdash^e C'$.*

3.4.2 Basic Definitions

Definition 16 (Realizable) *Given a constraint C and a type scheme σ. Then σ is* realizable *with respect to C iff there is a τ such that $C \vdash^i \sigma \preceq \tau$.*

The set $\mathcal{S}$ of solved forms is not necessarily closed under substitutions. The next definition introduces the notion of consistency which ensures that all substitutions applied to constraints preserve the property of being in solved form.

Definition 17 (Consistent) *Given an HM(X) instance, a constraint C, type scheme σ and substitution ϕ. Then substitution ϕ is* consistent *with respect to a constraint C iff $\phi C \in \mathcal{S}$. Substitution ϕ is* consistent *with respect to type scheme $\sigma = \forall \bar{\alpha}.D \Rightarrow \tau$ iff $\phi D \in \mathcal{S}$ where w.l.o.g. there are no name clashes between $\bar{\alpha}$ and ϕ. This extends naturally to type environments*

3.4.3 Derived Typing Rules

The first lemma states that derivation $\vdash$ is closed under substitution.

Lemma 12 (Substitution) *Given $C, \Gamma \vdash e : \sigma$ and a substitution ϕ such that ϕ is consistent with respect to C and Γ. Then $\phi C, \phi\Gamma \vdash e : \phi\sigma$.*

Surprisingly, the following result does not hold in general.

Lemma 13 (Weakening) *Given $C, \Gamma \vdash e : \sigma$, a constraint $C' \in \mathcal{S}$ and a type environment Γ' such that $(C', \Gamma') \leq (C, \Gamma)$. Then $C, \Gamma' \vdash e : \sigma$.*

It states that given a judgment $C, \Gamma \vdash e : \sigma$, expression e is still derivable with type σ if a stronger constraint and a stronger type environment is used. Remember, constraints in typing judgments need to be always in solved form. The problem occurs in case of the ($\forall$ Intro) rule. I assume there is the following derivation:

$$(\forall \text{ Intro}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin \mathit{fv}(C) \cup \mathit{fv}(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

Furthermore, I assume there is a constraint C' in solved form such that $C' \vdash^e C \wedge \exists \bar{\alpha}.D$. In order to deduce the judgment $C', \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau$, I need to show that $C' \wedge D$ is in solved form. But this does not necessarily hold. Because there might exist constraints C', C and D such that $D \in \mathcal{S}$, $C \wedge D \in \mathcal{S}$, $C \wedge \exists \bar{\alpha}.D \in \mathcal{S}$, $C' \in \mathcal{S}$, $C' \vdash^e C \wedge \exists \bar{\alpha}.D$ but $C' \wedge D \notin \mathcal{S}$.

There are two ways to fix this problem. I could either add the following additional rule to the typing rules in Figure 3.1:

$$(\text{Weakening}) \quad \frac{C, \Gamma \vdash e : \sigma \qquad C' \vdash^e C \qquad C' \in \mathcal{S}}{C', \Gamma \vdash e : \sigma}$$

In fact, this rule needs then also to be added to the derivations in the next section. The addition of the (Weakening) rule does not contradict any of the previous results, nor yields any new surprising results. Another possibility would be to put sufficient conditions on the set $\mathcal{S}$. Given C', C and D such that $D \in \mathcal{S}$, $C \wedge D \in \mathcal{S}$, $C \wedge \exists\bar{\alpha}.D \in \mathcal{S}$, $C' \in \mathcal{S}$, $C' \vdash^e C \wedge \exists\bar{\alpha}.D$. Then $C' \wedge D \in \mathcal{S}$. The Weakening Lemma holds under this condition. All sensible definitions of solved forms satisfy this condition. Henceforth, I assume that this condition either holds or there is the additional (Weakening) rule present in the type system.

To establish soundness of the inference system in Chapter 4, I need the following lemma which is a combination of the Substitution and Weakening Lemma.

Lemma 14 (Substitution and Weakening) *Given $C, \Gamma \vdash e : \sigma$, a constraint $C' \in \mathcal{S}$ and a substitution ϕ such that ϕ consistent in Γ and σ and $C' \vdash^e \phi C$. Then $C', \phi\Gamma \vdash e : \phi\sigma$.*

I can make the same observations as in case of the Weakening Lemma. Therefore, this lemma holds under the same conditions. I omit the straightforward proof.

3.4.4 Normalized Derivations

I introduce a derivation $\vdash^n$ which is based on derivation $\vdash$ in Figure 3.1. Instead of rule ($\forall$ Elim) there is the following new rule:

$$\text{(Inst)} \quad C, \Gamma \vdash^n x : \tau \quad (x : \sigma \in \Gamma \quad C \vdash^i \sigma \preceq \tau)$$

All other rules stay unchanged. Derivation $\vdash^n$ simply enforces ($\forall$ Elim) steps as early as possible.

Next, I consider a syntax directed derivation $\vdash^s$. The ($\forall$ Intro) rule is combined with the (Let) rule. Furthermore, the (Var) and (Inst) rules are combined in the

$$\text{(Var–Inst)} \quad C, \Gamma \vdash^s x : \tau \quad (x : \sigma \in \Gamma \quad C \vdash^i \sigma \preceq \tau)$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x . x : \tau \vdash^s e : \tau'}{C, \Gamma_x \vdash^s \lambda x.e : \tau \to \tau'}$$

$$\text{(App)} \qquad \frac{C, \Gamma \vdash^s e_1 : \tau_1 \to \tau_2 \quad C, \Gamma \vdash^s e_2 : \tau_1}{C, \Gamma \vdash^s e_1 e_2 : \tau_2}$$

$$\text{(Sub)} \qquad \frac{C, \Gamma \vdash^s e : \tau \qquad C \vdash^e (\tau \preceq \tau')}{C, \Gamma \vdash^s e : \tau'}$$

$$\text{(Let)} \qquad \frac{\begin{array}{c} C, \Gamma_x \vdash^s e : \tau \quad C, \Gamma_x, \tau \vdash^{gen} (C', \sigma) \\ C'', \Gamma_x . x : \sigma \vdash^s e' : \tau' \end{array}}{C' \wedge C'', \Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

Figure 3.2: Normalized typing rules

(Var–Inst) rule. The rules can be found in Figure 3.2. In the (Let) rule, I implicitly require that the constraint $C' \wedge C''$ is in solved form. Remember, the set of constraints of solved forms is not necessarily closed under $\wedge$.

The next lemmas state how these derivations are connected.

Lemma 15 (Equivalence of $\vdash$ and $\vdash^n$) *Given a type environment Γ, a constraint C, a term e and a type scheme σ. Then $C, \Gamma \vdash e : \sigma$ iff $C, \Gamma \vdash^n e : \sigma'$ where $\vdash^i \sigma \equiv \sigma'$.*

Lemma 16 (Soundness of $\vdash^s$) *Given $C, \Gamma \vdash^s e : \tau$. Then $C, \Gamma \vdash e : \tau$.*

Both lemmas can be proven by straightforward induction.

In the following lemma, it is essential that the type environment Γ' is realizable. Recall, a type environment Γ' is realizable in a constraint C if for every $x : \sigma \in \Gamma'$ there is a τ such that $C \vdash^i \sigma \preceq \tau$.

Furthermore, I introduce a generalization relation $\vdash^{gen}$. Given constraints $C, C_o \in \mathcal{S}$, a type environment Γ and types σ, σ_o. Then $C, \Gamma, \sigma \vdash^{gen} (C_o, \sigma_o)$ iff $C_o =^e \exists\bar{\alpha}.C =^e C_1 \wedge \exists\bar{\alpha}.C_2$, $\sigma_o = \forall\bar{\alpha}.C_2 \Rightarrow \sigma$, $\alpha \notin fv(C_1) \cup fv(\Gamma)$ for some constraints C_1, C_2.

Lemma 17 (Completeness of $\vdash^s$) *Given $C', \Gamma' \vdash^n e : \sigma'$, $C' \vdash^i \Gamma \preceq \Gamma'$ and Γ' is realizable in C'. Then*

$$\begin{array}{lll} \textit{(a)} \quad \sigma' = \tau : & C, \Gamma \vdash^s e : \tau & C' \vdash^e C \\ \textit{(b) otherwise :} & C, \Gamma \vdash^s e : \tau & C, \Gamma, \tau \vdash^{gen} (\sigma_o, C_o) \\ & & C' \vdash^e C_o \quad C' \vdash^i \sigma_o \preceq \sigma \end{array}$$

Lemma 18 (Weakening) *Given $C, \Gamma \vdash^s e : \tau$ and $(C', \Gamma') \leq (C, \Gamma)$. Then $C', \Gamma' \vdash^s e : \tau$.*

3.4.5 Nested Forall Quantifiers

The careful reader might have noticed that nested type schemes such as

$$\forall\alpha.(\alpha <: \tau_1) \Rightarrow (\forall\beta.(\beta <: \tau_2) \Rightarrow \alpha \rightarrow \beta)$$

do not appear in the type system. Although, perfectly well–formed according to the syntactic domain of type schemes as defined in Section 3.2, such type schemes with nested forall quantifiers are ruled out. Please consult the typing rules in Figure 3.1. Rules (∀ Intro) and (∀ Elim) assume that all type schemes are of the form $\forall\bar{\alpha}.C \Rightarrow \tau$, i.e. type schemes appearing in typing judgments do not contain nested forall quantifiers. [5]

The following formulation of quantifier introduction and elimination rule might be logically more pleasing:

[5] Note, higher–order types such as $(\forall\alpha.\alpha) \rightarrow (\forall\beta.\beta \rightarrow \beta)$ are not permitted at all.

$$(\forall \text{ Intro'}) \qquad \frac{C \wedge D, \Gamma \vdash e : \sigma \qquad \alpha \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \alpha.D, \Gamma \vdash e : \forall \alpha.D \Rightarrow \sigma}$$

$$(\forall \text{ Elim'}) \qquad \frac{C, \Gamma \vdash e : \forall \alpha.D \Rightarrow \sigma \qquad C \vdash^e [\tau/\alpha]D \qquad [\tau/\alpha] \text{ consistent in } \sigma}{C, \Gamma \vdash e : [\tau/\alpha]\sigma}$$

where $[\tau/\alpha]$ consistent in σ ensures that all constraints in $[\tau/\alpha]\sigma$ remain in solved form.

Rules ($\forall$ Intro') and ($\forall$ Elim') allow step by step introduction and elimination of quantifiers. I omit to formally prove that both styles of formulations are equivalent and rather give some intuition that it is sufficient to deal only with type schemes of the form $\forall \bar{\alpha}.C \Rightarrow \tau$.

Consider the following derivation which involves two applications of the ($\forall$ Intro') rule where α and β are not free in Γ and expression e has type $\alpha \to \beta$.

$$\frac{\dfrac{(\alpha <: \beta), \Gamma \vdash e : \alpha \to \beta}{\exists \beta.(\alpha <: \beta), \Gamma \vdash e : \forall \beta.(\alpha <: \beta) \Rightarrow \alpha \to \beta}}{\mathsf{true}, \Gamma \vdash e : \forall \alpha.(\exists \beta.(\alpha <: \beta)) \Rightarrow (\forall \beta.(\alpha <: \beta) \Rightarrow \alpha \to \beta)}$$

Application of the ($\forall$ Intro) rule would yield:

$$\frac{(\alpha <: \beta), \Gamma \vdash e : \alpha \to \beta}{\mathsf{true}, \Gamma \vdash e : \forall \alpha, \beta.(\alpha <: \beta) \Rightarrow \alpha \to \beta}$$

Both type schemes in the final judgment are equivalent:

$$\vdash^i \ (\forall\alpha, \beta.(\alpha <: \beta) \Rightarrow \alpha \to \beta) \equiv (\forall\alpha.(\exists\beta.(\alpha <: \beta)) \Rightarrow (\forall\beta.(\alpha <: \beta) \Rightarrow \alpha \to \beta))$$

Similar observations hold for the quantifier elimination rule. Hence, it is sufficient to deal only with type schemes of the form $\forall\bar{\alpha}.C \Rightarrow \tau$.

3.4.6 Summary

I introduced an instance relation $C \ \vdash^i \ \sigma \preceq \sigma'$ which allows to compare type schemes σ, σ' with respect to a constraint C. I introduced the definitions of realizability and consistency. Furthermore, I discussed derived typing rules and normalized derivations. The normalized derivations will provide to be helpful for the completeness proof in Section 4.3. In addition, I discussed issues concerning nested forall quantifiers.

3.5 Constrained Polymorphism

This section considers previous Hindley/Milner style systems that deal with constrained types. All such type systems extend the type judgments $\Gamma \vdash e : \sigma$ of the original Hindley/Milner system with a constraint hypothesis on the left side of the turnstile, written $C, \Gamma \vdash e : \sigma$. Type schemes $\forall\bar{\alpha}.\tau$ of the Hindley/Milner system are extended with a constraint component; written $\forall\bar{\alpha}.C \Rightarrow \tau$ to express that the constraint C restricts the types that can legally be substituted for the bound variables $\bar{\alpha}$. All type systems have essentially the same rule for eliminating quantifiers,

$$(\forall \text{ Intro–Jones}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

$$(\forall \text{ Intro–Aiken/Wimmers}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \exists D \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

$$(\forall \text{ Intro–Smith}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad C \vdash [\bar{\tau}/\bar{\alpha}]D \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

$$(\forall \text{ Intro–HOG}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

Figure 3.3: Versions of the quantifier introduction rule

which is written as follows:

$$(\forall \text{ Elim}) \quad \frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash [\bar{\tau}/\bar{\alpha}]D}{C, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'}$$

The rule is a refinement of the corresponding rule in the Hindley/Milner system. It states that, when instantiating a type scheme $\forall \bar{\alpha}.D \Rightarrow \tau'$, the only valid instances are those instances $[\bar{\tau}/\bar{\alpha}]\tau'$ which satisfy the constraint part D of the type scheme.

3.5.1 A Comparison between Different Quantifier Introduction Rules

The HM(X) framework comes with a novel formulation of the quantifier introduction. Constraints are not only bound in type schemes but are additionally bound by the

projection operator in the constraint part of typing judgments:

$$(\forall \text{ Intro}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

This differs to previous formulations. Figure 3.3 shows four different rules that have all been proposed in the literature.

I repeat the discussion about different formulations of the quantifier introduction found in [OSW99]. In [OSW99], the examples are mainly due to Martin Odersky. In my dissertation, I give further examples of typing situations where problems in the context of constrained polymorphism might occur and discuss in further detail the role of the projection operator.

Jones [Jon92] uses a general framework of qualified types for type qualification with a rule equivalent to rule (∀ Intro–Jones). Any constraint can be shifted from the assumption on the left to the type scheme on the right of the turnstile; it is not checked whether the traded constraint is satisfiable. This might lead to programs that are well-typed as a whole, even though some parts have unsatisfiable constraints.

To give an example, assume that constraints are subtyping constraints ($\leq$) in a type system with classes and a subtyping relation determined by programmer declarations. I assume that there is a parametrized class $\mathsf{List}\,\alpha$ which is a subtype of type $\mathsf{Comparable}\,(\mathsf{List}\,\alpha)$, where $\mathsf{Comparable}$ is declared as follows:

$$\mathsf{type\ Comparable}\ \alpha = \{\mathsf{less} : \alpha \to \mathsf{Bool}\}$$

I go on to assume that there is a value Nil of type $\forall\alpha.\mathsf{true} \Rightarrow \mathsf{List}\,\alpha$ that represents the empty list. Consider the following (nonsensical) program.

Example 10

```
let
 f: ∀α.(List α ≤ Comparable Bool) ⇒ List α → List α
 f x = if x.less(true) then x else Nil
in 1
```

I use a Haskell-style notation, adding type annotations for illustration purposes. Using rule (∀ Intro-1), the program in Figure 10 is well-typed, even though one would not expect the constraint in function f's type scheme to have a solution, since the function type List α would not be a subtype of Comparable Bool.

In the ideal semantics of types [MPS86] which represents universal quantification by intersection, f's type would be an empty intersection. This is equal to the whole type universe including the error element *wrong*. However, the whole program in Figure 10 is still sound because every application of f must provide a valid instantiation of the constraint. Since the constraint is unsatisfiable, no application is possible. In essence, Jones treats constraints as proof obligations that have to be fulfilled by presenting "evidence" at the *instantiation* site. This scheme is clearly inspired by Haskell's implementation of overloading by dictionary passing. It runs into problems if one ever wants to compute a value of a constrained type without any instantiation sites, as in the following slight variation of Example 10.

Example 11

```
let
   y: ∀α.(List α ≤ Comparable α) ⇒ Bool
   y = Nil.less(true)
in 1
```

Jones excludes this code on the grounds that y's type is ambiguous, but it is unclear how to generalize this restriction to arbitrary constraint systems.

Nevertheless, it is possible to integrate Jones' approach into the HM(X) framework, thus giving it a semantic basis independent of dictionary passing. The essential idea is to consider only constraint systems in which projections of solved constraints are trivial, i.e $\vdash^e \exists\alpha.C$, for all constraints C that can appear on the left hand side of the turnstile, and for all type variables $\alpha \in fv(C)$. In this case, the ($\forall$ Intro) in HM(X) simplifies to ($\forall$ Intro–Jones).

Note that trivial projections correspond well to Haskell's "open world" assumption, which states that the range of possible instance types for an overloaded operation is not fixed in advance. Therefore, one can never rule out that a given constraint which still has free variables might have a solution.

In the type system of Aiken/Wimmers [AW93], moving a constraint from the left hand side of the turnstile to the right-hand side is allowed only if the constraint is satisfiable (i.e. has a solution). Hence, none of the previous examples would be typable using rule ($\forall$ Intro-2), which they use. However, this example is typable.

Example 12

```
let
    f: ∀β.β → Int
    f x =
        let y: ∀α.(List α ≤ Comparable β) ⇒ Bool
            y = Nil.less(x)
        in 1
in f true
```

The constraint $\mathsf{List}\,\alpha \leq \mathsf{Comparable}\,\beta$ has a solution, namely $\beta = \mathsf{List}\,\alpha$. Therefore,

using rule (∀ Intro-2) I can generalize y's type to

$$\forall\alpha.(\mathsf{List}\,\alpha \leq \mathsf{Comparable}\,\beta) \Rightarrow \mathsf{Bool}.$$

On the other hand, if I substitute the actual parameter true in f's definition, I get again Example 11 which is not typable under the system with (∀ Intro-2). Hence, the system with (∀ Intro-2) does not enjoy the property of subject reduction, which states that if a term is typable then its reduction instances are typable as well. In a later version, they use rule (∀ Intro–HOG) instead.

In contrast, G. Smith [Smi91] proposed rule (∀ Intro–Smith). The traded constraint D must be solvable by instantiation of only the quantified variables $\bar{\alpha}$. Hence, the previous mentioned problems would disappear under Smith's system. Smith's formulation seems however overly restrictive. Consider the following example. I assume that Comparable has precisely two instances:

Int $\leq$ Comparable Int

Char $\leq$ Comparable Char

Now consider the following program:

Example 13

```
let
    f: ∀β.β → Int
    f x =
        let g y = y.less(x)
        in 1
in 1
```

When typing the definition of g, Smith's system requires a solution of the constraint $\tau \leq \mathsf{Comparable}\,\beta$, where τ is y's type. But none of the two possible substitutions $\tau = \mathsf{Int}$ or $\tau = \mathsf{Char}$ satisfy the constraint. This shows that Smith's formulation has problems when typing definitions when no instantiation sites are present.

Another quantifier introduction rule has been suggested by the Hopkins Objects Group [EST95b]. Constrained D is not only moved from the left–hand side of the turnstile to the right–hand side, but a copy of D is kept on the left–hand side. This seems to overcome all previously mentioned problems. However, rule ($\forall$ Intro–HOG) seems strange in that its conclusion contains two copies of the constraint D: one in which the type variables α are bound and one in which they are free. Actually, the Hopkins Objects Group uses a slightly different system in which generalization is coupled with the let rule and one of the two constraints undergoes a variable renaming. Nevertheless, rule ($\forall$ Intro–HOG) does not seem to apply to systems which follow the open world approach.

I observe that Jones's formulation of the quantifier introduction rule works well in systems which follow the open world approach and which use a lazy semantics. But this formulation has problems when dealing with a strict semantics. Hence, Jones's formulation supports a "lazy" inference approach. Constraints need only to be satisfied at instantiation sites. Aiken/Wimmers first formulation of the quantifier introduction rule does not enjoy a subject reduction property. Smith's and the Hopkins Object Group's formulation of the quantifier introduction rule follow the closed world approach. Smith's formulation has problems when typing definitions first, because then no knowledge about possible instantiation sites is available yet. This can be seen as a "strict" inference approach, constraints need additionally satisfied at definition sites. The ($\forall$ Intro) rule in HM(X) is capable of modeling all approaches, simply by the way projection is defined on constraints in solved form.

In the next section, I will study the role of projection in case of an open and a closed world approach in more detail. In addition, I will point out limitations of the quantifier introduction rule in the overloading system [OWW95] and Ohori's record calculus [Oho95]. I will also study the role of the projection operator in solved forms.

3.5.2 Open versus Closed World Systems

This decision between an open and a closed world approach is usually reflected in the formulation of the quantifier introduction rule. In an open world approach, one does not have full knowledge about the whole world. There might be some facts which are not known yet. To be more specific, I consider Haskell style type classes [VHJW96, Jon92, NP95]. There are constraints of the form $\mathrm{C}\,\tau$ which denotes that τ belongs to the class C. Haskell's type class system follows an open world approach, i.e. even empty type classes are considered legal. Consider the following typing judgment:

$$\mathrm{Eq}\,\alpha, \emptyset \vdash \lambda \mathsf{x}.\lambda \mathsf{y}.\mathit{eq}\ \mathsf{x}\ \mathsf{y} : \alpha \to \alpha \to \mathsf{Bool}$$

where I assume the type class declaration for the class $\mathrm{Eq}\,\alpha$ defines an equality function *eq* on type α. I consider the type class approach in style of Nipkow/Prehofer [NP95]. Their quantifier introduction rule essentially reads as follows:

$$(\forall \text{ Intro-Nipkow/Prehofer}) \quad \frac{D \wedge \mathrm{C}\,\alpha, \Gamma \vdash e : \sigma \quad \alpha \notin \mathit{fv}(\Gamma) \cup \mathit{fv}(D)}{D, \Gamma \vdash e : \forall \alpha.\mathrm{C}\,\alpha \Rightarrow \sigma}$$

Then application of the quantifier introduction rule results in the typing judgment:

$$\mathsf{true}, \emptyset \vdash \lambda \mathsf{x}.\lambda \mathsf{y}.eq\ \mathsf{x}\ \mathsf{y} : \forall\alpha.\mathrm{Eq}\,\alpha \Rightarrow \alpha \to \alpha \to \mathsf{Bool} \tag{3.1}$$

I do not know yet whether there is an instance declaration for the type class Eq. Therefore, the constraint $\mathrm{Eq}\,\alpha$ is simply traded from the left hand side of the turnstile into the constraint component of the type scheme.

This is in contrast to a closed world approach. There, I need full knowledge about the whole world at type checking time. In case of the the above example, I need to provide an actual instance of class Eq in order to trade the constraint $\mathrm{Eq}\,\alpha$ from the left hand side to the right hand side of the turnstile. Smith [Smi91] gave a formulation of a quantifier introduction rule that models a closed world approach:

$$(\forall \text{ Intro-Smith}) \quad \frac{C \wedge D, \Gamma \vdash e : \sigma \quad \alpha \notin fv(\Gamma) \quad C \vdash^e [\tau/\alpha]D}{C, \Gamma \vdash e : \forall\alpha.D \Rightarrow \sigma}$$

The decision for an open or a closed world approach is fixed by the kind of quantifier introduction rule. Recall the ($\forall$ Intro–HM(X)) rule. The quantified constraint is kept in projected form on the left hand side of the turnstile. For instance, in the above example I keep the constraint $\exists\alpha.\mathrm{Eq}\,\alpha$. Depending on how projection is defined on constraints, an open or a closed world approach is modeled. For instance, an open world approach can be modeled by a constraint rule of the form: $\exists\alpha.\mathrm{Eq}\,\alpha =^e \mathsf{true}$. In such a system, typing judgment 3.1 is derivable. A closed world can also be modeled easily. I do not specify the above mentioned constraint rule. But then the constraint $\exists\alpha.\mathrm{Eq}\,\alpha$ is only satisfiable if there is an actual instance of class $\mathrm{Eq}\,\alpha$. Remember, constraints appearing in typing judgments and type schemes have to be at least satisfiable. Assume there is an instance of class Eq defined on inte-

gers. This is reflected in the constraint system by a rule of the form: $\mathsf{true} \vdash^e \mathrm{Eq}\,\mathsf{Int}$. The conditions put on constraint systems in HM(X) yield $\mathrm{Eq}\,\mathsf{Int} \vdash^e \exists\alpha.\mathrm{Eq}\,\alpha$. Then typing judgment 3.1 is derivable.

I observe that both approaches can easily be modeled within one framework. Depending on how projection is defined, one or the other approach can be modeled. Trivial projection leads to an open world approach, whereas non–trivial projection leads to a closed world approach.

3.5.3 Odersky/Wadler/Wehr Overloading

I consider the overloading system [OWW95] of Odersky/Wadler/Wehr, henceforth referred to as *SystemO*, which follows an open world approach. A formal treatment of their system can be found in Section 5.2. Odersky/Wadler/Wehr consider overloading of functions where overloading is restricted to the argument type. Hence, it is forbidden to overload numeric constants because the overloaded identifier always has to be a function type and overloading is restricted to the first argument. Constraints of the form $(o :: \alpha \to \tau)$ restrict the possible instances of a type variable α for a given overloaded identifier o.

The quantifier introduction rule in *SystemO* reads essentially as:

$$(\forall \text{ Intro–}\mathit{SystemO}) \quad \frac{C \wedge \pi_\alpha, \Gamma \vdash e : \sigma \qquad \alpha \notin \mathit{fv}(\Gamma) \cup \mathit{fv}(C)}{C, \Gamma \vdash e : \forall\alpha.\pi_\alpha \Rightarrow \sigma}$$

and π_α stands for the constraint $(o_1 :: \alpha \to \tau_1) \wedge \ldots \wedge (o_n :: \alpha \to \tau_n)$. Please note, this rule allows step by step quantification which allows construction of type schemes of the form

$$\forall\alpha.(o :: \alpha \to \tau_1) \Rightarrow \forall\beta.(o :: \beta \to \tau_2) \Rightarrow \tau$$

where $\beta \notin fv(\tau_1)$. The following formulation is equivalent where all free type variables are bound in one step.

$$(\forall \text{ Intro–}\mathit{SystemO}) \quad \frac{C \wedge \pi_{\bar{\alpha}}, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(\Gamma) \cup fv(C) \qquad \text{there are no } i, j \in \{1, \ldots, n\} \text{ such that } j < i \text{ and } \alpha_i < \alpha_j}{C, \Gamma \vdash e : \forall \bar{\alpha}.\pi_\alpha \Rightarrow \tau}$$

where $\pi_{\bar{\alpha}}$ stands for the constraint

$$\begin{array}{l} (o_{1_1} :: \alpha_1 \to \tau_{1_1}) \wedge \ldots \wedge (o_{1_m} :: \alpha_1 \to \tau_{1_m}) \wedge \\ \vdots \\ (o_{n_1} :: \alpha_n \to \tau_{n_1}) \wedge \ldots \wedge (o_{n_m} :: \alpha \to \tau_{n_m}) \end{array}$$

where $\alpha_i < \alpha_j$ iff $\alpha_j \notin fv(\tau_{i_1}, \ldots, \tau_{i_m})$ and $\alpha_i \neq \alpha_j$. The side conditions ensure that both formulations are equivalent.

This rule can be seen equivalent to the ($\forall$ Intro) rule in HM(X). The HM(X) formulation of *SystemO* contains constraint rules of the form

$$\exists \alpha.(o :: \alpha \to \tau) =^e \mathsf{true}$$

which models the open world assumption of *SystemO*. If projection is not allowed to appear in syntactic form in $\mathcal{S}$, the formulations of the quantifier introduction rule in *SystemO* and the corresponding HM(X) instance are equivalent.

Consider the following program where I assume that there is a function

$$\mathrm{eq} : \forall \alpha.\alpha \to \alpha \to \mathsf{Bool}$$

in the initial type environment Γ_0.

Example 14

```
f: ∀β.∀α.(o :: α → β) ⇒ α → Int
f x =
    let g: β → Bool
        g = λ y. eq (o x) y
    in 1
```

Function g can not be given a polymorphic type. Projection would not be trivial if one quantifies first over the free type variable β. In HM(X) projection needs not necessarily to be trivial. I can then type Example 14 in the following way.

```
f: ∀α.∃β.(o :: α → β) ⇒ α → Int
f x =
    let g: ∀β.(o :: α → β) ⇒ β → Bool
        g = λ y. eq (o x) y
    in 1
```

Function g is now of polymorphic type but can not be applied to arguments with different types. Every overloaded instance is uniquely determined by its argument type. In the above example, this means that for every α there can be only one β such that we can provide an overloaded instance of the form $(o :: \alpha \to \beta)$. In Section 5.2, I state that the projection operator does not add any additional expressiveness to the overloading system *SystemO*.

Projection becomes useful in an extension of *SystemO* with curried overloaded functions. Such a calculus proves to be useful in dealing with printer functions

where one not only can specify the format but also the device. For instance, $print$: HP–Printer $\rightarrow$ Postscript $\rightarrow$ () specifies a print function that supports the Postscript–format for a specific printer. In the following example, function f takes a device as a parameter and implements a local IO function.

Example 15

f x =

let g y = $print$ x "Header Information"; $print$ x y

in (g 1, g true)

where the sequencing operator _;_ is contained as a primitive in the initial type environment Γ_0.

It is not possible to give function g a polymorphic type based on a system using rule ($\forall$ Intro–*SystemO*), but it is now possible to apply g to arguments of different type. In HM(X) this example can be typed as follows.

Example 16

f : $\forall\alpha.(print :: \alpha \rightarrow \mathsf{String} \rightarrow ()) \wedge (print :: \alpha \rightarrow \mathsf{Int} \rightarrow ()) \wedge$

$(print :: \alpha \rightarrow \mathsf{Bool} \rightarrow ()) \Rightarrow \alpha \rightarrow \mathsf{Int}$

f x =

let g : $\forall\beta.(print :: \alpha \rightarrow \beta \rightarrow ()) \Rightarrow \beta \rightarrow ()$

g y = $print$ x "Header Information"; $print$ x y

in (g 1, g true)

In the next section, I make similar observations in case of Ohori–style records and provide more details of how the typing judgments can be derived.

I maintain that the projection operator in HM(X) allows to give a semantic

explanation for the open world approach by constraint rules of the form

$$\exists\alpha.(o :: \alpha \to \tau) =^e \mathsf{true}$$

This was already observed in the previous section. Projection is not necessarily trivial on all constraints in solved forms, for example consider $\exists\beta.(\mathit{print} :: \alpha \to \beta \to ()))$. If projection has to be trivial when quantifying over free type variables, this imposes an ordering relation on the free type variables, see the quantifier introduction rule in *SystemO*. This can lead to some limitations. Because of the ordering relation imposed on the free type variables, less programs are typable. The HM(X) formulation of the quantifier introduction rule overcomes such problems. Similar observations can be made in case of Ohori–style records.

3.5.4 Ohori Records

I consider Ohori's record calculus [Oho95] [6] which deals with record types of the form $\{l_1 : \tau_1, \ldots, l_n : \tau_n\}$. Polymorphic type variables are kinded, restricting the possible instantiation types. Type schemes are of the form $\forall\alpha.(\alpha :: k) \Rightarrow \tau$ where type variable α has kind k. A kind k is either the universal kind U denoting the set of all types or a record kind $\{\!\{l : \tau'\}\!\}$ denoting the set of all record types that contain a field with label l and type τ'. This allows to type various record operations in a concise way. For instance, the type of the record selection operator is written as follows:

$$(_.l) : \forall\alpha, \beta.(\beta :: U) \wedge (\alpha :: \{\!\{l : \beta\}\!\}) \Rightarrow \alpha \to \beta$$

[6] A technical presentation of his calculus can be found in Section 5.1.

The function $(_.l)$ takes as an argument any record that contains a field with label l and extracts the field value associated to l.

Kinded record types have to fulfill some conditions. For instance, each field label must have a unique type, see Section 5.1 for details.

Ohori's formulation of the quantifier introduction rule is written as follows:

$$(\forall \text{ Intro–Ohori}) \quad \frac{C \wedge (\alpha :: k), \Gamma \vdash e : \sigma \quad \alpha \notin fv(\Gamma) \cup fv(C) \quad \alpha \notin fv(k)}{C, \Gamma \vdash e : \forall \alpha.(\alpha :: k) \Rightarrow \sigma}$$

where the function *fv* computes the free variables. Ohori's formulation of the quantifier introduction rule is a close cousin to the formulation found in [OWW95]. Hence, I can apply a similar reasoning.

Ohori records with overloaded field labels

Given a record calculus where one can overload field labels [7]. Then one can deal with record types of the form

$$\{print : \mathsf{Int} \to (), print : \mathsf{Bool} \to (), print : \mathsf{String} \to ()\}$$

where () represents the unit type. Each field represents a different implementation of the *print* function on integers, booleans and strings. Such a record calculus might be useful to group related functions into a record. Hence, a record type can be seen now as the specification for a library. Record selection corresponds to accessing a specific function supplied in a given library. The above record type could represent the specification for a library that supplies IO–operations. The technical presentation of such a calculus can be found in Section 5.1.4.

[7] The technical presentation of such a calculus can be found in Section 5.1.4.

Consider now the following scenario. The IO–library is passed to a function which implements a local IO function based on the given IO–library functions. Please note, the following example is similar to Example 14. This is not surprising because the record application with overloaded field labels can be encoded in terms of the overloading application with curried overloaded functions.

Example 17

```
f x =
  let g y = (x.print) "Header Information"; (x.print) y
  in (g 1, g true)
```

where I assume that the pairing operator $(_,_)$ and the sequencing operator $_;_$ are primitives contained in an initial type environment Γ_0. Function g has to be of polymorphic type because one might want to apply function g to arguments with different types in the let-body. In the above example, I apply g to an integer and a boolean.

But such a program is not typable in a calculus based on the ($\forall$ Intro–Ohori) rule. Consider an intermediate derivation. Typing judgments are now extended with a constraint component on the left hand side of the turnstile.

$$(\alpha :: \{\!\{print : \beta \to ()\}\!\}) \wedge (\alpha :: \{\!\{\mathsf{String} \to ()\}\!\}) \wedge (\beta :: U),$$
$$\Gamma_0.\mathsf{x} : \alpha$$
$$\vdash$$
$$\lambda \mathsf{y}.(\mathsf{x}.print)\,\text{"Header Information"}; (\mathsf{x}.print)\,\mathsf{y} : \beta \to ()$$

In order to give function g a polymorphic type one has to quantify over type variable β. But the side conditions of the ($\forall$ Intro–Ohori) rule forbid to quantify over β because $\beta \in fv((\alpha :: \{\!\{print : \beta \to ()\}\!\}))$ (or expressed in terms of the constraint

language, projection would not be trivial). Hence, Example 17 is not typable in a system based on the (∀ Intro–Ohori) rule.

Remember the quantifier introduction rule in HM(X):

$$(\forall \text{ Intro}) \quad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

In HM(X) it is now possible to quantify over β. Quantification over β results in the constraint [8] $\exists \beta.((\alpha :: \{\!\{print : \beta \to ()\}\!\}) \wedge (\beta :: U))$ which is kept on the left hand side of the turnstile. In the let–body I apply function g to two arguments which results in the constraint

$$\begin{aligned} C \;\; = \;\; & (\alpha :: \{\!\{print : \mathsf{String} \to ()\}\!\}) \wedge (\alpha :: \{\!\{print : \mathsf{Int} \to ()\}\!\}) \wedge \\ & (\alpha :: \{\!\{print : \mathsf{Bool} \to ()\}\!\}) \end{aligned}$$

because the constraint in the type scheme of function g has to be fulfilled. In HM(X) I can type Example 17 in the following way:

```
f : ∀α.C ⇒ α → ((), ())
f x =
  let g : ∀β.(α :: {{print : β → ()}}) ∧ (β :: U) ⇒ β → ()
      g y = (x.print) "Header Information"; (x.print) y
  in (g 1, g true)
```

Note, the constraint $\exists \beta.((\alpha :: \{\!\{print : \beta \to ()\}\!\}) \wedge (\beta :: U))$ disappears because the constraint $(\exists \beta.((\alpha :: \{\!\{print : \beta \to ()\}\!\}) \wedge (\beta :: U))) \wedge C$ is semantically equivalent to the constraint C.

[8] Note, $(\alpha :: \{\!\{print : \beta \to ()\}\!\}) \wedge (\beta :: U)$ corresponds to the constraint D in the above formulation of rule (∀ Intro).

I can conclude that the (∀ Intro–Ohori) rule has some limitations, if Ohori's record calculus is extended with some advanced features such as overloaded field labels. There are programs where rule (∀ Intro–Ohori) forbids quantification over a type variable. Such programs are now typable within the HM(X) framework.

Nishimura message passing

Recently, there have been a proposal by Nishimura [Nis98] for a calculus where method names can be passed around as arguments. Method names are treated as field labels in a record. In a later section, I will show how to encode Ohori's record calculus based on the overloading approach by Odersky et al. Furthermore, I present an extension of Odersky et al.'s overloading approach with first class overloaded identifiers, i.e. overloaded identifier can now be passed around as arguments, see Section 5.2.2. Nishimura's record calculus is based on Ohori's record calculus. More specifically, Nishimura's formulation of the quantifier introduction rule is equivalent to Ohori's formulation. Hence, I can encode his calculus in terms of the extended overloading application where overloaded identifiers are first class values. The encoding is done along the lines as for Ohori's calculus.

In Nishimura's calculus, there are problems which are similar to ones discussed in the previous section. Essentially, I give a recast of Example 17 in terms of a record calculus with first class field labels. The following program is typable in HM(X) but not in a calculus based on the (∀ Intro–Ohori) rule. The reason is the same as in Example 17. Projection always has to be trivial in the (∀ Intro–Ohori) rule which does not hold in the following example. Hence, this example would not be typable in Nishimura's calculus.

Example 18

$$\mathsf{f} : \forall\alpha.(\alpha :: \{\!\{\mathit{print_int} : \mathsf{Int} \to ()\}\!\}) \wedge (\alpha :: \{\!\{\mathit{print_bool} : \mathsf{Bool} \to ()\}\!\}) \Rightarrow \alpha \to ((), ())$$

```
f x =
    let g: ∀print, β.(α :: {{print : β → ()}}) ⇒ print → β → ()
        g = λ print, y . (x.print) y
    in (g print_int 1, g print_bool true)
```

The function g takes as an additional parameter now an overloaded identifier. Note, I distinguish between the term variable print and the type variable *print* (The same holds for print_int and print_bool.). The term print has type *print*. The type variable *print* ranges only over overloaded identifiers (Specifically, *print* ranges only over field labels, but I assume that the record calculus with first class field labels is encoded in terms of an overloading application with first class overloaded identifiers.). The constraint $(_ :: \{\{_ : _\}\})$ takes now three arguments, a record, a field and the type of the field in the given record. I use sorts to ensure that the second argument always has to be an overloaded identifier. This is reflected in the term algebra which is now multi–sorted.

In fact, Nishimura's calculus is a little bit more restrictive. A method name (which corresponds to a field label resp. an overloaded identifier) needs always to be attached to its arguments. I could put stronger conditions on the extended overloading application to enforce this. In the above example, a method name is always passed with its argument, hence this example still applies to Nishimura's calculus.

3.5.5 Projection in Solved Form

I observe that the projection–operator does not need to be present in syntactic form in case of let–realizable judgments. An expression e is let–*realizable* iff every variable x defined by a let–statement appears in a monomorphic context. A judgment $C, \Gamma \vdash e : \tau$ is let–realizable iff expression e is let–realizable. A derivation $C, \Gamma \vdash e : \sigma$ is

projection–free iff constraint C is projection–free and in all sub–derivations $C', \Gamma' \vdash e' : \sigma'$ constraint C' is projection–free as well.

Theorem 6 (Projection in Solved Form) *Given a* let*–realizable judgment* $C, \Gamma \vdash e : \tau$ *where constraints appearing in* Γ *are projection–free. Then one can derive* $C', \Gamma \vdash e : \tau$ *such that* $C', \Gamma \vdash e : \tau$ *is projection–free and* $C' =^e C$ *for some constraint* C'*.*

Proof: Building of type schemes becomes only important in case of the let–rule. Given the expression $\mathsf{let}\ f = e_1\ \mathsf{in}\ e_2$. Assume typing of e_1 yields the type scheme $\forall\bar{\alpha}.C \Rightarrow \tau$, then the ($\forall$ Intro) rule requires that at least the constraint $\exists\bar{\alpha}.C$ must be kept on the left–hand side of the turnstile. By assumption expression e is let–realizable. This implies that the type scheme $\forall\bar{\alpha}.C \Rightarrow \tau$ needs to be instantiated to type $[\bar{\tau}/\bar{\alpha}]\tau$ for some $\bar{\tau}$. But then the constraint, let us refer to it as D, on the left–hand side of the judgment entails the constraint $[\bar{\tau}/\bar{\alpha}]C$ and the constraint $\exists\bar{\alpha}.C$. It holds that $[\bar{\tau}/\bar{\alpha}]C \vdash^e \exists\bar{\alpha}.C$. Hence, the constraint $\exists\bar{\alpha}.C$ is redundant. Constraints on the left–hand side of the turnstile can be replaced by a semantically equivalent constraint,see Lemma 13 in Section 3.4.3. Hence, the constraint $\exists\bar{\alpha}.C$ needs not to be present in syntactic form on the left–hand side of the typing judgment. This implies projection needs not to appear in syntactic form. This holds for all sub–derivations.

■

Note that the above lemma crucially relies on the fact that we consider constraints in typing judgments and in type schemes as an equivalence class, i.e. we identify $C, \Gamma \vdash e : \sigma$ with the equivalence class $\{D, \Gamma \vdash e : \sigma \,|\, \exists C' \in \mathcal{S}.D =^e C =^e C'\}$ (see Section 3.2).

An important implication of the above theorem is summarized in the following corollary. Projection does not influence typability.

Corollary 3 *Given an instance $\mathcal{X}$ of the* HM(X) *framework where $\mathcal{S}$ denotes the set of solved forms. We denote by $\mathcal{S}^{\exists}$ the smallest superset of $\mathcal{S}$ that is closed under projection. We consider a variation of $\mathcal{X}$, denoted by $\mathcal{X}^{\exists}$ where the set of solved forms equals $\mathcal{S}^{\exists}$. Then $\mathcal{X}$ and $\mathcal{X}^{\exists}$ type the same set of* let*–realizable programs.*

Proof: We need to transform judgments in $\mathcal{X}^{\exists}$ into judgments in $\mathcal{X}$. This can be achieved by applying the above theorem. ■

Note that the above statement does not imply that projection in syntactic form can always be omitted. This might affect the expressiveness of the type system. I only can transform derivable judgments into projection–free judgments. The examples in Section 3.5.3 and 3.5.4 showed that projection appearing in syntactic form is crucial in some typing situations.

Projection becomes also important when performing type inference. Definitions commonly are typed first before possible instantiation sites. Hence, projection allows to overcome problems as exemplified in Example 13 in Section 3.5.1.

3.5.6 Summary

The HM(X) formulation of the quantifier introduction rule improves over previous formulations. This is achieved by the projection operator which allows to bind type variables additionally in the constraint part. Previous syntactically formulated versions of the quantifier introduction rule such as [OWW95, Oho95] gain from the HM(X) formulation.

3.6 Terms versus Constraints

In this section, I elaborate on two possible representation forms of types. Types in *term* form are represented in the language of types (specified by a term algebra) whereas types in *constraint* form are expressed in the language of constraints (specified by a term constraint system).

3.6.1 A Motivating Example

I give a type system [9] which is close in spirit to the dimension system given in Section 3.2. There are list data types which are parameterized in the length of the list. Compared to other systems, the list constructor List takes as an additional parameter the length of the list. For simplicity, I assume that all list elements are of the same type, e.g. Int. Then, I write $\mathsf{List}\, i$ to denote an integer list of length i. This is quite similar to the dimension type system. Instead of dimensions, there are now indices.

$$\begin{array}{lrcl} \textbf{Indices} & i & ::= & \alpha \mid +(i,i) \mid 0 \mid 1 \\ \textbf{Types} & \tau & ::= & \alpha \mid \mathsf{List}\, i \mid \tau \to \tau \end{array}$$

The constant constructor 0 represents the zero, and 1 represents the one index. The constructor $+(\cdot,\cdot)$ allows to add indices and obeys the expected algebraic laws, see Section 3.1 for details. Commonly, I abbreviate $+(1,1)$ by 2 and so forth.

Consider now the following initial type environment:

[9] The interested reader can find another example in the Appendix B.2.

$$\begin{array}{rcrl} \Gamma & = & \{\mathsf{l}_1 & : \mathsf{List}\ 1, \\ & & \mathsf{l}_2 & : \mathsf{List}\ 2, \\ & & \mathit{remove} & : \forall n, m.\mathsf{List}\ +(n,m) \to \mathsf{List}\ n \to \mathsf{List}\ m, \\ & & \mathit{pair} & : \forall t_1, t_2.t_1 \to t_2 \to (t_1, t_2)\} \end{array}$$

where l_1 and l_2 are some arbitrary lists of length one and two, *remove* is a primitive operation which takes two lists and removes as many elements from the first list as the second list contains and *pair* is the pairing operator. Consider the following expression:

$$e \quad = \quad \lambda x.\mathsf{let}\ y = \mathit{remove}\, x\ \mathsf{in}\ \mathit{pair}(y\,\mathsf{l}_1)(y\,\mathsf{l}_2)$$

I can not give function y a polymorphic type if the set $\mathcal{S}$ of solved forms consists of only the true element. In an intermediate step, I find the following judgment:

$$\Gamma.x : \mathsf{List}\ +(n,m) \vdash \mathit{remove}\, x : \mathsf{List}\ n \to \mathsf{List}\ m \tag{3.2}$$

Indices n and m appear on the left–hand side of the turnstile which prohibits to quantify over them. Let us assume that $\mathcal{S}$ consists now of all satisfiable constraints. Then I can derive

$$(\alpha = +(n,m)), \Gamma.x : \mathsf{List}\ \alpha \vdash \mathit{remove}\, x : \mathsf{List}\ n \to \mathsf{List}\ m$$

and it is now possible to quantify over indices m and n. Expression e is now typable:

$$(\alpha = +(n', m')) \wedge (n' = 1) \wedge (m' = \beta) \wedge$$
$$(\alpha = +(n'', m'')) \wedge (n'' = 2) \wedge (m'' = \gamma),$$
$$\Gamma \vdash e : \mathsf{List}\ \alpha \rightarrow (\mathsf{List}\ \beta, \mathsf{List}\ \gamma)$$

Expression e is not typable if $\mathcal{S}$ consists only of the true element because the lambda–variable x with type α has to satisfy the constraint

$$(\alpha = +(n', m')) \wedge (n' = 1) \wedge (m' = \beta) \wedge$$
$$(\alpha = +(n'', m'')) \wedge (n'' = 2) \wedge (m'' = \gamma)$$

But there is no term representation for α covering all solutions which are apparent in the constraint representation. Therefore, I can not give function y a polymorphic type, hence e is not typable in a system where types are represented in term form. The questions arises over which conditions both views, term respectively constraint form, are equivalent. It will turn out that unitary unification is the key to establishing a one–to–one correspondence between both representation forms.

3.6.2 Formalizing a Connection between Terms and Constraints

Given a constraint system X over a term algebra $\mathcal{T}$. I define $\mathcal{X}^c$ as the system obtained from X where the set $\mathcal{S}$ consists of all satisfiable constraints. I refer to such a system as a *constraint*–based system and say that the system (respectively the types) is in constraint form/presentation. I define $\mathcal{X}^t$ as obtained from X where the set $\mathcal{S}$ might be possibly more restricted. I refer to such a system as a *term*–based system and say that the system (respectively the types) is in term form/presentation.

I refer to $\vdash^{\mathcal{X}^c}$ as the derivation in the constraint–based fragment $\mathcal{X}^c$ and to $\vdash^{\mathcal{X}^t}$ as the derivation in the term–based fragment $\mathcal{X}^t$. Please note, both systems, $\mathcal{X}^c$ and $\mathcal{X}^t$, share the underlying constraint system. There is only a difference in the set $\mathcal{S}$ of solved forms.

I can transform judgments in constraint form to judgments in term form, if there is a minimal term solution to each satisfiable constraint. I refer to the process of finding solutions to constraint problems as *normalization*. I repeat Definitions 23 and 25 found in Section 4.2.

Definition 23 *Let* X *be a term constraint system over a term algebra* $\mathcal{T}$ *and* $\mathcal{S}$ *be the set of solved constraints in* X. *Let* $C \in \mathcal{S}$ *and* $D \in$ X *be constraints and let* ϕ,ψ *be substitutions such that* (D, ϕ) *is a constraint problem. Then* (C, ψ) *is a* normal form *of* (D, ϕ) *iff* $\phi \leq^{\phi'} \psi$, $C \vdash^e \phi' D$ *and* $\psi C = C$.

(C, ψ) *is* principal *if for all normal forms* (C', ψ') *of* (D, ϕ) *it holds that* $\psi \leq^{\phi'} \psi'$ *and* $C' \vdash^e \phi' C$.

Definition 25 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set of solved constraints* $\mathcal{S}$ *in* X. *The constraint system* X *has the* principal constraint property *if for every constraint problem* (D, ϕ) *in* X, *either* (D, ϕ) *does not have a normal form or* (D, ϕ) *has a principal normal form.*

The following definition approximates type schemes in constraint form by type schemes in term form.

Definition 18 *Given a type* σ *in* $\mathcal{X}^c$ *and a type* σ' *and a constraint* C *in* $\mathcal{X}^t$. *Then* $C \vdash \sigma \mapsto \sigma'$ *iff the following two conditions hold*

- $\vdash^i \sigma \preceq \sigma'$

- *Given a type σ'' in $\mathcal{X}^t$ such that $C \vdash^i \sigma \preceq \sigma''$. Then $C \vdash^i \sigma' \preceq \sigma''$.*

The judgment $\mapsto$ extends to type environments in the usual way.

Theorem 7 (Transformation) *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set $\mathcal{S}$ of solved forms such that $\mathcal{S}$ is closed under projection and $\mathcal{X}^t$ enjoys the principal constraint property, a type environment Γ' in $\mathcal{X}^t$ and a constraint C in $\mathcal{X}^t$ where $C \vdash \Gamma \mapsto \Gamma'$. Then the following holds.*

- *If $C, \Gamma \vdash^{\mathcal{X}^c} e : \tau$ then $C, \Gamma' \vdash^{\mathcal{X}^t} e : \tau$.*
- *If $C, \Gamma \vdash^{\mathcal{X}^c} e : \sigma$ where σ realizable in C then $C, \Gamma' \vdash^{\mathcal{X}^t} e : \sigma'$ where $C \vdash \sigma \mapsto \sigma'$.*

An immediate consequence is the following corollary. Please note, a substitution $\phi = [\bar{\tau}/\bar{\alpha}]$ can also be regarded as the constraint $(\alpha_1 = \tau_1) \wedge \ldots \wedge (\alpha_n = \tau_n)$.

Corollary 4 (Substitutions in Constraint Form) *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set $\mathcal{S}$ of solved forms such that $\mathcal{S}$ is closed under projection and $\mathcal{X}^t$ enjoys the principal constraint property, a type environment Γ' in $\mathcal{X}^t$and a judgment $D \wedge \phi, \Gamma \vdash^{\mathcal{X}^c} e : \alpha$ such that (D, ϕ) is a principal normal form. Then $D, \phi\Gamma \vdash^{\mathcal{X}^t} e : \phi(\alpha)$.*

I am now in the position to state under which conditions there is a correspondence between judgments in constraint and term form.

Theorem 8 (Full and Faithful) *Given a constraint system X over a term algebra $\mathcal{T}$. If $\mathcal{X}^t$ satisfies the principal constraint property then every program typable in $\mathcal{X}^c$ is typable in $\mathcal{X}^t$ and vice versa.*

In the special case of equational theories, the principal constraint property corresponds to unitary unification. For instance, the motivating example does not satisfy the principal constraint property. The constraint

$$(\alpha = +(n', m')) \wedge (n' = 1) \wedge (m' = \beta) \wedge$$
$$(\alpha = +(n'', m'')) \wedge (n'' = 2) \wedge (m'' = \gamma)$$

does not have a most general unifier. Remember, there is not a minus constructor in the index theory. Adding the minus constructor to the theory would allow to represent α in term form. But this would only fix the problem for this specific example. The index theory still does not satisfy the principal constraint property.

3.6.3 Summary

The constraint–based formulation of type systems is more expressive than the term–based formulation. That means, typing information represented in constraint form improves over the term presentation. Principal normal forms represent best solutions of constraint problems (D, ϕ). In the presence of best solutions (the principal constraint property holds), the constraint–based and term–based formulation of types systems are equivalently expressive. Another observation is that given a derivation $D \wedge \phi, \Gamma \vdash^{\mathcal{X}^c} e : \alpha$ in the constraint–based system where (D, ϕ) is a principal normal form. Then also $D, \phi\Gamma \vdash^{\mathcal{X}^t} e : \phi(\alpha)$ is derivable in the term–based system.

3.7 Logical System in Constraint Form

I present a variation of the HM(X) framework where all typing information is represented in constraint form. The motivation is to give the cleanest possible presentation for such a system. I only highlight the main differences to the original presentation.

I commonly refer to the original presentation as the term presentation and to the new presentation as the constraint presentation.

I work with the following syntactic domains.

$$
\begin{array}{lrcl}
\textbf{Values} & v & ::= & x \mid c \mid \lambda x.e \\
\textbf{Expressions} & e & ::= & v \mid e\, e \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \\
\textbf{Types} & \tau & ::= & \alpha \mid\ \tau \to \tau \\
\textbf{Type schemes} & \sigma & ::= & \tau \mid \forall \alpha.C \Rightarrow \sigma
\end{array}
$$

The set of values and expressions remains unchanged, whereas in the syntactic domain of types I only admit type variables and function types. All other types must be represented in the constraint language. I require that constraints on the left-hand side of typing judgments and in type schemes must be satisfiable. Recall, a constraint C is satisfiable if $\vdash^e \exists fv(C).C$. This is the only requirement which I put on constraints which differs to the term presentation. There, I could possibly further restrict the set of constraints which appear in typing judgments and type schemes. For instance, in order to model the Hindley/Milner only the empty constraint $\{\}$ (representing true) is allowed to appear in typing judgments. This was motivated by the term representation of types in the Hindley/Milner system. Here, I prefer a constraint presentation of types. This change implies the occurrence of explicit equality constraints in the HM(X) system in constraint form, which were represented as substitutions in the term presentation.

The typing rules can be found in Figure 3.4. I assume that type schemes are equivalent up to renaming of bound type variables. Technically, I consider a type scheme $\forall \bar{\alpha}.C \Rightarrow \tau$ as an equivalence class where the equivalence relation $\sim$ is defined as follows: $(\forall \bar{\alpha}.C_1 \Rightarrow \tau_1) \sim (\forall \bar{\beta}.C_2 \Rightarrow \tau_2)$ if $[\bar{\gamma}/\bar{\alpha}]C_1 =^e [\bar{\gamma}/\bar{\beta}]C_2$ and $[\bar{\gamma}/\bar{\alpha}]\tau_1 = [\bar{\gamma}/\bar{\beta}]\tau_2$ where $\bar{\gamma}$ are fresh type variables and $=$ stands for syntactic equality.

$$\text{(Var)} \qquad C, \Gamma \vdash x : \sigma \quad (x : \sigma \in \Gamma)$$

$$\text{(Sub)} \qquad \frac{C, \Gamma \vdash e : \tau \quad C \vdash^e (\tau \preceq \tau')}{C, \Gamma \vdash e : \tau'}$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x.x : \tau \vdash e : \tau'}{C, \Gamma_x \vdash \lambda x.e : \tau \to \tau'}$$

$$\text{(App)} \qquad \frac{C, \Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad C, \Gamma \vdash e_2 : \tau_1}{C, \Gamma \vdash e_1 e_2 : \tau_2}$$

$$\text{(Let)} \qquad \frac{C, \Gamma_x \vdash e : \sigma \qquad C, \Gamma_x.x : \sigma \vdash e' : \tau'}{C, \Gamma_x \vdash \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

$$\text{(}\forall\text{ Intro)} \qquad \frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

$$\text{(}\forall\text{ Elim)} \qquad \frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash^e \exists \bar{\alpha}.D}{C \wedge D, \Gamma \vdash e : \tau'}$$

$$\text{(}\exists\text{ Intro)} \qquad \frac{C, \Gamma \vdash e : \sigma \qquad \bar{\alpha} \notin fv(\Gamma) \cup fv(\sigma)}{\exists \bar{\alpha}.C, \Gamma \vdash e : \sigma}$$

Figure 3.4: Logical type system in constraint form

If not otherwise stated, most of the properties carry over from the HM(X) system in term form to the HM(X) system in constraint form. For instance, judgments are still closed under strengthening the constraint.

Lemma 19 (Weakening) *Given a judgment $C, \Gamma \vdash e : \sigma$ and a satisfiable constraint D such that $D \vdash^e C$. Then $D, \Gamma \vdash e : \sigma$.*

I give a brief description of the typing rules and point out only the changes compared to the formulation in term form. Rules (Var) to (∀ Intro) remain unchanged

whereas I reformulated rule (∀ Elim) and introduced the new rule (∃ Intro).

Rule (∀ Elim) differs compared to the formulation in term form which was previously as follows:

$$\frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash^e [\bar{\tau}/\bar{\alpha}]D}{C, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'}$$

This rule exhibits the influence of the term–based approach. Type instances are generated by substituting bound type variables. Here, I give up the concept of substitutions (besides variable renaming). Satisfiability of the constraint part of the constraint is ensured via the side condition $C \vdash^e \exists \alpha.D$. It would be possible to adopt the constraint–based formulation of the (∀ Elim) rule to the original HM(X) formulation, i.e. the HM(X) system in term form. Both formulations are essentially equivalent as stated in the following lemma.

Lemma 20 (Equivalence of Quantifier Introduction Rules) *Given an instance of the HM(X) system in term form. If the judgment $C, \Gamma \vdash e : \sigma$ is derivable using the constraint–based (∀ Elim) rule then $C, \Gamma \vdash e : \sigma$ is derivable using the substitution–based formulation of the (∀ Elim) rule instead. The reverse direction holds, too.*

Rule (∃ Intro) is motivated by the following logical rule: $\forall \alpha.(P \to Q) = (\exists \alpha.P) \to Q$ where $\alpha \notin fv(Q)$. The outermost forall quantification of the free type variable α in the judgment $C, \Gamma \vdash e : \sigma$ can be moved to the constraint part, if α does only appear in the constraint part. This hides type variables which only appear in the constraint part of typing judgments.

Rules (∀ Elim) and (∃ Intro) evolved out of a discussion with Martin Müller.

Rules (Abs) and (App) require the presence of function types in the syntactic domain of types. It would be possible to give a formulation of the typing rules where only type variables appear in the syntactic domain of types. However, the current formulation looks more aesthetic, I need to admit function types in the domain of types. There is no difference in the logical properties of the two formulations. It is only a matter of taste whether to allow additionally function types in the domain of types or not. The following lemma shows that one can always put typing judgments in canonical form.

Lemma 21 (Canonical Form) *Given a constraint C, a type environment Γ, a term e and a type τ. Then $C, \Gamma \vdash e : \tau$ iff $C \wedge (\alpha = \tau), \Gamma \vdash e : \alpha$ where α is a fresh type variable.*

3.7.1 Summary

I gave a logical presentation of the HM(X) system where all typing information is represented in constraint form. The motivation was to give the cleanest possible presentation for such a system.

Chapter 4

Type Inference in HM(X)

The task of the inference system is to infer a type for a given term e. I consider an inference system in compositional style. Expressions are composed of smaller subexpressions for which the inference system already inferred its types. As an example, consider the function application $e_1 e_2$ where e_1 has the inferred type τ_1 and e_2 has the inferred type τ_2. For the moment, let us consider the standard Hindley/Milner system. The typing problem $e_1 e_2$ is transformed into the constraint problem $(\tau_1 = \tau_2 \to \alpha)$. The fresh type variable α corresponds to the yet unspecified result type of the application $e_1 e_2$. Constraint solving in the standard Hindley/Milner system corresponds to Herbrand unification. The situation is different in case of an extension of the Hindley/Milner system with subtypes. The typing problem $e_1 e_2$ is transformed into the constraint problem $(\tau_1 <: \tau_2 \to \alpha)$. Then solving a constraint problem means simply whether the constraint is satisfiable or not.

In both cases, constraint generation remains similar. It involves accumulation of either equality or subtype constraints. In general, constraint generation means accumulation of subsumption constraints. But the form of constraint solving differs. Depending on the structure of the set $\mathcal{S}$ of solved forms, constraint solving might

involve more sophisticated methods than Herbrand unification or a satisfiability test. In the latter, I refer to constraint solving as constraint normalization.

Hence, type inference consists of two phases: constraint generation and constraint normalization. I provide a generic type inference algorithm which is parameterized in the constraint solver. A characterization of minimal constraint solutions is given in Section 4.2. In such cases, I say the constraint system possess most general normalizers. One would expect to infer principal types if the constraint solvers computes most general normalizers. But there is a subtlety first discovered by Andrew Kennedy [Ken96] who gave the following example.

4.1 A Problem when Inferring Principal Types

Kennedy introduces a Hindley/Milner style system that deals with physical dimensions. In addition to the usual types, there are dimension types. The term algebra of dimension types is defined as follows:

$$\begin{array}{lrcl} \textbf{Dimensions} & d & ::\supseteq & \alpha \mid i(d) \mid \mathit{prod}(d,d) \mid 1 \mid \mathrm{M} \mid \mathrm{T} \\ \textbf{Types} & \tau & ::= & \alpha \mid \dim(d) \mid \tau \to \tau \end{array}$$

The dimension constructor $i(\cdot)$ corresponds to the inverse of a dimension and $\mathit{prod}(\cdot,\cdot)$ to the product of two dimensions. Dimension constants are 1 for the unit measure, M for the mass dimension and T for the time dimension. For specific applications there might be further dimension constants. Please consult Section 3.1 and Section 3.2 for further description of the dimension type system.

Consider the following initial type environment:

$$\begin{aligned}\Gamma \;=\; \{kg \;&:\; \dim \mathrm{M},\\ s \;&:\; \dim \mathrm{T},\\ div \;&:\; \forall d_1, d_2.\, \dim prod(d_1, d_2) \to \\ &\qquad \dim d_1 \to \dim d_2,\\ pair \;&:\; \forall t_1, t_2. t_1 \to t_2 \to (t_1, t_2)\}\end{aligned}$$

Here, kg and s are some basic dimensions, $pair$ is the pairing operator and div is a primitive operation on dimensions. Consider the following expression:

$$e \qquad = \qquad \lambda x.\mathsf{let}\ y = div\, x\ \mathsf{in}\ pair(y\, kg)(y\, s)$$

Function y is applied to arguments of different type, hence needs to be of polymorphic type. The subexpression $div\, x$ has the following type under type environment $\Gamma.x : \dim prod(d_1, d_2)$:

$$\Gamma.x : \dim prod(d_1, d_2) \vdash div\, x : \dim d_1 \to \dim d_2 \tag{4.1}$$

It is not possible to quantify over the type variables d_1 and d_2 because d_1 and d_2 also appear in the type environment. But I can derive another type for $div\, x$ under the same type environment:

$$\begin{gathered}\Gamma.x : \dim prod(d_1, d_2)\\ \vdash \\ div\, x : \dim prod(d_1, d_3) \to \dim prod(i(d_3), d_2)\end{gathered} \tag{4.2}$$

Judgment 4.2 is obtained from 4.1 by instantiating d_1 with $prod(d_1, d_3)$ and d_2

with $prod(i(d_3), d_2)$. Dimension types obey the laws of an abelian group. For instance, it holds that

$$\vdash^e \ (prod(prod(d_1, d_3), prod(i(d_3), d_2)) = prod(d_1, d_2))$$

Therefore, the type environment in 4.2 remains the same and now it is possible to quantify over the free type variable d_3. Then program e becomes typable.

$$\begin{gathered} \Gamma \vdash e : \\ \forall d_1, d_2. \dim prod(d_1, d_2) \rightarrow \\ (\dim prod(i(\mathrm{M}), prod(d_1, d_2)), \\ \dim prod(i(\mathrm{T}), prod(d_1, d_2))) \end{gathered} \tag{4.3}$$

The theory of dimension types possesses most general unifiers. There is a decidable algorithm that computes the most general unifier, see [LBB84]. Hence, one would expect that type inference yields principal types. Unfortunately, it turns out that the standard type inference algorithm fails to infer principal types. In case of dimension types, there are several different representation types for a given term under the same type environment, see judgments (4.1) and (4.2). If one chooses a wrong representation, it might not be possible to type the rest of the program. In the above example, I first had to apply a substitution to reveal the free type variable d_3. It it crucial that this substitution leaves the type environment unchanged.

Consider now a variation of the dimension type system where dimension types are allowed to appear in constraint form, i.e. the set $\mathcal{S}$ consists now of all satisfiable constraints. Then, I find the following variation of typing judgment (4.2).

$$(\alpha = prod(d_1, d_2)), \Gamma.x : \alpha \vdash \ div\, x : \dim d_1 \rightarrow \dim d_2$$

If I would unify the constraint $(\alpha = \mathit{prod}(d_1, d_2))$, I would obtain judgment (4.2) again. But I keep constraints in explicit form and then type variables d_1 and d_2 do not appear anymore in the type environment. I can quantify over these type variables. The function y has now the desired polymorphic type

$$\forall d_1, d_2.(\alpha = \mathit{prod}(d_1, d_2)) \Rightarrow \dim d_1 \rightarrow \dim d_2$$

The following typing judgment is a variation of judgment (4.3).

$$\begin{gathered}(\alpha = \mathit{prod}(d_1', d_2')) \wedge (d_1' = \mathrm{M}) \wedge (d_2' = \beta) \wedge \\ (\alpha = \mathit{prod}(d_1'', d_2'')) \wedge (d_1'' = \mathrm{T}) \wedge (d_2'' = \gamma), \\ \Gamma \vdash e : \dim \alpha \rightarrow (\dim \beta, \dim \gamma)\end{gathered}$$

For instance, the constraint

$$(\alpha = \mathit{prod}(d_1', d_2')) \wedge (d_1' = \mathrm{M}) \wedge (d_2' = \beta)$$

appears by applying the term y to kg.

In a system in constraint form, e has type $\dim \alpha \rightarrow (\dim \beta, \dim \gamma)$ under the constraint

$$\begin{gathered}(\alpha = \mathit{prod}(d_1', d_2')) \wedge (d_1' = \mathrm{M}) \wedge (d_2' = \beta) \wedge \\ (\alpha = \mathit{prod}(d_1'', d_2'')) \wedge (d_1'' = \mathrm{T}) \wedge (d_2'' = \gamma)\end{gathered}$$

It must be noted that unification of the above constraint would yield judgment 4.3

4.1.1 Summary

I observe that standard inference performs unification eagerly. After each inference step the unification procedure is invoked. Then standard inference fails to infer

principal types even in the presence of most general unifiers. This is exemplified by Kennedy's example. It turns out that this is only a problem for regular theories. A theory is *non–regular* if there exists types τ, τ' such that $\vdash^e (\tau = \tau')$ and $fv(\tau) \neq fv(\tau')$. Dimension types form a non–regular theory. It holds that $\vdash^e (prod(\alpha, i(\alpha)) = 1)$ but $fv(1) = \emptyset$. The idea is now to present types in constraint form which overcomes the above mentioned problems.

4.1.2 Outline

In Section 4.3, I consider an extension of the standard inference algorithm to more general constraint domains. Instead of unification, inference is based on the concept of normalization. Normalization is performed in each inference step. This implies that type terms are constructed eagerly. I also call this form of inference term–based inference respectively inference in term form. [1] Complete inference can then only be achieved for regular theories. Section 4.4 considers inference in constraint form. The constraint–based inference system leads to an adaption of the term–based inference system. The basic idea is to represent types in constraint form in intermediate inference step. The actual normalization step (that means, switching from the constraint representation of types back to the term representation) happens only at the end. This allows to infer principal types in term form even for non–regular theories. The technical difficulty is then to show that type inference is still sound.

Preliminary results can already be found in [OSW99]. The main improvement is the introduction of the purely constraint–based inference approach and the solution of the inference problem for non–regular theories.

[1] It would also be appropriate to refer to this kind of inference as substitution–based inference because a type term τ can always be seen as the substitution $[\tau/\alpha]$ where α is a fresh type variable.

4.2 Normalization

I start with some preliminary definitions. The main novelty is the introduction of the concept of normalization and most general normalizers to which I also refer as principal normal forms.

Definition 19 (Equational Theory) *An* equational theory E *over a term algebra* $\mathcal{T}$ *is a constraint system where for each* $\tau, \tau' \in \mathcal{T}$ *there is an equality predicate* $(\tau = \tau')$ *and* $(\cdot = \cdot)$ *is a congruence relation. An equational theory* E *is* regular *iff for each* $\tau, \tau' \in \mathcal{T}$ *such that* $\vdash^e (\tau = \tau')$*, then* $fv(\tau) = fv(\tau')$*.*

Please note, a term constraint system X implies an equational theory E. I refer to E as the underlying equational theory of X.

Definition 20 (Substitutions) *Given an equational theory* E *over a term algebra* $\mathcal{T}$*. Given a substitution* ϕ *and a domain* U *of variables. Then* $dom(\phi) = \{\alpha \,|\, \phi(\alpha) \neq \alpha\}$ *denotes the domain of* ϕ *and* $codom(\phi) = \{\beta \,|\, \exists \alpha \in dom(\phi).\beta \in fv(\phi(\alpha))\}$ *denotes the codomain of* ϕ*. Let* $\phi_{|U}$ *be the restriction of the substitution* ϕ *to the domain* U*. That is,* $\phi_{|U}(x) = \phi(x)$ *iff* $x \in U$ *and* $\phi_{|U}(x) = x$ *otherwise. For substitutions* ϕ *and* ψ*, I write* $\psi =_U \phi$ *iff* $\vdash^e (\psi(x) = \phi(x))$ *for all* $x \in U$*. I write* $\psi \leq_U^{\phi'} \phi$ *iff* $\phi' \circ \psi =_U \phi$*. I write* $\psi \leq_U \phi$ *iff there exists* ϕ' *such that* $\psi \leq^{\phi'} \phi$*.* $\psi =_U \phi$ *iff* $\psi \leq_U \phi$ *and* $\phi \leq_U \psi$*. In general, I omit the set* U*.*

It follows from [LMM87] that $\leq_U$ is a complete lower semi–lattice where least upper bounds, if they exist, correspond to unifications and greatest lower bounds correspond to anti–unifications.

Definition 21 (Unitary unification) *Given an equational theory* E*. Then* E *enjoys* unitary unification *iff for substitutions* ϕ *and* ψ*, either their least upper bound*

exists, or there is no upper bound at all. The least upper bound of ϕ *and* ψ *is denoted by* $\phi \sqcup \psi$.

I also state that a constraint system X *enjoys* unitary unification *if the underlying equational theory in* X *enjoys unitary unification.*

The Herbrand constraint system HERBRAND enjoys unitary unification as shown by Robinson [Rob65]. Another example is the dimension system described in Section 3.1. In [Ken94], Kennedy shows (based on work by [LBB84]) that the dimension system enjoys unitary unification.

I extend the relation $\leq$ on substitutions to constraint problems.

Definition 22 (Constraint Problems) *Given a constraint system* X *over a term algebra* $\mathcal{T}$. *Given a constraint* C *in* X *and a substitution* ψ. *The pair* (C, ψ) *is called a* constraint problem *iff* $\psi C = C$. $(D, \phi) = (D_1, \phi_1) \sqcup (D_2, \phi_2)$ *is the* combination *of the constraint problems* (D_1, ϕ_1) *and* (D_2, ϕ_2) *iff* $\phi = \phi_1 \sqcup \phi_2$ *and* $D = \phi(D_1 \wedge D_2)$. $(C, \phi) \leq (C', \phi')$ *iff there exists a substitution* ψ *such that* $\phi \leq^{\psi} \phi'$ *and* $C' \vdash^{e} \psi C$, *and* $(C, \phi) = (C', \phi')$ *iff* $(C, \phi) \leq (C', \phi')$ *and* $(C', \phi') \leq (C, \phi)$. *This makes* $\leq$ *a partial ordering relation on constraint problems.*

I refer to $\mathcal{CP}$ *as the set of constraint problems in* X.

Lemma 22 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *such that* X *enjoys unitary unification. Then* $(\mathcal{CP}, \leq)$ *is a complete lower semi–lattice where least upper bounds exist.*

Normalization means then computation of a *normal form* of a constraint problem (C, ψ).

Definition 23 (Principal Normal Form) *Let* X *be a term constraint system over a term algebra* $\mathcal{T}$ *and* $\mathcal{S}$ *be the set of solved constraints in* X. *Let* $C \in \mathcal{S}$ *and* $D \in$ X

be constraints and let ϕ,ψ be substitutions such that (D, ϕ) is a constraint problem. Then (C, ψ) is a normal form *of (D, ϕ) iff $\phi \leq^{\phi'} \psi$, $C \vdash^e \phi' D$ and $\psi C =^e C$, or equivalently $(D, \phi) \leq (C, \psi)$.*

(C, ψ) is principal *if for all normal forms (C', ψ') of (D, ϕ) it holds that $\psi \leq^{\phi'} \psi'$ and $C' \vdash^e \phi' C$, or equivalently $(C, \psi) \leq (C', \psi')$.*

The principal normal form represents the *best* solution of a constraint problem. As an example, consider the constraint system HERBRAND. In it, a principal normal form corresponds to a most general unifier and a normal form corresponds to a unifier of a constraint problem.

Lemma 23 (Uniqueness) *Given a constraint problem (D, ϕ) and two principal normal forms (C, ψ) and (C', ψ') of (D, ϕ). Then $(C, \psi) = (C', \psi')$.*

The above lemma follows trivially. Please note, uniqueness only states uniqueness modulo semantic equivalence. Hence, a principal normal form should be seen as an equivalent class. Nevertheless, it is possible to define a well–defined function *normalize* from constraint problems (D, ϕ) to normal forms:

$$
\begin{aligned}
& \mathit{normalize}(D, \phi) \\
= \;& (C, \psi) \quad \text{if } (C, \psi) \text{ principal normal form of } (D, \phi) \\
= \;& \mathit{fail} \qquad \text{otherwise}
\end{aligned}
$$

Principal normal forms are syntactically unique if the set of solved forms satisfies the syntactic uniqueness property.

Definition 24 (Syntactic Uniqueness Property) *Given a set $\mathcal{S}$ of solved forms. Then $\mathcal{S}$ satisfies the* syntactic uniqueness property *if given $C \in \mathcal{S}$ there is no other constraint $D \in \mathcal{S}$ such that $C =^e D$.*

Lemma 24 (Syntactic Uniqueness) *Given a set $\mathcal{S}$ of solved forms such $\mathcal{S}$ satisfies the syntactic uniqueness property, a constraint problem (D, ϕ) and two principal normal forms (C, ψ) and (C', ψ') of (D, ϕ). Then (C', ψ) can be retrieved from (C, ψ) simply by variable renaming.*

Please note, checking whether a set of solved forms satisfies the syntactic uniqueness property is a non–trivial task. It requires a decidable entailment test. For instance, in case of subtyping, it is not know yet whether entailment is decidable or not [Pot98].

The property of having a principal normal form extends to constraint systems.

Definition 25 (Principal Constraint Property) *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set of solved constraints $\mathcal{S}$ in* X*. The constraint system* X *has the* principal constraint property *if for every constraint problem (D, ϕ) in* X*, either (D, ϕ) does not have a normal form or (D, ϕ) has a principal normal form.*

I state that a constraint system possesses *most general normalizers* if the constraint system satisfies the principal constraint property. I also state that the HM(X) type system has the principal constraint property if X has the principal constraint property.

In Section 5.1, I discuss in detail a type system for Ohori-style records that satisfies the principal constraint property. This example belongs to a class of constraint systems where constraint solving involves some form of unification. Further examples of constraint systems of this kind are HERBRAND and DIM. In general, there are already a wide range of unification procedures [BS94]. But mostly they work on more restricted constraint domains, e.g. the projection operator does not appear in the constraint language. In the next section, I show how these unification procedures can be lifted to become normalization procedures in HM(X).

The situation is different in case of the constraint system $\mathcal{SC}$. The set $\mathcal{S}$ of solved forms consists of all satisfiable constraints. Given a constraint problem (D, ϕ) I distinguish between two cases. If D (note that $\phi D = D$) is unsatisfiable then (D, ϕ) does not have a normal form. Assume D is satisfiable then (D, ϕ) is the (trivial) principal normal form of (D, ϕ). I conclude that the constraint system $\mathcal{SC}$ satisfies the principal constraint property, and that a *normalize* function can be defined as follows:

$$\begin{aligned} & \mathit{normalize}(C, \phi) \\ = \;& (C, \phi) \quad \text{if } C \text{ is satisfiable} \\ = \;& \mathit{fail} \quad \text{otherwise} \end{aligned}$$

The normalization function is computable since satisfiability in $\mathcal{SC}$ is decidable. This follows easily by adapting techniques developed in [TS96].

I elaborate more on the algebraic properties of the computation of principal normal forms in case the principal constraint property holds and unification is unitary. Systems where the set of solved forms is equation–free represent an important class because they enable type inference based on unification.

Definition 26 (Equation–free) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms. Then* $\mathcal{S}$ *is in* equation–free *iff for each* $C \in \mathcal{S}$ *if* $C \vdash^e (\tau = \tau')$ *then* $\vdash^e (\tau = \tau')$. *In such a situation, I refer to* $\mathcal{S}$ *as the set of solved and equation–free forms.*

Lemma 25 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* $\mathcal{S}$ *is equation–free. Then* X *enjoys unitary unification.*

The next lemmas lead to the conclusion that normalization can be performed in arbitrary order.

Lemma 26 *Given two constraint problems (D, ϕ) and (D', ϕ'), a principal normal form (C, ψ) of (D', ϕ') and a principal normal form (C', ψ') of (D, ϕ) such that (C, ψ) is also a normal form of (D, ϕ) and (C', ψ') is also a normal form of (D', ϕ'). Then (C, ψ) is a principal normal form of (D, ϕ) and (C', ψ') is principal normal form of (D', ϕ').*

Lemma 27 *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set of solved constraints $\mathcal{S}$ in* X *such that* X *enjoys the principal constraint property and unitary unification. Given constraint problems (D_1, ϕ_1) and (D_2, ϕ_2) such that $dom(\phi_1) \cap codom(\phi_2) = \emptyset$, $dom(\phi_2) \cap codom(\phi_1) = \emptyset$, $fv(D_1) \cap dom(\phi_2) = \emptyset$ and $fv(D_2) \cap dom(\phi_1) = \emptyset$. Let (C, ψ) be the principal normal form of $(D_1, \phi_1) \sqcup (D_2, \phi_2)$. Then the principal normal form (C_1, ψ_1) of (D_1, ϕ_1) exists and (C, ψ) is a normal form of $(C_1, \psi_1) \sqcup (D_2, \phi_2)$.*

Lemma 28 *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set of solved constraints $\mathcal{S}$ in* X *such that* X *enjoys the principal constraint property and unitary unification. Given constraint problems (D_1, ϕ_1) and (D_2, ϕ_2) such that $dom(\phi_1) \cap codom(\phi_2) = \emptyset$, $dom(\phi_2) \cap codom(\phi_1) = \emptyset$, $fv(D_1) \cap dom(\phi_2) = \emptyset$ and $fv(D_2) \cap dom(\phi_1) = \emptyset$. Let (C_1, ψ_1) be the principal normal form of (D_1, ϕ_1) and (C, ψ) be the principal normal form of $(C_1, \psi_1) \sqcup (D_2, \phi_2)$. Then (C, ψ) is a normal form of $(D_1, \phi_1) \sqcup (D_2, \phi_2)$.*

Theorem 9 (Eager versus by Need Normalization) *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set of solved constraints $\mathcal{S}$ in* X *such that* X *enjoys the principal constraint property and unitary unification. Given constraint problems (D, ϕ) and (D', ϕ') such that $dom(\phi_1) \cap codom(\phi_2) = \emptyset$, $dom(\phi_2) \cap codom(\phi_1) = \emptyset$,*

$fv(D_1) \cap dom(\phi_2) = \emptyset$ and $fv(D_2) \cap dom(\phi_1) = \emptyset$. Then

$$normalize((D, \phi) \sqcup (D', \phi'))$$
$$=$$
$$normalize(normalize(D, \phi) \sqcup D', \phi')$$

Proof: The equation $normalize((D, \phi) \sqcup (D', \phi')) = normalize(normalize(D, \phi) \sqcup D', \phi')$ states that if the left–side holds the right–side follows and vice versa. Please note, Lemma 26 allows to strengthen the result of Lemmas 27 and 28 to principal normal forms. ■

4.2.1 Lifting of Normalization

One of the improvement of the HM(X) framework is the projection operator which allows for a binding of type variables in the constraint part. The normalization procedure in HM(X) has to deal with projected constraints. whereas previously normalization only had to deal with projection–free constraints.

Definition 27 (Projection–free) *A constraint C is* projection–free *iff C considered as a set consists only of primitive predicates.*

Normalization in the special case of equality constraints has already been widely studied in the literature. There exits a wide range of results, I only mention [BS94]. In the following, I give a method on how to lift normalization procedures for projection–free constraints to the general case. Lifting holds under some sufficient conditions.

Definition 28 (Lifting Property) *Given a constraint system* X *over a term algebra $\mathcal{T}$ and a set $\mathcal{S}$ of solved forms.* X *satisfies the* lifting *property iff the following conditions hold:*

1. *There exists a computable procedure normalize such that for each constraint problem* (D, ϕ) *where* D *is projection–free, normalize*$(D, \phi) = (C, \psi)$ *and* (C, ψ) *is the principal normal form of* (D, ϕ)*, or normalize*(D, ϕ) *reports failure if* (D, ϕ) *does not have a normal form at all.*

2. $\mathcal{S}$ *is equation–free and this is the only condition imposed on* $\mathcal{S}$*.*

3. *Given a constraint problem* (D, ϕ) *and a normal form* (C, ψ) *then* $C = \psi D$*.*

4. *For each* $\exists\bar{\alpha}.C \in \mathcal{S}$ *there exist* $\bar{\tau}$ *and* $D \in \mathcal{S}$ *such that* $[\bar{\tau}/\bar{\alpha}]C = D \in \mathcal{S}$*.*

Remark 3 *Condition (2.) states that the set of solved forms only property is being equation–free (being satisfiable is always implicitly assumed). In case of the Hindley/Milner system, this would allow the constraint* $\exists\alpha.(\alpha = \beta)$ *(which is equation–free) being in solved form. Obviously, such constraints are redundant. Lifting still applies if the set of solved forms satisfies the syntactic uniqueness property, see Definition 24.*

The next two lemmas are the key lemmas to lift normalization procedures on projection–free constraints to the general case.

Lemma 29 (Lifting of Normal Forms) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *satisfies the lifting property. Given a constraint problem* $(\exists\bar{\alpha}.D, \phi)$*. Then* $(\exists\bar{\alpha}.D, \phi)$ *does have a normal form iff* (D, ϕ) *does have a normal form.*

Lemma 30 (Lifting of Principal Normal Forms) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *satisfies the lifting property. Given a constraint problem* $(\exists\bar{\alpha}.D, \phi)$*. Let* (C, ψ) *be the principal normal form of* (D, ϕ)*. Then* $(\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D, (\pi \circ \psi_{\setminus\bar{\alpha}}))$ *is the principal normal form of* $(\exists\bar{\alpha}.D, \phi)$ *where* $\pi = [\bar{\beta}/\bar{\alpha}]$ *and* $\bar{\beta}$ *are new.*

Lemma 29 allows a reduction of the existence check of normal forms to the base case of projection–free constraint problems. Lemma 30 allows a computation of principal normal forms based on the normalization procedure for projection–free constraints.

Theorem 10 (Lifting of Normalization) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *satisfies the lifting property. Then* X *satisfies the principal constraint property.*

The dimension system in Section 3.1 is an example of a system which satisfies the lifting property. Further examples are discussed in Section 5.1.

4.3 Inference in Term Form

I connect the principal constraint property of a constraint system with the principal types property of a type system. Figure 4.1 gives a generic type inference algorithm that computes principal types, if the constraint system satisfies the principal constraint property and some additional restrictions. [2] The algorithm is formulated as a deduction system over clauses of the form $\psi, C, \Gamma \vdash^W e : \tau$ with type environment Γ, expression e as input values and substitution ψ, constraint C, type τ as output values. For each syntactic construct of expressions e there is one clause. The deduction rules can be interpreted operationally, as a logic program that constructs a bottom–up derivation of $\vdash^W$ clauses.

In the (Var) rule, I assume that an unqualified type τ can be represented as $\forall\emptyset.\mathsf{true} \Rightarrow \tau$. This avoids a separate case of this rule for unqualified types. In rules (App) and (Let) I combine the results of the premises and build the conclusion

[2] Here, I consider inference in term form which has some problems as pointed out earlier, see Section 4.1

$$\text{(Var)}\quad \frac{x : (\forall \bar{\alpha}.D \Rightarrow \tau) \in \Gamma \qquad \bar{\beta} \text{ new} \qquad (C, \psi) = \mathit{normalize}([\bar{\beta}/\bar{\alpha}]D, [\bar{\beta}/\bar{\alpha}])}{\psi_{|fv(\Gamma)}, C, \Gamma \vdash^W x : \psi\tau}$$

$$\text{(Abs)}\quad \frac{\psi, C, \Gamma_x.x : \alpha \vdash^W e : \tau \qquad \alpha \text{ new}}{\psi_{\setminus\{\alpha\}}, C, \Gamma_x \vdash^W \lambda x.e : \psi(\alpha) \to \tau}$$

$$\text{(App)}\quad \frac{\begin{array}{c}\psi_1, C_1, \Gamma \vdash^W e_1 : \tau_1 \qquad \psi_2, C_2, \Gamma \vdash^W e_2 : \tau_2 \\ (D, \psi') = (C_1, \psi_1) \sqcup (C_2, \psi_2) \sqcup ((\tau_1 \preceq \tau_2 \to \alpha), \mathit{id}) \quad \alpha \text{ new} \\ (C, \psi) = \mathit{normalize}(D, \psi')\end{array}}{\psi_{|fv(\Gamma)}, C, \Gamma \vdash^W e_1 e_2 : \psi(\alpha)}$$

$$\text{(Let)}\quad \frac{\begin{array}{c}\psi_1, C_1, \Gamma_x \vdash^W e : \tau \qquad (C_2, \sigma) = \mathit{gen}(C_1, \psi_1\Gamma, \tau) \\ \psi_2, C_3, \Gamma_x.x : \sigma \vdash^W e' : \tau' \\ (D, \psi') = (C_2, \psi_1) \sqcup (C_3, \psi_2) \\ (C, \psi) = \mathit{normalize}(D, \psi')\end{array}}{\psi_{|fv(\Gamma_x)}, C, \Gamma_x \vdash^W \mathsf{let}\ x = e\ \mathsf{in}\ e' : \psi\tau'}$$

Figure 4.1: Type inference in term form

if normalization succeeds. All three rules make use of the function *normalize*, specified in the last subsection. The deduction rules yield an algorithm only if *normalize* is computable. In the following, I assume that the normalization functions is computable.

The type inference algorithm $\vdash^W$ is a straightforward extension of algorithm W, see [DM82]. The algorithm $\vdash^W$ consists of the following three basic components: constraint generation, constraint normalization and generalization of unbound type variables. All three components can already be found in the original algorithm W but are now extended to deal with constraints. I already discussed constraint generation and normalization. In Section 3.4, I introduced a generalization relation. Given constraints $C, C_o \in \mathcal{S}$, a type environment Γ and types σ, σ_o. Then $C, \Gamma, \sigma \vdash^{gen}$

(C_o, σ_o) iff $C_o =^e \exists\bar{\alpha}.C =^e C_1 \wedge \exists\bar{\alpha}.C_2$, $\sigma_o = \forall\bar{\alpha}.C_2 \Rightarrow \sigma$, $\alpha \notin fv(C_1) \cup fv(\Gamma)$ for some constraints C_1, C_2.

I make two observations. First, the set $\bar{\alpha}$ of quantified type variables is only a subset of the free type variables in $(fv(\sigma) \cup fv(C))\backslash fv(\Gamma)$. Remember, the set $\mathcal{S}$ of solved forms is not necessarily closed under projection (in most cases the set $\mathcal{S}$ will be closed under projection). Therefore, I have to ensure that the constraint $\exists\bar{\alpha}.C$ is in solved form. I only require that the set $\bar{\alpha}$ is maximal such that $\exists\bar{\alpha}.C$ is still in solved form. In general, I do not know how to determine the maximal set of variables $\bar{\alpha}$, nor whether there exist a least upper bound between all maximal sets of variables $\bar{\alpha}$. Hence, $\vdash^W$ yields an algorithm only if this property is decidable. Second, generalization splits a constraint into two parts. Generalized variables can be free only in one of the two parts, C_1, but not the other, C_2. Only the C_2 part ends up as a constraint in the generalized type scheme. Note that the above requirement can always be fulfilled by taking C_1 to be true. However, depending on the actual constraint system used, there might exist better strategies, which keep the constraint in the generalized type scheme smaller.

The type inference algorithm interleaves constraint generation and normalization. Each inference rule combines the constraint problems of the premises and performs then a normalization step. That means I perform *eager* normalization during type inference. In essence, I only need to perform normalization right before a (Let) rule (because the constraint in a type scheme needs to be in normal form) or at the end. This corresponds to *by need* normalization. An example of a by need formulation of type inference for the Hindley/Milner type system can already be found in [Wan87]. Figure 4.2 contains a by need formulation of the type inference algorithm in HM(X). If the constraint X enjoys the principal constraint property and unitary unification then both formulations are equivalent. This is justified by Theorem 9 in Section 4.2.

$$\text{(Var)} \qquad \frac{x : (\forall \bar{\alpha}.D \Rightarrow \tau) \in \Gamma \qquad \bar{\beta} \text{ new}}{[\bar{\beta}/\bar{\alpha}], D, \Gamma \vdash^{W'} x : \tau}$$

$$\text{(Abs)} \qquad \frac{\psi, C, \Gamma_x.x : \alpha \vdash^{W'} e : \tau \qquad \alpha \text{ new}}{\psi, C, \Gamma_x \vdash^{W'} \lambda x.e : \alpha \to \tau}$$

$$\text{(App)} \qquad \frac{\psi_1, C_1, \Gamma \vdash^{W'} e_1 : \tau_1 \qquad \psi_2, C_2, \Gamma \vdash^{W'} e_2 : \tau_2 \qquad \alpha \text{ new}}{\psi_1 \sqcup \psi_2, C_1 \wedge C_2 \wedge (\tau_1 \preceq \tau_2 \to \alpha), \Gamma \vdash^{W'} e_1 e_2 : \alpha}$$

$$\text{(Let)} \qquad \frac{\begin{array}{c}\psi_1, C_1, \Gamma_x \vdash^{W_{byneed}} e : \tau \qquad (C_2, \sigma) = gen(C_1, \psi_1 \Gamma_x, \tau) \\ \psi_2, C_3, \Gamma_x.x : \sigma \vdash^{W'} e' : \tau'\end{array}}{\psi_1 \sqcup \psi_2, C_2 \wedge C_3, \Gamma_x \vdash^{W'} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \psi'\tau'}$$

$$\text{(Normalize)} \qquad \frac{\phi, C, \Gamma \vdash^{W'} e : \tau \qquad (D, \psi) = normalize(C, \phi)}{\psi_{|fv(\Gamma)}, D, \Gamma \vdash^{W_{byneed}} e : \psi\tau}$$

Figure 4.2: By need type inference in term form

Theorem 11 (Eager versus by Need Inference) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *enjoys the principal constraint property and unitary unification. Then* $\psi, C, \Gamma \vdash^{W} e : \tau$ *iff* $\psi, C, \Gamma \vdash^{W_{byneed}} e : \tau$.

4.3.1 Soundness and Completeness Results

The result triple of the type inference algorithm $\vdash^{W}$ forms a *typing configuration* (C, σ, ψ), which consists of a constraint $C \in \mathcal{S}$, a type scheme σ and a substitution ψ such that $\psi C = C$, $\psi\sigma = \sigma$ and ψ is consistent with respect to Γ. Recall, a substitution ϕ is consistent with respect to a type scheme $\sigma = \forall \bar{\alpha}.D \Rightarrow \tau$ if $\psi D \in \mathcal{S}$ where I assume there are no name clashes between $\bar{\alpha}$ and ψ. This extends naturally to type environments. Given two typing configurations (C, σ, ψ) and (C', σ', ψ'), I

say (C, σ, ψ) is *more general* than (C', σ', ψ') with respect to Γ iff there exists a ϕ' such that $\psi \leq^{\phi'}_{fv(\Gamma)} \psi$, $C' \vdash^e \phi' C$ and $C' \vdash^i \phi'\sigma \preceq \sigma'$. In such a situation I write $(C, \sigma, \psi) \leq_\Gamma (C', \sigma', \psi)$.

Lemma 31 (Typing Configuration) *Given a type environment Γ and a term e. If $\psi, C, \Gamma \vdash^W e : \tau$ then (C, τ, ψ) is a typing configuration.*

Furthermore, this typing configuration always represents a valid typing of the given term under the given type environment.

Theorem 12 (Soundness of Inference) *Given a term e and a type environment Γ. If $\psi, C, \Gamma \vdash^W e : \tau$ then $C, \psi\Gamma \vdash e : \tau$, $\psi C = C$ and $\psi\tau = \tau$.*

To achieve complete inference, I require that an HM(X) type system has to fulfill the principal constraint property. But Kennedy's example shows that inference is not complete even if the constraint system possesses most general normalizers. To achieve complete inference in term form, I have to restrict the completeness results to regular theories. Recall, an equational theory is regular if $\vdash^e (\tau = \tau')$ implies $fv(\tau) = fv(\tau')$. I say a constraint system X is regular if the underlying equational theory is regular. An example of a non–regular theory is the dimension constraint system DIM.

Type inference in term form also includes inference in constraint form (more about this in the next section) as a special case if normalization is trivial, i.e. normalization represents only a satisfiability test. The following completeness results holds for two classes of constraint systems. By $\mathcal{X}^c$ I denote the class of constraint systems where $\mathcal{S}$ equals to the set of all satisfiable constraints, hence normalization only involves a satisfiability test. For the second class, denoted by $\mathcal{X}^{reg}$, I consider only regular constraint systems.

To obtain a completeness result for type inference, I assume that there is an HM(X) type system where X either belongs to $\mathcal{X}^c$ or $\mathcal{X}^{reg}$. In both cases the principal constraint property must hold. In case X belongs to $\mathcal{X}^{reg}$, X must in addition enjoy unitary unification. This is a technical assumption and is usually fulfilled if the principal constraint property holds.

Furthermore, I consider only those typing judgments $C, \Gamma \vdash e : \sigma$ where the type environment and the constraint on the left hand side of the turnstile are realizable, i.e. have a type instance. Recall, a type environment Γ is realizable in a constraint C if for every $x : \sigma \in \Gamma$ there is a τ such that $C \vdash^i \sigma \preceq \tau$.

Another technical assumption is that the set $\mathcal{S}$ of solved forms is closed under projection. In case of the (Let) rule, I first type the definition and then the body of the let–statement. Therefore, projection becomes essential. As stated in Theorem 6 in Section 3.5.2 whether the set $\mathcal{S}$ is projection–closed or not, does not change the set of typable programs. Hence, w.l.o.g. I can assume that the set $\mathcal{S}$ is closed under projection.

Theorem 13 (Completeness of Inference) *Let $C', \phi\Gamma \vdash e : \sigma'$ be a typing judgment such that $\phi\Gamma$ is realizable in C'. Then*

$$\psi, C, \Gamma \vdash^W e : \tau$$

for some substitution ψ, constraints C, C_o, types τ, σ_o, such that

$$\begin{array}{rcl} C, \psi\Gamma, \tau & \vdash^{gen} & (C_o, \sigma_o) \\ (C_o, \sigma_o, \psi) & \leq_\Gamma & (C', \sigma', \phi) \end{array}$$

4.3.2 Principal Types

The completeness theorem can be simplified for top–level programs to the following corollary, which states that type inference yields principal types.

Definition 29 (Principal types) *Given an instance $\mathcal{X}$ of the* HM(X) *type system. $\mathcal{X}$ satisfies the* principal type property *iff the following holds. Given a pair (Γ, e) such that $C', \Gamma \vdash e : \sigma'$ for some constraint C' and type scheme σ'. Then there is a constraint C and a type scheme σ such that*

- $C, \Gamma \vdash e : \sigma$
- *If $C', \Gamma \vdash e : \sigma'$ then $C' \vdash^e C$ and $C' \vdash^i \sigma \preceq \sigma'$.*

The pair (C, σ) is called a principal type. *Commonly, the constraint component equals* true.

Corollary 5 (Principal Types) *Let $\mathsf{true}, \Gamma \vdash e : \sigma$ be a closed typing judgment such that Γ is realizable in* true. *Then $\phi, C, \Gamma \vdash^W e : \tau$ for some substitution ϕ, constraint C, types τ, σ_o such that*

$$\begin{array}{ccc} C, \phi\Gamma, \tau & \vdash^{gen} & (\mathsf{true}, \sigma_o) \\ \vdash^i \sigma_o & \preceq & \sigma \end{array}$$

Please note, this is a straightforward extension of the notion of principal types found in the Hindley/Milner system. In this system, principality states there is one unique type (a principal type) from which all other types can be generated (by building generic instances). In contrast, a principal type in HM(X) represents rather a class of types which are semantically equivalent. As an example, consider the program $\lambda x.x$ in a variation of the Hindley/Milner system where the set of solved forms equals all satisfiable constraints. One obvious principal type would be

$\sigma_1 = \forall\alpha.\alpha \to \alpha$. But the type $\sigma_2 = \forall\alpha,\beta.(\alpha = \beta) \Rightarrow \alpha \to \beta$ would be another possible candidate. It holds that $\mathsf{true}, \emptyset \vdash \lambda x.x : \sigma_1$ and $\mathsf{true}, \emptyset \vdash \lambda x.x : \sigma_2$ and $\vdash^i \sigma_1 \preceq \sigma_2$ but also $\vdash^i \sigma_2 \preceq \sigma_1$. This shows there is no unique principal type in a variation of the Hindley/Milner type system where the set of solved forms equals all satisfiable constraints.

I observe that uniqueness of principal types is usually lost in cases where set of solved forms equals all satisfiable constraints. This is the approach usually taken in subtyping systems. In subtyping systems, authors [AW93, TS96, Pot98] rather prefer the term *most general type* than principal type. Nevertheless, I use the term principal type bearing in mind that no unique principal type might exist. Unique principal types possibly exist in case the set of solved form is further restricted, e.g. equation–free. A unique principal type can be computed if principal normal forms are syntactically unique, see Lemma 24 in Section 4.2.

4.4 Inference in Constraint Form

This section provides a inference system for the HM(X) system in constraint form. To achieve complete inference, I only need a satisfiability check. Furthermore, I present an adapted version of the inference system in term form which solves the problem with non–regular theories.

4.4.1 Constraint Inference

The inference system, see Figure 4.3, is formulated as a deduction system with clauses of the form $C, \Gamma \vdash^{Wc} e : \alpha$ where Γ and e are input values and C and α are output values. Please note, the satisfiability check can be performed at any time because all inference rules preserve satisfiability. Satisfiability must be decidable to obtain a

$$\text{(Var)} \qquad \frac{x : (\forall \bar{\alpha}.D \Rightarrow \tau) \in \Gamma \qquad \beta \text{ new}}{\exists \bar{\alpha}.((\beta = \tau) \wedge D), \Gamma \vdash^{Wc} x : \beta}$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x.x : \alpha \vdash^{Wc} e : \gamma \qquad \beta \text{ new}}{\exists \alpha, \gamma.((\beta = \alpha \to \gamma) \wedge C), \Gamma_x \vdash^{Wc} \lambda x.e : \beta}$$

$$\text{(App)} \qquad \frac{C_1, \Gamma \vdash^{Wc} e_1 : \alpha_1 \qquad C_2, \Gamma \vdash^{Wc} e_2 : \alpha_2 \qquad \alpha \text{ new}}{\exists \alpha_1, \alpha_2.(C_1 \wedge C_2 \wedge (\alpha_1 \preceq \alpha_2 \to \alpha)), \Gamma \vdash^{Wc} e_1 e_2 : \alpha}$$

$$\text{(Let)} \qquad \frac{C_1, \Gamma_x \vdash^{Wc} e : \alpha \qquad C_3, \Gamma_x.x : \forall \alpha.C_1 \Rightarrow \alpha \vdash^{Wc} e' : \beta}{(\exists \alpha.C_1) \wedge C_3, \Gamma_x \vdash^{Wc} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \beta}$$

$$\text{(Equ)} \qquad \frac{C, \Gamma \vdash^{Wc} e : \alpha \qquad C =^{e} D}{D, \Gamma \vdash^{Wc} e : \alpha}$$

Figure 4.3: Type inference in constraint form

decidable inference algorithm.

Judgments in the inference system are kept in normalized form, only type variables appear on the right–hand side of $\vdash^{Wc}$. Unnecessary type variables are hidden by the projection operator (this corresponds to the ($\exists$ Intro) rule in the logical system). It holds the invariant, given $C, \Gamma \vdash^{Wc} e : \alpha$ then $fv(C) \subseteq fv(\Gamma, \alpha)$ and $\alpha \notin fv(\Gamma)$. This simplifies generalization in case of the (Let) rule. I immediately can build the type scheme $\forall \alpha.C \Rightarrow \alpha$. However, depending on the actual constraint system, there might be a way to split constraint C_1 into two parts D_1, D_2 such that $C_1 =^{e} D_1 \wedge D_2$ and $\alpha \notin fv(D_2)$. Then, only D_1 needs to be moved into the type scheme. But whether or not such a splitting is possible depends on the specific constraint system.

Rule (Equ) allows equivalent transformations of constraints in case one prefers certain canonical/minimal representation of constraints. This rule can be applied at

any time, e.g. after each inference step or only at the end. I note that constraint simplification is especially important in case of the (Let) rule. The careful reader might notice that keeping types in constraint form might lead to a blow–up. Assume I have the let–bound variable x with type $\forall\bar{\alpha}.C \Rightarrow \tau$. Applying x to different monomorphic arguments results [3] into the two constraints $\exists\bar{\alpha}.((\beta = \tau) \wedge C)$ and $\exists\bar{\alpha}.((\beta' = \tau) \wedge C)$. I assume further that the constraint C contains a subpart which is independent of the free variables $\bar{\alpha}$, i.e. $C = C_1 \wedge C_2$ where $\bar{\alpha} \cap fv(C_1) = \emptyset$. Then, the two constraints above are equivalent to

$$C_1 \wedge (\exists\bar{\alpha}.((\beta = \tau) \wedge C_2)) \wedge (\exists\bar{\alpha}.((\beta' = \tau) \wedge C_2))$$

Hence, not the whole constraint C gets duplicated. In general, there is a blow–up of subsumption constraints which leads to exponential complexity of the inference problem. This topic is well–studied for the special case of the standard Hindley/Milner system, see [KMM91]. Constraint simplification is also important before generating type schemes. Simplification in general fits naturally into the HM(X) framework. The set $\mathcal{S}$ of solved forms equals all satisfiable constraints in the constraint–based approach. Further restrictions of the set $\mathcal{S}$ leads to a term–based approach which can be seen as a simplified representation of constraints. Another form of simplification would be to enforce the set of solved forms to be in syntactically unique form (see Definition 24 in Section 4.2). Simplification strategies in the context of subtyping were studied in [Pot98, EST95a].

I can state the following results. I omit to give proofs of the soundness and completeness results. Both proofs are almost identical to the soundness and completeness results of the class $\mathcal{X}^c$ in case of inference in term form.

[3] Before I apply x I generate a generic instance of x, see the (Var) rule.

Theorem 14 (Soundness of Inference) *Given a term e and a type environment Γ. If $C, \Gamma \vdash^{W_c} e : \alpha$ then $C, \Gamma \vdash e : \alpha$.*

For the completeness result, I only consider those typing judgments $C, \Gamma \vdash e : \sigma$ where the type environment and the constraint on the left hand side of the turnstile are realizable, i.e. have a type instance. Recall, a type environment Γ is realizable in a constraint C if for every $x : \sigma \in \Gamma$ there is a τ such that $C \vdash^{i} \sigma \preceq \tau$.

Theorem 15 (Completeness of Inference) *Given a typing judgment $C, \Gamma \vdash e : \sigma$ such that Γ is realizable in C. Then $C', \Gamma \vdash^{W_c} e : \alpha$ such that*

$$C \vdash^{e} \exists\alpha.C'$$
$$C \vdash^{i} \forall\alpha.C' \Rightarrow \alpha \preceq \sigma.$$

I need the realizability condition to state the property about the constraint. Otherwise, I could not state that $C \vdash^{e} \exists\alpha.C'$.

The completeness result could be stated in a much more concise way than in the term–based HM(X) system. To achieve principal types I only need satisfiability. A decidable satisfiability test results into decidable type inference.

Corollary 6 (Principal Types) *Let $\mathsf{true}, \Gamma \vdash e : \sigma$ be a closed typing judgment such that Γ is realizable in* true*. Then $C, \Gamma \vdash^{W_c} e : \alpha$ such that*

$$\vdash^{e} \exists\alpha.C$$
$$\vdash^{i} \forall\alpha.C \Rightarrow \alpha \preceq \sigma.$$

4.4.2 Term Inference

I turn my attention to the inference problem in the term–based system as described in Section 3.2. Kennedy's example shows that the standard inference algorithm does

not infer principal types, even in the presence of unitary unification. The standard inference algorithm performs unification eagerly, after each step. This can be a problem in case of non–regular equational theories. An equational theory is *non–regular* if there exists types τ, τ' such that $(\tau = \tau')$ and $fv(\tau) \neq fv(\tau')$. For instance, dimension types form a non–regular theory. It holds that $(prod(\alpha, i(\alpha)) = 1)$ but $fv(1) = \emptyset$. In Section 4.3, I could already show that regular theories always have principal types under the assumption that the constraint system satisfies the principal constraint property (which corresponds to unitary unification in case of an equational theory). A similar result in the special case of regular equational theories was also shown by Rémy [Rém92a].

Kennedy could give an adapted version of his inference algorithm which infers principal types for non–regular theories. But he can only state principal types under some additional conditions. I provide more details in the next section where I also discuss another alternative approach given by Valery Trifonov [TS96].

In my dissertation, I present a revised term–based inference system which always infers principal types if the constraint system satisfies the principal constraint property. The revised inference system can be found in Figure 4.4. It consists of two parts. The purely constraint–based inference system introduced in Section 4.4.1 and a term reconstruction step. In the standard formulation of the inference system, term reconstruction is done in each step. In Section 4.3, I observed that term reconstruction only needs to be done right before a let–rule. Because then the constraint is moved in the type scheme, and constraints in the logical system have to be in solved form. But Kennedy's example shows that in the presence of non–regular theories inference in term form is not necessarily complete. In the revised inference formulation, term reconstruction is postponed to the very end. Then complete inference can be retained even for non–regular theories. The difficulty is now to show that this

$$\text{(Var)} \qquad \frac{x : (\forall \bar{\alpha}.D \Rightarrow \tau) \in \Gamma \qquad \beta \text{ new}}{\exists \bar{\alpha}.((\beta = \tau) \wedge D), \Gamma \vdash^{Wc} x : \beta}$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x.x : \alpha \vdash^{Wc} e : \gamma \qquad \beta \text{ new}}{\exists \alpha, \gamma.((\beta = \alpha \to \gamma) \wedge C), \Gamma_x \vdash^{Wc} \lambda x.e : \beta}$$

$$\text{(App)} \qquad \frac{C_1, \Gamma \vdash^{Wc} e_1 : \alpha_1 \qquad C_2, \Gamma \vdash^{Wc} e_2 : \alpha_2 \qquad \alpha \text{ new}}{\exists \alpha_1, \alpha_2.(C_1 \wedge C_2 \wedge (\alpha_1 \preceq \alpha_2 \to \alpha)), \Gamma \vdash^{Wc} e_1 e_2 : \alpha}$$

$$\text{(Let)} \qquad \frac{C_1, \Gamma_x \vdash^{Wc} e : \alpha \qquad C_3, \Gamma_x.x : \forall \alpha.C_1 \Rightarrow \alpha \vdash^{Wc} e' : \beta}{(\exists \alpha.C_1) \wedge C_3, \Gamma_x \vdash^{Wc} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \beta}$$

$$\text{(Equ)} \qquad \frac{C, \Gamma \vdash^{Wc} e : \alpha \qquad C =^e D}{D, \Gamma \vdash^{Wc} e : \alpha}$$

$$\text{(Term reconstruction)} \qquad \frac{C, \Gamma \vdash^{Wc} e : \alpha \qquad (D, \psi) = \mathit{normalize}(C, \mathit{id})}{\psi_{|fv(\Gamma)}, D, \Gamma \vdash^{W_{term}} e : \psi(\alpha)}$$

Figure 4.4: Revised type inference in term form

yields a sound inference system, see the proofs of the soundness and completeness results.

There is also the (Equ) rule which allows to transform constraints into a semantically equivalent but better suitable representation.

In the following two theorems, I assume that HM(X) instances are given as described in Section 3.2.

Theorem 16 (Soundness of Inference) *Given an instance of the* HM(X) *system where* X *satisfies the principal constraint property and the set* $\mathcal{S}$ *is closed under projection. Given a type environment* Γ *and a term* e*. If* $\psi, C, \Gamma \vdash^{W_{term}} e : \tau$ *then* $C, \psi\Gamma \vdash e : \tau$ *where* $\psi C =^e C$ *and* $\psi\tau = \tau$*.*

Please note, compared to the original soundness result (see Theorem 12) I need stronger assumptions; the principal constraint property and $\mathcal{S}$ is closed under projection. These assumptions are necessary to apply the Transformation Theorem 7 which transforms judgments in constraint form to judgments in term form.

The following completeness results extends the previous completeness result for term–based HM(X) systems. Completeness can be achieved under the assumption that the constraint system satisfies the principal constraint property. I do not need to restrict the result to regular theories or require additional conditions as done in [Ken96].

Theorem 17 (Completeness of Inference) *Given an* HM(X) *instance where* X *satisfies the principal constraint property. Given* $C, \phi\Gamma \vdash e : \sigma$ *such that* $\phi\Gamma$ *is realizable in* C. *Then* $\psi, D, \Gamma \vdash^{W_{term}} e : \tau$ *for some substitution* ψ, *constraints* D, C_o *and types* τ, σ_o *such that*

$$D, \psi\Gamma, \tau \vdash^{gen} (\sigma_o, C_o)$$
$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \qquad C \vdash^{i} \phi'\sigma_o \preceq \sigma \qquad C \vdash^{e} \phi' C_o.$$

To achieve complete inference, I need less assumptions compared to the completeness result for the inference system $\vdash^{W}$ (see Theorem 13). Unitary unification is not necessary because during the actual inference process no substitutions are computed.

I can furthermore simplify the completeness results to top–level programs which states that the inference system computes principal types.

Corollary 7 (Principal types) *Given an* HM(X) *instance where* X *satisfies the principal constraint property. Given a closed typing judgment* $\mathsf{true}, \Gamma \vdash e : \sigma$ *where* Γ *is realizable in* true. *Then* $\psi, D, \Gamma \vdash^{W_{term}} e : \tau$ *for some substitution* ψ, *constraint*

D and types τ, σ_o such that

$$D, \psi\Gamma, \tau \vdash^{gen} (\sigma_o, \mathsf{true})$$
$$\vdash^{i} \sigma_o \preceq \sigma.$$

The basic trick I apply lies in postponing the term reconstruction step to the very end. In intermediate steps I always keep inference judgments in normalized form $C, \Gamma \vdash^{Wc} e : \alpha$. Then generalization is unproblematic. Term reconstruction is only performed at the very end. This achieves sound and complete type inference. The drawback is that constraints in intermediate steps might become big. In case of the dimension type system, equality constraints are not resolved via unification but only accumulated and checked whether they are satisfiable or not. Fortunately, for a large class of systems it is even possible to perform normalization in intermediate steps while retaining complete inference.

I consider a variation of the inference system $\vdash^{W_{term}}$ where normalization is allowed to be performed in intermediate steps. Rule (Normalize) is added to the inference system $\vdash^{W_{term}}$.

$$\text{(Normalize)} \quad \frac{C, \Gamma \vdash^{Wc} e : \alpha \qquad (D, \phi) = \mathit{normalize}(C, \mathit{id})}{D \wedge \phi, \Gamma \vdash^{Wc} e : \alpha}$$

The idea is to represent normal forms in constraint form. The normalization procedure states that $D \vdash^{e} \phi C$. It follows that $D \wedge \phi \vdash^{e} C$ where the substitution $\phi = [\bar{\tau}/\bar{\alpha}]$ is considered as the constraint $(\alpha_1 = \tau_1) \wedge \ldots \wedge (\alpha_n = \tau_n)$. Please note, given $C, \Gamma \vdash^{Wc} e : \alpha$ the invariant $\mathit{fv}(C) \subseteq \mathit{fv}(\Gamma, \alpha)$ and $\alpha \notin \mathit{fv}(\Gamma)$ still holds. The substitution ϕ in the (Normalize) rule operates only on the free variables $\mathit{fv}(\Gamma, \alpha)$.

In inference system $\vdash^{W_{normalize}}$ (see Figure 4.5) the (Normalize) rule is applied

$$\text{(Var)} \qquad \frac{x : (\forall \bar{\alpha}.D \Rightarrow \tau) \in \Gamma \qquad \beta \text{ new}}{\exists \bar{\alpha}.((\beta = \tau) \wedge D), \Gamma \vdash^{W_{c2}} x : \beta}$$

$$\text{(Abs)} \qquad \frac{C, \Gamma_x.x : \alpha \vdash^{W_c} e : \gamma \qquad \beta \text{ new}}{\exists \alpha, \gamma.((\beta = \alpha \to \gamma) \wedge C), \Gamma_x \vdash^{W_{c2}} \lambda x.e : \beta}$$

$$\text{(App)} \qquad \frac{C_1, \Gamma \vdash^{W_c} e_1 : \alpha_1 \qquad C_2, \Gamma \vdash^{W_c} e_2 : \alpha_2 \qquad \alpha \text{ new}}{\exists \alpha_1, \alpha_2.(C_1 \wedge C_2 \wedge (\alpha_1 \preceq \alpha_2 \to \alpha)), \Gamma \vdash^{W_{c2}} e_1 e_2 : \alpha}$$

$$\text{(Let)} \qquad \frac{C_1, \Gamma_x \vdash^{W_c} e : \alpha \qquad C_3, \Gamma_x.x : \forall \alpha.C_1 \Rightarrow \alpha \vdash^{W_c} e' : \beta}{(\exists \alpha.C_1) \wedge C_3, \Gamma_x \vdash^{W_{c2}} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \beta}$$

$$\text{(Normalize)} \qquad \frac{C, \Gamma \vdash^{W_{c2}} e : \alpha \qquad (D, \phi) = \mathit{normalize}(C, \mathit{id})}{D \wedge \phi, \Gamma \vdash^{W_c} e : \alpha}$$

$$\text{(Term reconstruction)} \qquad \frac{C, \Gamma \vdash^{W_c} e : \alpha \qquad (D, \psi) = \mathit{normalize}(C, \mathit{id})}{\psi_{|fv(\Gamma)}, D, \Gamma \vdash^{W_{\mathit{normalize}}} e : \psi(\alpha)}$$

Figure 4.5: Revised type inference in term form II

after each step. The addition of rule (Normalize) obviously retains soundness. If the constraint system satisfies the lifting property (see Definition 28 in Section 4.2.1), inference systems $\vdash^{W_{term}}$ and $\vdash^{W_{normalize}}$ are equivalent.

Theorem 18 (Equivalence of $\vdash^{W_{term}}$ and $\vdash^{W_{normalize}}$) *Given an* HM(X) *instance where* X *satisfies the lifting property. Then* $\phi, D, \Gamma \vdash^{W_{term}} e : \tau$ *iff* $\phi, D, \Gamma \vdash^{W_{normalize}} e : \tau$.

Inference system $\vdash^{W_{normalize}}$ can be seen as a constraint–based formulation of inference system $\vdash^{W}$. Specifically, it follows that $\vdash^{W_{normalize}}$ and $\vdash^{W}$ have the same complexity.

The dimension system DIM satisfies the lifting property. Hence, principal types

can be inferred by the inference system $\vdash^{W_{normalize}}$ as efficient as in the term–based inference system $\vdash^{W}$. In addition, I note that the lifting property holds usually for systems following a unification–based approach towards normalization, hence covering a large class of systems.

4.4.3 Discussion

In [Ken96], Kennedy discovered the problem of performing type inference in non–regular theories. The central problem seems that in non–regular theories there might be some implicitly free type variables. Such type variables need to be revealed when generalizing over free type variables before applying the let–rule. For instance, consider the motivating example at the beginning of this chapter. In it, I applied a substitution to the whole judgment which revealed an implicitly free type variable, leaving the type environment unchanged. The question is whether such a treatment is always possible. Kennedy gave a sufficient axiomatization under which such implicitly free type variables can always be revealed. But it is not known yet whether there is a decidable algorithm which satisfies the axiomatization. For some specific theories he could give algorithms which satisfy the axiomatization. If additional unification is unitary then a principal types theorem holds. In [Ken96], Kennedy posed the question whether these extra assumptions are necessary. My revised term–based inference algorithm always infers principal types giving unitary unification. There are no further additional conditions necessary. Moreover, inference systems $\vdash^{W}$ and $\vdash^{W_{normalize}}$ have the same complexity.

Valery Trifonov [TS96] introduces what he calls λ–lifting which also seems to overcome Kennedy's problem. He writes judgments in the form

$$\Gamma' \vdash e : \forall\bar{\alpha}.\Gamma \Rightarrow \tau \,|\, C$$

The type environment Γ' contains only let–bound variables whereas all other free variables are contained in Γ. Employing his notation would allow to type function y in the example at the beginning of this chapter as follows:

$$\emptyset \vdash y : \forall d_1, d_2.\Gamma.x : \dim(prod(d_1, d_2)) \Rightarrow \dim(d_1) \rightarrow \dim(d_2)$$

Now, y has the desired polymorphic type. His notation is quite different from the standard notation. For details, see [TS96]. The λ–lifting notation was also employed by Francois Pottier [Pot98]. Both used λ–lifting in the context of subtyping where normalization corresponds only to a satisfiability test. The λ–lifting notation allows for certain simplification strategies which would not be able to perform with the standard notation. But there seems to be no gain of the λ–lifting notation in case of unification–based approaches.

4.5 Summary

I sum up the results of this section. The following two figures reflect the structure of the soundness/completeness proofs. Figure 4.6 presents the results of the term–based inference system and shows the connections between the intermediate derivations to achieve the completeness result. Lemma 37 can be found in the appendix (see Proofs of Section 4.3). The proof follows the standard technique to establish the completeness result, though a proof in such a generality has not been found in the literature yet. Figure 4.7 presents the results of the constraint–based inference system and revised inference system in term form. I apply a new proof technique. The straightforward completeness result of the constraint–based system is lifted to the term–based system by application of the Transformation Theorem. In-

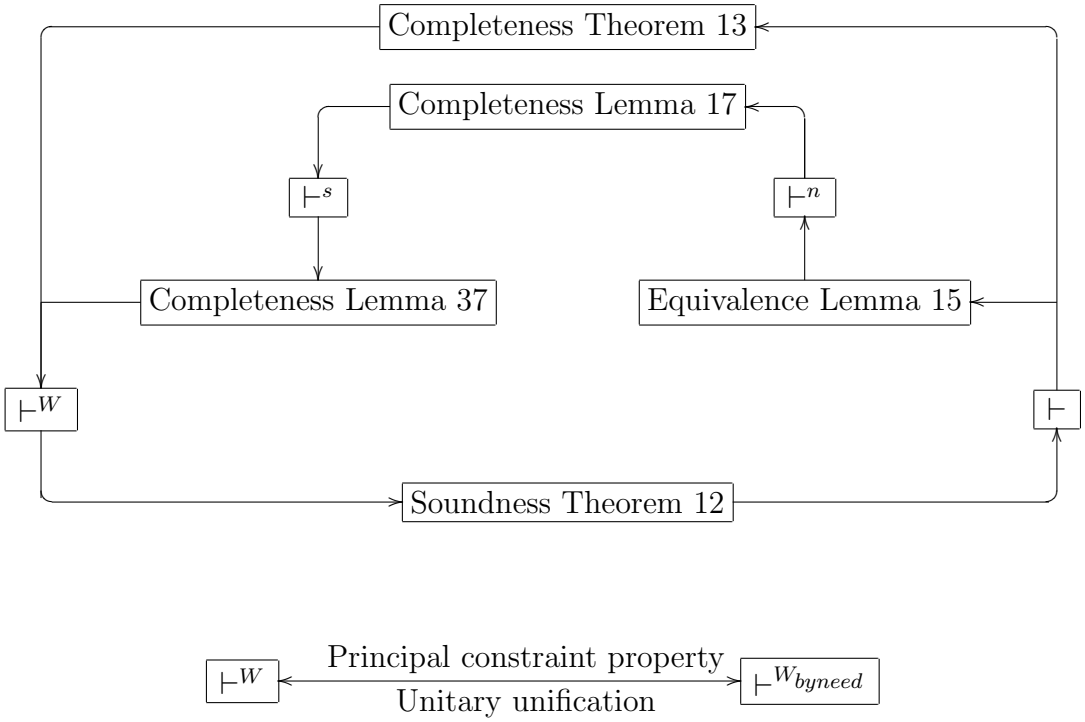


Figure 4.6: Type inference in term form

---

ference systems $\vdash^{W_{term}}$ and $\vdash^{W_{normalize}}$ are equivalent if the lifting property holds. The inference system $\vdash^{W_{normalize}}$ is a constraint–based formulation of the term–based inference system $\vdash^{W_{term}}$. Furthermore, the results of Section 4.2.1 (lifting of normalization) allow to reuse a wide range of already existing constraint solvers to perform type inference in HM(X).

In summary, type inference consists of the following three phases:

1. Constraint solving in its simplest form only involves a satisfiability test. This ensures correctness of type inference.

2. Constraint simplification is relevant to ensure efficient type inference.

3. Term reconstruction is relevant for user interaction.

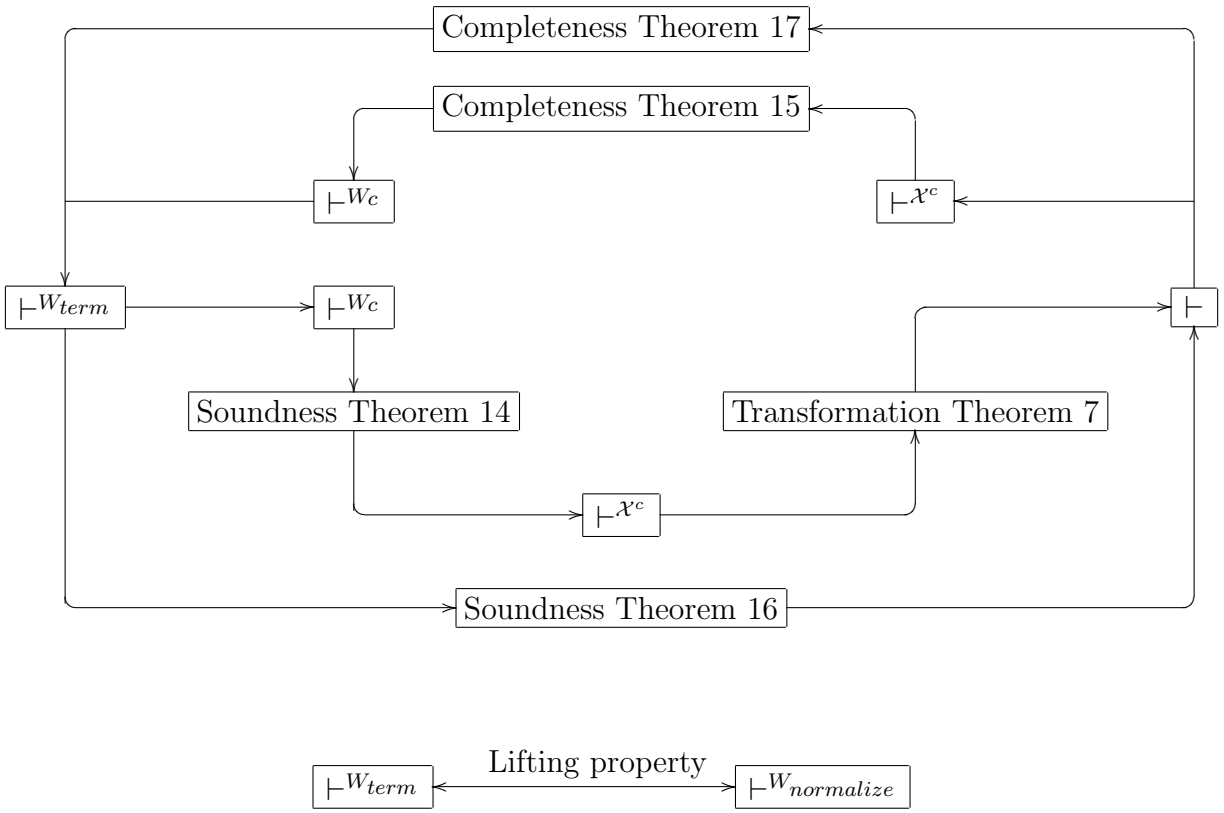


Figure 4.7: Type inference in constraint form

The above classification evolved out of fruitful and inspiring discussions with Martin Müller. Note that constraint solving might involve more sophisticated methods such as unification. Hence, we sometimes find a mix of the above described inference phases.

Chapter 5

Applications

This chapter of my dissertation considers applications of the HM(X) framework. In the first part, I consider Ohori's record calculus [Oho95] as an instance of the HM(X) system. The corresponding type system instance of HM(X) models Ohori's calculus models fully and faithfully, i.e. exactly the same set of programs are typable in both record calculi. Furthermore, I describe some useful extensions of Ohori's calculus, such as extensible records, record concatenation, overloaded field labels and a calculus where field labels become first–class values. The record calculus with first–class labels turns out to be especially interesting and very expressive. This calculus provides the basis to encode other features such as type case.

The second part considers extensions of Odersky/Wadler/Wehr's overloading system [OWW95]. The development is similar to the previous part. I consider extensions such as overloaded curried functions and overloaded identifiers as first–class values.

5.1 Ohori Records

Following ideas of Ohori [Oho95], I give an instance of the HM(X) system which deals with polymorphic records. Ohori's system, abbreviated *Ohori* in the following, has besides type variables and function types also record types denoted by $\{l_1 : \tau_1, \ldots, l_n : \tau_n\}$, where l_i is an element of an enumerable set of record labels. I assume that there is an ordering relation between all field labels. All record fields are ordered with respect to this ordering relation. Because there is a fixed ordering of record fields, Herbrand unification applies to resolve equality constraints between records, as shown by Ohori in [Oho95].

Type quantification in *Ohori* is kinded; in the type scheme $\forall\alpha.\alpha :: \kappa \Rightarrow \sigma$ the type variable α ranges only over kind κ. A kind is of the form $\{\!\{l_1 : \tau_1, \ldots, l_n : \tau_n\}\!\}$; it comprises all records that contain at least fields $l_1, \ldots, l_n$ with types $\tau_1, \ldots, \tau_n$.

Instead of a constraint on the left hand side of a typing judgment, Ohori uses a *kind assignment* $\mathcal{K}$ which can be considered as a function that assigns each type variable α its kind k. He writes $\mathcal{K} \wedge (\alpha :: k)$ for the disjoint extension of $\mathcal{K}$ with a new type variable α with kind k.

Here is an example of a program typed in *Ohori*.

Example 19

```
f: ∀α, β.(α :: {{l : β}}) ⇒ α → Int
f x =
    let g: β → Bool
        g = λ y. eq y (x.l)
    in 1
```

I use a Haskell-style notation, with type scheme annotations added for illustration purposes. The program assumes that there is a function

$$\text{eq} : \forall\alpha.\alpha \to \alpha \to \mathsf{Bool}$$

in the initial type environment.

Type System

I now translate *Ohori* into the HM(X) framework. I add to the initial type environment Γ_0 primitive constructs that deal with record formation, selection and update. I assume for every ordered sequence of record labels $l_1, \ldots, l_n$ there exists an n–ary parameterized data type $R_{l_1,\ldots,l_n}$. The record type $\{l_1 : \alpha_1, \ldots, l_n : \alpha_n\}$ is then represented as $R_{l_1,\ldots,l_n}\tau_1 \ldots \tau_n$. For simplicity, I will keep the record type notation as a synonym for the data type notation. The initial environment contains data type constructors

$$l_1_\ldots_l_n : \alpha_1 \to \ldots \alpha_n \to \{l_1 : \alpha_1, \ldots, l_n : \alpha_n\}$$

Then, $l_1_\ldots_l_n\, e_1 \ldots e_n$ represents record formation $\{l_1 = e_1, \ldots, l_n = e_n\}$. For each field label l, I add to the initial type environment Γ_0 the two functions

$$(_.l) : \forall\alpha, \beta.(\alpha :: \{\!\{l : \beta\}\!\}) \Rightarrow \alpha \to \beta$$
$$\mathit{modify}_l : \forall\alpha, \beta.(\alpha :: \{\!\{l : \beta\}\!\}) \Rightarrow \alpha \to \beta \to \alpha$$

The first function corresponds to record selection, the second to record update.

Kinded quantification in REC is modeled by primitive constraints of the form $(\tau :: k)$ where τ is a type and k is a kind. Technically, this means, I add $(\tau :: k)$ to the set Ω of primitive constraints where $(::)$ is a primitive predicate of arity 2. I define REC as the smallest term constraint system that satisfies the following additional rules:

$$\begin{array}{ll} \textbf{REC1} & \vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\}) \\ & \text{where } l_1, \ldots, l_n \text{ are distinct} \\ \textbf{REC2} & (\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2) \end{array}$$

It must be noted that rules **REC1** and **REC2** specify a set of rules for each record $\{l_1 : \tau_1, \ldots l_n : \tau_n\}$ and field label l. In a later variation of Ohori's record calculus, field labels become first–class, hence the set of rules specified by rule **REC2** can be described by only one rule.

The following are derived rules:

$$\begin{array}{ll} \textbf{REC3} & (\{\ldots, l : \tau_1, \ldots\} :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2) \\ \textbf{REC4} & \exists\alpha.(\alpha :: k) =^e \mathsf{true} \\ & \text{where } \alpha \notin fv(k) \end{array}$$

Please note, these conditions rule out recursive records, since the type algebra does not have recursive types. On the other hand, I do allow recursive constraints between type variables in REC. For instance, the constraint $(\alpha :: \{\!\{l : \alpha \to \alpha\}\!\})$ is well-formed. But that constraint is not satisfiable and therefore cannot appear as a solved form. Also ruled out (by conditions **REC2** and **REC3**) is overloading of field labels.

The subsumption predicate ($\preceq$) corresponds to syntactic equality and the set $\mathcal{S}$ of solved forms in HM(REC) consists of all satisfiable constraints of the form

$$C \quad ::= \quad \{\} \mid (\alpha :: \{\!\{l : \tau\}\!\}) \mid C \wedge C \mid \exists\bar{\alpha}.C$$

where I take the empty token set as a representation of true. Furthermore, I require

that the constraints in $\mathcal{S}$ are equation–free, i.e.$C \vdash^e (\tau = \tau')$ must imply $\vdash^e (\tau = \tau')$. For instance,

$$(\alpha :: \{\!\{l : \beta\}\!\}) \wedge (\alpha :: \{\!\{l : \gamma \to \gamma\}\!\})$$

is not equation–free and is therefore excluded.

The type system HM(REC) is as given in Section 3.2, with subsumption ($\preceq$) being modeled by (=). As an example, here the annotated program from Example 19 re-formulated in HM(REC):

Example 20

```
f: ∀α.(∃β.(α :: {{l : β}})) ⇒ α → Int
f x =
  let g : ∀β.(α :: {{l : β}}) ⇒
          β → Bool
      g = λ y. eq y (x.l)
  in 1
```

In HM(REC), I quantify in the innermost let over type variable β, leaving just α to be quantified in the top level function f. This is not possible in *Ohori*, since α's kind depends on β. The question arises whether this makes HM(REC) a more permissive type system than *Ohori*. Specifically, are there examples where I can use function g polymorphically? The answer is no. Every instance of g has to satisfy the constraint $\exists\beta.(\alpha :: \{\!\{l_1 : \beta\}\!\})$. But α can only have one field entry with label l_1. Therefore, I can use g in the let-body only monomorphically. In general, I observe that *Ohori* and HM(REC) type exactly the same programs, but the types are more precise in HM(REC). I refer to Section 3.5 for an in–depth treatment of constrained polymorphism.

Theorem 19 (Full and Faithful) *Every program typable in Ohori is typable in* HM(REC) *and vice versa.*

Ohori's record calculus can be encoded in terms of the overloading application. This was already observed by Odersky/Wadler/Wehr [OWW95]. The basic idea is to consider a record constraint of the form $(\alpha :: \{\!\{l : \tau\}\!\})$ as an overloaded constraint of the form $(l :: \alpha \to \tau)$ where the field label l becomes now an overloaded identifier. Record creation corresponds to providing instance declarations and record selection corresponds to applying an overloaded identifier (which corresponds to a field label) to a record. For more details, I refer to [OWW95]. But then the above results follows easily from the full– and faithfulness result of the overloading application in Section 5.2.

Type Inference

Ohori already showed that unification is unitary in his system. Type inference in HM(REC) can easily achieved by application of the lifting result in Section 4.2.1.

5.1.1 Extensible Records

The record calculus of Ohori lacks some important features such as extensible records. I show now how to add record extension to HM(REC).

For each label l, I add primitive constraints of the form $\mathit{extend}_l(\tau_1, \tau_2, \tau_3)$ to the set of primitive constraints Ω. The predicate $\mathit{extend}_l(\tau_1, \tau_2, \tau_3)$ expresses adding a field with label l and type τ_2 to the record τ_1 where τ_3 is the resulting record. REC^e is the smallest term constraint system that fulfills the following rules:

REC1 $\vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\})$

where $l_1, \ldots, l_n$ are distinct

REC2 $(\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2)$

REC3 $extend_l(\alpha, \beta, \gamma) \vdash^e (\gamma :: \{\!\{l : \beta\}\!\})$

REC4 $extend_l(\alpha, \beta, \gamma) \wedge (\alpha :: \{\!\{l' : \tau\}\!\}) \vdash^e$

$(\gamma :: \{\!\{l' : \tau\}\!\})$

REC5 $extend_l(\alpha, \beta, \gamma) \wedge (\gamma :: \{\!\{l' : \tau\}\!\}) \vdash^e (\alpha :: \{\!\{l' : \tau\}\!\})$

where $l \neq l'$

REC6 $\vdash^e extend_{l_i}(\{l_1 : \tau_1, \ldots, l_{i-1} : \tau_{i-1}, l_{i+1} : \tau_{i+1}, \ldots, l_n : \tau_n\}, \tau_i,$

$\{l_1 : \tau_1, \ldots, l_n : \tau_n\})$

Remark 4 *Conditions* **REC3** *and* **REC4** *state information about the extended record given the original record. The opposite is done by condition* **REC5**. *Condition* **REC6** *represents the base case. Please note, the conditions put on* $extend_l$ *prohibit extending a record with an already existing field. I also find the derived constraint rules mentioned in the previous section.*

Jones [Jon92] uses a predicate $(\alpha \backslash l)$ *to express that record* α *does not contain a field label* l. *I can define* $(\alpha \backslash l)$ *as an abbreviation for* $\exists \beta, \gamma.extend_l(\alpha, \beta, \gamma)$. *Furthermore, I can define* $remove_l(\alpha, \gamma) = \exists \beta.extend_l(\alpha, \beta, \gamma)$ *which expresses removing the field with label* l *from record* γ *resulting in record* α. *Please note, the field label* l *must be present in* γ *because record extension does not allow overriding of existing fields.*

For each field label l, I add

$$extend_l : \forall \alpha, \beta, \gamma.extend_l(\alpha, \beta, \gamma) \Rightarrow \alpha \rightarrow \beta \rightarrow \gamma$$

1. $D \subseteq Cl(D)$
2 If $(\alpha = \{l_1 : \tau_1, \ldots, l_n : \tau_n\}) \in Cl(D)$
then $(\alpha :: \{\!\{l_1 : \tau_1\}\!\}), \ldots, (\alpha :: \{\!\{l_n : \tau_n\}\!\}) \in Cl(D)$
3 If $(\alpha :: \{\!\{l : \tau_1\}\!\}), (\alpha :: \{\!\{l : \tau_2\}\!\}) \in Cl(D)$ then $(\tau_1 = \tau_2) \in Cl(D)$
4. If $extend_l(\alpha_1, \alpha_2, \alpha_3) \in Cl(D)$ then $(\alpha_3 :: \{\!\{l : \alpha_2\}\!\}) \in Cl(D)$
5. If $extend_l(\alpha_1, \alpha_2, \alpha_3), (\alpha_1 :: \{\!\{l' : \beta\}\!\}) \in Cl(D)$ then $(\alpha_3 :: \{\!\{l' : \beta\}\!\}) \in Cl(D)$
6. If $extend_l(\alpha_1, \alpha_2, \alpha_3), (\alpha_3 :: \{\!\{l' : \beta\}\!\}) \in Cl(D)$, $l \neq l'$ then $Cl(D) \in (\alpha_1 :: \{\!\{l' : \beta\}\!\})$
7. If $extend_l(\alpha_1, \alpha_2, \alpha_3)$ and $(\alpha_1 :: \{\!\{l : \beta\}\!\}) \in Cl(D)$ then $\bot \in Cl(D)$

Figure 5.1: Closure for extensible records

---

to the initial type environment Γ_0. The construct

$$remove_l : \forall\alpha, \gamma.\exists\beta.extend_l(\alpha, \beta, \gamma) \Rightarrow \gamma \to \alpha$$

handles removal of the field label l.

The set $\mathcal{S}$ of solved forms in HM(REC^e) consists of all satisfiable and equation–free constraints of the form

$$\begin{aligned} C \quad ::= \quad & \{\} \mid (\alpha :: \{\!\{l : \tau\}\!\}) \mid extend_l(\tau_1, \tau_2, \tau_3) \\ \mid \quad & C \wedge C \mid \exists\bar{\alpha}.C \end{aligned}$$

In addition, I consider type inference for this record application. The development is similar to type inference in HM(REC). The lifting property holds also in this case, hence it is sufficient to consider normalization only for projection–free constraints.

Given a projection–free constraint D. W.l.o.g., I assume all kind constraints are of the form $(\alpha :: \kappa)$ because $(\alpha :: \kappa) =^e \exists\alpha.((\alpha = \tau) \wedge (\alpha :: \kappa))$. I assume that the set E_1 contains all such predicates $(\alpha = \tau)$. The same assumptions can by made about constraints $extend_l(\alpha_1, \alpha_2, \alpha_3)$. Then the closure $Cl(D)$ of a constraint D is the smallest constraint which satisfies the conditions in Figure 5.1.

The $\perp$ element signals immediate failure. Remember, I do not allow overriding of existing fields. The closure of D contains all predicates $(\tau :: \{\!\{l : \tau'\}\!\})$ which might cause any inconsistencies. Given all such predicates, I can generate all unification problems $(\tau = \tau')$ which have to be resolved. The following lemma states that I really have generated all such predicates.

Lemma 32 *Given a field label l and types τ, τ'. If $\not\vdash^e (\tau :: \{\!\{l : \tau'\}\!\})$ then $(\tau :: \{\!\{l : \tau'\}\!\}) \in Cl(D)$ iff $D \vdash^e (\tau :: \{\!\{l : \tau'\}\!\})$. Furthermore, if $\not\vdash^e (\tau = \tau')$ then $(\tau = \tau') \in Cl(D)$ iff $D \vdash^e (\tau = \tau')$.*

If $\perp \in Cl(D)$ the constraint D does not have a normal form at all. Otherwise, I assume that E_2 contains all equality predicates in $Cl(D)$. Then, I can apply unification over Herbrand terms [Rob65] to resolve all equality predicates $(=)$ in $E_1 \cup E_2$. If no unifier exists then D does not have a normal form at all, otherwise Herbrand unification yields a most general unifier ϕ of the $E_1 \cup E_2$ and then $(\phi D, \phi)$ represents the principal normal form of (D, id). Please note, I assume that constraints $C \in \phi D$ where $\vdash^e C$ are disregarded.

It follows that REC^e satisfies the principal constraint property by application of the lifting method.

Theorem 20 *The constraint system* REC^e *satisfies the principal constraint property.*

The careful reader might ask whether constraints of the form

$$extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\})$$

are in $\mathcal{S}$ or not. I conjecture that

$$extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\}) \vdash^e (\alpha = \{l_1 : \tau_1\}) \tag{5.1}$$

does not hold in REC^e. This would justify that $extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\})$ belongs to $\mathcal{S}$. I can prove that

$$extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\}) \vdash^e (\alpha :: \{\!\{l_1 : \tau_1\}\!\})$$

and obviously the only satisfying instance of the constraint $extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\})$ instantiates α to $\{l_1 : \tau_1\}$. But I believe that 5.1 is not derivable with the logical rules of REC^e. However, I do not have a formal proof for this conjecture yet.

I redefine the set $\mathcal{S}$ of solved forms in HM(REC^e) to consist of all satisfiable and equation–free constraints of the form

$$\begin{aligned} C \quad ::= \quad & \{\} \mid (\alpha :: \{\!\{l : \tau\}\!\}) \mid extend_l(\alpha, \tau, \beta) \\ \mid \quad & C \wedge C \mid \exists \bar{\alpha}.C \end{aligned}$$

Please note, this excludes constraints of the form $extend_{l_2}(\alpha, \tau_2, \{l_1 : \tau_1, l_2 : \tau_2\})$. Obviously, the principal constraint property still holds. After the above described normalization step, one simply needs to resolve predicates of the form $extend_l(\{\ldots\}, \tau, \alpha)$ and $extend_l(\alpha, \tau, \{\ldots\})$. That means, substituting type variable α with the appropriate record.

5.1.2 Record Concatenation

I consider adding record concatenation to Ohori's record calculus. I add the predicate $concat(\alpha, \beta, \gamma)$ to the set of primitive constraints Ω. This predicate states concate-

nating records α and β results into the record γ. REC^c is the smallest term constraint system that fulfills the following rules:

REC1 $\vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\})$
where $l_1, \ldots, l_n$ are distinct

REC2 $(\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2)$

REC3 $concat(\alpha, \beta, \gamma) \wedge (\alpha :: \{\!\{l : \delta\}\!\}) \vdash^e (\gamma :: \{\!\{l : \delta\}\!\})$

REC4 $concat(\alpha, \beta, \gamma) \wedge (\beta :: \{\!\{l : \delta\}\!\}) \wedge (\alpha \backslash l) \vdash^e (\gamma :: \{\!\{l : \delta\}\!\})$

REC5 $concat(\alpha, \beta, \gamma) \wedge (\gamma :: \{\!\{l : \delta\}\!\}) \wedge (\beta \backslash l) \vdash^e (\alpha :: \{\!\{l : \delta\}\!\})$

REC6 $concat(\alpha, \beta, \gamma) \wedge (\gamma :: \{\!\{l : \delta\}\!\}) \wedge (\alpha \backslash l) \vdash^e (\beta :: \{\!\{l : \delta\}\!\})$

REC7 $\vdash^e concat(\{\bar{l} : \bar{\tau}\}, \{\bar{l}' : \bar{\tau}'\}, \{\bar{l}'' : \bar{\tau}''\})$
where $\bar{l}'' = (\bar{l} \backslash \bar{l}') \cup \bar{l}'$ and $\tau_i'' = typeof(l_i'')$

REC8 $\vdash^e (\{l_1 : \tau_1, \ldots, l_n : \tau_n\} \backslash l')$
where $l' \neq l_i$

Remark 5 *I model asymmetric concatenation. If fields with the same label are present in both input records, the first record takes precedence. This is enforced by condition* **REC8** *where* $(\bar{l} \backslash \bar{l}') \cup \bar{l}'$ *builds the set of labels in the output record and* $typeof(\cdot)$ *is a function from labels to its associated types. In fact,* $typeof(\cdot)$ *should also be seen as parameterized in the record because the same label might possibly have different types in different records. Therefore, the record should always be seen as an implicit parameter for the* $typeof(\cdot)$ *function.*

The primitive

$$concat : \forall \alpha, \beta, \gamma. concat(\alpha, \beta, \gamma) \Rightarrow \alpha \to \beta \to \gamma$$

allows concatenation of records.

1. $D \subseteq Cl(D)$
2. If $(\{\ldots, l : \tau_1, \ldots\} :: \{\!\{l : \tau_2\}\!\}) \in Cl(D)$ then $(\tau_1 = \tau_2) \in Cl(D)$
3. If $(\alpha :: \{\!\{l : \tau_1\}\!\})$ and $(\alpha :: \{\!\{l : \tau_2\}\!\}) \in Cl(D)$ then $(\tau_1 = \tau_2) \in Cl(D)$
4. If $concat(\alpha, \beta, \gamma), (\alpha :: \{\!\{l : \delta\}\!\}) \in Cl(D)$ then $(\gamma :: \{\!\{l : \delta\}\!\}) \in Cl(D)$
5. If $concat(\alpha, \beta, \gamma), (\beta :: \{\!\{l : \delta\}\!\}), (\alpha \backslash l) \in Cl(D)$ then $(\gamma :: \{\!\{l : \delta\}\!\}) \in Cl(D)$
6. If $concat(\alpha, \beta, \gamma), (\gamma :: \{\!\{l : \delta\}\!\}), (\beta \backslash l) \in Cl(D)$ then $(\alpha :: \{\!\{l : \delta\}\!\}) \in Cl(D)$
7. If $concat(\alpha, \beta, \gamma), (\gamma :: \{\!\{l : \delta\}\!\}), (\alpha \backslash l)$ then $(\beta :: \{\!\{l : \delta\}\!\})$
8. If $(\alpha \backslash l), (\alpha :: \{\!\{l : \tau\}\!\}) \in Cl(D)$ then $\bot \in Cl(D)$
9. If $(\{\bar{l} : \bar{\tau}\} \backslash l) \in Cl(D)$, $l \notin \bar{l}$ then $\bot \in Cl(D)$
10. If $concat(\{\bar{l_1} : \bar{\tau_1}\}, \{\bar{l_2} : \bar{\tau_2}\}, \gamma) \in Cl(D)$
    then $\bar{l_3} = \bar{l_2} \backslash \bar{l_1}$, $(\gamma = \{\bar{l_1} : \bar{\tau_1}, \bar{l_3} : typeof(\bar{l_3})\}) \in Cl(D)$
11. If $concat(\{\bar{l_1} : \bar{\tau_1}\}, \beta, \{\bar{l_2} : \bar{\tau_2}\}) \in Cl(D)$ and $\bar{l_1} \subseteq \bar{l_2}$
    then $(typeof(\bar{l_1}) = typeof(\bar{l_2} \cap \bar{l_1})) \in Cl(D)$
12. If $concat(\{\bar{l_1} : \bar{\tau_1}\}, \beta, \{\bar{l_2} : \bar{\tau_2}\}) \in Cl(D)$ and $\bar{l_1} \not\subseteq \bar{l_2}$ then $\bot \in Cl(D)$
13. If $concat(\alpha, \cdot, \{\bar{l} : \bar{\tau}\}), (\alpha :: \{\!\{l : \tau\}\!\}) \in Cl(D)$ and $l \in \bar{l}$
    then $(\tau = typeof(\bar{l} \cap \{l\})) \in Cl(D)$
14. If $concat(\alpha, \cdot, \{\bar{l} : \bar{\tau}\}), (\alpha :: \{\!\{l : \tau\}\!\}) \in Cl(D)$ and $l \notin \bar{l}$ then $\bot \in Cl(D)$
15. If $concat(\alpha, \{\bar{l_1} : \bar{\tau_1}\}, \{\bar{l_2} : \bar{\tau_2}\}) \in Cl(D)$ and $\bar{l_1} \not\subseteq \bar{l_2}$ then $\bot \in Cl(D)$
16. If $concat(\{\bar{l} : \bar{\tau}\}, \cdot, \gamma), (\gamma :: \{\!\{l : \tau\}\!\}) \in Cl(D)$ and $l \in \bar{l}$
    then $(\tau = typeof(\bar{l} \cap \{l\})) \in Cl(D)$
17. If $concat(\{\bar{l_1} : \bar{\tau_1}\}, \{\bar{l_2} : \bar{\tau_2}\}, \{\bar{l_3} : \bar{\tau_3}\}) \in Cl(D)$ and $\bar{l_1} \subseteq \bar{l_3}, \bar{l_2} \subseteq \bar{l_3}$
    then $(typeof(\bar{l_1}) = typeof(\bar{l_3} \cap \bar{l_1})), (typeof(\bar{l_2} \backslash \bar{l_1}) = typeof(\bar{l_3} \cap (\bar{l_2} \backslash \bar{l_1}))) \in Cl(D)$
18. If $concat(\{\bar{l_1} : \bar{\tau_1}\}, \{\bar{l_2} : \bar{\tau_2}\}, \{\bar{l_3} : \bar{\tau_3}\}) \in Cl(D)$ and $(\bar{l_1} \not\subseteq \bar{l_3}$ or $\bar{l_2} \not\subseteq \bar{l_3})$
    then $\bot \in Cl(D)$

Figure 5.2: Closure for record concatenation

The set $\mathcal{S}$ of solved forms in HM(REC^c) consists of all satisfiable and equation–free constraints of the form

$$C \quad ::= \quad \{\} \mid (\alpha :: \{\!\{l : \tau\}\!\}) \mid concat(\tau_1, \tau_2, \tau_3) \mid C \wedge C \mid \exists \bar{\alpha}.C$$

Type inference for HM(REC^c) can be achieved in a similar way as for the record application with extensible records in the previous section. I only consider projection–free constraints and apply a closure algorithm, see Figure 5.2. In contrast to the previous section, constraints of the form $p(\bar{\tau})$ are not put into normal form, i.e.

transformed into $\exists\alpha.((\bar{\alpha} = \bar{\tau}) \wedge p(\bar{\tau})$. This would complicate closure rules such as rule (10.). Furthermore, it is necessary to perform several iterations of building the closure and unifying equality constraints. Unification problems resolved in one step might lead to new unification problems in the next step. Consider the constraint

$$\mathit{concat}(\{l_1 : \tau_1\}, \{l_2 : \tau_2\}, \gamma) \wedge \mathit{concat}(\gamma, \{l_3 : \tau_3\}, \{l_1 : \alpha_1, l_2 : \alpha_2, l_3 : \alpha_3\})$$

Building of the closure yields the unification problems

$$(\gamma = \{l_1 : \tau_1, l_2 : \tau_2\}) \wedge (\alpha_3 = \tau_3)$$

Computing the mgu and applying the mgu to the above constraint results in:

$$\begin{aligned}&\mathit{concat}(\{l_1 : \tau_1\}, \{l_2 : \tau_2\}, \{l_1 : \tau_1, l_2 : \tau_2\}) \wedge \\ &\mathit{concat}(\{l_1 : \tau_1, l_2 : \tau_2\}, \{l_3 : \tau_3\}, \{l_1 : \alpha_1, l_2 : \alpha_2, l_3 : \tau_3\})\end{aligned}$$

But two further unification problems

$$(\alpha_1 = \tau_1) \wedge (\alpha_2 = \tau_3)$$

need to be resolved until the whole constraint is in solved form.

Therefore, computation of the principal normal forms performs several iterations of building the closure and resolving all unification problems. Given a projection–free constraint D. I set $D_1 = D, \phi_0 = id$ and $D_{i+1} = \phi_i D_i$ where ϕ_i is the mgu of all equality problems in $\mathit{Cl}(D_i)$ for $i > 0$. The sequence of $D_1, D_2, \ldots$ has a fixed point. Either all unification problems get resolved or the element $\perp$ appears at one step. Therefore, the iteration process terminates at a step i. Then $(\phi D, \phi)$ where $\phi = \phi_i \circ \ldots \circ \phi_1$ represents the principal normal form.

Please note, in rule (11.) the constraint $concat(\{\bar{l_1} : \bar{\tau_1}\}, \beta, \{\bar{l_2} : \bar{\tau_2}\})$ does not allow to determine the record β because asymmetric record concatenation is modeled. Nevertheless, the equality problems ($typeof(\bar{l_1}) = typeof(\bar{l_2} \cap \bar{l_1})$) ensure that matching fields in the records $\{\bar{l_1} : \bar{\tau_1}\}$ and $\{\bar{l_2} : \bar{\tau_2}\}$ have the same type. Please note, the side condition $\bar{l_1} \subseteq \bar{l_2}$ ensures that all fields of the former record are also contained in the latter. Also, the constraint $concat(\alpha, \cdot, \{\bar{l} : \bar{\tau}\})$ ($\cdot$ stands for a hole which can either be filled up by a variable or record) in itself is in solved form. Further information is needed (see rule (13.)) to generate unification problems.

Application of the lifting method allows to establish the principal constraint property for REC^c.

Theorem 21 *The constraint system* REC^c *satisfies the principal constraint property.*

I conjecture that the set of solved forms satisfies the syntactic uniqueness property, hence type inference infers unique principal types.

5.1.3 Polymorphic Field Labels

In the previous application, field labels were considered as constants. I extend the term algebra in REC. I distinguish now between labels and types:

$$\begin{array}{lrcl} \textbf{Labels} & l & ::= & \alpha \,|\, l_1 \,|\, \ldots \\ \textbf{Types} & \tau & ::= & \alpha \,|\, label(l) \,|\, \{l_1 : \tau_1, \ldots, l_n : \tau_n\} \,|\, \tau \to \tau \end{array}$$

I distinguish between label constants and label variables. Commonly, I refer to label constants as l_i and assume that for each constant label l_i there is a primitive

$$\mathrm{l}_i \;:\; label(l_i)$$

in the initial type environment. Primitives for record creation remain the same. Please note, only constant labels appear in record types $\{l_1 : \tau_1, \ldots, l_n : \tau_n\}$.

The primitive constructs for field selection and record update are now polymorphic in the field labels. This means, there is no longer a special construct for each label l needed.

$$\begin{aligned} (_._) \quad &: \forall \alpha, \beta, l. \\ &(\alpha :: \{\!\{l : \beta\}\!\}) \Rightarrow \\ &\alpha \to \mathit{label}(l) \to \beta \end{aligned}$$

$$\begin{aligned} \mathit{modify} \quad &: \forall \alpha, \beta, l. \\ &(\alpha :: \{\!\{l : \beta\}\!\}) \Rightarrow \\ &\alpha \to \mathit{label}(l) \to \beta \to \alpha \end{aligned}$$

The predicate $(r :: \{\!\{l : \tau\}\!\})$ takes now a record r, a label l and a type τ as its arguments. The conditions put on REC^l are exactly the same found in REC. The constraint system REC^l fulfills is the smallest term constraint system satisfying the following rules:

$$\begin{aligned} \textbf{REC1} \quad &\vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\}) \\ &\text{where } l_1, \ldots, l_n \text{ are distinct} \\ \textbf{REC2} \quad &(\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2) \end{aligned}$$

The constraint system REC^l is left under–specified. For instance, consider the constraint

$$(\{l_1 : \tau_1, l_2 : \tau_2\} :: \{\!\{l : \alpha\}\!\})$$

where l_1 and l_2 are constant field labels with types τ_1 and τ_2 but l is a field label

variable with type variable α. There is not enough information given to determine whether l equals to l_1 or l_2. This represents a form of indeterministic choice.

Satisfiability in REC^l is decidable but NP–complete. A proof can be found in Appendix B.1. This result is not new and can already be found in similar form in [Tre93]. Complete and sound inference could then be achieved by taking the set $\mathcal{S}$ to consist of all satisfiable constraints.

Compared to REC constraints of the form $(\{\ldots\} :: \{\!\{l : \tau\}\!\})$ can no longer be resolved anymore because label l might be a variable. Nevertheless, it is possible to impose stronger conditions on the set $\mathcal{S}$ while retaining the principal constraint property.

The set $\mathcal{S}$ of solved forms in HM(REC^l) consists of all satisfiable and equation–free constraints of the form

$$C \quad ::= \quad \{\} \mid (\tau :: \{\!\{l : \tau'\}\!\}) \mid C \wedge C \mid \exists \bar{\alpha}.C$$

In HM(REC^l) constraints of the form $(\tau :: \{\!\{l : \tau'\}\!\})$ where τ is not necessarily a type variable are allowed which are disallowed in HM(REC). Please note, it is crucial that label variables are not allowed to appear within record types. This would immediately prevent a resolution of equality predicates. For instance, consider the constraint

$$(\{l : \alpha, l' : \beta\} = \{l_1 : \mathsf{Int}, l_2 : \mathsf{Bool}\})$$

No most general unifier exists to resolve such an equality constraint.

To establish the principal constraint property in HM(REC^l), I follow the development in HM(REC^e). Given a projection–free constraint D where all kind constraints are of the form $(\alpha :: \kappa)$. W.l.o.g. D is satisfiable, otherwise it follows immediately that no normal form exists all. The closure $\mathcal{C}l(D)$ of a constraint D is the smallest

constraint which fulfills the following conditions:

1. $D \subseteq \mathit{Cl}(D)$
2. If $(\alpha = \{l_1 : \tau_1, \ldots, l_n : \tau_n\}) \in \mathit{Cl}(D)$
   then $(\alpha :: \{\!\{l_1 : \tau_1\}\!\}), \ldots, (\alpha :: \{\!\{l_n : \tau_n\}\!\}) \in \mathit{Cl}(D)$
3. If $(\alpha :: \{\!\{l : \tau_1\}\!\})$ and $(\alpha :: \{\!\{l : \tau_2\}\!\}) \in \mathit{Cl}(D)$
   then $(\tau_1 = \tau_2) \in \mathit{Cl}(D)$

Herbrand unification resolves then all equality constraints in $\mathit{Cl}(D)$. But this does not immediately lead to a principal normal form of D. Consider the constraint

$$(\{l_1 : \tau_1, l_2 : \tau_2\} :: \{\!\{l : \alpha\}\!\}) \wedge (\mathit{label}(l_1) = \mathit{label}(l))$$

Then the above describe normalization procedure would yield the constraint

$$(\{l_1 : \tau_1, l_2 : \tau_2\} :: \{\!\{l_1 : \alpha\}\!\})$$

which is not in solved form yet. Hence, it is necessary to perform several iterations until the principal normal form is achieved. This is similar to record concatenation in the previous section.

Given D as described above. I set $D_1 = D, \phi_0 = id$ and $D_{i+1} = \phi_i D_i$ where ϕ_i is the mgu of all equality problems in $\mathit{Cl}(D_i)$ for $i > 0$. It is easy to see that the sequence of $D_1, D_2, \ldots$ has a fixed point. Therefore, the iteration process terminates at a step i. Then $(\phi D, \phi)$ where $\phi = \phi_i \circ \ldots \circ \phi_1$ represents the principal normal form.

The lifting method leads to the following theorem.

Theorem 22 *The constraint system* REC^l *satisfies the principal constraint property.*

Encoding Type Case

I apply the record calculus with polymorphic field labels to encode a form of type case. The type case rule is usually written as follows:

$$\frac{\Gamma \vdash e_i : \tau_i' \quad \text{for } i = 1, \ldots, n \qquad \Gamma \vdash e : \tau_j \quad \text{for some } j = 1, \ldots, n}{\Gamma \vdash \begin{array}{ll} \mathsf{typecase}\ e\ \mathsf{of} & \tau_1 \Rightarrow e_1 \\ & \vdots \\ & \tau_n \Rightarrow e_n \quad : \tau_j' \end{array}}$$

Depending on e's type, the corresponding branch is chosen. The denotational semantics of typecase contains no surprises:

$$\begin{array}{lcl} [\![\mathsf{typecase}\ e\ \mathsf{of} \quad \tau_1 \Rightarrow e_1 & & \\ \qquad\qquad \vdots & = & \mathsf{if}\ [\![e]\!]\eta \in [\![\tau_1]\!]\mathsf{then}\ [\![e_1]\!]\eta \\ \tau_n \Rightarrow e_n]\!]\eta & & \mathsf{else\ if}\ \ldots \\ & & \mathsf{else\ if}\ [\![e]\!]\eta \in [\![\tau_n]\!]\mathsf{then}\ [\![e_n]\!]\eta \\ & & \mathsf{else}\ \mathbf{W} \end{array}$$

I show now how to represent type case as a primitive function and not as a typing rule. The exercise here is not to give applications of the type case statement. Possible applications could involve system to model overloading respectively a form of dynamic typing. I rather show how the type case rule can be neatly be represented through a primitive function based on the record calculus with polymorphic field labels.

I assume that there is additionally the following primitive in the initial type

environment Γ_0.

$$typecase \quad : \quad \forall\alpha,\beta,\gamma.\exists l.(\alpha :: \{\!\{l : \beta \to \gamma\}\!\}) \Rightarrow \beta \to \alpha \to \gamma$$

Furthermore, I assume that there are annotated function abstractions of the form $\lambda x : \tau.e$ which states that function parameter x has type τ. Then, the type case statement

$$\begin{array}{rl} \mathsf{typecase}\ e\ \mathsf{of} & \tau_1 \Rightarrow e_1 \\ & \vdots \\ & \tau_n \Rightarrow e_n \end{array}$$

can be represented as

$$typecase \quad e \quad \{l_1 = \lambda x : \tau_1.e_1, \ldots, l_n = \lambda x : \tau_n.e_n\} \tag{5.2}$$

where $\{l_1 = \lambda x : \tau_1.e_1, \ldots, l_n = \lambda x : \tau_n.e_n\}$ is a record with distinct constant labels $l_1, \ldots, l_n$ and x is a fresh type variable.

Assume e has type τ and

$$\{l_1 = \lambda x : \tau_1.e_1, \ldots, l_n = \lambda x : \tau_n.e_n\}$$

has type

$$\{l_1 : \tau_1 \to \tau_1', \ldots, l_n : \tau_n \to \tau_n'\}$$

where field labels l_i have label type l_i and expression e_i have type τ_i'. Expression 5.2 is well–typed if the constraint

$$(\{l_1 : \tau_1 \to \tau_1', \ldots, l_n : \tau_n \to \tau_n'\} :: \{\!\{l : \tau \to \gamma\}\!\})$$

is fulfilled. The constraint rules enforce type τ to match any of the types τ_j's. This automatically selects one of the branches and determines result type γ to be τ'_j.

5.1.4 Records with Overloaded Field Labels

I have already noted that Ohori's record calculus can be encoded in terms of the Odersky/Wadler/Wehr overloading system. In Section 5.2.1, I give an extension of the Odersky/Wadler/Wehr overloading system which allows to encode Ohori's system extended with overloaded field labels.

5.1.5 Summary

There is already lots of work done in the area of record calculi, see [HP91, Car84, CM89, EST95b, Rém95a, Jon92, Oho95, Wan88, Wan89, Rém95b, Rém89]. In the present work, I showed how to encode Ohori's record calculus as an HM(X) instance and provided various useful extensions such as extensible records, record concatenation and first–class labels. The HM(X) system proved to be useful in designing such extensions in a systematic way. In turn, I relate the HM(X) record applications to some previous work.

Extensible records were already considered by [Gas98, HP90, Rém93]. In my dissertation, I show that Ohori's work can be extended to deal with extensible records. Gaster's work [Gas98] improves compared to others in that he can provide an efficient compilation method for extensible records. In [Dri93, DH95] efficient implementation methods for dynamic dispatch are studied in the context of object–oriented languages with multiple inheritance. This suggests that the HM(X) applications with extensible records and record concatenation can be implemented efficiently.

Record concatenation was studied in [Rem92b, Wan93, Ken96]. But only the

work by Kennedy [Ken96] achieves principal types with a unification–based approach. His form of record concatenation is limited to fields that must be of the same type. The HM(X) application HM(REC^c) provides record concatenation while still retaining principal types. The crucial difference compared to previous applications lies in representing information by constraints of the form $concat(\alpha, \beta, \gamma)$ rather than representing typing information in the language of types. For instance, Wand [Wan93] can not give a principal type to the following expression:

$$\mathsf{f\ x\ y} = (concat(\mathsf{x,y})).\mathsf{l} + 1$$

This should be applicable to any pair of records x and y in which x has an integer field l or in which y has an integer field l and field l is absent in x. The best type Wand can give to the above expression is a disjunction of two types. Such problems occur due to a term representation of typing information in Wand's system. In contrast, the HM(X) application with record concatenation can give the following principal type:

$$\mathsf{f} \ : \ \forall \alpha, \beta, \gamma.(\gamma :: \{\!\{l : \mathsf{Int}\}\!\}) \wedge concat(\alpha, \beta, \gamma) \Rightarrow \alpha \to \beta \to \mathsf{Int}$$

Typing information is now represented in constraint form which allows an achievement of principal types while still following a unification–based approach.

A calculus similar to HM(REC^l) where labels are first–class has already been proposed by Gaster [Gas98] and Nishimura [Nis98]. In Section 3.5.4, I have already pointed out the shortcomings of Nishimura's formulation of the quantifier introduction rule which can be solved by the HM(X) formulation. In Appendix B.2, I give an example which shows limitations of Gaster's record calculus with first–class labels. I give a program which is not typable in his system. This is due to the fact that typing

information about records is represented in the type language. For instance, the type $\{l : \alpha | r\}$ represents all values which contain at least a field with label l and type α. Such values are characterized by the type β under the constraint $(\beta :: \{\!\{l : \alpha\}\!\})$ in REC. In general, more programs are typable in HM(REC^l) than in Gaster's calculus.

Polymorphic field labels seem to be especially interesting. In Section 5.1.3, I showed how to encode a type case construct in terms of a record calculus with polymorphic field labels. Hence, records with first–class labels provide a form of overloading. This suggests that there must a close relationship [1] between records and overloading.

5.2 Odersky/Wadler/Wehr Overloading

I give a translation of the overloading approach by Odersky et al [OWW95] (to which I refer as *SystemO*) in terms of an HM(X) application. Odersky et al. consider the overloading of functions where overloading is restricted to the argument type. Hence, it is forbidden to overload numeric constants because the overloaded identifier always has to be a function type, and overloading is restricted to the first argument.

I define a constraint system OVER which captures the properties of the overloading system *SystemO* as described in [OWW95]. Essentially, there are two different kinds of constraints. Constraints of the form $(o :: \alpha \to \tau)$ restrict the possible instances of a type variable α for a given overloaded identifier o. For instance, consider the following program:

$\mathsf{f} : \forall \alpha, \beta.(o :: \alpha \to \beta) \Rightarrow \alpha \to \beta$

$\mathsf{f}\ \mathsf{x} = o\ \ \mathsf{x}$

[1] Note that Odersky et al. could show that their overloading system is able to express Ohori style records.

The function f takes only arguments for which an instance declaration of an overloaded identifier o with the same argument type is given.

The other kind of constraints are of the form $(o :: \sigma_T)$. Constraints of this kind are used to describe instance declarations of overloaded identifiers. I use the subscript T on the type scheme σ_T to describe the head type constructor of that type scheme. I put the following conditions on σ_T:

$$\begin{array}{rcll} \sigma_T & = & T\bar{\alpha} \to \tau & fv(\tau) \subseteq \bar{\alpha} \\ & | & \forall\alpha.\pi_\alpha \Rightarrow \sigma'_T & fv(\pi_\alpha) \subseteq fv(\sigma'_T) \end{array}$$

where I assume that the constraint π_α only contains predicates of the form $(o :: \alpha \to \tau)$. Furthermore, I assume that σ_T is a closed type scheme. Please note, as observed in Section 3.4.5 and in Section 3.5.3 type schemes of the form

$$\forall\alpha_1.\pi_{\alpha_1} \Rightarrow \ldots \forall\alpha_n.\pi_{\alpha_n} \Rightarrow \tau$$

can be seen equivalent to

$$\forall\alpha_1, \ldots, \alpha_n.\pi_{\alpha_1} \wedge \ldots \wedge \pi_{\alpha_n} \Rightarrow \tau$$

Therefore, it is possible to see the first type scheme as an abbreviation for the other type scheme.

Constraint system OVER is defined as follows:

$$\textbf{OVER1} \quad (o :: \alpha \to \tau) \wedge (o :: \alpha \to \tau') \vdash^e (\tau = \tau')$$

$$\textbf{OVER2} \quad \frac{(o :: T\bar{\tau}_1 \to \tau_1) \wedge (o :: T\bar{\tau}_2 \to \tau_2)}{(\bar{\tau}_1 = \bar{\tau}_2) \wedge (\tau_1 = \tau_2)} \vdash^e$$

$$\textbf{OVER3} \quad C \wedge (o :: \forall\bar{\alpha}.D \Rightarrow \tau) \vdash^e (o :: [\bar{\tau}/\bar{\alpha}]\tau) \quad \text{if } C \vdash^e [\bar{\tau}/\bar{\alpha}]D$$

$$\textbf{OVER4} \quad \exists\alpha.(o :: \alpha \to \tau) =^e \mathsf{true}$$

Rules **OVER1** and **OVER2** enforce that the argument type uniquely determines the result type. Rule **OVER4** reflects the fact that I implement an open world approach. This is similar to type classes.

The set $\mathcal{S}$ of solved forms in HM(OVER) consists of all satisfiable constraints of the form

$$C \quad ::= \quad \{\} \mid (o :: \alpha \to \tau) \mid C \wedge C \mid \exists\bar{\alpha}.C$$

Furthermore, I put two additional conditions on the set $\mathcal{S}$. First, I require that the constraints in $\mathcal{S}$ are equation–free, i.e. $C \vdash^e (\tau = \tau')$ must imply $\vdash^e (\tau = \tau')$.

For instance,

$$(o :: \alpha \to \beta) \wedge (o :: \alpha \to \gamma \to \gamma)$$

is not equation–free because

$$(o :: \alpha \to \beta) \wedge (o :: \alpha \to \gamma \to \gamma) \vdash^e (\beta = \gamma \to \gamma)$$

Finally, I impose an ordering relation on the type variables in a constraint in solved form. For every $C \in \mathcal{S}$ there is an ordering $<$ on the type variables in C such that for all predicates $(\alpha :: k)$ and $\beta \in \mathit{fv}(k)$ with $C \vdash^e (\alpha :: k)$ it holds that $\alpha < \beta$. For instance, consider the constraint $(o :: \alpha \to \beta)$. which is in solved form. It holds that $\alpha < \beta$. This condition prohibits cyclic dependencies between type variables that appear in constraints.

For each overloaded identifier o, I now add primitives of the form

$$o : \forall \alpha, \beta.(o :: \alpha \to \beta) \Rightarrow \alpha \to \beta$$

to the initial type environment Γ_0. Note, the overloaded identifier o appears as a term variable but also in the constraint $(o :: \alpha \to \beta)$.

There are additionally instance declarations of the form $\mathsf{inst}\ o : \sigma_T = e\ \mathsf{in}\ p$ in the term language. This the meaning of the identifier o with the function given by e on all arguments that are constructed from the type constructor T. The extended term language is as follows:

$$\textbf{Declarations} \quad p \ ::= \ e \,|\, \mathsf{inst}\ o : \sigma_T = e\ \mathsf{in}\ p$$

where the term e stands for an expression.

There is one additional typing rule for creation of new instance declarations.

$$\mathrm{X'} \ = \ \mathrm{X} \cup \ \vdash^e \ (o :: \sigma_T)$$

(Inst)

$$\frac{C, \Gamma, \mathrm{X} \vdash \ e : \sigma_T \qquad C, \Gamma, \mathrm{X'} \vdash \ p : \sigma'}{C, \Gamma, \mathrm{X'} \vdash \ \mathsf{inst}\ o : \sigma_T = e\ \mathsf{in}\ p : \sigma'}$$

The derivation $\vdash$ is now additionally parameterized in the constraint system because new facts are added with each instance declaration. Please note, assume there are two instance declarations for an overloaded identifier o with the same head type constructor. Then, there are two new constraint rules $\vdash^e (o :: \sigma'_T)$ and $\vdash^e (o :: \sigma_T)$. The rules put on the constraint system OVER ensure that the two type schemes σ'_T and σ_T have to be equivalent [2]. But this does not ensure that the meaning of the two instances are the same. Hence, I require that there is only one instance declaration for overloaded identifier o with head type constructor T.

Another issue is the semantic meaning of such programs. I assume that first all instance declarations are processed which allows the constraint system to be fixed. In [OWW95] a denotational semantics for *SystemO* is given from which the denotational semantics in HM(X) originated.

I establish now a connection between HM(OVER) and the original approach *SystemO*. In Section 3.5.3, I already discussed differences between the formulation of the quantifier introduction rule in *SystemO* and HM(X). Please note, in HM(OVER) projection does not need to be trivial. This does not extend the expressiveness of the language as stated in the following theorem. I refer to the appendix for a proof sketch.

Theorem 23 (Full and Faithful) HM(OVER) *models SystemO fully and faithfully.*

5.2.1 Overloaded Curried Functions

I extend the overloading approach *SystemO* with curried overloaded functions. In such an extension there are constraints of the form $(o :: \alpha \to \beta \to \tau)$. The idea is now that not only can the first argument be overloaded but also the second argument as well.

[2] See the subsection about ordering relations in Section 3.4

Such a calculus proves to be useful in dealing with printer functions where one not only can specify the format but also the device. For instance, *print* : HP–Printer $\rightarrow$ Postscript $\rightarrow$ () specifies a print function that supports the Postscript–format for a specific printer. One could also think of print function that supports a specific file format for a hard disk, specified by *print* : Harddisk $\rightarrow$ Unix $\rightarrow$ ().

Furthermore, it follows immediately that the record calculus with overloaded field labels can be encoded in such an extended overloading application.

An overloaded identifier might now depend on two different type constructors.

$$\begin{array}{rcll} \sigma_{T_1T_2} & = & T_1\bar{\alpha} \rightarrow T_2\bar{\beta} \rightarrow \tau & fv(\tau) \subseteq \bar{\alpha} \cup \bar{\beta} \\ & | & \forall\alpha,\beta.\pi_{\alpha\beta} \Rightarrow \sigma'_{T_1T_2} & fv(\pi_{\alpha\beta}) \subseteq fv(\sigma'_{T_1T_2}) \end{array}$$

where I assume that the constraint $\pi_{\alpha\beta}$ only contains predicates of the form $(o :: \alpha \rightarrow \beta \rightarrow \tau)$. Furthermore, I assume that $\sigma_{T_1T_2}$ is a closed type scheme. The other constraints are of the form $(o :: \alpha \rightarrow \beta \rightarrow \tau)$. Compared to the original overloading approach condition **OVER2** is split into two conditions to deal with curried overloaded functions.

$$(o :: \alpha \to \beta \to \tau) \wedge (o :: \alpha \to \beta \to \tau')$$

OVER1 $$\vdash^e$$

$$(\tau = \tau')$$

$$(o :: T\bar{\tau}_1 \to S\bar{\tau}'_1 \to \tau_1) \wedge (o :: T\bar{\tau}_2 \to S\bar{\tau}'_2 \to \tau_2)$$

OVER2 $$\vdash^e$$

$$(\bar{\tau}_1 = \bar{\tau}_2) \wedge (\bar{\tau}'_1 = \bar{\tau}'_2) \wedge (\tau_1 = \tau_2)$$

$$(o :: T\bar{\tau}_1 \to \alpha \to \tau_1) \wedge (o :: T\bar{\tau}_2 \to \alpha \to \tau_2)$$

OVER3 $$\vdash^e$$

$$(\bar{\tau}_1 = \bar{\tau}_2) \wedge (\tau_1 = \tau_2)$$

$$(o :: \alpha \to T\bar{\tau}_1 \to \tau_1) \wedge (o :: \alpha \to T\bar{\tau}_2 \to \tau_2)$$

OVER4 $$\vdash^e$$

$$(\bar{\tau}_1 = \bar{\tau}_2) \wedge (\tau_1 = \tau_2)$$

OVER5 $C \wedge (o :: \forall\alpha.D \Rightarrow \tau) \vdash^e (o :: [\tau/\alpha]\tau)$
if $C \vdash^e [\tau/\alpha]D$

OVER6 $\exists\alpha.(o :: \alpha \to \beta \to \tau) =^e \mathsf{true}$

Note, condition **OVER2** states that as soon as the head type constructors of the two argument types are fixed, the result type is uniquely determined but not earlier. Conditions **OVER3** and **OVER4** state that if one argument type is fixed and the head type constructor of the other argument type is known, then the result type is uniquely determined. Condition **OVER1** covers the case if the two argument types

are type variables.

The set $\mathcal{S}$ of solved forms consists of all satisfiable and equation–free constraints of the form

$$C \quad ::= \quad \{\} \mid (o :: \alpha \to \beta \to \tau) \mid C \wedge C$$

Wehr [Weh97] also introduces curried overloaded functions. The idea is to encode curried overloaded functions in terms of the basic overloading system. For instance, consider two overloaded instances

$$o : \forall \bar{\alpha}.C \Rightarrow T \to S \to \tau_1$$
$$o : \forall \bar{\beta}.C' \Rightarrow T \to T \to \tau_2$$

where there are two constant type constructors T and S. Instead of these two instances he introduces

$$o : \forall \alpha, \beta.(o' :: \alpha \to \beta) \Rightarrow T \to \alpha \to \beta$$

where for each argument type there are instance declarations

$$o' : \forall \bar{\alpha}.C \Rightarrow S \to \tau_1$$
$$o' : \forall \bar{\beta}.C' \Rightarrow T \to \tau_2$$

Further details can be found in [Weh97].

In Wehr's curried overloaded function approach [Weh97], it is not obvious how to encode records with overloaded field labels. For instance, in his system it is not possible to type the program [3] in Example 17 in Section 3.5.4.

[3]Remember, this example is based on the curried overloading application.

5.2.2 Overloaded Identifiers as First–Class Values

In the previous applications, overloaded identifiers were considered as constants. Now, overloaded identifiers become first class values that can be passed around as arguments. This requires a couple of changes. In the term language, I distinguish now between overloaded variables and constants. This is in contrast to the previous applications which considered overloaded identifiers as primitives. This change requires to distinguish between overloaded constants and variables. The development is similar to the record calculus with first–class labels.

$$\begin{array}{lrcl} \textbf{Over} & o & ::= & \alpha \mid o_1 \mid \ldots \\ \textbf{Types} & \tau & ::= & \alpha \mid \mathit{over}(o) \mid \tau \to \tau \end{array}$$

The constraint $(_ :: _)$ takes additionally an overloaded identifier type as a parameter. I assume that the signature of the predicate $(_ :: _)$ specifies that the first argument always has to be an overloaded identifier.

The constraint rules are almost the same as in the original application. In the following rules the term o stands for an overloaded type variable.

$$\textbf{OVER1} \quad (o :: \alpha \to \tau) \wedge (o :: \alpha \to \tau') \vdash^e (\tau = \tau')$$

$$\textbf{OVER2} \quad \begin{array}{c} (o :: T\bar{\tau}_1 \to \tau_1) \wedge (o :: T\bar{\tau}_2 \to \tau_2) \\ \vdash^e \\ (\bar{\tau}_1 = \bar{\tau}_2) \wedge (\tau_1 = \tau_2) \end{array}$$

$$\begin{array}{ll} \textbf{OVER3} & C \wedge (o :: \forall\alpha.D \Rightarrow \tau) \vdash^e (o :: [\tau/\alpha]\tau) \\ & \text{if } C \vdash^e [\tau/\alpha]D \\ \textbf{OVER4} & \exists\alpha.(o_i :: \alpha \to \tau) =^e \mathsf{true} \end{array}$$

The definition of solved forms is standard. The set $\mathcal{S}$ of solved forms consists of all satisfiable and equation–free constraints of the form

$$C \quad ::= \quad \{\} \mid (o :: \alpha \to \tau) \mid C \wedge C$$

I also do not need anymore primitives for each overloaded constants. There are the following primitive in the initial type environment Γ_0:

$$@ : \forall o, \alpha, \beta.(o :: \alpha \to \beta) \Rightarrow over(o) \to \alpha \to \beta$$

This primitive can be read as invoke overloaded identifier o at argument α. Instead of $o\,x$ I write now $@\,o\,x$. Furthermore, I require that instance declarations are only allowed for overloaded constants.

Note, in order to model the Example 18 in Section 3.5.4, I actually need an overloading application that models first class overloaded identifiers combined with curried overloading. For instance, the record constraint $(\alpha :: \{\!\{print : \beta \to ()\}\!\})$ in function g translates to the overloaded constraint $(print :: \alpha \to \beta \to ())$ where *print* is an overloaded type variable and I additionally overload the second argument type. It is straightforward to combine the curried and first class overloading approach into an overloading approach that combines these two features.

5.2.3 Summary

The overloading application does not fit as nicely as the record application into the HM(X) system. An additional language construct must be declared which does not seem to definable by a primitive. Further extensions have been proposed in [Weh97] which might be worthwhile to express in terms of the HM(X) system.

5.3 Program checking

Here comes some additional stuff added after submitting my thesis.

5.3.1 Problems

In this section, I point out some of the difficulties that arise in the context of index systems. As an example, I consider the dimension type system in style of Kennedy [Ken94] described as an HM(X) instance in Section 3.2. Kennedy gave the following example of a function for raising real numbers to integral powers:

```
fun power 0 x = 1.0
  | power n x = x*power (n-1) x
```

The dimension of the result depends on an integer value.

But his system does not deal with dependent types. Hence, the best type he can give is the following dimensionless type:

$$\mathsf{power} : \mathsf{int} \to \mathsf{dim}(1) \to \mathsf{dim}(1)$$

This situation looks familiar to the matrix multiplication example. In an extended dimension type system it would be possible to type the power function as follows:

$$\mathsf{power} : \mathsf{index}(\mathsf{n}) \to \mathsf{dim}(\mathsf{d}) \to \mathsf{dim}(\mathsf{d}^{\mathsf{n}})$$

I employ a finer–grained versions of integers called indices. The type $\mathsf{index}(n)$ indicates that the first parameter of the power function is an index type and even gives information about the value. This idea is further exemplified in the following section. Here, it is only important to point out that indices obey certain arithmetic laws. In general, arithmetic is undecidable unless we rely on variations such as Presburger arithmetic. This is an important issue and was already observed by

Zenger [Zen99a] and Xi and Pfenning [XP98]. In case of Presburger arithmetic it is crucial to deal only with liner equations. But quite easily we can end up in a system with non–linear equations. I repeat the addition function in the dimension calculus:

$$\mathsf{plus} : \forall d.\mathsf{dim}(d) \to \mathsf{dim}(d) \to \mathsf{dim}(d)$$

The combination of the plus and the power functions leads to non–linear equations. Consider the following program:

```
plus (power l x) (power m (power k x))
```

This program yield the constraint $(\mathsf{dim}(\alpha^l) = \mathsf{dim}((\alpha^k)^m))$ which results in the non–linear equation $(l = *(k, m))$. Unfortunately, solving such non–linear equations is undecidable. Hence, we end up in an undecidable type system.

I observe that extending the static error detecting abilites leads easily to an undecidable system. In type system research retaining a decidable system is an important issue. In the following section, I present a calculus to perform array–bounds checking. The array calculus is decidable and therefore has some obvious limitations.

5.3.2 Array–bounds checking

I employ a variation of the record calculus with polymorphic field labels to perform array–bounds checking. The basic idea is to represent arrays as a restricted form of records. Array indices are represented as field labels. The development here differs from previous approaches [Zen99a, XP98, JS96]. There, arrays were considered as lists attached with a length parameter, e.g. $\mathsf{List}\,\alpha\,\mathsf{index}(i)$. In contrast, I represent arrays as a special form of records. This shift implies that I can not rely on pattern

matching to iterate through lists. The size parameters of arrays are now implicit. I introduce primitives to retrieve the size information of an array. Then I introduce a new typing rule that allows to iterate through arrays. In this setting, I am not only interested in detecting array–bounds failure already at compile–time. Moreover, I enrich the type system to deal with poly–dimensional arrays. This allows us to write functions that take as arguments arrays of any dimension.

In my array calculus, I treat the array

$$[\text{"}b\text{"}, \text{"}d\text{"}, \text{"}x\text{"}]$$

as a short–hand for the record

$$\{1 = \text{"}b\text{"}, 2 = \text{"}d\text{"}, 3 = \text{"}x\text{"}\}$$

where index 1 has type $\mathsf{index}(1)$, 2 has type $\mathsf{index}(2)$ and so–forth. Silently, I assume that every arrays starts at position 1. Instead of labels, I use now indices which obey some arithmetic laws. The type language is as follows:

$$\begin{array}{lrcl} \textbf{Indices} & i & ::= & \alpha \mid +(i,i) \mid 0 \mid 1 \\ \textbf{Arrays} & r & ::\supseteq & \{l_1 : \tau_1, \ldots, l_n : \tau_n\} \\ \textbf{Types} & \tau & ::= & \alpha \mid \mathsf{index}(i) \mid r \mid \tau \to \tau \end{array}$$

The index constants 0 and 1 are constant type constructors and represent the corresponding values. Constructors $+(\cdot,\cdot)$ has the obvious meaning. The constructors obey the usual laws of arithmetic. This can be specified by constraint rules of the form:

$$\textbf{INDEX1} \quad \vdash^e \ (+(1,0) = 1)$$
$$\textbf{INDEX2} \quad \vdash^e \ (+(i_1,i_2) = +(i_2,i_1))$$
$$\textbf{INDEX3} \quad \vdash^e \ (+(+(i_1,i_2),i_3) = +(i_1,+(i_2,i_3)))$$

The kind of arithmetic I employ is very simple. There is only addition but for instance no multiplication. More complex versions of arithmetic are possible but one has to keep in mind that more expressive forms of arithmetics can possibly lead to an undecidable type system. For instance, Zenger [Zen99a] employs a Presburger arithmetic which is still decidable whereas Xi/Pfenning [XP98] use a richer form of arithmetic which is undecidable. For the purpose of this applicatio I want to keep the presentation simple. The task here is to present the main ideas of an instance of the HM(X) system which is capable to perform array–bounds checking.

Corresponding to the type constructors there are some value constructors. For constant type 0 there is a constant value $\mathsf{0}$ respectively the constant value $\mathsf{1}$ has the constant type 1. The names chosen for the constant values are irrelevant but I have chosen names that reflect the intended meaning. For the type constructor $+(\cdot,\cdot)$ there is the following primitive:

$$\mathsf{plus} : \forall i_1, i_2.\mathsf{index}(i_1) \to \mathsf{index}(i_2) \to \mathsf{index}(+(i_1,i_2))$$

I observe that indices appear now in two different contexts. As types and as terms. The properties of an index system are captured by constraint rules. The user has access to the index system via primitive terms, consider e.g. the constant value $\mathsf{1}$ or the primitive function plus.

Further predicates can be specified such as $(\cdot \leq \cdot)$ to specify an ordering relation

on indices.

Arrays are represented as a special kind of record. The array

$$[\text{"}b\text{"}, \text{"}d\text{"}, \text{"}x\text{"}]$$

has type

$$\{1 : \mathsf{String}, 2 : \mathsf{String}, 3 : \mathsf{String}\}.$$

where I assume that arrays are sorted with respect to their index. Furthermore, I assume that 2 is an abbreviation for $+(1, 1)$. Similar holds for 3 and so–forth.

To initialize arrays I write

$$\mathsf{a}[3] = [\text{"}b\text{"}, \text{"}d\text{"}, \text{"}x\text{"}].$$

The notation $\mathsf{a}[3]$ stands for a one–dimensional array whose first dimension has length 3. This notations extends to multi–dimensional arrays such as $\mathsf{a}[3][40]$. I overload notation and write e.g. $\mathsf{a}[1]$ to access the first element in array a. Usually, I use array notation but one has to be aware that essentially we deal with a special form of records. For instance, the array index operation $\mathsf{a}[i]$ corresponds to record selection, usually written as $\mathsf{a}.i$. I impose conditions on arrays to ensure that arrays are always regular. An array is *regular* if given an array a where $\mathsf{a}[i]$ and $\mathsf{a}[j]$ are also arrays then these arrays must have the same length. Again, this can be enforced by a constraint system. I assume that there is a primitive constraints of the form $\mathit{array}(\alpha, i, \beta)$ which denotes that array α at index i has type β which obeys the following rules:

$$\textbf{ARRAY1} \quad (i_1 \leq i_2) \wedge \ldots \wedge (i_{n-1} \leq i_n) \vdash^e \mathit{array}(\{i_1 : \tau, \ldots i_n : \tau\}, i_i, \tau)$$

$$\textbf{ARRAY2} \quad \mathit{array}(\alpha, i, \beta) \wedge \mathit{array}(\alpha, j, \gamma) \vdash^e (\beta = \gamma)$$

Rule **ARRAY2** enforces regularity. Primitives like the array index operation are typed as follows:

$$\cdot[\cdot] : \forall \alpha, i, \beta. \mathit{array}(\alpha, i, \beta) \Rightarrow \alpha \rightarrow \mathsf{index}(i) \rightarrow \beta$$

In order to do something meaningful with this array calculus we need some further primitives such as accessing the length of an array. The properties of the lenght function are described by the primitive constraint $\mathit{length}(\cdot)$ which denotes the length of an array. The rules for $\mathit{length}(\cdot)$ are straightforward.

$$\textbf{LEN1} \quad \vdash^e (\mathit{length}(\{i_1 : \tau, \ldots, i_n : \tau\}) = i_n)$$

$$\textbf{LEN2} \quad (\mathit{length}(\alpha) = i) \wedge (\mathit{length}(\alpha) = i') \vdash^e (i = i')$$

Then, I can give the following primitive function:

$$\mathsf{length} : \forall \alpha, n.(l = \mathit{length}(\alpha)) \Rightarrow \alpha \rightarrow \mathsf{index}(l)$$

The type ensures that the implementation satisfies the desired behavior. But still, we are not in the position to write interestng programs in our array calculus. Assume I want to write a function which interates through a list given the lowest

and the highest index. The approaches by [XP98, Zen99a] employ pattern matching to check whether one has already reached the end of the list or not. In contrast, I employ primitive functions to check such properties. Then the term $\mathsf{length}\ \mathsf{x} = 1$ states that array x has length 1. Depending on whether this is true or not I want to execute a specific branch. I want to write programs of the following form:

$$\begin{array}{ll} \mathsf{case} & \mathsf{length}\ \mathsf{x} = 1 \Rightarrow e_1 \\ & \mathsf{length}\ \mathsf{x} > 1 \Rightarrow e_n \end{array}$$

The terms $\mathsf{length}\ \mathsf{x} = 1$ and $\mathsf{length}\ \mathsf{x} > 1$ can be seen as a form of *guards*. The idea is that a guard ensures that one safely can execute the corresponding branch. In contrast to the $\mathsf{if–then–else}$ statement I want to be more specific under which conditions a guard holds. Usually, guards are represented by boolean values. The idea is to use a finer–grained kind of booleans. Such a treatment can already be found in [Zen99a]. In the following, I borrow ideas from Zenger's thesis. He introduces an additional type constructor $\mathsf{bool}(\cdot)$ which takes constraints between indices as arguments. Then the type $\mathsf{bool}(\mathsf{true})$ describes then true values. For instance, the equality function between indices is typed as follows in Zenger's system:

$$= \ : \ \forall\alpha,\beta.\mathsf{index}(\alpha) \to \mathsf{index}(\beta) \to \mathsf{bool}((\alpha = \beta))$$

That means, in Zengers system constraints appear within types. This implies that it becomes necessary to quantify suddenly over constraints, e.g. consider typing of the conjunctive function.

$$\mathsf{and} : \ : \ \forall\alpha,\beta.\mathsf{bool}(\alpha) \to \mathsf{bool}(\beta) \to \mathsf{bool}(\alpha \wedge \beta)$$

where α and β range over constraints. His system still remains decidable because he only quantifies over constraints containing indices. In the HM(X) system, constraints are not allowed to appear within types. Hence, I need to find a way to express something similar. In the following I describe a method to express properties of index constraints in the type language to stay within the HM(X) framework. This allows to express similar properties as Zenger.

First, I additionally introduce booleans to the type language.

$$\begin{array}{lrcl} \textbf{Booleans} & b & ::= & \alpha \mid \mathsf{T} \mid \mathsf{F} \\ \textbf{Types} & \tau & ::= & \alpha \mid \mathsf{index}(i) \mid r \mid \mathsf{bool}(b) \mid \tau \to \tau \end{array}$$

The type $\mathsf{bool}(\mathsf{T})$ represents now true values. Zenger's type language contains types of the form $\mathsf{bool}((\alpha = \beta))$. Such a type can not directly be represented in the HM(X) system. But I can express something similar. I introduce a predicate $(\cdot < \cdot \mapsto \cdot)$ which takes three parameters. The first two parameters are indices and the last one is a boolean. The index parameters can be seen as the input and the boolean parameter as the output parameters. The predicate $(\alpha < \beta \mapsto \gamma)$ states that depending whether or not index α is smaller than index β, boolean γ is either true or false. Then the type $\forall\gamma.(\alpha < \beta \mapsto \gamma) \Rightarrow \mathsf{bool}(\gamma)$ corresponds to the type $\mathsf{bool}((\alpha = \beta))$ in Zenger's system. I assume there are two further predicates $(\cdot = \cdot \mapsto \cdot)$ and $(\cdot > \cdot \mapsto \cdot)$. The constraint rules are of the form:

$$\textbf{SMALL1} \quad \vdash^e (0 < 1 \mapsto \mathsf{T})$$

$$\textbf{SMALL2} \quad (\alpha < \beta \mapsto \mathsf{T}) \vdash^e (\alpha < +(\beta, \beta') \mapsto \mathsf{T})$$

$$\textbf{SMALL3} \quad (\alpha = \beta \mapsto \mathsf{T}) \vdash^e (\alpha < \beta \mapsto \mathsf{F})$$

$$\textbf{SMALL4} \quad (\alpha > \beta \mapsto \mathsf{T}) \vdash^e (\alpha < \beta \mapsto \mathsf{F})$$

$$\textbf{GREAT1} \quad \vdash^e (1 > 0 \mapsto \mathsf{T})$$

$$\textbf{GREAT2} \quad (\alpha > \beta \mapsto \mathsf{T}) \vdash^e (+(\alpha, \alpha') < \beta \mapsto \mathsf{T})$$

$$\textbf{GREAT3} \quad (\alpha = \beta \mapsto \mathsf{T}) \vdash^e (\alpha > \beta \mapsto \mathsf{F})$$

$$\textbf{GREAT4} \quad (\alpha < \beta \mapsto \mathsf{T}) \vdash^e (\alpha > \beta \mapsto \mathsf{F})$$

$$\textbf{EQUAL1} \quad \vdash^e (\alpha = \alpha \mapsto \mathsf{T})$$

$$\textbf{EQUAL2} \quad (\alpha < \beta \mapsto \mathsf{T}) \vdash^e (\alpha = \beta \mapsto \mathsf{F})$$

$$\textbf{EQUAL3} \quad (\alpha > \beta \mapsto \mathsf{T}) \vdash^e (\alpha = \beta \mapsto \mathsf{F})$$

$$\textbf{EQUAL4} \quad (\alpha = \beta \mapsto \gamma) =^e (\beta = \alpha \mapsto \gamma)$$

In case of $(\cdot < \cdot \mapsto \cdot)$, rules **SMALL1** and **SMALL2** express when an index is acutally smaller than another index. Rules **SMALL3** and **SMALL4** cover the failure cases. The rules for the other predicates are similar.

Then, I can type the is–smaller, is–equal and is–greater functions between indices as follows:

$$\begin{aligned} < &: \forall\alpha, \beta, \gamma.(\alpha < \beta \mapsto \gamma) \Rightarrow \mathsf{index}(\alpha) \to \mathsf{index}(\beta) \to \mathsf{bool}(\gamma) \\ = &: \forall\alpha, \beta, \gamma.(\alpha = \beta \mapsto \gamma) \Rightarrow \mathsf{index}(\alpha) \to \mathsf{index}(\beta) \to \mathsf{bool}(\gamma) \\ > &: \forall\alpha, \beta, \gamma.(\alpha > \beta \mapsto \gamma) \Rightarrow \mathsf{index}(\alpha) \to \mathsf{index}(\beta) \to \mathsf{bool}(\gamma) \end{aligned}$$

The result type $\mathsf{bool}(\gamma)$ signals whether indexα is smaller (equal or greater) than β or not. The connection is established through the constraint $(\alpha < \beta \mapsto \gamma)$.

It remains to define the conjunctive and disjunctive primitives. The development

is similar to the comparison primitives. The predicate $(\cdot \wedge \cdot \mapsto \cdot)$ takes two booleans as inputs and reports one boolean as an output. The constraint rules are as follows:

$$\textbf{AND1} \quad \vdash^e (\mathsf{T} \wedge \mathsf{T} \mapsto \mathsf{T})$$

$$\textbf{AND2} \quad (\alpha \wedge \beta \mapsto \gamma) =^e (\beta \wedge \alpha \mapsto \gamma)$$

$$\textbf{AND3} \quad (\alpha \wedge \beta \mapsto \gamma) \wedge (\gamma \wedge \delta \mapsto \epsilon) =^e (\alpha \wedge \delta' \mapsto \epsilon) \wedge (\beta \wedge \delta \mapsto \delta')$$

Rule **AND1** handles the base case. Rules **AND2** and **AND3** state commutativity and associativity laws. In a similar way, a predicate $(\cdot \vee \cdot \mapsto \cdot)$ can be defined. The conjunctive and disjunctive primitives are as follows:

$$\mathsf{and} \quad : \quad \forall \alpha, \beta, \gamma.(\alpha \wedge \beta \mapsto \gamma) \Rightarrow \mathsf{bool}(\alpha) \to \mathsf{bool}(\beta) \to \mathsf{bool}(\gamma)$$

$$\mathsf{or} \quad : \quad \forall \alpha, \beta, \gamma.(\alpha \vee \beta \mapsto \gamma) \Rightarrow \mathsf{bool}(\alpha) \to \mathsf{bool}(\beta) \to \mathsf{bool}(\gamma)$$

The following case rule was motivated by a discussion with Zenger [Zen99a] who also pointed out problems in earlier formulations:

$$\text{(Case)} \quad \frac{\begin{array}{c} D_i, \Gamma \vdash g_i : \mathsf{bool}(\gamma_i) \qquad D_i \wedge D, \Gamma \vdash e_i : \tau \qquad \gamma_i \notin fv(\tau, \Gamma) \\ C = \bigwedge_i D_i \wedge D \wedge (\gamma_1 \vee \ldots \vee \gamma_n \mapsto \mathsf{T}) \end{array}}{\exists \bar{\gamma}.C, \Gamma \vdash \begin{array}{ll} \mathsf{case} & g_1 \Rightarrow e_1 \\ & \ldots \\ & g_n \Rightarrow e_n \quad : \tau \end{array}}$$

The constraint $(\gamma_1 \vee \ldots \vee \gamma_n \mapsto \mathsf{T})$ states that at least one guard has to be true. The predicate is a generalization of the tenary predicate $(\cdot \vee \cdot \mapsto \cdot)$. The side condition $\gamma_i \notin fv(\tau)$ is actually not necessary. By using only the provided primitive functions on indices this invariant can always be achieved. Because type variables

γ_i's only appear in the constraint part of the conclusion, I can apply the projection operator which yields the constraint $\exists\bar{\gamma}.C$. This furthermore allows to represent the guard constraints in a more readable form, e.g. the constraint $\exists\gamma.(\alpha < \beta \mapsto \gamma)$ is semantically equivalent to the constraint $(\alpha \leq \beta)$. Predicates such as $(\cdot = \cdot \mapsto \cdot)$ and $(\cdot \vee \cdot \mapsto \cdot)$ are only necessary because it is not possible to represent constraints in the type language. Semantically, both forms are equivalent. Henceforth, I will always use the simpler representation.

Soundness for this rule is discussed in Appendix A.4. The type soundness result can be extended to the array calculus and yields the following theorem.

Theorem 24 *Given a well–typed array program. Then there will be no out–of bounds failures at run–time.*

As already pointed out in the previous sections, there are certain limitations, i.e. there are array programs which can not be statically checked.

♠Limitations, which errors can't we catch at compile–time?♣

But there are still enough interesting programs which can be statically checked. As an example, I consider a function that allows to iterate through an array. This is similar to the fold function.

$$\begin{aligned}
&\mathsf{arrfold} :: \forall, \alpha, \beta, \gamma, i.\mathit{array}(\alpha, i, \beta) \wedge (1 \leq i) \wedge (i \leq \mathit{length}(\alpha)) \Rightarrow \\
&\qquad \alpha \to \mathsf{index}(i) \to \gamma \to (\beta \to \gamma \to \gamma) \to \gamma \\
&\mathsf{arrfold\ x\ i\ y\ g} = \\
&\quad \mathsf{case\ i} = 1 \Rightarrow \mathsf{y} \\
&\qquad \mathsf{i} = \mathsf{lenght\ x} \Rightarrow \mathsf{y} \\
&\qquad \mathsf{i} < \mathsf{length\ x} \Rightarrow \mathsf{arrfold\ x\ (plus\ i\ 1)\ (g\ x[i]\ y)\ g}
\end{aligned}$$

Here, the constraints $(1 \leq i) \wedge (i \leq \mathit{length}(\alpha))$ represents the constraint

$$\exists\bar{\gamma}.(1 = i \mapsto \gamma_1) \wedge (i = \mathit{length}(\alpha) \mapsto \gamma_2) \wedge (i < \mathit{length}(\alpha) \mapsto \gamma_3) \wedge (\gamma_1 \vee \gamma_2 \vee \gamma_3 \mapsto \mathsf{T})$$

The constraint $(\gamma_1 \vee \gamma_2 \vee \gamma_3 \mapsto \mathsf{T})$ ensures that the constraint is satisfiable. This the miminal conditions put on constraints appearing in typing judgments and type schemes.

In order to type the third branch, I need some further constraint rules such as

$$\begin{array}{rcl} (\alpha < \beta) & \vdash^e & (+(\alpha, 1) \leq \beta) \\ (i < \mathit{length}(\alpha)) \wedge \mathit{array}(\alpha, i, \beta) & \vdash^e & \mathit{array}(\alpha, +(i, 1), \beta) \end{array}$$

When typing arrfold x (plus i 1) (g x[i] y) g the constraint part of arrfold's type gets instantiated to

$$\mathit{array}(\alpha, +(i, 1), \beta) \wedge (1 \leq +(i, 1)) \wedge (+(i, 1) \leq \mathit{length}(\alpha))$$

On the left–hand side of the typing judgment we find the constraint

$$\mathit{array}(\alpha, i, \beta) \wedge (1 \leq i) \wedge (i \leq \mathit{length}(\alpha))$$

which entails the above constraint under the given constraint rules.

Given an array x of integers. Then

```
f x = arrfold x 1 x[1] max
```

computes the maximum value of array x where max is a primitive to compute the maximum of two integer values. Remember, I assume that the array starts at position 1.

Another important observation is that I need to annotate arrfold with its type because we are faced with polymorphic recursion [Hen93] for which type inference is undecidable in general. Therefore, I require to annotate recursive functions with its type. In Appendix A.3 I consider an extension of the HM(X) system with the polymorphic recursion rule.

The inference problem turns out to be a hard one. In Appendix A.4, I give some initial results which let me conjecture that under certain assumptions type inference is feasible. It remains to be seen whether it is possible to adopt inference techniques described in [Zen99a].

The purpose of the array calculus is to detect more errors already at compile–time which have been previously postponed to run–time. But my treatment of arrays even allows to deal with poly–dimensional arrays. That means, I am able to write functions that take arrays but also arrays of arrays and so–forth as arguments. Hence, we can type more programs statically.

I introduce some additional primitives to type poly–dimensional programs. I already introduced the primitive constraint $\mathit{length}(\cdot)$ which denotes the length of an array. Additionally, I introduce the primitive constraint $\mathit{dim}(\cdot)$ which denotes the dimension of an array.

$$\textbf{DIM1} \quad \mathit{array}(\alpha, i, \mathsf{String}) \vdash^e (1 = \mathit{dim}(\alpha))$$

$$\textbf{DIM2} \quad \begin{array}{c} (d = \mathit{dim}(\beta)) \wedge \mathit{array}(\alpha, i, \beta) \wedge \mathit{array}(\beta, j, \gamma) \\ \vdash^e \\ (+(d, 1) = \mathit{dim}(\alpha)) \end{array}$$

The first rule covers the base case for a one–dimensional array of strings. Rules for arrays of ints, booleans, etc are similar. The second rule defines inductively how to

compute the dimensions for multi–dimensional arrays. I add the following primitive to compute the dimension of an array to the initial type environment Γ_0.

$$\mathsf{dim} : \forall\alpha, n.(d = \mathit{dim}(\alpha)) \Rightarrow \alpha \rightarrow \mathsf{index}(d)$$

I discuss now the expressiveness of the array calculus which allows to deal with poly–dimensional arrays. To my knowledge only the system by Jay [Jay98] allows to type programs that deal with poly–dimensional arrays. First, I introduce one further primitive constraint $\mathit{basetype}(\cdot)$ to compute the base type of a given array.

$$\textbf{BASE1} \quad \frac{\mathit{array}(\alpha, i, \beta) \wedge (\mathit{dim}(\alpha) = 1)}{\vdash^e \quad (\beta = \mathit{basetype}(\alpha))}$$

$$\textbf{BASE2} \quad \frac{\mathit{array}(\alpha, i, \beta) \wedge \mathit{array}(\beta, j, \gamma)}{\vdash^e \quad (\mathit{basetype}(\alpha) = \mathit{basetype}(\beta))}$$

As an example, I consider the following function maxpoly which computes the maximum of a poly–dimensional array whereas maxone computes the maximum of a one–dimensional array. I consider poly–dimensional arrays which have integers as its base type.

```
maxpoly :: ∀α, β, i, γ.array(α, i, β)) ∧ (Int = basetype(α))∧
           (1 ≤ dim α) ⇒ α → Int
maxpoly x =
  case dim x = 1 ⇒ maxone x
       dim x > 1 ⇒ arrfold x 1 (maxpoly x[1]) g
```

$$\mathsf{where\ g\ y\ z} = \mathsf{max\ (maxpoly\ y)\ z}$$

$$\mathsf{maxone} :: \forall\alpha, i.\, array(\alpha, i, \mathsf{Int}) \Rightarrow \alpha \rightarrow \mathsf{Int}$$

$$\mathsf{maxone\ x} = \mathsf{arrfold\ x\ 1\ x[1]\ max}$$

Typing of the two functions require further constraint rules. In case of maxone I need the two further rules

$$\begin{aligned} array(\alpha, i, \beta) \quad &\vdash^e \quad array(\alpha, 1, \beta) \\ &\vdash^e \quad (1 \leq length(\alpha)) \end{aligned}$$

In case of maxpoly I need additionally the rule

$$array(\alpha, i, \beta) \wedge (dim(\alpha) = 1) \wedge (\mathsf{Int} = basetype(\alpha)) \quad \vdash^e \quad array(\alpha, i, \mathsf{Int})$$

Other examples of poly–dimensional arrays include solving of equations and similar problems, see [Jay98]. Such a treatment can not be found in [Zen99a, XP98, JS96]. I furthermore observe that the given constraint rules might not be complete to type further programs. In case of the arrfold and the maxpoly function I had to add new constraint rules to make typing possible. The added constraint rules were not surprising but it is not obvious which additionally rules might still be necessary.

The exercise here was to show that based on the HM(X) system, it is possible to develop an array calculus which allows to detect more errors at compile–time and even makes it possible to statically type programs which deals with poly–dimensional arrays. It proved to be necessary to add the case rule to the logical typing rules to allow typing of interesting programs. The HM(X) system proved to be flexible enough to allow formulation of the case rule. The type soundness result could be reestablished whereas the type inference problem turned out to be a hard one.

Chapter 6

Conclusion

6.1 Achievements

One of the main goals of my dissertation was to come up with the cleanest possible notation to achieve the most general results, following Einstein's principle: keep it as simple as possible but not simpler.

I provided a general framework HM(X) to design Hindely/Milner style type systems in a concise way. The design of a new type system (or static analysis) starts with the definition of the appropriate domain of types and constraints. Basic operations can then be added as primitive functions. For details see Section 3.2. Type system instances of HM(X) are sound provided the underlying constraint system is sound. See Section 3.3 for details. HM(X) can be seen as the proper logical framework to deal with constraints and Hindley/Milner style polymorphism. This issue is studied in Section 3.5. Constraints improve over types as shown in Section 3.6. For this purpose, I provided a purely constraint–based formulation of the HM(X) framework in Section 3.7. Furthermore, I provided a generic inference system. Principal types can be inferred if the constraint system satisfies the principal constraint property. I

introduced various inference systems and proved soundness as well as completeness results. For an overview of the achieved results see Section 4.5. The main novelty is a constraint–based inference system which improves significantly over previous formulations. Specifically, I solve the open problem of type inference for non–regular theories. In addition, I showed that several variations of the Hindley/Milner type system can be seen as instances of the HM(X) framework. I focused on applications which deal with records. One novelty is a record application which allows concatenation of records and still achieves principal types based on unification. See Chapter 5 for details.

6.2 Implementation

In Appendix D, I give a specification of the generic HM(X) inference system in terms of the functional language Haskell. The goal was rather to provide a rigorous specification of the essential inference components than to provide an optimized implementation. A similar goal was pursued in case of Haskell's type system [Jon99]. The applicability of the HM(X) systems was demonstrated in [PS99] and [Zen99b]. In the first paper, the authors demonstrated how some of the ideas presented in my dissertation can be applied to other settings such as architecture description languages [GMW97]. The other paper provides an implementation of the dimension analysis in Pizza [OW97] which is an extension of the Java language [GJS96] with polymorphism, higher–order types and algebraic data types.

An efficient implementation method for general HM(X) application is still missing. Efficient compilation methods are available for more specific applications such as records [Oho95, Gas98]. I believe it is rather difficult to give an efficient compilation method for general HM(X) applications.

I believe that an efficient compilation method is less important in the light of applying the HM(X) systems to perform static analyses such as dimension analysis. Modelling static analyses in terms of the HM(X) system is a promising area for future work.

6.3 Future Work

Recently, Foster, Fähndrich and Aiken [FFA99] described a framework for adding type qualifiers to a language. An example of a type qualifier form the C world is the ANSI C qualifier `const`. Yet another example of type qualifiers comes from binding–time analysis [DHM95]. There, qualifier `static` denotes values know at compile–time whereas `dynamic` denotes run–time values. While Foster, Fähndrich and Aiken have not yet formalized their system in terms of the HM(X) system, their inference system already borrows ideas from the HM(X) system. I believe their whole framework can be readily described in terms of the HM(X) system.

Another interesting topic for future work is to look at systems which deal with indexed data structures. I am most interested in systems which deal with indexed data structures such as arrays and matrices. Previous approaches mostly rely on a form of dependent types [XP98, Zen99a] to deal with indexed data structures. The concept of dependent types allows a consideration of types which not only have type parameters, but additionally have value parameters, examples being matrices and arrays. It remains to be seen whether a constraint–based formulation in terms of HM(X) has advantages.

It is also interesting to extend the HM(X) framework to richer type systems, incorporating advanced features such as existential types and first–class polymorphism. This seems to be especially necessary when modelling index systems [XP98, Zen99a].

Bibliography

[AW93] A. Aiken and E. L. Wimmers. Type inclusion constraints and type inference. In *FPCA '93: Conference on Functional Programming Languages and Computer Architecture, Copenhagen, Denmark*, pages 31–41, New York, June 1993. ACM Press.

[BS94] F. Baader and J.H. Siekmann. Unification theory. In D.M. Gabbay, C.J. Hogger, and J.A. Robinson, editors, *Handbook of Logic in Artificial Intelligence and Logic Programming*, pages 41–125. Oxford University Press, Oxford, UK, 1994.

[Car84] Luca Cardelli. A semantics of multiple inheritance. In Gilles Kahn, David B. MacQueen, and Gordon D. Plotkin, editors, *Semantics of Data Types*, pages 51–67. Springer-Verlag, June 1984. Lecture Notes in Computer Science 173.

[Car97] Cardelli. Type systems. In Jr. Allen B. Tucker, editor, *The Computer Science and Engineering Handbook*. CRC Press, 1997.

[CM89] Luca Cardelli and John C. Mitchell. Operations on records. In M. Main, A. Melton, M. Mislove, and David Schmidt, editors, *Proceedings Mathematical Foundations of Programming Semantics, 5th International Conference, Tulane University, New Orleans, Louisiana, USA*, pages 22–52.

Springer-Verlag, March/April 1989. Lecture Notes in Computer Science 442.

[Dam85] L. Damas. *Type Assignment in Programming Languages*. PhD thesis, Edinburgh University, 1985.

[DH95] K. Driesen and U. Hoelzle. Minimizing row displacement dispatch tables. *ACM SIGPLAN Notices*, 30(10):141–155, October 1995.

[DHM95] Dirk Dussart, Fritz Henglein, and Christian Mossin. Polymorphic binding-time analysis in polynomial time. In *Proceedings of SAS*, pages 118–135. Springer Verlag, September 1995.

[DM82] L. Damas and R. Milner. Principal type-schemes for functional programs. In *Conference Record of the Ninth Annual ACM Symposium on Principles of Programming Languages*, pages 207–212. ACM, ACM, January 1982.

[Dri93] Karel Driesen. Selector table indexing and sparse arrays. In Andreas Paepcke, editor, *Proceedings of the 8th Annual Conference on Object-Oriented Programming Systems, Languages and Applications*, pages 259–270, Washington, DC, USA, September 26October –1 1993. ACM Press.

[EST95a] J. Eifrig, S. Smith, and V. Trifonov. Sound polymorphic type inference for objects. In *OOPSLA '95 Conference Proceedings*, volume 30(10) of *ACM SIGPLAN Notices*, pages 169–184, 1995.

[EST95b] Jonathan Eifrig, Scott Smith, and Valery Trifonov. Type inference for recursively constrained types and its application to object oriented programming. In *Electronic Notes in Theoretical Computer Science*, volume 1, 1995.

[FFA99] Jeffrey S. Foster, Manuel Fähndrich, and Alexander Aiken. A theory of type qualifiers. *ACM SIGPLAN Notices*, 34(5):192–203, May 1999. Proceedings of the 1999 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI).

[Gas98] Benedict R. Gaster. *Records, variants, and qualified types*. PhD thesis, Computer Science, University of Nottingham, July 1998.

[GJS96] James Gosling, Bill Joy, and Guy Steele. *The Java Language Specification*. Java Series, Sun Microsystems, 1996. ISBN 0-201-63451-1.

[GMW97] David Garlan, Robert T. Monroe, and David Wile. ACME: An Architecture Description Interchange Language. In *Proceedings of CASCON'97*, pages 169–183, Toronto, Ontario, November 1997.

[Hen93] Fritz Henglein. Type inference with polymorphic recursion. *ACM Transactions on Programming Languages and Systems*, 15(1):253–289, April 1993.

[HMT71] L. Henkin, J.D. Monk, and A. Tarski. *Cylindric Algebra*. North-Holland Publishing Company, 1971.

[HP90] Robert W. Harper and Benjamin C. Pierce. Extensible records without subsumption. Technical Report CMU-CS-90-102, School of Computer Science, Carnegie Mellon University, Feburary 1990.

[HP91] Robert Harper and Benjamin Pierce. A record calculus based on symmetric concatenation. In *Conference Record of the Eighteenth Annual ACM Symposium on Principles of Programming Languages, Orlando, Florida*, pages 131–142. ACM Press, January 1991.

[HW73] C. A. R. Hoare and Niklaus Wirth. An axiomatic definition of the programming language Pascal. *Acta Informatica*, 2:335–355, 1973.

[Jay98] C.B. Jay. Poly-dimensional regular arrays in fish, April 1998. submitted to ICFP'98.

[JM94] Joxan Jaffar and Michael Maher. Constraint logic programming: A survey. *Journal of Logic Programming*, 19(20):503–581, 1994.

[Jon92] Mark P. Jones. *Qualified Types: Theory and Practice*. D.phil. thesis, Oxford University, September 1992.

[Jon99] Mark P. Jones. Typing Haskell in Haskell. In *Haskell Workshop*, September 1999.

[JS96] C.B. Jay and M. Sekanina. Shape checking of array programs. Technical Report 96.09, University of Technology, Sydney, 1996.

[Kae92] S. Kaes. Type inference in the presence of overloading, subtyping and recursive types. In *Proceedings of the 1992 ACM Conference on LISP and Functional Programming*, pages 193–204, 1992.

[Ken94] A. Kennedy. Dimension types. In Donald Sannella, editor, *Programming Languages and Systems—ESOP'94, 5th European Symposium on Programming*, volume 788 of *Lecture Notes in Computer Science*, pages 348–362, Edinburgh, U.K., 11–13 April 1994. Springer.

[Ken96] A. J. Kennedy. Type inference and equational theories. Technical Report LIX/RR/96/09, LIX, Ecole Polytechnique, 91128 Palaiseau Cedex, France, September 1996.

[KMM91] P.C. Kanellakis, H.G. Mairson, and J.C. Mitchell. Unification and ML type reconstruction. In *Computational Logic, Essays in Honor of Alan Robinson*, pages 444–478. MIT Press, 1991.

[LBB84] D. S. Lankford, G. Butler, and B. Brady. Abelian group unification algorithms for elementary terms. In W. Bledsoe and W. Loveland, editors, *Automated Theorem Proving: After 25 Years*. AMS, 1984.

[LMM87] J. Lassez, M. Maher, and K. Marriott. Unification revisited. In J. Minker, editor, *Foundations of Deductive Databases and Logic Programming*. Morgan Kauffman, 1987.

[Mil78] R. Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17:348–375, Dec 1978.

[MPS86] D. MacQueen, G. Plotkin, and R. Sethi. An ideal model for recursive polymorphic types. *Information and Control*, 71:95–130, 1986.

[MTHM97] Robin Milner, Mads Tofte, Robert Harper, and David MacQueen. *The Definition of Standard ML (Revised)*. The MIT Press, 1997.

[Nis98] Susumu Nishimura. Static typing for dynamic messages. In *Conference Record of POPL '98: 25th Annual ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages, San Diego, Calif.* ACM, January 1998.

[NP95] T. Nipkow and C. Prehofer. Type reconstruction for type classes. *Journal of Functional Programming*, 5(2):201–224, 1995.

[Oho95] A. Ohori. A polymorphic record calculus and its compilation. *ACM TOPLAS*, 6(6):805–843, November 1995.

[Oho97] A. Ohori. Type system for specializing polymorphism. *Lecture Notes in Computer Science*, 1281:107–122, 1997.

[OL96] Martin Odersky and Konstantin Läufer. Putting type annotations to work. In *Proc. 23rd ACM Symposium on Principles of Programming Languages (POPL)*, pages 65–67, January 1996.

[OSW99] M. Odersky, M. Sulzmann, and M Wehr. Type inference with constrained types. *Theory and Practice of Object Systems*, 5(1):35–55, 1999.

[OW97] Martin Odersky and Philip Wadler. Pizza into Java: Translating theory into practice. In *Conference Record of POPL '97: The 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 146–159, Paris, France, 15–17 January 1997.

[OWW95] M. Odersky, P. Wadler, and M. Wehr. A second look at overloading. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture (FPCA'95)*, pages 135–146, La Jolla, California, June 25–28, 1995. ACM SIGPLAN/SIGARCH and IFIP WG2.8, ACM Press.

[PH97] John Peterson and Kevin Hammond. Report on the programming language Haskell: a non-strict, purely functional language, version 1.4. Technical Report YALEU/DCS/RR-1106, Yale University Department of Computer Science, March 1997.

[Pot98] Francois Pottier. A framework for type inference with subtyping. In *Proceedings of the third ACM SIGPLAN International Conference on Functional Programming (ICFP '98)*, pages 228–238, September 1998.

[PS99] John Peterson and Martin Sulzmann. Analysis of architectures using constraint–based types. In *WICSA1 First Working IFIP Conference on Software Architecture*, February 1999. position paper.

[Rém89] Didier Rémy. Typechecking records and variants in a natural extension of ML. In *Conference Record of POPL '89: The 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, January 1989.

[Rém92a] D. Rémy. Extending ML type system with a sorted equational theory. Technical Report 1766, INRIA, October 1992.

[Rem92b] D. Remy. Typing record concatenation for free. In ACM, editor, *Conference record of the Nineteenth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages: papers presented at the symposium, Albuquerque, New Mexico, January 19–22, 1992*, pages 166–176, New York, NY, USA, 1992. ACM Press.

[Rém93] Didier Rémy. Type inference for records in a natural extension of ML. In Carl A. Gunter and John C. Mitchell, editors, *Theoretical Aspects Of Object-Oriented Programming. Types, Semantics and Language Design.* MIT Press, 1993.

[Rém95a] Didier Rémy. A case study of typechecking with constrained types: Typing record concatenation. Presented at the workshop on *advances in types for computer science* at the Newton Institute, Cambridge, UK. Available electronically at `http://pauillac.inria.fr/~remy`, August 1995.

[Rém95b] Didier Rémy. Refined subtyping and row variables for record types. Presented at the workshop on *advances in types for computer science* at the Newton Institute, Cambridge, UK. Avaliable electronically at `html://pauillac.inria.fr/~remy`, August 1995.

[Rob65] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the Association for Computing Machinery*, 12:23–41, 1965.

[Sar93] Vijay A. Saraswat. *Concurrent Constraint Programming.* Logic Programming Series, ACM Doctoral Dissertation Award Series. MIT Press, Cambridge, Massachusetts, 1993.

[Smi91] Geoffrey S. Smith. *Polymorphic type inference for languages with overloading and subtyping.* PhD thesis, Cornell University, Ithaca, NY, August 1991.

[SOW97] Martin Sulzmann, Martin Odersky, and Martin Wehr. Type inference with constrained types. In *FOOL4: 4th. Int. Workshop on Foundations of Object-oriented programming Languages*, January 1997.

[Str67] Christopher Strachey. Fundamental concepts in programming languages. unpublished lecture notes, August 1967.

[Tre93] Ralf Treinen. Feature constraints with first-class features. In Andrzej M. Borzyszkowski and Stefan Sokołowski, editors, *Mathematical Foundations of Computer Science*, Lecture Notes in Artificial Intelligence, vol. 711, pages 734–743, Gdańsk, Poland, 30 August–3 September 1993. Springer-Verlag.

[TS96] Valery Trifonov and Scott Smith. Subtyping constrained types. In *Proceedings of the Third Internationl Static Analysis Symposium*, volume 1145 of *LNCS*, pages 349–365, 1996.

[VHJW96] Cordelia V.Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in Haskell. *ACM TOPLAS*, 18(2):109–138, March 1996.

[Wan87] Mitchell Wand. A simple algorithm and proof for type inference. In *Fundamenta Informaticae X*, pages 115–122. North-Holland, 1987.

[Wan88] Mitchell Wand. Corrigendum: Complete type inference for simple objects. In *Proceedings of the IEEE Symposium on Logic in Computer Science*, page 132, 1988.

[Wan89] Mitchell Wand. Type inference for record concatenation and multiple inheritance. In *Proceedings of the IEEE Symposium on Logic in Computer Science*, pages 92–97, June 1989.

[Wan93] Mitchell Wand. Type inference for record concatenation and multiple inheritance. *Information and Computation*, pages 1–15, 93.

[Weh97] Martin Wehr. *Überladung in Typsystemen.* PhD thesis, Universität Karlsruhe, October 1997.

[WF91] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. Technical Report Rice COMP TR91-160, Rice University, Houston, Texas, 1991.

[XP98] Hongwei Xi and Frank Pfenning. Eliminating array bound cecking through dependent types. In *Proceedings of the ACM SIGPLAN '98*

Conference on Programming Language Design and Implementation, New York, June 1998. ACM Press.

[Zen99a] Christoph Zenger. *Indizierte Typen*. PhD thesis, Universität Karlsruhe, 1999.

[Zen99b] Matthias Zenger. Implementing equational theories in Pizza, August 1999. personal communication.

Appendix A

Extensions

A.1 Extensible Records and Polymorphic Field Labels

The predicate $extend(\cdot,\cdot,\cdot,\cdot)$ takes now as an additional parameter a field label. The constraint rules are as follows:

REC1 $\vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\})$
where $l_1, \ldots, l_n$ are distinct

REC2 $(\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2)$

REC3 $extend(l, \alpha, \beta, \gamma) \vdash^e (\gamma :: \{\!\{l : \beta\}\!\})$

REC4 $extend(l, \alpha, \beta, \gamma) \wedge (\alpha :: \{\!\{l' : \tau\}\!\}) \vdash^e (\gamma :: \{\!\{l' : \tau\}\!\})$

REC5 $\vdash^e extend(l_{n+1}, \{l_1 : \tau_1, \ldots, l_n : \tau_n\}, \tau, \{l_1 : \tau_1, \ldots, l_n : \tau_n, l_{n+1} : \tau\})$
where $l_1, \ldots, l_{n+1}$ are distinct

The primitive functions for record extension is now written as follows:

$$extend : \forall \alpha, \beta, \gamma, l.extend(l, \alpha, \beta, \gamma) \Rightarrow label(l) \to \alpha \to \beta \to \gamma$$

The set $\mathcal{S}$ of solved forms contains all satisfiable constraints. It is straightforward to prove that satisfiability is decidable. I leave the question open whether more a restricted set of solved forms still allows to retain principality of the constraint system.

A.2 Symmetric Record Concatenation

I give a variation REC^{c^s} of REC^c where record concatenation is symmetric. The constraint rules are as follows:

$$
\begin{array}{ll}
\textbf{REC1} & \vdash^e (\{l_1 : \tau_1, \ldots l_n : \tau_n\} :: \{\!\{l_i : \tau_i\}\!\}) \\
 & \text{where } l_1, \ldots, l_n \text{ are distinct} \\
\textbf{REC2} & (\tau :: \{\!\{l : \tau_1\}\!\}) \wedge (\tau :: \{\!\{l : \tau_2\}\!\}) \vdash^e (\tau_1 = \tau_2) \\
\textbf{REC3} & \mathit{concat}(\alpha, \beta, \gamma) \wedge (\alpha :: \{\!\{l : \delta\}\!\}) \wedge (\beta \backslash l) \vdash^e (\gamma :: \{\!\{l : \delta\}\!\}) \\
\textbf{REC4} & \mathit{concat}(\alpha, \beta, \gamma) \wedge (\beta :: \{\!\{l : \delta\}\!\}) \wedge (\alpha \backslash l) \vdash^e (\gamma :: \{\!\{l : \delta\}\!\}) \\
\textbf{REC5} & \mathit{concat}(\alpha, \beta, \gamma) \wedge (\gamma :: \{\!\{l : \delta\}\!\}) \wedge (\beta \backslash l) \vdash^e (\alpha :: \{\!\{l : \delta\}\!\}) \\
\textbf{REC6} & \mathit{concat}(\alpha, \beta, \gamma) \wedge (\gamma :: \{\!\{l : \delta\}\!\}) \wedge (\alpha \backslash l) \vdash^e (\beta :: \{\!\{l : \delta\}\!\}) \\
\textbf{REC7} & \vdash^e \mathit{concat}(\{\bar{l} : \bar{\tau}\}, \{\bar{l}' : \bar{\tau}'\}, \{\bar{l}'' : \bar{\tau}''\}) \\
 & \text{where } \bar{l}'' = (\bar{l} \backslash \bar{l}') \cup \bar{l}' \text{ and } \tau_i'' = \mathit{typeof}(l_i'') \\
\textbf{REC8} & \vdash^e (\{l_1 : \tau_1, \ldots, l_n : \tau_n\} \backslash l') \\
 & \text{where } l' \neq l_i
\end{array}
$$

Please note the differences in rules **REC3** and **REC8** compared to the rules given in Section 5.1.2. The principal constraint property can be established along the lines of the asymmetric record concatenation application.

Here comes some additional stuff added after submitting my thesis.

A.3 Polymorphic Recursion

The rule for polymorphic recursion is as follows.

$$\text{(Rec)} \quad \frac{C, \Gamma_x.x : \sigma \vdash e : \sigma}{C, \Gamma_x \vdash e : \mathsf{rec}\ x : \sigma\ \mathsf{in}\ e : \sigma}$$

It is straightforward to extend the type soundness result stated in Theorem 5 in Section 3.3.1. The semantics of the polymorphic recursion rule is as follows.

$$[\![\mathsf{rec}\, x : \sigma e]\!]\eta \;=\; \mathit{fix}(\lambda v.[\![e]\!]\eta[x \mapsto v])$$

where *fix* stands for the fixpoint operator.

A.4 Case

Soundness

I extend the type soundness result for case expressions along the lines of the denotational semantics in Section 3.3.1. I define $[\![\mathsf{bool(T)}]\!] = \{\mathsf{true}\}$ and $[\![\mathsf{bool(F)}]\!] = \{\mathsf{false}\}$. The semantics for case is as follows:

$$\begin{array}{lll} [\![\mathsf{case} & g_1 \Rightarrow e_1 & \\ & \ldots & = \ \mathsf{if}\ [\![g_1]\!]\eta \ = \{\mathsf{true}\}\mathsf{then}\ [\![e_1]\!]\eta \\ & g_n \Rightarrow e_n]\!]\eta & \mathsf{else\ if}\ \ldots \\ & & \mathsf{else\ if}\ [\![g_n]\!]\eta \ = \{\mathsf{true}\}\mathsf{then}\ [\![e_n]\!]\eta \\ & & \mathsf{else}\ \mathbf{W} \end{array}$$

I extend the type soundness result stated in Theorem 5. Consider the case rule.

$$\frac{\begin{array}{c} D_i, \Gamma \vdash g_i : \mathsf{bool}(\gamma_i) \qquad D_i \wedge D, \Gamma \vdash e_i : \tau \qquad \gamma_i \notin fv(\tau, \Gamma) \\ C = \bigwedge_i D_i \wedge D \wedge (\gamma_1 \vee \ldots \vee \gamma_n \mapsto \mathsf{T}) \end{array}}{\exists \bar{\gamma}.C, \Gamma \vdash \begin{array}{ll} \mathsf{case} & g_1 \Rightarrow e_1 \\ & \ldots \\ & g_n \Rightarrow e_n \quad : \tau \end{array}} \text{(Case)}$$

By assumption there is a substitution ϕ such that $\phi\Gamma$ and $\phi\tau$ closed and such that $\vdash^e \phi\exists\bar{\gamma}.C$. The array constraint system is sound, i.e. there exists monotypes $\bar{\mu}$ such that $\vdash^e (\phi \circ [\bar{\mu}/\bar{\gamma}])C$. At least one of the $\bar{\mu}$ equals T. This ensures that at least one branch is executable. I extend ϕ to the substitution $\phi' = \phi \circ [\bar{\mu}/\bar{\gamma}]$. Application of the induction hypothesis yields $[\![e_i]\!]\eta \in [\![\phi'\tau]\!]$. The side condition $\gamma_i \notin fv(\tau, \Gamma)$ ensures that $\phi_{|fv(\tau)} = \phi'_{|fv(\tau)}$. Then I holds that

$$[\![\begin{array}{ll} \mathsf{case} & g_1 \Rightarrow e_1 \\ & \ldots \\ & g_n \Rightarrow e_n]\!]\eta \in [\![\phi\tau]\!] \end{array}$$

which concludes the proof.

Inference

The inference problem for case turns out to be a difficult one. I need to put further restrictions on the case rule to achieve decidable inference. I can not state soundness and completeness yet, but I conjecture that both results hold.

I assume that polymorphic recursion statements are always combined with case

statements. Such statements are of the form

$$\mathsf{rec}\ f\bar{x}\ (f:\sigma)\mathsf{in} \quad \mathsf{case}\ \ g_1 \Rightarrow e_1 \ \ldots \ g_n \Rightarrow e_n$$

where $\bar{x}$ are the only free variables (besides primitive functions) in the body and σ is a closed type scheme. As an example consider the arrfold function.

Inference for such expressions is as follows:

$$\text{(Rec-Case)}\quad \frac{\begin{array}{c} I_i, \Gamma \vdash^W g_i : \mathsf{bool}(\gamma_i) \qquad \psi_i, D_i \wedge I_i', \Gamma \vdash^W e_i : \tau_i' \\ \sigma = \forall \bar{\alpha}. I_c \wedge D_c \Rightarrow \tau_1 \to \ldots \to \tau_n \to \tau \qquad fv(\sigma) = \emptyset \\ \Gamma = \{x_1 : \tau_1, \ldots, x_n : \tau_n\} \cup \Gamma_0 \\ (\psi, D) = normalize(\textstyle\bigwedge_i D_i \wedge \bigwedge_i (\alpha = \tau_i'), \bigsqcup_i \psi_i) \qquad \alpha \text{ fresh} \\ \phi = match(\psi(\alpha), \tau) \\ I_c \vdash^e \textstyle\bigvee_i I_i \\ I_c \wedge I_i \vdash^e I_i' \\ I_c \wedge D_c \wedge I_i \vdash^e (\phi \circ \psi) D_i \end{array}}{\psi, \mathsf{true}, \Gamma_0 \vdash^W \begin{array}{ll} \mathsf{rec}\ f\bar{x}\ (f:\sigma)\mathsf{in} & \mathsf{case}\ \ g_1 \Rightarrow e_1 \\ & \ldots \qquad : \sigma \\ & g_n \Rightarrow e_n \end{array}}$$

I give a brief discussion of this rule. W.l.o.g. I assume that I can always split a constraint in a part which only contains index constraints and the rest. For instance, consider in the premise the judgment $\psi_i, D_i \wedge I_i', \Gamma \vdash^W e_i : \tau_i$ where I_i' denots index

constraints and D_i the rest of the constraints. The same holds for type scheme σ. I further assume that index constraints (such as I_i, I_i' and I_c) are always represented in the simpler notation. The judgment $I_i, \Gamma \vdash^W g_i : \mathsf{bool}(\gamma_i)$ implicitely states that expression g_i contains only subexpressions operating on indices. I perform normalization on non–index constraints, stated by $(\psi, D) = \mathit{normalize}(\bigwedge_i D_i \wedge \bigwedge_i (\alpha = \tau_i'), \bigsqcup_i \psi_i)$. Here, normalization involves Herbrand unification and resolution of constraints such as $\mathit{array}(\alpha, i, \beta) \wedge \mathit{array}(\alpha, j, \gamma)$ which yields the constraint $\mathit{array}(\alpha, i, \beta)$ plus the substitution $[\beta/\gamma]$. Normalization is decidable by applying adopting standard techniques such as those in case of Ohori's record calculus. One implicit assumptions is that index types do not appear in the result type of case expressions. This holds for all example programs I gave. By $\phi = \mathit{match}(\psi(\alpha), \tau)$ I compute the smallest *matching* substitution such that $(\phi \circ \psi)(\alpha) = \tau$ but $\phi\tau = \tau$. It remains to perform three entailment tests. The two tests $I_c \vdash^e \bigvee_i I_i$ and $I_c \wedge I_i \vdash^e I_i'$ can be done by translating entailment into a satisfiability problem. The satisfiability problem for the given arithmetic is decidable. The last test, $I_c \wedge D_c \wedge I_i \vdash^e (\phi \circ \psi) D_i$, can also easily be peformed. All constraint rules are in horn clausal form. Then, the test $I_c \wedge D_c \wedge I_i \vdash^e (\phi \circ \psi) D_i$ can be translated into a Prolog inference problem.

Appendix B

Miscellaneous

B.1 Satisfiability in REC^l is NP–complete

To prove NP–completeness, I have to show that satisfiability in REC^l is NP–time and NP–hard. It is easy to see that satisfiability is NP–time. To prove NP–hardness, I reduce the maximum–cover problem to satisfiability in REC^l.

Definition 30 (Maximum–Cover Problem) *Given n sets $S_1, \ldots, S_n$ such that $\bigcup_i S_i = \{x_1, \ldots, x_m\}$ and a natural number k. Find a set $I \subseteq \{1, \ldots, n\}$ such that $|I| \leq k$ and $\bigcup_{j \in I} S_j = \bigcup_i S_i$.*

I refer to the $(n+1)$–tuple $(S_1, \ldots, S_n, k)$ as the maximum–cover problem.

I give a constraint $C \in \mathrm{REC}^l$ such that C is satisfiable iff $(S_1, \ldots, S_n, k)$ holds. The reduction goes as follows. I represent an element x_i as types x_i and a set S_i as a record type $\{l_{i_1} : x_{i_1}, \ldots, l_{i_r} : x_{i_r}\}$. The record variable α_j represents a $S_j, j \in I$ stated by $C_j = (\{l_1 : S_1, \ldots, l_n : S_n\} :: \{\!\{\beta_j : \alpha_j\}\!\})$ where β_j's are label variables. Then $D_1 = \bigcup_{j=1}^{k} C_j$ states that there are k sets S_j.

Given an element x_i. The constraint

$$C_i' = \exists l, \alpha.((\alpha :: \{\!\{l : x_i\}\!\}) \wedge (\exists l'.(\{l_1 : \alpha_1, \ldots, l_k : \alpha_k\} :: \{\!\{l' : \alpha\}\!\})))$$

states each element x_i belongs to at least one S_j.

Then $(\bigwedge_{i=1}^{m} C_i') \wedge (\bigwedge_{j=1}^{k} C_j)$ is satisfiable iff $(S_1, \ldots, S_n, k)$ holds.

B.2 A Motivating Example II

I give another example which points out problems when representing typing information in term form. I consider a record calculus with first–class field labels in style of Gaster [Gas98]. There are record types of the form $\{l : \alpha | r\}$ where l is a field label with type α and r is a row variable. The novelty is that field labels are now first–class. Consider the following expression:

```
f x =
    let g = λ y z . (x.y) z
    in (g p1 1, g p2 true)
```

Function g takes as first argument a field label. It selects the field from the given record and applies it to the second argument. In the let–body I apply g to arguments of different types, therefore g must be of polymorphic type.

I consider now possible typings for the above program. Record selection can be provided as a primitive operation which writes as follows:

$$(\cdot.\cdot) : \forall r, \beta, l.l \; lacks \, r \Rightarrow label(l) \rightarrow \{l : \beta | r\} \rightarrow \beta$$

where the constraint $l\, lacks\, r$ states that row r does not contain label l and the constructor $label(\cdot)$ introduces the sub–language of label types. Note, because labels are first–class I can pass label l as an argument to the record selection function. Furthermore, the label primitives p1 and p2 are typed as follows:

$$\mathsf{p1} : label(\rho_1)$$
$$\mathsf{p2} : label(\rho_2)$$

where $label(\rho_1)$ (resp $label(\rho_2)$) ensures that ρ_1 (resp ρ_2) is a label constant.

I consider a variation of Gaster's system where types are allowed to appear in constraint form. The type () represents the unit type. Then, the above program is typable in the following way:

$$\begin{array}{l}
\mathsf{f} : \forall \alpha, r', r''.\ \rho_1\, lacks\, r' \wedge \rho_2\, lacks\, r'' \wedge \\
\qquad\qquad (\alpha = \{\rho_1 : \mathsf{Int} \to () | r'\}) \wedge (\alpha = \{\rho_2 : \mathsf{Bool} \to () | r''\}) \\
\qquad\qquad \Rightarrow \alpha \to ((), ()) \\
\mathsf{f\ x} = \\
\quad \mathsf{let\ g}\text{: } \forall \rho, \beta, r.\rho\, lacks\, r \wedge (\alpha = \{\rho : \beta \to () | r\}) \Rightarrow label(\rho) \to \beta \to () \\
\qquad\qquad \mathsf{g} = \lambda\ \mathsf{y\ z}\ .\ \mathsf{(x.y)\ z} \\
\quad \mathsf{in\ (g\ p1\ 1,\ g\ p2\ true)}
\end{array}$$

In Gaster's original system explicit equality constraints are disallowed. Hence, I need to resolve such constraints. But a problem occurs similar to the previous example in Section 3.6.1. There is no term representation of type variable α covering all cases of the constraint representation. Hence, it is not possible to give function g a type in Gaster's system.

Appendix C

Proofs

Quite often I apply induction on a given derivation. But in order to apply the induction hypothesis, I sometimes need to rewrite the premise. But this would break the inductive proof. The trick is to first perform induction on the *size* of expressions and then on the number of non–structural rules. In Figure C.1, I introduce a measure on the size of expressions which is parameterized on a map w from identifiers to positive integers. This method appears also in a technical report by Kennedy [Ken96]. He mentions that the original idea is due to Tobias Nipkow.

Many proofs involves constraint reasoning. I will occasionally abbreviate constraint entailment $\vdash^e$ with $\vdash$ and constraint equality $=^e$ with $=$.

C.1 Proofs of Section 3.3.1

Lemma 7 *Given a sound constraint system* X *and a closed type scheme* σ*. Then* $\mathbf{W} \notin [\![\sigma]\!]$.

Proof: This is true for all monotypes μ. Consider now a type scheme $\sigma = (\forall \bar{\alpha}.C \Rightarrow \tau)$. The type scheme σ is closed and all constraints have to be at least satisfiable, hence $\vdash^e \exists \bar{\alpha}.C$. By assumption, the constraint system X is sound. Then I can follow

$$\begin{array}{rcl}
\delta(w, x) & = & w(x) \\
\delta(w, \lambda x.e) & = & 1 + \delta(w, e) \\
\delta(w, e_1\, e_2) & = & \delta(w, e_1) + \delta(w, e_2) \\
\delta(w, \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2) & = & \delta(w, e_1) + \delta(w[x \mapsto \delta(w, e_1)], e_2)
\end{array}$$

Figure C.1: Measure of expression size

---

there are monotypes $\bar{\mu}$ such that $\vdash^e [\bar{\mu}/\bar{\alpha}]C$. Then the denotation of $[\![\sigma]\!]$ is not an empty intersection. $\mathbf{W}$ is not contained in the denotation of any monotype, thus $\mathbf{W}$ is not contained in $[\![\sigma]\!]$. ∎

Theorem 5 (Type Soundness) *Let $C, \Gamma \vdash e : \sigma$ be a valid typing judgment in* HM(X)*, where* X *is a sound and coherent constraint system. Let ϕ be a substitution such that $\phi\Gamma$ and $\phi\sigma$ are closed and such that $\vdash^e \phi C$ and ϕ is consistent in Γ. Let η be a variable environment such that $\eta \models \phi\Gamma$. Then*

$$\begin{array}{ll}
(1) & \mathbf{W} \notin [\![\phi\sigma]\!] \\
(2) & [\![e]\!]\eta \in [\![\phi\sigma]\!]
\end{array}$$

Proof: Please note, the assumptions ensure that ϕ is consistent in σ which implies that the constraint component of $\phi\sigma$ is satisfiable. Then (1) follows immediately from Lemma 7. I prove now (2) by a structural induction on typing derivations. There are three interesting cases.

Case (Var) The last step of the derivation is:

$$C, \Gamma \vdash x : \sigma \qquad (x : \sigma \in \Gamma)$$

Therefore $x : \phi\sigma \in \phi\Gamma$. Since $\eta \models \phi\Gamma$, $[\![x]\!]\eta = \eta(x) \in [\![\phi\sigma]\!]$.

Case ($\forall$ Intro) The last step of the derivation is:

$$\frac{C \wedge D, \Gamma \vdash e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists\bar{\alpha}.D, \Gamma \vdash e : \forall\bar{\alpha}.D \Rightarrow \tau}$$

Let ϕ be such that $\phi\Gamma$ and $\phi(\forall\bar{\alpha}.D \Rightarrow \tau)$ are closed and such that $\vdash^e \phi(C \wedge \exists\bar{\alpha}.D)$. Furthermore, I assume there are no name clashes between ϕ and $\bar{\alpha}$. Let $\bar{\mu}$ be an arbitrary vector of monotypes such that

$$\vdash^e \exists\bar{\alpha}.((\bar{\alpha} = \bar{\mu}) \wedge \phi D)$$

Since $\mathcal{C}$ is sound there is at least one such vector $\bar{\mu}$. Let $\phi' = [\bar{\mu}/\bar{\alpha}] \circ \phi$. Then since $\bar{\alpha} \notin fv(C)$, $\phi'(C \wedge D) = \phi C \wedge \phi' D$, which expands to $\phi C \wedge \exists\bar{\alpha}.((\bar{\mu} = \bar{\alpha}) \wedge \phi D)$. By our assumption this constraint is valid. Furthermore, $\phi'\Gamma$ and $\phi'\tau$ are both closed. By the induction hypothesis, $[\![e]\!]\eta \in [\![\phi'\tau]\!]$. Since $\bar{\mu}$ was arbitrary such that $\vdash^e [\bar{\mu}/\bar{\alpha}](\phi D)$,

$$\begin{aligned} [\![e]\!]\eta &\in \bigcap\{[\![[\bar{\mu}/\bar{\alpha}](\phi\tau)]\!] \mid \vdash^e [\bar{\mu}/\bar{\alpha}](\phi D)\} \\ &= [\![\phi(\forall\bar{\alpha}.D \Rightarrow \tau)]\!]. \end{aligned}$$

Case (Sub) The last step of the derivation is:

$$\frac{C, \Gamma \vdash e : \tau \qquad C \vdash^e (\tau \preceq \tau')}{C, \Gamma \vdash e : \tau'}$$

I know that there is a substitution ϕ such that $\phi\Gamma$ and $\phi\tau'$ are closed and such that $\vdash^e \phi C$. It follows that $\vdash^e (\phi\tau \preceq \phi\tau')$. It might be the case that $\phi\tau$ still contains

some free variables. I can extend ϕ to a substitution ϕ' such that $\phi'\tau$ is closed. Because ϕ' is an extension of ϕ I get that $\phi'\Gamma$ is closed and $\vdash^e \phi' C$. Applying the induction hypothesis, I get that $[\![e]\!]\eta \in [\![\phi'\tau]\!]$. Because X is coherent I know that $[\![\phi'\tau]\!] \subseteq [\![\phi'\tau']\!]$. Because $\phi\tau'$ is a closed type and ϕ' extends ϕ I get that $[\![\phi'\tau']\!] = [\![\phi\tau']\!]$ and this yields $[\![e]\!]\eta \in [\![\phi\tau']\!]$. ■

C.2 Proofs of Section 3.4.1

Lemma 10 *The following inference rules are derivable.*

$$\textit{(Ref-}\sigma\textit{)} \qquad C \vdash^i \sigma \preceq \sigma$$

$$\textit{(Weaken)} \qquad \frac{D \vdash^e C \qquad C \vdash^i \sigma \preceq \sigma'}{D \vdash^i \sigma \preceq \sigma'}$$

$$\textit{(Trans)} \qquad \frac{C \vdash^i \sigma_1 \preceq \sigma_2 \qquad C \vdash^i \sigma_2 \preceq \sigma_3}{C \vdash^i \sigma_1 \preceq \sigma_3}$$

Proof: I only prove the (Trans) rule. W.l.o.g. (I further assume that all derivations are normalized as stated in Lemma 11)

$$\begin{aligned} \sigma_1 &= \forall\alpha.D_1 \Rightarrow \tau_1, \\ \sigma_2 &= \forall\beta.D_2 \Rightarrow \tau_2 \quad \text{and} \\ \sigma_3 &= \forall\gamma.D_3 \Rightarrow \tau_3 \end{aligned}$$

where α, β and γ are distinct and new.

W.l.o.g.

$$C \vdash^i \sigma_1 \preceq \sigma_2$$

is derived from

$$C \wedge D_2 \vdash^i \phi_1\tau_1 \preceq \tau_2 \quad C \wedge D_2 \vdash^e \phi_1 D_1$$

where $\phi_1 = [\tau/\alpha]$ and

$$C \vdash^i \sigma_2 \preceq \sigma_3$$

is derived from

$$C \wedge D_3 \vdash^i \phi_2\tau_2 \preceq \tau_3 \quad C \wedge D_3 \vdash^e \phi_2 D_2$$

where $\phi_2 = [\tau'/\beta]$.

It holds that

$$\phi_1 C =^e C \text{ and } \phi_2 C =^e C.$$

Because $\vdash^i$ and $\vdash^e$ are closed under substitution it follows that

$$C \wedge \phi_2 D_2 \vdash^e (\phi_2 \circ \phi_1) D_1 \quad C \wedge \phi_2 D_2 \vdash^i (\phi_2 \circ \phi_1)\tau_1 \preceq \phi_2\tau_2.$$

Also,

$$C \wedge D_3 \vdash^e C \wedge \phi_2 D_2$$

holds. Then it follows that

$$C \wedge D_3 \vdash^e (\phi_2 \circ \phi_1) D_1 \quad C \wedge D_3 \vdash^i (\phi_2 \circ \phi_1)\tau_1 \preceq \tau_3$$

and this yields

$$C \vdash^i \sigma_1 \preceq \sigma_3.$$

■

Lemma 11 *Let C be a constraint and σ, σ' be type schemes. Then $C \vdash^{i} \sigma \preceq \sigma'$ iff $C \vdash^{i_2} \sigma \preceq \sigma'$.*

Proof: That $\vdash^{i_2} \subseteq \vdash^{i}$ holds is not difficult to see.. In order to prove $\vdash^{i} \subseteq \vdash^{i_2}$ I show that it is always possible to switch the order of consecutive $(\preceq \forall)$, $(\forall \preceq)$ steps. I assume there is the following derivation

$$\frac{\dfrac{C \wedge D' \vdash^{i} [\tau/\alpha]\sigma \preceq \sigma' \qquad \beta \notin tv([\tau/\alpha]\sigma) \cup tv(C)}{C \wedge \exists\beta.D' \vdash^{i} [\tau/\alpha]\sigma \preceq \forall\beta.D' \Rightarrow \sigma' \qquad C \wedge \exists\beta.D' \vdash^{e} [\tau/\alpha]D}}{C \wedge \exists\beta.D' \vdash^{i} \forall\alpha.D \Rightarrow \sigma \preceq \forall\beta.D' \Rightarrow \sigma'}$$

Starting from

$$C \wedge D' \vdash^{i} [\tau/\alpha]\sigma \preceq \sigma'.$$

I can perform a $(\forall \preceq)$ step and get

$$C \wedge D' \vdash^{i} \forall\alpha.D \Rightarrow \sigma \preceq \sigma'.$$

I observe that $\beta \notin fv(\forall\alpha.D \Rightarrow \sigma)$. That means, I can perform a $(\preceq \forall)$ step and obtain

$$C \wedge \exists\beta.D' \vdash^{i} (\forall\alpha.D \Rightarrow \sigma) \preceq (\forall\beta.D' \Rightarrow \sigma').$$

I have shown now that I always can delay $(\preceq \forall)$ steps. By a straightforward induction it can be shown that $\vdash^{i} \subseteq \vdash^{i_2}$ which concludes the proof. ■

C.3 Proofs of Section 3.4.4

Lemma 17 (Completeness of $\vdash^{s}$) *Given $C', \Gamma' \vdash^{n} e : \sigma'$, $C' \vdash^{i} \Gamma \preceq \Gamma'$ and Γ' is*

realizable in C'. Then

$$\begin{array}{lll} \textit{(a)} \quad \sigma' = \tau : & C, \Gamma \vdash^s e : \tau & C' \vdash^e C \\ \textit{(b) otherwise :} & C, \Gamma \vdash^s e : \tau & C, \Gamma, \tau \vdash^{gen} (\sigma_o, C_o) \\ & C' \vdash^e C_o & C' \vdash^i \sigma_o \preceq \sigma \end{array}$$

Proof: I use induction on the derivation $\vdash^n$.

Case (Inst) I know that $C', \Gamma' \vdash^n x : \sigma'$ where $x : \sigma' \in \Gamma'$. By assumption I know there is a $x : \sigma$ in Γ such that $C' \vdash^i \sigma \preceq \sigma'$. If $\sigma' = \tau$ then I can immediately apply the (Inst) rule and I am done. Otherwise, w.l.o.g. I can assume that $\sigma = \forall \bar{\alpha}.D \Rightarrow \tau'$. I set $C = [\bar{\beta}/\bar{\alpha}]D$ and $\tau = [\bar{\beta}/\bar{\alpha}]\tau'$ where $\bar{\beta}$ are fresh type variables. I apply again the (Inst) rule and find $C, \Gamma \vdash^s x : \tau$. I set $(\sigma_o, C_o) = gen(C, \Gamma, \tau)$ where σ_o is essentially a renamed version of σ. I find that $C' \vdash^i \sigma_o \preceq \sigma'$. By assumption Γ' is realizable in C', hence there is a τ such that $C' \vdash^e [\bar{\tau}/\bar{\alpha}]D$. This leads us to the conclusion that $C' \vdash^e C_o$ and I am done.

Case (Abs) I find the following situation:

$$\frac{C', \Gamma'_x.x : \tau' \vdash^n e : \tau'}{C', \Gamma'_x \vdash^n \lambda x.e : \tau' \to \tau''}$$

Application of the induction hypothesis yields

$$C, \Gamma_x.x : \tau' \vdash^s e : \tau'' \qquad C' \vdash^e C$$

Then I can apply the (Abs) rule and get

$$C, \Gamma_x \vdash^s \lambda x.e : \tau' \to \tau''.$$

Case (App) I find the following situation:

$$\frac{C', \Gamma' \vdash^n e_1 : \tau_1 \to \tau_2 \quad C', \Gamma' \vdash^n e_2 : \tau_1}{C', \Gamma' \vdash^n e_1 e_2 : \tau_2}$$

I can apply the induction hypothesis to the left and right premise and get

$$C_1, \Gamma \vdash^s e_1 : \tau_1 \to \tau_2$$

and

$$C_2, \Gamma \vdash^s e_2 : \tau_1$$

where $C' \vdash^e C_1$ and $C' \vdash^e C_2$. Because $\vdash^s$ is closed under strengthening the constraint I get

$$C', \Gamma \vdash^s e_1 : \tau_1 \to \tau_2$$

and

$$C', \Gamma \vdash^s e_2 : \tau_1.$$

Application of the (App) yields

$$C', \Gamma \vdash^s e_1 e_2 : \tau_2.$$

Case (Sub) I find the following situation:

$$\frac{C', \Gamma' \vdash^n e : \tau \qquad C' \vdash^e (\tau \preceq \tau')}{C', \Gamma' \vdash^n e : \tau'}$$

Here, only case (a) is possible. By induction I find that $C, \Gamma \vdash^s e : \tau$ and $C' \vdash^e C$. Then I also find $C', \Gamma \vdash^s e : \tau$. I apply the (Sub) rule and obtain $C', \Gamma \vdash^s e : \tau'$.

Case ($\forall$ Intro) I find the following situation:

$$\frac{C' \wedge D, \Gamma' \vdash^n e : \sigma \quad \bar{\alpha} \notin \mathit{fv}(C') \cup \mathit{fv}(\Gamma')}{C' \wedge \exists \bar{\alpha}.D, \Gamma' \vdash^n e : \forall \bar{\alpha}.D \Rightarrow \sigma}$$

I consider two cases. First, assume $\sigma = \tau$. W.l.o.g. I can assume that $\bar{\alpha} \notin \mathit{fv}(\Gamma)$. Application of the induction hypothesis yields

$$C, \Gamma \vdash^s e : \tau \qquad C' \wedge D \vdash^e C$$

I define $(\sigma_o, C_o) = \mathit{gen}(C, \Gamma, \tau)$. It holds that $\bar{\alpha} \notin \mathit{fv}(\sigma_o) \cup \mathit{fv}(C_o)$ because w.l.o.g. $\bar{\alpha} \notin \mathit{fv}(\Gamma)$. Therefore, I can deduce that

$$C' \wedge \exists \bar{\alpha}.D \vdash^e C_o$$

and

$$C' \wedge \exists \bar{\alpha}.D \vdash^i \sigma_o \preceq \forall \bar{\alpha}.D \Rightarrow \tau.$$

Otherwise, I apply the induction hypothesis and get

$$C, \Gamma \vdash^s e : \tau$$
$$C' \wedge D \vdash^e C_o \qquad C' \wedge D \vdash^i \sigma_o \preceq \sigma \qquad \text{where}$$
$$C, \Gamma, \tau \vdash^{gen} (\sigma_o, C_o).$$

Because $\bar{\alpha} \notin fv(\Gamma)$ I get that

$$C' \wedge \exists \bar{\alpha}.D \vdash^e C_o$$

and

$$C' \vdash^i \sigma_o \preceq \forall \bar{\alpha}.D \Rightarrow \sigma.$$

Then it is easy to see that

$$C' \wedge \exists \bar{\alpha}.D \vdash^i \sigma_o \preceq \forall \bar{\alpha}.D \Rightarrow \sigma$$

which establishes the induction step.

Case (Let) I find the following situation:

$$\frac{C', \Gamma'_x \vdash^n e : \sigma \quad C', \Gamma'_x.x : \sigma \vdash^n e' : \tau'}{C', \Gamma'_x \vdash^n \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

First, I consider the case if σ is a type τ. I apply the induction hypothesis to the left premise and obtain $C_1, \Gamma_x \vdash^s e : \tau$ and $C' \vdash^e C_1$. I find σ_o, C_o such that $C_1, \Gamma_x, \tau \vdash^{gen} (\sigma_o, C_o)$. It is an easy observation that $C' \vdash^i \sigma_o \preceq \tau$ holds. Now, I apply the induction hypothesis to the right premise. This yields $C_2, \Gamma_x.x : \sigma_o \vdash^s e' :$

τ' and $C' \vdash^e C_2$. I weaken the constraint and find

$$C', \Gamma_x.x : \sigma_o \vdash^s e' : \tau'.$$

It holds that $C' \vdash^e C_o \wedge C_2$. I apply the (Let) rule and obtain $C', \Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'$.

Now, let us consider the case if σ is a type scheme. Application of the induction hypothesis to the left premise yields:

$$C_1, \Gamma_x \vdash^s e : \tau \quad C_1, \Gamma_x, \tau \vdash^{gen} (\sigma_o, C_o)$$
$$C' \vdash^i \sigma_o \preceq \sigma \quad C' \vdash^e C_o.$$

To apply the induction hypothesis to the right premise I have to show that $\Gamma'_x.x : \sigma$ is realizable in C'. I know that $C', \Gamma'_x.x : \sigma \vdash^n e : \tau'$ holds. If x does not appear in the free variables of e it is sufficient to consider only Γ'_x which is by assumption realizable. Otherwise I know that that the type of x must have been instantiated to a monomorphic type which shows that $\Gamma'_x.x : \sigma$ is realizable in C'. Then I can apply the induction hypothesis to the right and find

$$C_2, \Gamma_x.x : \sigma_o \vdash^s e : \tau' \quad C' \vdash^e C_2.$$

I weaken the constraint and find

$$C', \Gamma_x.x : \sigma_o \vdash^s e : \tau'.$$

It holds that $C' \vdash^e C_o \wedge C_2$. I apply the (Let) rule and find

$$C', \Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'.$$

■

Lemma 18 (Weakening) *Given* $C, \Gamma \vdash^s e : \tau$ *and* $(C', \Gamma') \leq (C, \Gamma)$. *Then* $C', \Gamma' \vdash^s e : \tau$.

Proof: I use induction on the derivation $\vdash^s$. There is only one interesting case.

Case (Let) I find the following situation:

$$\frac{C_1, \Gamma_x \vdash^s e : \tau \quad C_1, \Gamma_x, \tau \vdash^{gen} (C_2, \sigma) \qquad C_3, \Gamma_x.x : \sigma \vdash^s e' : \tau'}{C_2 \wedge C_3, \Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

W.l.o.g. $(C_2, \sigma) = (\exists \bar{\alpha}.C_1, \forall \bar{\alpha}.C_1 \Rightarrow \tau)$ where I assume that $\bar{\alpha}$ are such that $\bar{\alpha} \notin fv(\Gamma'_x)$. This can always be achieved through α–renaming. It holds that $(C' \wedge C_1, \Gamma'_x) \leq (C_1, \Gamma_x)$. I apply the induction hypothesis to the left premise and find

$$C' \wedge C_1, \Gamma'_x \vdash^s e : \tau.$$

I set $C'' = C' \wedge C_1$, $(C'_2, \sigma') = gen(C'', \Gamma'_x, \tau)$ where $C'_2 = \exists \bar{\beta}.C''$, $\sigma' = \forall \bar{\beta}.C'' \Rightarrow \tau$ and $\bar{\alpha} \notin fv(\sigma')$. It is an easy observation that

$$C' \vdash^i \sigma' \preceq \sigma.$$

I can apply the induction hypothesis to the right premise and obtain

$$C', \Gamma'_x.x : \sigma' \vdash^s e' : \tau'.$$

Because of the assumptions it holds that $\bar{\alpha} \notin fv(\Gamma'_x)$ which implies that $\bar{\alpha} \subseteq \bar{\beta}$. Then

I can deduce that $C' \vdash^e C_2 \wedge C_3$. Finally, I find that $C' \wedge C'_2 =^e C'$. It remains to apply the (Let) rule and I obtain

$$C', \Gamma'_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'.$$

■

C.4 Proofs of Section 3.6

Theorem 7 (Transformation) *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* $\mathcal{S}$ *is closed under projection and* $\mathcal{X}^t$ *enjoys the principal constraint property, a type environment* Γ' *in* $\mathcal{X}^t$ *and a constraint* C *in* $\mathcal{X}^t$ *where* $C \vdash \Gamma \mapsto \Gamma'$*. Then the following holds.*

- *If* $C, \Gamma \vdash^{\mathcal{X}^c} e : \tau$ *then* $C, \Gamma' \vdash^{\mathcal{X}^t} e : \tau$*.*
- *If* $C, \Gamma \vdash^{\mathcal{X}^c} e : \sigma$ *where* σ *realizable in* C *then* $C, \Gamma' \vdash^{\mathcal{X}^t} e : \sigma'$ *where* $C \vdash \sigma \mapsto \sigma'$*.*

Proof: The proof proceeds by induction on the number of occurrences of (Sub), (∀ Elim) and (∀ Intro) and then on the expression size. I only show the interesting cases.

Case (Let) I find the following situation:

$$\frac{C, \Gamma_x \vdash^{\mathcal{X}^c} e : \sigma \qquad C, \Gamma_x.x : \sigma \vdash^{\mathcal{X}^c} e' : \tau'}{C, \Gamma_x \vdash^{\mathcal{X}^c} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

W.l.o.g. I can assume that σ is a polymorphic type. The same argument would hold for monomorphic types. By induction I find

$$C, \Gamma'_x \vdash^{\mathcal{X}^t} e : \sigma' \qquad C \vdash \sigma \mapsto \sigma'.$$

Then it holds that $C \vdash \Gamma_x.x : \sigma \mapsto \Gamma'_x.x : \sigma'$. I can now apply the induction hypothesis and I obtain

$$C, \Gamma'_x.x : \sigma' \vdash^{\mathcal{X}^t} e' : \tau'.$$

Finally, I apply the (Let) rule and I find

$$C, \Gamma'_x \vdash^{\mathcal{X}^t} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'.$$

Case ($\forall$ Elim) I find the following situation:

$$\frac{C, \Gamma \vdash^{\mathcal{X}^c} e : \forall \bar{\alpha}.D \Rightarrow \tau \qquad C \vdash^{e} [\bar{\tau}/\bar{\alpha}]D}{C, \Gamma \vdash^{\mathcal{X}^c} e : [\bar{\tau}/\bar{\alpha}]\tau}$$

By induction I find

$$C, \Gamma' \vdash^{\mathcal{X}^t} e : \sigma'$$

where $C \vdash (\forall \bar{\alpha}.D \Rightarrow \tau) \mapsto \sigma'$. I also know that

$$C \vdash^{i} \forall \bar{\alpha}.D \Rightarrow \tau \preceq [\bar{\tau}/\bar{\alpha}]\tau.$$

Because of the properties put on $\mapsto$, it holds that $C \vdash^{i} \sigma' \preceq [\bar{\tau}/\bar{\alpha}]\tau$. But then I obtain

$$C, \Gamma' \vdash^{\mathcal{X}^t} e : [\bar{\tau}/\bar{\alpha}]\tau.$$

Case (∀ Intro) I find the following situation:

$$\frac{C \wedge D, \Gamma \vdash^{\mathcal{X}^c} e : \tau \qquad \bar{\alpha} \notin fv(C) \cup fv(\Gamma)}{C \wedge \exists \bar{\alpha}.D, \Gamma \vdash^{\mathcal{X}^c} e : \forall \bar{\alpha}.D \Rightarrow \tau}$$

I know $\forall \bar{\alpha}.D \Rightarrow \tau$ is realizable in $C \wedge \exists \bar{\alpha}.D$. That means, there is a ϕ ($\phi = [\bar{\tau}/\bar{\alpha}]$ for some $\bar{\tau}$) which only acts on $\bar{\alpha}$ such that

$$C \wedge \exists \bar{\alpha}.D \vdash^{e} \phi D.$$

I can conclude that

$$C \wedge \exists \bar{\alpha}.D \vdash^{e} C \wedge \phi D.$$

By assumption $C \wedge \exists \bar{\alpha}.D$ is in $\mathcal{X}^t$. But that means $(C \wedge \exists \bar{\alpha}.D, \phi)$ (please note $\phi(C \wedge \exists \bar{\alpha}.D) = C \wedge \exists \bar{\alpha}.D$) is a normal form of $(C \wedge D, id)$. $\mathcal{X}^t$ fulfills the principal constraint property. This implies the principal normal form of $(C \wedge D, id)$ exists, Then $(C', \psi') = \mathit{normalize}(C \wedge D, id)$ such that $id \leq \psi'$ and $C' \vdash^{e} \psi'(C \wedge D)$. (C', ψ') is principal, therefore $\psi' \leq^{\phi'} \phi$ and $C \wedge \exists \bar{\alpha}.D \vdash^{e} \phi' C'$. It follows that ψ' acts only on $\bar{\alpha}$, hence $\psi'(C \wedge D) = C \wedge \psi' D$. I apply ψ' (please note, $\bar{\alpha} \notin fv(\Gamma)$) to the premise and find

$$C \wedge \psi' D, \Gamma \vdash^{\mathcal{X}^c} e : \psi' \tau.$$

Weakening yields

$$C', \Gamma \vdash^{\mathcal{X}^c} e : \psi' \tau.$$

Application of the induction hypothesis yields

$$C', \Gamma' \vdash^{\mathcal{X}^t} e : \psi' \tau$$

where w.l.o.g. $\bar{\alpha} \notin fv(\Gamma')$. I apply the ($\forall$ Intro) rule and I find

$$\exists \bar{\alpha}.C', \Gamma' \vdash^{\mathcal{X}^t} e : \forall \bar{\alpha}.C' \Rightarrow \psi'\tau.$$

Please note, the set $\mathcal{S}$ of solved forms is closed under projection. It also holds that $C \wedge \exists \bar{\alpha}.D \vdash^e \exists \bar{\alpha}.C'$. Weakening yields

$$C \wedge \exists \bar{\alpha}.D, \Gamma' \vdash^{\mathcal{X}^t} e : \forall \bar{\alpha}.C' \Rightarrow \psi'\tau.$$

It remains to show that $C \wedge \exists \bar{\alpha}.D \vdash (\forall \bar{\alpha}.D \Rightarrow \tau) \mapsto (\forall \bar{\alpha}.C' \Rightarrow \psi'\tau)$. There are two cases.

1. $\vdash^i (\forall \bar{\alpha}.D \Rightarrow \tau) \preceq (\forall \bar{\alpha}.C' \Rightarrow \psi'\tau)$ because $C' \vdash^e \psi' D$.

2. Given $\forall \bar{\beta}.D' \Rightarrow \tau'$ in $\mathcal{X}^t$ such that $C \wedge \exists \bar{\alpha}.D \vdash^i (\forall \bar{\alpha}.D \Rightarrow \tau) \preceq (\forall \bar{\beta}.D' \Rightarrow \tau')$. Then, there exist $\bar{\psi}$ such that $C \wedge (\exists \bar{\alpha}.D) \wedge D' \vdash^e \bar{\psi} D$ and $C \wedge (\exists \bar{\alpha}.D) \wedge D' \vdash^i \bar{\psi}\tau \preceq \tau'$. That means, $(C \wedge (\exists \bar{\alpha}.D) \wedge D', \bar{\psi})$ is a normal form of $(C \wedge D, id)$. Principality yields, there exist $\bar{\phi}$ such that $\psi' \leq^{\bar{\phi}} \bar{\psi}$ and $C \wedge (\exists \bar{\alpha}.D) \wedge D' \vdash^e \bar{\phi} C'$. Then, I can conclude $C \wedge \exists \bar{\alpha}.D \vdash^i (\forall \bar{\alpha}.C' \Rightarrow \psi'\tau) \preceq (\forall \bar{\beta}.D' \Rightarrow \tau')$.

■

Please note, implicitly I assumed that all constraints appearing in a derviation $C \vdash^i \sigma \preceq \sigma'$ are in solved form. A similar reasoning as in the Transformation Theorem 7could be applied to enforce this invariant.

Theorem 8 (Full and Faithful) *Given a constraint system X over a term algebra $\mathcal{T}$. If $\mathcal{X}^t$ satisfies the principal constraint property then every program typable in $\mathcal{X}^c$ is typable in $\mathcal{X}^t$ and vice versa.*

Proof: The proof proceeds by transforming typing judgments in $\mathcal{X}^t$ to judgments in $\mathcal{X}^c$ and vice versa. I annotate derivations $\vdash$ with the type system to distinguish between derivations in $\mathcal{X}^t$ and $\mathcal{X}^c$.

Given a judgment $C, \Gamma \vdash {}^{\mathcal{X}^t} e : \tau$. I observe that it is possible to replace applications of the term–based elimination rule by the constraint–based elimination rule. This is stated by Lemma 20 in Section 3.7. Furthermore, judgments can always be put into canonical form. It is an easy observation that the above judgment can be transformed into a judgment $C \wedge (\alpha = \tau), \Gamma \vdash {}^{\mathcal{X}^c} e : \alpha$ in the constraint–based fragment of HM(X).

Given a judgment $C, \Gamma \vdash {}^{\mathcal{X}^c} e : \tau$. I make one further simplifying assumption. I consider only closed type environments where all type schemes are valid in the term–based as well as the constraint–based HM(X) fragment, i.e. $\Gamma \in \mathcal{X}^t$. This omits transforming the type environment as well, and simplifies the presentation of the essential proof parts. I also assume that the set $\mathcal{S}$ of solved forms is closed under projection. As stated in Theorem 6 this does not affect the expressiveness of the type system. I apply normalization to achieve a term–based representation. This is possible because the underlying constraint system satisfies the principal constraint property. I set $(D, \phi) = \mathit{normalize}(C, \mathit{id})$. Application of the weakening and substitution lemma to the above judgment yields $D, \Gamma \vdash {}^{\mathcal{X}^c} e : \phi\tau$. I apply the Transformation Theorem 7 which yields $D, \Gamma \vdash {}^{\mathcal{X}^t} e : \phi\tau$ and this concludes the proof. ∎

C.5 Proofs of Section 3.7

Lemma 20 *Given an instance of the HM(X) system in term form. If the judgment* $C, \Gamma \vdash e : \sigma$ *is derivable using the constraint–based (∀ Elim) rule then* $C, \Gamma \vdash e : \sigma$ *is*

derivable using the substitution–based formulation of the (∀ Elim) rule instead. The reverse direction holds too.

Proof: The proof proceeds by induction on the derivation. It is sufficient to consider only applications of the elimination rules. Consider the last application of the constraint–based (∀ Elim) rule in the derivation tree:

$$\frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash^e \exists \bar{\alpha}.D}{C \wedge D, \Gamma \vdash e : \tau'}$$

By induction, the premise can be transformed in such a way that all applications of the constraint–based elimination rule are replaced by the term–based elimination rule. Weakening applied to the premise yields $C \wedge D, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau'$. Finally, application of the term–based elimination rule yields $C \wedge D, \Gamma \vdash e : \tau'$. and this concludes the inductive proof.

For the other direction, consider the last application of the term–based elimination rule in the derivation tree:

$$\frac{C, \Gamma \vdash e : \forall \bar{\alpha}.D \Rightarrow \tau' \qquad C \vdash^e [\bar{\tau}/\bar{\alpha}]D}{C, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'}$$

Application of the induction hypothesis to the premise allows to replace all term–based elimination rule applications by constraint–based rule applications. It holds that $C \vdash^e [\bar{\tau}/\bar{\alpha}]D \vdash^e \exists \bar{\alpha}.D$. This allows to apply the constraint–based elimination rule which yields $C \wedge D, \Gamma \vdash e : \tau'$. Remember, I consider equivalence of the two formulations of the elimination rules only in the term–based fragment of HM(X). Hence, it is possible to apply the substitution lemma. I get the judgment $C \wedge [\bar{\tau}/\bar{\alpha}]D, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'$ which is equivalent to $C, \Gamma \vdash e : [\bar{\tau}/\bar{\alpha}]\tau'$. This concludes the

proof. ■

Lemma 21 *Given a constraint C, a type environment Γ, a term e and a type τ. Then $C, \Gamma \vdash e : \tau$ iff $C \wedge (\alpha = \tau), \Gamma \vdash e : \alpha$ where α is a fresh type variable.*

Proof: Given $C, \Gamma \vdash e : \tau$. By weakening I get that $C \wedge (\alpha = \tau), \Gamma \vdash e : \tau$ where α is a fresh type variable. It remains to apply the (Sub) rule which yields $C \wedge (\alpha = \tau), \Gamma \vdash e : \alpha$.

I consider now the other direction. Remember, that the type τ is a member of the type language in the HM(X) system in constraint form. A restricted substitution lemma holds under the assumption that type τ consists only of type variables or function types. Hence, from $C \wedge (\alpha = \tau), \Gamma \vdash e : \alpha$ it follows that $\phi(C \wedge (\alpha = \tau)), \phi\Gamma \vdash e : \phi(\alpha)$ where $\phi = [\tau/\alpha]$. The last judgment is equivalent to $C, \Gamma \vdash e : \tau$.

■

C.6 Proofs of Section 4.2

Lemma 26 *Given two constraint problems (D, ϕ) and (D', ϕ'), a principal normal form (C, ψ) of (D', ϕ') and a principal normal form (C', ψ') of (D, ϕ) such that (C, ψ) is also a normal form of (D, ϕ) and (C', ψ') is also a normal form of (D', ϕ'). Then (C, ψ) is a principal normal form of (D, ϕ) and (C', ψ') is principal normal form of (D', ϕ').*

Proof: I abbreviate normal form by nf and principal normal form by pnf. By assumption, (C, ψ) is a pnf and (C', ψ') is a nf of (D, ϕ). Then $(C, \psi) \leq (C', \psi')$ and also $(C', \psi') \leq (C, \psi)$. This implies (C, ψ) pnf of (D, ϕ) and (C', ψ') pnf of (D', ϕ').

■

Lemma 27 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set of solved constraints* $\mathcal{S}$ *in* X *such that* X *enjoys the principal constraint property and unitary unification. Given constraint problems* (D_1, ϕ_1) *and* (D_2, ϕ_2) *such that* $dom(\phi_1) \cap codom(\phi_2) = \emptyset$, $dom(\phi_2) \cap codom(\phi_1) = \emptyset$, $fv(D_1) \cap dom(\phi_2) = \emptyset$ *and* $fv(D_2) \cap dom(\phi_1) = \emptyset$. *Let* (C, ψ) *be the principal normal form of* $(D_1, \phi_1) \sqcup (D_2, \phi_2)$. *Then the principal normal form* (C_1, ψ_1) *of* (D_1, ϕ_1) *exists and* (C, ψ) *is a principal normal form of* $(C_1, \psi_1) \sqcup (D_2, \phi_2)$.

Proof: By assumption $\phi_1 \sqcup \phi_2 \leq^{\phi'} \psi$ and $C \vdash^e \phi'((\phi_1 \sqcup \phi_2)(D_1 \wedge D_2))$. Then $\phi_1 \leq^{\phi'_1} \psi$ where ϕ'_1 is defined on the following domains:

1. $U_1 = codom(\phi_{1|dom(\phi_2)})$:

   For each $\beta \in U_1$ there exists a unique $\alpha \in dom(\phi_2)$ such that $\beta \in fv(\phi_1(\alpha))$. This can be achieved by imposing an ordering relation on the variables. Then for each $\beta \in U_1$ define $\phi'_1(\beta) = \phi' \circ \phi_\alpha(\beta)$ where $\phi_\alpha = (\phi_1 \sqcup \phi_2)(\alpha)$.

2. $U_2 = codom(\phi_{1 \setminus dom(\phi_2)}) \setminus U_1$:

   Define $\phi'_{1|U_2} = \phi'$.

3. $U_3 = dom(\phi_2) \setminus dom(\phi_1)$:

   Define $\phi'_{1|U_3} = \phi' \circ \phi_2$.

4. otherwise :

   Define $\phi'_1 = \psi$.

Then ϕ'_1 is well–defined by construction and $\phi_1 \leq^{\phi'_1} \psi$. From the assumption it follows that $C \vdash^e \phi' \circ (\phi_1 \sqcup \phi_2) D_1$. The conditions put on the domain, codomain and free variables of ϕ_1, ϕ_2, D_1 yield that $(\phi' \circ (\phi_1 \sqcup \phi_2))_{|fv(D_1)} = (\phi'_1 \circ \phi_1)_{|fv(D_1)}$. Then

I find that $C \vdash^e \phi_1' D_1$ (remember, $\phi_1 D_1 = D_1$), hence (C, ψ) is a normal form of (D_1, ϕ_1). The principal constraint property holds, therefore I can assume that the principal normal form (C_1, ψ_1) of (D_1, ϕ_1) exists.

It remains to show that $(D', \phi') = (C_1, \psi_1) \sqcup (D_2, \phi_2)$, $\phi' \leq^{\bar{\phi}} \psi$ and $C \vdash^e \bar{\phi} D'$ for some $\bar{\phi}$.

1. $\phi' = \psi_1 \sqcup \phi_2$ exists:

   (C_1, ψ_1) principal, hence $\psi_1 \leq^{\phi_2'} \psi$ and $C \vdash^e \phi_2' C_1$ for some ϕ_2'. A similar construction as before gives (C, ψ) normal form of (D_2, ϕ_2), then $\phi_2 \leq^{\phi_3'} \psi$ and $C \vdash^e \phi_3' D_2$. ψ_1 and ϕ_2 have a common upper bound, hence the least upper bound $\psi_1 \sqcup \phi_2$ exists. $\psi_1 \sqcup \phi_2 \leq^{\bar{\phi}} \psi$ for some substitution $\bar{\phi}$. Moreover, there are ρ_1, ρ_2 such that $\rho_1 \circ \phi_2' = \bar{\phi}$ and $\rho_2 \circ \phi_3' = \bar{\phi}$.

2. $C \vdash^e (\bar{\phi} \circ \phi') C_1$:

   $$\begin{aligned} (\bar{\phi} \circ (\psi_1 \sqcup \phi_2))_{|fv(C_1)} &= (\rho_1 \circ \phi_2' \circ (\psi_1 \sqcup \phi_2))_{|fv(C_1)} \\ &= (\rho_1 \circ \phi_2' \circ \psi_1)_{|fv(C_1)} \quad (fv(C_1) \cap dom(\phi_2) = \emptyset) \end{aligned}$$

   I know $C \vdash^e \phi_2' C_1$, $\rho_1 C = C$. Then $C \vdash^e (\rho_1 \circ \phi_2') C_1 = (\bar{\phi} \circ (\psi_1 \sqcup \phi_2)) C_1 = (\bar{\phi} \circ \phi') C_1$.

3. $C \vdash^e (\bar{\phi} \circ \phi') D_2$:

   $$\begin{aligned} (\bar{\phi} \circ (\psi_1 \sqcup \phi_2))_{|fv(D_2)} &= (\rho_2 \circ \phi_3' \circ (\psi_1 \sqcup \phi_2))_{|fv(D_2)} \\ &= (\rho_2 \circ \phi_3' \circ \phi_2)_{|fv(D_2)} \quad (fv(D_2) \cap dom(\psi_1) = \emptyset) \end{aligned}$$

   I know $C \vdash^e \phi_3' D_2$, $\rho_2 C = C$. Then $C \vdash^e (\rho_2 \circ \phi_3') D_2 = (\rho_2 \circ \phi_3' \circ \phi_2) D_2 = (\bar{\phi} \circ \phi') D_2$.

The last two proof steps showed that $C \vdash^e (\bar{\phi} \circ \phi')(C_1 \wedge D_2)$, therefore (C, ψ) is a

normal form of (D', ϕ'). ■

Lemma 28 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set of solved constraints* $\mathcal{S}$ *in* X *such that* X *satisfies the principal constraint property and unitary unification. Given constraint problems* (D_1, ϕ_1) *and* (D_2, ϕ_2) *such that* $dom(\phi_1) \cap codom(\phi_2) = \emptyset$, $dom(\phi_2) \cap codom(\phi_1) = \emptyset$, $fv(D_1) \cap dom(\phi_2) = \emptyset$ *and* $fv(D_2) \cap dom(\phi_1) = \emptyset$. *Let* (C_1, ψ_1) *be the principal normal form of* (D_1, ϕ_1) *and* (C, ψ) *be the principal normal form of* $(C_1, \psi_1) \sqcup (D_2, \phi_2)$. *Then* (C, ψ) *is a normal form of* $(D_1, \phi_1) \sqcup (D_2, \phi_2)$.

Proof: I set $(D', \phi') = (C_1, \psi_1) \sqcup (D_2, \phi_2)$. By assumption $\psi_1 \sqcup \phi_2 = \phi' \leq^{\phi'_1} \psi$ and $C \vdash^e \phi'_1 D'$ for some substitution ϕ'_1 and $\phi_1 \leq^{\phi'_2} \psi_1$ and $C_1 \vdash^e \phi'_2 D_1$ for some substitution ϕ'_2. I have to show that $\phi_1 \sqcup \phi_2 \leq^{\bar{\phi}} \psi$ and $C \vdash^e (\bar{\phi} \circ (\phi_1 \sqcup \phi_2))(D_1 \wedge D_2)$ for some substitution $\bar{\phi}$.

1. $\phi_1 \sqcup \phi_2$ exists:

   $\psi_1 \sqcup \phi_2$ exists, therefore $(\phi'_2 \circ \phi_1) \leq^{\rho_1} \bar{\phi}_1$ and $\phi_2 \leq^{\rho_1} \bar{\phi}_1$. It follows that ϕ_1, ϕ_2 have a unifier because $\phi_1 \leq^{\rho_1 \circ \phi'_2} \bar{\phi}_1$ and $\phi_2 \leq^{\rho_1} \bar{\phi}_1$. Then $\phi_1 \sqcup \phi_2$ exists. Specifically, it follows that $\phi_1 \sqcup \phi_2 \leq^{\rho} \psi_1 \sqcup \phi_2$ for some substitution ρ.

2. $\phi_1 \sqcup \phi \leq^{\phi'_1 \circ \rho} \psi$ because $\phi_1 \sqcup \phi_2 \leq^{\rho} \psi_1 \sqcup \phi_2 \leq^{\phi'_1} \psi$. Especially, it holds that $\phi'_1 \circ \rho \circ (\phi_1 \sqcup \phi_2) = \phi'_1 \circ (\psi_1 \sqcup \phi_2)$.

3. $C \vdash^e \phi'_1 \circ \rho \circ (\phi_1 \sqcup \phi_2) D_2$ because of $C \vdash^e \phi'_1 \circ (\psi_1 \sqcup \phi_2) D_2$ and the above equation.

4. $C \vdash^e \phi'_1 \circ \rho \circ (\phi_1 \sqcup \phi_2) D_1$. From the assumptions it follows that $C \vdash^e \phi'_1 \circ (\psi_1 \sqcup \phi_2) \circ \phi'_2 D_1$. The substitution ϕ'_2 operates on $codom(\phi_1)$ and $dom(\psi_1)$ and $fv(D_1) \cap dom(\phi_2) = \emptyset$. Then $(\phi'_2 \circ (\psi_1 \sqcup \phi_2))_{|fv(D_1)} = \phi'_{2|fv(D_1)}$. Therefore,

ρ can be chosen (take $\rho' = \phi'_2 \circ \rho$) such that $(\phi'_1 \circ \rho \circ (\phi_1 \sqcup \phi_2))_{|fv(D_1)} = (\phi'_1 \circ (\psi_1 \sqcup \phi_2) \circ \phi'_2)_{|fv(D_1)}$. Then it follows that $C \vdash^e \phi'_1 \circ \rho \circ (\phi_1 \sqcup \phi_2)D_1$.

I find that (C, ψ) is a normal form of $(D_1, \phi_1) \sqcup (D_2, \phi_2)$. ■

Lemma 29 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *satisfies the lifting property. Given a constraint problem* $(\exists\bar{\alpha}.D, \phi)$*. Then* $(\exists\bar{\alpha}.D, \phi)$ *does have a normal form iff* (D, ϕ) *does have a normal form.*

Proof: W.l.o.g. $\bar{\alpha} \notin dom(\phi) \cup codom(\phi)$.

"$\Leftarrow$" : Assume (C, ψ) is a normal form of (D, ϕ). I set $C' = \exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D$ and $\psi' = \pi \circ \psi_{\setminus\bar{\alpha}}$ where $\pi = [\bar{\beta}/\bar{\alpha}]$ and $\bar{\beta}$ new. By assumption, there exist ϕ' such that $\phi \leq^{\phi'} \psi$ and because of the assumption put on ϕ it follows that $\phi \leq^{\phi'} \psi_{\setminus\bar{\alpha}}$. Please note, $\psi'(\exists\bar{\alpha}.D) = C'$. It remains to show that $(\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D) \in \mathcal{S}$. By assumption, $\mathcal{S}$ is equation–free. Assume $\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D \vdash^e (\tau = \tau')$. Then $(\pi \circ \psi)D \vdash^e (\tau = \tau')$ because $[\tau/\alpha]D = \exists\alpha.((\alpha = \tau) \wedge D) \vdash^e \exists\alpha.D$. By assumption $\psi D \in \mathcal{S}$ but then also $(\pi \circ \psi)D \in \mathcal{S}$. It follows that $\vdash^e (\tau = \tau')$, thus $\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D \in \mathcal{S}$. That means, (C', ψ') is a normal form of $(\exists\bar{\alpha}.D, \phi)$.

"$\Rightarrow$" : Assume (C, ψ) normal form of $(\exists\bar{\alpha}.D, \phi)$. Then $\exists\bar{\alpha}.\psi D \in \mathcal{S}$. The lifting property states there exist $\bar{\tau}$ such that $([\bar{\tau}/\bar{\alpha}] \circ \psi)D \in \mathcal{S}$. This shows (D, ϕ) does have a normal form as well. ■

Lemma 30 *Given a constraint system* X *over a term algebra* $\mathcal{T}$ *and a set* $\mathcal{S}$ *of solved forms such that* X *satisfies the lifting property. Given a constraint problem* $(\exists\bar{\alpha}.D, \phi)$*. Let* (C, ψ) *be the principal normal form of* (D, ϕ)*. Then* $(\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D, (\pi \circ \psi_{\setminus\bar{\alpha}}))$ *is the principal normal form of* $(\exists\bar{\alpha}.D, \phi)$ *where* $\pi = [\bar{\beta}/\bar{\alpha}]$ *and* $\bar{\beta}$ *are new.*

Proof: Please note, w.l.o.g. $\bar{\alpha} \notin dom(\phi) \cup codom(\phi)$. I set $C' = \exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D$ and $\psi' = \pi \circ \psi_{\setminus\bar{\alpha}}$.

1. (C', ψ') is a normal form of $(\exists\bar{\alpha}.D, \phi)$:

   By assumption, there exist ϕ' such that $\phi \leq^{\phi'} \psi$ and because of the assumption put on ϕ it follows that $\phi \leq^{\phi'} \psi_{\setminus\bar{\alpha}}$. Please note, $\psi'(\exists\bar{\alpha}.D) = C'$. It remains to show that $(\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D) \in \mathcal{S}$. By assumption, $\mathcal{S}$ is equation–free. Assume $\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D \vdash^e (\tau = \tau')$. Then $(\pi \circ \psi)D \vdash^e (\tau = \tau')$ because $[\tau/\alpha]D = \exists\alpha.((\alpha = \tau) \wedge D) \vdash^e \exists\alpha.D$. By assumption $\psi D \in \mathcal{S}$ but then also $(\pi \circ \psi)D \in \mathcal{S}$. It follows that $\vdash^e (\tau = \tau')$, thus $\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D \in \mathcal{S}$.

2. (C', ψ') is principal: Assume (C'', ψ'') is a normal form of $(\exists\bar{\alpha}.D, \phi)$ where $C'' = \psi''(\exists\bar{\alpha}.D) = \exists\bar{\alpha}.\psi''D$ (w.l.o.g. $\bar{\alpha} \notin dom(\psi'') \cup codom(\psi'')$). Then $\exists\bar{\alpha}.\psi''D \in \mathcal{S}$. The lifting property states there exist $\bar{\tau}$ such that $([\bar{\tau}/\bar{\alpha}] \circ \psi'')D \in \mathcal{S}$. I set $\psi''' = [\bar{\tau}/\bar{\alpha}] \circ \psi''$ and $C''' = \psi'''D$. Then (C''', ψ''') is a normal form of (D, ϕ). (C, ψ) is principal, hence there exists ϕ' such that $\psi \leq^{\phi'} \psi'''$. Then $\pi \circ \psi_{\setminus\bar{\alpha}} \leq^{\phi' \circ \pi^{-1}} \psi''$ where $\bar{\alpha} \notin dom(\phi' \circ \pi^{-1}) \cup codom(\phi' \circ \pi^{-1})$. It remains to show that $C'' \vdash^e (\pi' \circ \pi^{-1})C'$.

$$
\begin{aligned}
(\phi' \circ \pi^{-1})C' &= (\phi' \circ \pi^{-1})\exists\bar{\alpha}.(\pi \circ \psi_{\setminus\bar{\alpha}})D \\
&= \exists\bar{\alpha}.(\phi' \circ \pi^{-1}) \circ (\pi \circ \psi_{\setminus\bar{\alpha}})D \\
&\quad \text{because } \bar{\alpha} \notin dom(\pi'' \circ \pi^{-1}) \cup codom(\pi' \circ \pi^{-1}) \\
&= \exists\bar{\alpha}.\psi''D \\
&= C''
\end{aligned}
$$

■

C.7 Proofs of Section 4.3

Theorem 12 (Soundness of Inference) *Given a type environment* Γ *and a term* e. *If* $\psi, C, \Gamma \vdash^W e : \tau$ *then* $C, \psi\Gamma \vdash e : \tau$, $\psi C = C$ *and* $\psi\tau = \tau$.

Proof: I apply induction on the derivation $\vdash^W$. I only consider two cases. The other cases can be proven in a similar style.

Case (App) I find the following situation:

$$\frac{\begin{array}{c}\psi_1, C_1, \Gamma \vdash^W e_1 : \tau_1 \qquad \psi_2, C_2, \Gamma \vdash^W e_2 : \tau_2 \\ (D, \psi') = (C_1, \psi_1) \sqcup (C_2, \psi_2) \sqcup ((\tau_1 \preceq \tau_2 \to \alpha), id) \quad \alpha \text{ new} \\ (C, \psi) = \mathit{normalize}(D, \psi')\end{array}}{\psi_{|fv(\Gamma)}, C, \Gamma \vdash^W e_1 e_2 : \psi(\alpha)}$$

I apply the induction hypothesis to the left and right premise and I obtain

$$C_1, \psi_1\Gamma \vdash e_1 : \tau_1 \quad \psi_1 C_1 = C_1 \quad \psi_1\tau_1 = \tau_1.$$

and

$$C_2, \psi_2\Gamma \vdash e_2 : \tau_2 \quad \psi_2 C_2 = C_2 \quad \psi_2\tau_2 = \tau_2.$$

W.l.o.g. I can assume that all identifier in Γ are contained in e_1 and e_2 and not more. This fact and normalization ensures that ψ is consistent in Γ. Furthermore, I know that $C \vdash^e \psi C_1$ and $C \vdash^e \psi C_2$. This ensures that ψC_1 and ψC_2 are in solved form. Then I apply the Weakening/Substitution Lemma 14 and obtain

$$C, \psi\Gamma \vdash e_1 : \psi\tau_1 \quad C, \psi\Gamma \vdash e_2 : \psi\tau_2.$$

I know that $C \vdash^{e} (\psi\tau_1 \preceq \psi\tau_2 \to \psi(\alpha))$ and apply the (Sub) rule to get $C, \psi\Gamma \vdash e : \psi\tau_2 \to \psi(\alpha)$. It remains to apply the (App) rule and I find

$$C, \psi\Gamma \vdash e_1 e_2 : \psi(\alpha).$$

Case (Let)

$$\frac{\begin{array}{c}\psi_1, C_1, \Gamma_x \vdash^{W} e : \tau \qquad C_1, \psi_1\Gamma_x, \tau \vdash^{gen} (C_2, \sigma) \\ \psi_2, C_3, \Gamma_x.x : \sigma \vdash^{W} e' : \tau' \\ (D, \psi') = (C_2, \psi_1) \sqcup (C_3, \psi_2) \\ (C, \psi) = \mathit{normalize}(D, \psi')\end{array}}{\psi_{|fv(\Gamma_x)}, C, \Gamma_x \vdash^{W} \mathsf{let}\ x = e\ \mathsf{in}\ e' : \psi\tau'}$$

I apply the induction hypothesis to both premises and I find

$$C_1, \psi_1\Gamma_x \vdash e : \tau$$

and

$$C_3, \psi_2\Gamma_x.x : \psi_2 : \sigma \vdash e' : \tau'.$$

I apply the ($\forall$ Intro) rule to the first judgment and find

$$\exists\bar{\beta}.C_1, \psi_1\Gamma_x \vdash e : \sigma$$

where $\sigma = \forall\bar{\beta}.C_1. \Rightarrow \tau$. Please note, the set $\mathcal{S}$ of solved forms needs not necessarily to be closed under projection but the conditions put on *gen* ensure that $\exists\bar{\beta}.C_1$ is in solved form. Now, I transform both judgments in a similar way as in the previous

case and get

$$C, \psi\Gamma_x \vdash e : \psi\sigma$$

and

$$C, \psi\Gamma_x.x : \psi\sigma \vdash e : \psi\tau'.$$

I apply the (Let) rule and obtain

$$C, \psi\Gamma_x \vdash e : \psi\tau'.$$

■

I now prove completeness for HM(X) type systems in term form where X in $\mathcal{X}^{reg}$ or in $\mathcal{X}^c$ and X satisfies the principal constraint property. In case X belongs to $\mathcal{X}^{reg}$, I additionally require that X enjoys unitary unification. Some technical lemmas (which I will point out) rely on the fact that X is regular. The proof for $\mathcal{X}^c$ is similar, but there I only need weaker versions of these technical lemmas which do not rely on the regularity of the constraint system. First, I prove completeness for X in $\mathcal{X}^{reg}$. Then I present a completeness proof for X in $\mathcal{X}^c$.

The generalization relation $\vdash^{gen}$ relates a constraint C, a type environment Γ and a type τ and to the generalized constraint and type, written $C, \Gamma, \tau \vdash^{gen} (C', \sigma)$. I use two specialized generalization versions which only relate either the specialized constraint or the specialized type. $C, \Gamma, \tau \vdash^{gen} (C', \sigma)$ iff $C, \Gamma, \tau \vdash^{gen_1} C'$ and $C, \Gamma, \tau \vdash^{gen_2} \sigma$.

The following two lemmas rely on the fact that I only consider regular theories. I give the proof for one lemma where one can see that X needs to be a regular theory. The first lemma states that I can lift entailment between two constraints to the generalized constraints. It is also essential that the set $\mathcal{S}$ of solved forms is closed

under projection.

Lemma 33 *Given a type context* Γ*, constraints* $C, \tilde{C}$*, types* τ, τ' *and substitutions* ϕ, ϕ', ψ *such that* $C \vdash^e \phi'\tilde{C}$ *and* $\psi \leq^{\phi'}_{fv(\Gamma)} \phi$*. Then* $C_o \vdash^e \phi'\tilde{C}_o$ *where* $C, \phi\Gamma, \tau' \vdash^{gen_2} C_o$ *and* $\tilde{C}, \psi\Gamma, \tau \vdash^{gen_2} \tilde{C}_o$*.*

Proof: W.l.o.g. I assume $C_o = \exists\bar{\alpha}.C$ and $\tilde{C}_o = \exists\bar{\beta}.\tilde{C}$. I show that $\bar{\alpha} \notin fv(\phi'\tilde{C}_o)$. Assume the contrary. W.l.o.g.

$$\bar{\alpha} \notin fv(\Gamma) \cup fv(\tilde{C}_o) \cup codom(\phi) \tag{C.1}$$

because I can always rename bound variables and during type inference always new type variables have been introduced. That means there is a $\gamma \in fv(\tilde{C}_o)$ such that $\bar{\alpha} \in fv(\phi'(\gamma))$. Further it holds that $\gamma \notin fv(\psi\Gamma)$. Assume $\gamma \in fv(\psi\Gamma)$ then there is a $\delta \in fv(\Gamma)$ such that $\delta \in fv(\psi(\gamma))$. I know that $\phi(\delta) = \phi' \circ \psi(\delta)$ (here I need the fact that X is regular, both sides of the equation contain the same set of free variables) and then I find $\bar{\alpha} \in codom(\phi)$ which is a contradiction to C.1. I get $\gamma \notin fv(\psi\Gamma)$ and $\gamma \in fv(\tilde{C}_o)$. But this is again a contradiction because $\tilde{C}_o$ is a generalized constraint. Our starting assumption was false and I find that $\bar{\alpha} \notin fv(\phi'\tilde{C}_o)$.

Now, I can conclude that $\tilde{C} \vdash^e \tilde{C}_o$. Then it follows that $\phi'\tilde{C} \vdash^e \phi'\tilde{C}_o$. This yields $C \vdash^e \phi'\tilde{C}_o$. Finally, I obtain $C_o \vdash^e \exists\bar{\alpha}.\phi'\tilde{C}_o$ and because $\bar{\alpha} \notin fv(\phi'\tilde{C}_o)$ that I means I get $C_o \vdash^e \phi'\tilde{C}_o$ as desired. ∎

Remark 6 *The proof of the previous lemma relies on the fact that X is regular. For X in* $\mathcal{X}^c$ *I only need a restricted version of this lemma. Therefore, I still can achieve complete type inference for X in* $\mathcal{X}^c$*.*

The next lemma is similar to the previous one, except that it compares types instead of constraints.

Lemma 34 *Given a type context* Γ*, constraints* $C, \tilde{C}$*, types* τ, τ' *and substitutions* ϕ, ϕ', ψ *such that* $C \vdash^e \phi'\tilde{C}$*,* $C \vdash^i \phi'\tau \preceq \tau'$ *and* $\psi \leq^{\phi'}_{fv(\Gamma)} \phi$*. Then* $\vdash^i \phi'\tilde{\sigma}_o \preceq \sigma_o$ *where* $C, \phi\Gamma, \tau' \vdash^{gen_1} \sigma_o$ *and* $\tilde{C}, \psi\Gamma, \tau \vdash^{gen_1} \tilde{\sigma}_o$*.*

The next lemma states that I can lift some properties about a constraint and a substitution to the same constraint but extended substitution.

Lemma 35 *Given a set* U *of variables, constraints* C_1, C' *and substitutions* $\psi, \psi_1, \psi_2, \phi, \phi_1$ *such that* $\psi_1 C_1 = C_1$*,* $C' \vdash^e \phi'_1 C_1$*,* $\psi = \psi_1 \sqcup \psi_2$*,* $\psi \leq^{\phi'}_U \phi$*,* $\psi_1 \leq^{\phi'_1}_U \phi$*,* $codom(\psi_2) \cap fv(C_1) \subseteq U$ *and* $codom(\psi_1) \cap fv(C_2) \subseteq U$*. Then* $C' \vdash^e (\phi' \circ \psi)C_1$*.*

Proof: From the assumption I can follow that there is a ρ_1 such that $\psi = \rho_1 \circ \psi_1$. Then I find that

$$\begin{aligned} \phi &= (\phi' \circ \psi)_{|U} \\ &= (\phi' \circ \rho_1 \circ \psi_1)_{|U} \\ &= (\phi'_1 \circ \psi_1)_{|U}. \end{aligned}$$

I can deduce that

$$(\phi' \circ \rho_1)_{|codom(\psi_1) \cup (U \setminus dom(\psi_1))} = \phi'_{1|codom(\psi_1) \cup (U \setminus dom(\psi_1))}.$$

W.l.o.g. I can assume that

$$\begin{aligned} dom(\phi'_1) &\subset codom(\psi_1) \cup (U \setminus dom(\psi_1)) \quad \text{and} \\ dom(\phi' \circ \rho_1) &\subset codom(\psi_1) \cup codom(\psi) \cup U. \end{aligned}$$

It is an easy observation that

$$codom(\psi) \subseteq codom(\psi_1) \cup codom(\psi_2).$$

Then I find that

$$dom(\phi' \circ \rho_1) \subset codom(\psi_1) \cup codom(\psi_2) \cup U.$$

This leads to

$$dom(\phi' \circ \rho_1) \cap fv(C_1) \subset codom(\psi_1) \cup (U \backslash fv(\psi_1)).$$

Then I can deduce that

$$(\phi' \circ \psi)C_1 = (\phi' \circ \rho_1)C_1 = \phi'_1 C_1.$$

From $C' \vdash^e \phi'_1 C_1$ I get as desired that

$$C' \vdash^e (\phi' \circ \psi)C_1.$$

■

The next Lemma is similar to the previous one but it is stated for the $\vdash^i$ relation.

Lemma 36 *Given a set U of variables, a constraint C', type schemes $\tilde{\sigma}, \sigma''$ and substitutions $\psi, \psi_1, \psi_2, \phi, \phi_1$ such that $\psi_1\tilde{\sigma} = \tilde{\sigma}$, $C' \vdash^i \phi'_1\tilde{\sigma} \preceq \sigma''$, $\psi = \psi_1 \sqcup \psi_2$, $\psi \leq^{\phi'}_U \phi$, $\psi_1 \leq^{\phi'_1}_U \phi$, $codom(\psi_2) \cap fv(C_1) \subseteq U$ and $codom(\psi_1) \cap fv(C_2) \subseteq U$. Then $C' \vdash^i (\phi' \circ \psi)\tilde{\sigma} \preceq \sigma''$.*

Lemma 37 (Completeness of $\vdash^W$) *Given $C', \phi\Gamma \vdash^s e : \tau'$. Then*

$$\psi, C, \Gamma \vdash^W e : \tau$$

for some substitutions ψ, ϕ', constraint C and type τ such that,

$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^e \phi' C \quad C' \vdash^i \phi' \tau \preceq \tau'.$$

Proof: In this case I can not use induction on the derivation $\vdash^s$. I run into problems in case of the (Let) rule. In order to apply the induction hypothesis I have to rewrite the right premise of the (Let) rule. But this would break the inductive proof on the derivation $\vdash^s$. The trick is that first I perform induction on the *size* of expressions and then on the number of (Sub) rules. In Figure C.1 I introduce a measure on the size of expressions which is parameterized on a map w from identifiers to positive integers. This method appears also in a technical report by Kennedy [Ken96]. He mentions that the original idea is due to Tobias Nipkow.

The proof proceeds first by induction on $\delta(w, e)$ and then on the number of occurrences of rule (Sub) in the original derivation.

Case (Var) I find the following situation:

$$C', \phi\Gamma \vdash^s e : \tau'$$

where $\sigma = \forall \bar{\alpha}.D \Rightarrow \tau \in \Gamma$ and $C' \vdash^i \phi\sigma \preceq \tau'$. I set $\pi = [\bar{\beta}/\bar{\alpha}]$ as a variable renaming substitution where $\bar{\beta}$ are fresh type variables. I get that

$$C' \vdash^e (\delta \circ \phi \circ \pi)D \qquad C' \vdash^i (\delta \circ \phi \circ \pi)\tau \preceq \tau'$$

where δ is a solution to the bound type variables. Then, $(C', \delta \circ \phi \circ \pi)$ is a normal form of (D, π). Because HM(X) satisfies the principal constraint property I get that

(C, ψ) is a principal normal form of (D, π). It holds that

$$\psi \leq^{\phi'} \delta \circ \phi \circ \pi \qquad C' \vdash^{e} \phi' C.$$

Furthermore, I can conclude that

$$C' \vdash^{i} (\phi' \circ \psi)\tau \preceq \tau' \qquad (\delta \circ \phi \circ \pi)_{|fv(\Gamma)} = \phi.$$

I get that $\psi \leq^{\phi'}_{fv(\Gamma)} \phi$ and I am done.

Case (Sub) I find the following situation:

$$\frac{C', \phi\Gamma \vdash^{s} e : \tau' \qquad C' \vdash^{e} (\tau' \preceq \tau'')}{C', \phi\Gamma \vdash^{s} e : \tau''}$$

By induction I find that

$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^{e} \phi' C \quad C' \vdash^{i} \phi'\tau \preceq \tau'$$

for some substitution ϕ' and ψ. It follows immediately that $C' \vdash^{i} \phi'\tau \preceq \tau''$ and I am done.

Case (Abs) I find the following situation:

$$\frac{C', \phi\Gamma_x.x : \tau' \vdash^{s} e : \tau''}{C', \phi\Gamma_x \vdash^{s} \lambda x.e : \tau' \to \tau''}$$

I extend the substitution ϕ and define $\tilde{\phi} = \phi \circ [\tau'/\alpha]$ where α is new. Obviously, it

holds that

$$\vdash^i \tilde{\phi}(\Gamma.x : \alpha) \leq \Gamma'.x : \tau'.$$

Application of the induction hypothesis yields

$$\begin{gathered} \psi, C, \Gamma_x.x : \alpha \vdash^W e : \tau \quad \psi \leq^{\phi'}_{fv(\Gamma)\cup\{\alpha\}} \tilde{\phi} \\ C' \vdash^e \phi' C \quad C' \vdash^i \phi'\tau \preceq \tau'. \end{gathered} \tag{C.2}$$

Obviously, it holds that

$$\psi_{\setminus\{\alpha\}} \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^i \phi'(\alpha \to \tau) \preceq \tau' \to \tau''.$$

Then, I apply the (Abs) rule and get

$$\psi_{\setminus\{\alpha\}}, C, \Gamma_x \vdash^W \lambda x.e : \psi(\alpha) \to \tau.$$

Case (App) I find the following situation:

$$\frac{C', \phi\Gamma \vdash^s e_1 : \tau_1' \to \tau_2' \quad C', \phi\Gamma \vdash^s e_2 : \tau_1'}{C', \phi\Gamma \vdash^s e_1 e_2 : \tau_2'}$$

Application of the induction hypothesis yields

$$\begin{gathered} \psi_1, C_1, \Gamma \vdash^W e_1 : \tau_1 \quad \psi_1 \leq^{\phi_1'}_{fv(\Gamma)} \phi \\ C' \vdash^e \phi_1' C_1 \quad C' \vdash^i \phi_1'\tau_1 \preceq \tau_1' \to \tau_2'. \end{gathered} \tag{C.3}$$

and

$$\begin{gathered} \psi_2, C_2, \Gamma \vdash^W e_2 : \tau_2 \quad \psi_2 \leq^{\phi_2'}_{fv(\Gamma)} \phi \\ C' \vdash^e \phi_2' C_2 \quad C' \vdash^i \phi_2'\tau_2 \preceq \tau_1' \end{gathered}$$

By assumption, unification is unitary. Then, I can set $\psi' = \psi_1 \sqcup \psi_2$ and find that $\psi' \leq^{\phi'}_{fv(\Gamma)} \phi$. I want to apply Lemmas 35, 36. I identify the set U in these lemmas with $fv(\Gamma)$. I assume that type variables introduced in one part of the inference tree do not appear in the other part. Formally, this means that

$$codom(\psi_2) \cap fv(C_1) \subseteq fv(\Gamma)$$

and

$$codom(\psi_1) \cap fv(C_2) \subseteq fv(\Gamma).$$

All preconditions of Lemmas 35, 36 are fulfilled. I can conclude that

$$C' \vdash^e (\phi' \circ \psi')C_1 \qquad C' \vdash^i (\phi' \circ \psi')\tau_1 \preceq \tau_1' \to \tau_2'$$
$$C' \vdash^e (\phi' \circ \psi')C_2 \qquad C' \vdash^i (\phi' \circ \psi')\tau_2 \preceq \tau_1'.$$

I set $D = C_1 \wedge C_2 \wedge (\tau_1 \preceq \tau_2 \to \alpha)$ where α is a fresh type variable. Then I obtain that $C' \vdash^e (\phi' \circ \psi' \circ [\tau_2'/\alpha])D$. I find that $(C', \phi' \circ \psi' \circ [\tau_2'/\alpha])$ is a normal form of (D, ψ'). By assumption HM(X) satisfies the principal constraint property. I obtain that (C, ψ) is the principal normal form of (D, ψ') where $\psi \leq^{\phi''} \phi' \circ \psi' \circ [\tau_2'/\alpha]$. Because (C, ψ) is principal I find that $C' \vdash^e \phi''C$. W.l.o.g. $(\phi' \circ \psi')\tau_2' = \tau_2'$. Then, I can conclude that $(\phi' \circ \psi' \circ [\tau_2'/\alpha])_{|fv(\Gamma)} = \phi$. This leads to $\psi \leq^{\phi''}_{fv(\Gamma)} \phi$. Furthermore, it holds that $\phi''(\alpha) = \tau_2'$ because

$$\tau_2' = \phi' \circ \psi' \circ [\tau_2'/\alpha](\alpha) = \phi'' \circ \psi(\alpha) = \phi''(\alpha).$$

The last reasoning steps holds because α is a new type variable therefore $\alpha \notin dom(\psi)$.

Finally, I apply the (App) rule and find

$$\psi_{|fv(\Gamma)}, C, \Gamma \vdash^W e_1 e_2 : \psi(\alpha)$$

which establishes the induction step.

Case (Let) I find the following situation:

$$\frac{C_1, \phi\Gamma_x \vdash^s e : \tau \quad C_1, \phi\Gamma_x, \tau \vdash^{gen} \sigma, C_2 \qquad C_3, \phi\Gamma_x.x : \sigma \vdash^s e' : \tau'}{C_2 \wedge C_3, \phi\Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

Induction hypothesis applied to the left part yields

$$\begin{array}{c} \psi_1, \tilde{C}_1, \Gamma_x \vdash^W e : \tau_1 \quad \psi_1 \leq^{\phi_1'}_{fv(\Gamma_x)} \phi \\ C_1 \vdash^e \phi_1' \tilde{C}_1 \quad C_1 \vdash^i \phi_1' \tau_1 \preceq \tau. \end{array} \tag{C.4}$$

From Lemma 34 and Lemma 33 and C.4 I obtain that

$$C_2 \vdash^e \phi_1' \tilde{C}_2 \qquad \vdash^i \phi_1' \sigma_1 \preceq \sigma \tag{C.5}$$

where $(\sigma_1, \tilde{C}_2) = gen(\tilde{C}_1, \psi\Gamma_x, \tau_1)$. I set $\tilde{\phi} = \phi_1' \circ \phi$. Then it holds that

$$\vdash^i \tilde{\phi}(\Gamma_x.x : \sigma_1) \preceq \phi\Gamma_x.x : \sigma \tag{C.6}$$

because

$$\tilde{\phi}\sigma_1 = (\phi_1' \circ \phi)\sigma_1 = (\phi_1' \circ (\phi_1' \circ \psi_1)_{|fv(\Gamma)})\sigma_1 = \phi_1'\sigma_1.$$

An easy observation yields

$$\tilde{\phi}_{|fv(\Gamma_x)} = \phi. \tag{C.7}$$

I rewrite the right premise with the stronger type environment in C.6 and find

$$C_3, \tilde{\phi}(\Gamma_x.x : \sigma_1) \vdash^s e' : \tau'.$$

Now, I am able to apply the induction hypothesis to the right part and find

$$\begin{gathered} \psi_2, \tilde{C_3}, \Gamma_x.x : \sigma_1 \vdash^W e' : \tau_1' \\ \psi_2 \leq^{\phi_2'}_{fv(\Gamma_x) \cup fv(\sigma_1)} \tilde{\phi} \\ C_3 \vdash^e \phi_2' \tilde{C_3} \quad C_3 \vdash^i \phi_2' \tau_1' \preceq \tau'. \end{gathered} \tag{C.8}$$

From C.4 I can deduce that

$$\psi_1 \leq^{\phi_1'}_{fv(\Gamma_x) \cup fv(\sigma_1)} \tilde{\phi}. \tag{C.9}$$

Because of C.4 and C.7 it holds that

$$(\phi_1' \circ \psi_1)_{|fv(\Gamma_x)} = \phi = \tilde{\phi}_{|fv(\Gamma_x)}$$

and if $\alpha \in fv(\sigma_1)$ I can assume that $\alpha \notin fv(\Gamma_x)$ then I know that

$$\phi(\alpha) = \alpha \quad \psi_1(\alpha) = \alpha.$$

I can deduce that

$$\phi_1' \circ \psi_1(\alpha) = \phi_1'(\alpha) = \tilde{\phi}(\alpha).$$

Then from C.8 and C.9 I find that ψ_1 and ψ_2 have an upper bound. By assumption,

unification is unitary, hence least upper bound exists. It holds that

$$\psi' \leq^{\phi'}_{fv(\Gamma_x) \cup fv(\sigma)} \phi \tag{C.10}$$

where $\psi' = \psi_1 \sqcup \psi_2$. With C.7 and from C.10 I find that

$$\psi' \leq^{\phi'}_{fv(\Gamma_x)} \phi.$$

From C.5 and C.4 I know that

$$C_2 \vdash^e \phi'_1 \tilde{C}_2 \quad C_3 \vdash^e \phi'_2 \tilde{C}_3 \quad C_3 \vdash^i \phi'_2 \tau'_1 \preceq \tau'.$$

As in the (App) case I can conclude from Lemmas 35, 36 that

$$C_2 \vdash^e (\phi' \circ \psi')\tilde{C}_2 \qquad C_3 \vdash^e (\phi' \circ \psi')\tilde{C}_3$$

and

$$C_3 \vdash^i (\phi' \circ \psi')\tau'_1 \preceq \tau'.$$

I set $D = \tilde{C}_2 \wedge \tilde{C}_3$. Then I obtain that $(C_2 \wedge C_3, \phi' \circ \psi')$ is a normal form of (D, ψ'). By assumption HM(X) satisfies the principal constraint property. Assume (C, ψ) is the principal normal form of (D, ψ') where $\psi \leq^{\phi''} \phi' \circ \psi'$. Now, I can apply the (Let) rule and find

$$\psi_{|fv(\Gamma)}, C, \Gamma_x \vdash^W \mathsf{let}\ x = e\ \mathsf{in}\ e' : \psi\tau'_1.$$

Furthermore, I obtain that

$$C_2 \wedge C_3 \vdash^e \phi'' C \qquad C_2 \wedge C_3 \vdash^i (\phi'' \circ \psi)\tau'_1 \preceq \tau'$$

where $\psi \leq^{\phi''}_{fv(\Gamma_x)} \phi$.

■

The following theorem states completeness of inference for the class $\mathcal{X}^{reg}$.

Theorem 25 *Given $C', \phi\Gamma \vdash e : \sigma'$ and $\phi\Gamma$ is realizable in C'. Then*

$$\psi, C, \Gamma \vdash^{W} e : \tau$$

for some substitutions ϕ', ψ, constraints C, C_o and types τ, σ_o such that,

$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^{e} \phi' C_o \quad C' \vdash^{i} \phi'\sigma_o \preceq \sigma' \; and C, \psi\Gamma, \tau \vdash^{gen} (\sigma_o, C_o).$$

Proof: First, I apply Lemma 15 in order to get a derivation in $\vdash^{n}$. Then, I can apply Lemma 17 (completeness of $\vdash^{s}$). This yields

$$\begin{array}{lll} \text{(a)} \quad \sigma' = \tau : & C, \phi\Gamma \vdash^{s} e : \tau \quad C' \vdash^{e} C \\ \text{(b) otherwise :} & C, \phi\Gamma \vdash^{s} e : \tau \quad C, \phi\Gamma, \tau \vdash^{gen} (\sigma_o, C_o) \\ & C' \vdash^{e} C_o \quad C' \vdash^{i} \sigma_o \preceq \sigma' \end{array} \tag{C.11}$$

After that I apply Lemma 37 (completeness of $\vdash^{W}$) and find

$$\begin{array}{c} \psi, \tilde{C}, \Gamma \vdash^{W} e : \tilde{\tau} \quad \psi \leq^{\phi'}_{fv(\Gamma)} \phi \\ C \vdash^{e} \phi' \tilde{C} \quad C \vdash^{i} \phi' \tilde{\tau} \preceq \tau \end{array}$$

I find $C, \psi\Gamma, \tau \vdash^{gen} (\tilde{\sigma}_o, \tilde{C}_o)$ for some $\tilde{\sigma}_o, \tilde{C}_o$. It remains to show

1. $C' \vdash^{i} \phi' \tilde{\sigma}_o \preceq \sigma'$
2. $C' \vdash^{e} \phi' \tilde{C}_o$.

This fact follows by application of the Lifting Lemmas 35, 36.

■

I now give a completeness proof for $\mathcal{X}^c$. I assume that I have an HM(X) type system where X in $\mathcal{X}^c$ and X satisfies the principal constraint property. I have already made the observation that in this case type inference only represents a satisfiability test and $\vdash^W$ returns always the identity substitution. The proof follows the same steps as in case of $\mathcal{X}^{reg}$. I show that all introduced derivations are equivalent. Most of the work is already done, I only have to restate Lemma 37.

I have already mentioned that the Lifting Lemmas 33 and 34 do not hold in general for non–regular theories. But I can state specialized versions of the Lifting Lemmas which do apply in this case. I use the convention that two substitutions are *syntactically* equal if they denote the same mapping. This is in contrast to the usual definition for equality between substitutions where equality breaks down to equality with respect to the underlying equational theory.

Lemma 38 *Given a type context* Γ*, constraints* $C, \tilde{C}$*, types* τ, τ' *and substitutions* ϕ, ϕ' *such that* $C \vdash^e \phi'\tilde{C}$ *and* $id \leq^{\phi'}_{fv(\Gamma)} \phi$ *where* ϕ' *is syntactically equal to* ϕ *on* $fv(\Gamma)$*. Then* $C_o \vdash^e \phi'\tilde{C}_o$ *where* $C, \phi\Gamma, \tau' \vdash^{gen_2} C_o$ *and* $\tilde{C}, \Gamma, \tau \vdash^{gen_2} \tilde{C}_o$*.*

Lemma 39 *Given a type context* Γ*, constraints* $C, \tilde{C}$*, types* τ, τ' *and substitutions* ϕ, ϕ' *such that* $C \vdash^e \phi'\tilde{C}$*,* $C \vdash^i \phi'\tau \preceq \tau'$ *and* $id \leq^{\phi'}_{fv(\Gamma)} \phi$ *where* ϕ' *is syntactically equal to* ϕ *on* $fv(\Gamma)$*. Then* $\vdash^i \phi'\tilde{\sigma}_o \preceq \sigma_o$ *where* $C, \phi\Gamma, \tau' \vdash^{gen_1} \sigma_o$ *and* $\tilde{C}, \Gamma, \tau \vdash^{gen_1} \tilde{\sigma}_o$*.*

Remark 7 *Both original lemmas can only be proven for non–regular theories. But it is interesting to point that both lemmas hold for arbitrary theories if the substitution* ψ *is equal to the identity substitution. In the previous proof I showed that* $\gamma \notin fv(\psi\Gamma)$*.*

Assume now that $\psi = id$ and $\gamma \in fv(\Gamma)$. But then I get $\phi'(\gamma) = \phi(\gamma)$ which yields that $\bar{\alpha} \in codom(\phi)$ (I can make this step because I know that ϕ' and ϕ are syntactically equal on $fv(\Gamma)$). This is a contradiction to our assumption. Hence, I find $\gamma \notin fv(\Gamma)$. The rest of the proof remains the same.

Lemma 40 (Completeness of $\vdash^W$) *Given $C', \phi\Gamma \vdash^s e : \tau'$. Then*

$$id, C, \Gamma \vdash^W e : \tau$$

for some constraint C, type τ and substitution ϕ' such that,

$$id \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^e C \quad C' \vdash^i \tau \preceq \tau'.$$

Proof: It is an easy observation that I can always choose ϕ' such that ϕ' and ϕ are syntactically equal on $fv(\Gamma)$. I need this condition to apply the Lifting Lemmas. As before the proof proceeds first by induction on $\delta(w, e)$ and then on the number of occurrences of rule (Sub) in the original derivation. There is only one interesting case.

Case (Let) I find the following situation:

$$\frac{C_1, \phi\Gamma_x \vdash^s e : \tau \quad C_1, \phi\Gamma_x, \tau \vdash^{gen} (\sigma, C_2) \qquad C_2, \phi\Gamma_x.x : \sigma \vdash^s e' : \tau'}{C_2 \wedge C_3, \Gamma_x \vdash^s \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

I apply the induction hypothesis to the right premise and obtain

$$id, \tilde{C}_1, \Gamma_x \vdash^W e : \tau_1$$
$$id \leq^{\phi_1'}_{fv(\Gamma)} \phi \quad C_1 \vdash^e \phi_1' \tilde{C}_1 \qquad C_1 \vdash^i \phi_1' \tau_1 \preceq \tau$$

I find $\tilde{C}_1, \Gamma_x, \tau_1 \vdash^{gen} (\sigma', \tilde{C}_2)$ for some $\sigma', \tilde{C}_2$. I get that $\vdash^i \phi_1' \sigma' \preceq \sigma$ and $C_2 \vdash^e \phi_1' \tilde{C}_2$ by application of the Lifting Lemmas 38 and 39. As before I set $\tilde{\phi} = \phi \circ \phi_1'$ I can rewrite the left premise and find

$$C_2, \tilde{\phi}(\Gamma_x.x : \sigma') \vdash^s e' : \tau'$$

I apply the induction hypothesis and obtain

$$id, \tilde{C}_3, \Gamma_x.x : \sigma' \vdash^W e' : \tau_2$$
$$id \leq^{\phi_2'}_{fv(\Gamma)} \phi \quad C_3 \vdash^e \phi_2' \tilde{C}_3 \quad C_3 \vdash^i \phi_2' \tau_2 \preceq \tau''$$

I can observe that ϕ_1' and ϕ_2' are identical on $fv(\Gamma)$ otherwise they act on different variables. Therefore, I can combine them and set $\phi' = \phi_1' \sqcup \phi_2'$. Furthermore, normalization succeeds because $C_2 \wedge C_3$ is satisfiable. I apply the (Let) rule and find

$$id, \tilde{C}_2 \wedge \tilde{C}_3, \Gamma_x \vdash^W \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau_2$$
$$id \leq^{\phi'}_{fv(\Gamma)} \phi \quad C_2 \wedge C_3 \vdash^e \phi'(\tilde{C}_2 \wedge \tilde{C}_3) \quad C_2 \wedge C_3 \vdash^i \phi' \tau_2 \preceq \tau''$$

This establishes the induction step. ∎

The following theorem can be proven similar in style as in case for $\mathcal{X}^{reg}$. It states completeness of inference for the class $\mathcal{X}^c$.

Theorem 26 *Given* $C', \phi\Gamma \vdash e : \sigma'$ *and* $\phi\Gamma$ *is realizable in* C'*. Then*

$$id, C, \Gamma \vdash^{W} e : \tau$$

for some constraints C, C_o*, types* τ, σ_o *and substitution* ϕ' *such that,*

$$id \leq^{\phi'}_{fv(\Gamma)} \phi \quad C' \vdash^{e} \phi' C_o \quad C' \vdash^{i} \phi'\sigma_o \preceq \sigma' \text{ and}$$
$$C, \Gamma, \tau \vdash^{gen} (\sigma_o, C_o).$$

Theorem 13 (Completeness of Inference) follows from Theorem 25 and 26.

C.8 Proofs of Section 4.4

I prove soundness and completeness of the revised term–based inference system, given in Figure 4.4. The proofs relies on sound and complete inference for the constraint–based system. I omit soundness and completeness proofs for the constraint–based system which are straightforward and analogous to the completeness result for the class $\mathcal{X}^c$ in case of the term–based inference system. The technical problem lies in transforming judgments in constraint form into judgments in term form. Remember, the inference system in term form in Section 4.4.2 performs inference first in the purely constraint–based fragment of HM(X). Term reconstruction is performed at the end. The output is a judgment in the term–based fragment of HM(X). Technically, I need to show how to transform judgments (derivation trees) of the constraint–based fragment of HM(X) into judgments (derivation trees) of the term–based fragment. This can be done by applying the Transformation Theorem 7 (see Section 3.6).

Theorem 16 (Soundness of Inference) *Given an instance of the* HM(X) *system*

where X *satisfies the principal constraint property and the set* $\mathcal{S}$ *is closed under projection. Given a type environment* Γ *and a term* e*. If* $\psi, C, \Gamma \vdash^{W_{term}} e : \tau$ *then* $C, \psi\Gamma \vdash e : \tau$ *where* $\psi C =^{e} C$ *and* $\psi\tau = \tau$*.*

Proof: I only consider the final step of the derivation $\vdash^{W_{term}}$.

$$\frac{C, \Gamma \vdash^{W_c} e : \alpha \qquad (D, \psi) = normalize(C, id)}{\psi_{|fv(\Gamma)}, D, \Gamma \vdash^{W_{term}} e : \psi(\alpha)}$$

From the premise, I obtain

$$C, \Gamma \vdash^{\mathcal{X}^c} e : \alpha$$

because constraint–based inference is sound. Furthermore, I know that $D \vdash^{e} \psi C$ which yields that

$$D, \psi\Gamma \vdash^{\mathcal{X}^c} e : \psi(\alpha)$$

Normalization ensures that $\psi\Gamma$ is in $\mathcal{X}^t$. I apply the Transformation Theorem 7 and obtain

$$D, \psi\Gamma \vdash^{\mathcal{X}^t} e : \psi(\alpha)$$

and I am done. ■

Theorem 17 (Completeness of Inference) *Given an instance of the* HM(X) *system where* X *satisfies the principal constraint property. Given* $C, \phi\Gamma \vdash^{\mathcal{X}^t} e : \sigma$ *such that* $\phi\Gamma$ *realizable in* C*. Then* $\psi, D, \Gamma \vdash^{W} e : \tau$ *for some substitution* ψ*,*

constraints D, C_o and types τ, σ_o such that

$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \qquad C \vdash^{i} \phi'\sigma_o \preceq \sigma \qquad C \vdash^{e} \phi' C_o$$
$$D, \psi\Gamma, \tau \vdash^{gen} (\sigma_o, C_o)$$

Proof: Given $C, \phi\Gamma \vdash^{\mathcal{X}^t} e : \sigma$. I perform type inference in $\mathcal{X}^c$ and obtain

$$\tilde{C}, \Gamma \vdash^{W_c} e : \alpha$$

such that

$$id \leq^{\phi''}_{fv(\Gamma)} \phi \quad C \vdash^{e} \phi'' C'_o \quad C \vdash^{i} \phi''\sigma'_o \preceq \sigma$$

where $\sigma'_o = \forall\alpha. \Rightarrow \tilde{C}$ and $C_o = \exists\alpha.\tilde{C}$. Furthermore, I assume that $\sigma = \forall\bar{\gamma}.D' \Rightarrow \tau'$. From $C \vdash^{i} \phi''\sigma_o \preceq \sigma$ I can deduce that there exists $\pi = [\tau/\alpha]$ such that

$$C \wedge D' \vdash^{e} (\pi \circ \phi'')\tilde{C} \qquad C \wedge D' \vdash^{i} (\pi \circ \phi'')\tau \preceq \tau'. \tag{C.12}$$

But then I know the principal normal form of $\tilde{C}$ exists. I set

$$(D, \psi) = normalize(\tilde{C}, id)$$

and find that

$$\psi \leq^{\phi'''} (\pi \circ \phi'') \qquad C \wedge D' \vdash^{e} \phi''' D. \tag{C.13}$$

I can apply the term reconstruction step and find that

$$\psi, D, \Gamma \vdash^{W_{term}} e : \psi(\alpha).$$

I find $D, \psi\Gamma, \psi(\alpha) \vdash^{gen} (\sigma_o, C_o)$ where I assume $\sigma_o = \forall\bar{\beta}.D \Rightarrow \psi(\alpha)$ and $C_o = \exists\bar{\beta}.D$. From C.12 and C.13 I follow that

$$C \vdash^{i} \phi'''(\forall\bar{\beta}.D \Rightarrow \psi(\alpha)) \preceq \sigma.$$

Furthermore, it holds that $\psi \leq^{\phi'''}_{fv(\Gamma)} \phi$.

It remains to prove that $C \vdash^{e} \phi'''C_o$. By assumption $\phi\Gamma$ is realizable in C, hence also σ is realizable in C. This implies there is a θ such that $C \vdash^{e} \theta D'$. I set $\phi' = \theta \circ \phi'''$. The substitution ϕ' satisfies the same properties as the substitution ϕ'''. From $C \wedge D' \vdash^{e} \phi'''D$ I can follow that $C \vdash^{e} \phi' C_o$ and find that

$$\psi \leq^{\phi'}_{fv(\Gamma)} \phi \qquad C \vdash^{i} \phi'\sigma_o \preceq \sigma \qquad C \vdash^{e} \phi' C_o$$

which establishes the completeness result.

■

Theorem 18 (Equivalence of $\vdash^{W_{term}}$ and $\vdash^{W_{normalize}}$) *Given an* HM(X) *instance where* X *satisfies the lifting property. Then* $\phi, D, \Gamma \vdash^{W_{term}} e : \tau$ *iff* $\phi, D, \Gamma \vdash^{W_{normalize}} e : \tau$.

Proof: The right–side follows trivially from the left–side. I give a proof sketch for the other direction. The essential observation is that application of the (Normalize) rule can be omitted and it is sufficient to perform only the term reconstruction step at the very end. I first make some observations about the normalization procedure. By assumption the lifting property holds. It follows that

$$\mathit{normalize}(D \wedge \phi, \mathit{id}) = \mathit{normalize}(\phi D, \phi).$$

This justifies to define $normalize(C) = D \wedge \phi$ where (D, ϕ) principal normal form of (C, id). By similar reasoning as in Lemma 30 and Theorem 9 it follows that

(1) $normalize(normalize(C_1) \wedge normalize(C_2)) = normalize(C_1 \wedge C_2)$

(2) $normalize(\exists \bar{\alpha}.normalize(C)) = normalize(\exists \bar{\alpha}.C)$

I consider now given the proof tree of a derivation $C, \Gamma \vdash^{Wc} e : \beta$ with possible applications of the (Normalize) rule. Constraints in judgments $C, \Gamma \vdash^{Wc} e : \beta$ can be of the following form:

$$C \quad ::= \quad p(\bar{\tau}) \mid C \wedge C \mid \exists.\bar{\alpha}.C \mid normalize(C)$$

A double induction over the number of (Normalize) and (Var) rule applications shows that judgment $C, \Gamma \vdash^{Wc} e : \beta$ can be transformed into a judgment $C', \Gamma \vdash^{Wc} e : \beta$ where all normalize functions in the constraint part of are omitted, i.e. C' is the constraint resulting from C by erasing all normalize functions in C. Because of equations (1) and (2) it holds that $normalize(C) = normalize(C')$.

This shows given judgment $\phi, D, \Gamma \vdash^{W_{term}} e : \tau$. Then I can derive the same judgment where all applications of the (Normalize) rule are omitted. ∎

C.9 Proofs of Chapter 5

Theorem 23 (Full and Faithful) HM(OVER) *(with or without projection appearing in syntactic form in $\mathcal{S}$) models SystemO fully and faithfully.*

Proof: I have to show that given an expression e and a type environment Γ. Then e is typable under Γ in *SystemO* iff e is typable under Γ in HM(OVER).

It is an easy observation that HM(OVER) models *SystemO* faithful. The ($\forall$ Intro–OVER)

rule in HM(OVER) subsumes the ($\forall$ Intro–*SystemO*) rule in *SystemO*. Hence, every judgment in *SystemO* is also derivable in HM(OVER).

It remains to proof the fullness direction. Given a judgment $C, \Gamma \vdash e : \sigma$ in HM(OVER), I have to show that e is typable under Γ in *SystemO*. I apply induction over the derivation tree of the judgment $C, \Gamma \vdash e : \sigma$. I consider the number of ($\forall$ Intro–OVER) rules with non–trivial projection. Recall, in Section 5.2 I have already observed that the ($\forall$ Intro–OVER) rule is strictly more powerful than the ($\forall$ Intro–*SystemO*) rule. There are cases where rule ($\forall$ Intro–*SystemO*) forbids to quantify over a free type variable. Such cases occur if projection is non–trivial. But this is possible under rule ($\forall$ Intro–OVER), see Example 14. The proof proceeds by rewriting the derivation tree and omitting application of the ($\forall$ Intro–OVER) rule with non–trivial projection. It turns out that application of the ($\forall$ Intro–OVER) with non–trivial projection does not allow to type more programs.

I consider the last occurrence of a ($\forall$ Intro–OVER) rule with non–trivial projection in the derivation tree. Note, I assume that the sub–term in the sub–judgment is used in a monomorphic context. That means, the polymorphic type of the sub–term will be instantiated to a monomorphic type in a later part of the derivation tree. Otherwise, I could immediately omit application of the ($\forall$ Intro–OVER) rule.

I examine the last ($\forall$ Intro–OVER) rule with non–trivial projection in the derivation tree of $C, \Gamma \vdash e : \sigma$. I find the following situation:

$$\frac{C' \wedge D, \Gamma' \vdash e' : \sigma' \qquad \beta \notin \mathit{fv}(\Gamma') \cup \mathit{fv}(C') \qquad \text{there is a } \alpha \text{ such that } \alpha < \beta}{C' \wedge \exists \beta.D, \Gamma' \vdash e' : \forall \beta.D \Rightarrow \sigma'}$$

I consider two cases. If $\alpha \notin \mathit{fv}(\Gamma')$ I could switch the order of quantification and first

quantify over α. Hence, I assume that I deal with a situation where I can not reorder the order of quantification. This enforces that $\alpha \in \mathit{fv}(\Gamma')$. Note, this hold only because I have imposed an ordering relation on the type variables in a constraint. Then, I can consider α as an arbitrary but fixed type variable. This observation is important in a later part of the proof. By assumption the polymorphic type $\forall\beta.D \Rightarrow \sigma'$ is used in a monomorphic context. I consider the possible instantiations for the type variable β. Assume, $\phi_1 = [\tau_1/\beta]$ and $\phi_2 = [\tau_2/\beta]$ are possible instantiations for the type variable β. Furthermore, I know that there must be a constraint of the form $(o :: \alpha \to \tau)$ in D with $\beta \in \mathit{fv}(\tau)$. Note, in general there is a sequence of constraints of the form $(o_1 :: \alpha_1 \to \tau_1), \ldots, (o_n :: \alpha_n \to \tau_n)$ with $\alpha_i \in \mathit{fv}(\tau_{i-1})$ and $\beta \in \mathit{fv}(\tau_n)$. W.l.o.g., I only consider the simplest possible case. Remember, for a fixed α (I know that α can not be a polymorphic type variable) the result type τ in the constraint $(o :: \alpha \to \tau)$ is uniquely determined. This is enforced by the rules put on the constraint system OVER. Hence, I find that $\tau_1 = \tau_2$. Therefore, I can omit application of the ($\forall$ Intro–OVER) rule. Because the polymorphic type variable β can not be instantiated to different types. Instead, I can use the sub–judgment $C' \wedge \exists\beta.D, \Gamma' \vdash e' : \phi_1\sigma$. This yields a derivation tree of judgment $C, \Gamma \vdash e : \sigma$ where the number ($\forall$ Intro–OVER) rules with non–trivial projection is reduced by one. Application of the induction hypothesis yields that the term e is typable under Γ in *SystemO*. Specifically, there is a derivation tree of the judgment $C, \Gamma \vdash e : \sigma$ where all applications of the quantifier introduction rule are instances of rule ($\forall$ Intro–*SystemO*). This concludes the proof.

■

Appendix D

HM(X) in Haskell

```
*-----------------------------------------------
* Specification of the generic HM(X)
* type inference system in Haskell
*
* Parts which need to be filled in to obtain
* a specific instance are highlighted
*
* As an example I consider the standard
* Hindley/Milner system and the dimension
* type system in style of A.J. Kennedy
*-----------------------------------------------

> module Main where
```

```
*---------------------------------------------
* Category: DATA DECLARATIONS
*---------------------------------------------

The type language

> type TVar       = Int
> type Var        = Int
> data Type       = TV TVar
>                 | Type `Fun` Type
>                     deriving Show

The constraint language

> data Predicate  = TRUE
>                 | Type `Subsume` Type
>                     deriving Show

> data Constraint = C [Predicate]
>                 | Proj [TVar] [Predicate]
>                      deriving Show

FILL IN: extend type and constraint language

Type schemes
```

```
> data TypeScheme = Mono Type
>                 | Poly [TVar] Constraint Type
>                      deriving Show
```

Substitutions and type environments

```
> type Subst      = [(TVar, Type)]
> type Typo       = [(Var,TypeScheme)]
```

The term language

```
> data Term       = Id Var | Abs Var Term | App Term Term
>                 | Let Var Term Term
>                      deriving Show
```

The error monad

```
> data E a        = Suc a | Err String
>                      deriving Show
```

The type inference monad - a typing problem consists of a three tupel

```
> data TP a = TP ( (TVar, Subst, Constraint) ->
>                  E (TVar, Subst, Constraint, a))
```

```
*--------------------------------------
```

```
* Category: CLASS DECLARATIONS
*---------------------------------------

> class TypesAndConstraints c where
>  apply :: Subst -> c -> TP c
>  tv    :: c -> [TVar]

apply -- applies a substitution to either a type or constraint
tv -- computes the free type variables

*---------------------------------------
* Category: INSTANCE DECLARATIONS
*---------------------------------------

> instance TypesAndConstraints Predicate where
>   apply s TRUE              = return TRUE
>   apply s (t1 `Subsume` t2) = do t1' <- apply s t1
>                                  t2' <- apply s t2
>                                  return (t1' `Subsume` t2')
>   tv TRUE                   = []
>   tv (t1 `Subsume` t2)      = (tv t1) ++ (tv t2)

> instance TypesAndConstraints c => TypesAndConstraints [c] where
>   apply s  = mapM (apply s)
>   tv  a    = foldl f [] a where
>                       f z x = z ++ (tv x)
```

```
> instance TypesAndConstraints Constraint where
>      apply s (C p)            = do p' <- apply s p
>                                    return (C p')
>      apply s (Proj tvl p)      = do p' <- apply s p
>                                     return (Proj tvl p')
>      tv (C p)                 = tv p
>      tv (Proj tvl p)           = without (tv p)  tvl

> instance TypesAndConstraints Type where
>   apply s (TV tvl)           = case lookup tvl s of
>                                     Just t  -> return t
>                                     Nothing -> return (TV tvl)
>   apply s (t1 `Fun` t2)     = do t1' <- apply s t1
>                                  t2' <- apply s t2
>                                  return (t1' `Fun` t2')
>   tv (TV tvl)                = [tvl]
>   tv (t1 `Fun` t2)          = (tv t1) ++ (tv t2)

> instance TypesAndConstraints TypeScheme where
>   apply s (Poly tvl c t)     = do c' <- apply s c
>                                   t' <- apply s t
>                                   return (Poly tvl c' t')
>   apply s (Mono t)          = do t' <- apply s t
>                                  return (Mono t')
>   tv (Poly tvl c t)          = without ((tv t) ++ (tv c)) tvl
```

```
>   tv (Mono t)               = tv t

FILL IN: instance declarations in case of extended type and
         constraint language

*-------------------------------------------------
* Category: MONAD DECLARATIONS
*-------------------------------------------------

> unTP (TP a) = a

> instance Monad TP where
>  m >>= k = TP ( \ (n,s,c) ->
>                      case unTP (m) (n,s,c) of
>                        Suc (n',s',c',x) -> unTP (k x) (n',s',c')
>                        Err s            -> Err s )
>  return x = TP ( \ (n,s,c) -> return (n,s,c,x) )

> instance Monad E where
>  m >>= k = case m of
>            Suc a -> k a
>            Err s -> Err s
>  return x = Suc x

> report_error :: String -> a -> TP a
> report_error msg x =  TP ( \ (n,s,c) -> Err msg)
```

```
*--------------------------------------

* Category: INITIALIZATIONS

*--------------------------------------


> null_subst :: Subst

> null_subst = []


> init_typo :: Typo


FILL IN: initial type environment


> true_cons :: Constraint

> true_cons = C [TRUE]


> init_tvar :: TVar

> init_tvar = 0


> new_tvar :: TVar -> TVar

> new_tvar n = n + 1


*--------------------------------------

* Category: Core type inferencer

*--------------------------------------


primitives for lists
```

```
> without :: [TVar] -> [TVar] -> [TVar]
> without [] a    = []
> without (x:a) b = if elem x b then without a b
>                   else x:(without a b)
```

primitives for dealing with constraints

```
> add_cons :: Constraint -> Constraint -> Constraint
> add_cons (C p1) (C p2)                = C (p1 ++ p2)
> add_cons (C p1) (Proj tv p2)          = Proj tv (p1 ++ p2)
> add_cons (Proj tv p1) (C p2)          = Proj tv (p1 ++ p2)
> add_cons (Proj tv1 p1) (Proj tv2 p2) = Proj (tv1 ++ tv2) (p1 ++ p2)

> projection :: Constraint -> [TVar] -> Constraint
```

FILL IN: definition of projection on constraints.
Projection might vary depending on different
kinds of constraint systems. A standard
definition can be found below.

```
projection (C p) tvl         = Proj tvl p
projection (Proj tv1 p) tv2 = Proj (tv1 ++ tv2) p
```

lifted functions

```
> tv_typo :: Typo -> [TVar]
> tv_typo env = foldl f [] env where
>           f z (v,ts) = z ++ (tv ts)

> add_constraint :: Constraint -> TP ()
> add_constraint c1 =
>      TP (\ (n,s,c) -> return (n,s,add_cons c c1,()))
```

handling of type variables and type environments

```
> newtvar :: TP TVar
> newtvar = TP (\ (n,s,c) -> return (new_tvar n,s,c,n) )

> insert :: Typo -> (Var, TypeScheme) -> Typo
> insert a (x,t) = (x,t):a

> mylookup :: Typo -> Var -> E TypeScheme
> mylookup [] y =
>     Err "undeclared variable"
> mylookup ((x,t):xs) y =
>       if x == y then return t
>       else mylookup xs y
```

instantiation and generalization

```
> rename :: TP Subst -> TVar -> TP Subst
```

```
> rename s x = do newtv <- newtvar
>                 s' <- s
>                 return ((x, TV newtv):s')

> inst :: Typo -> Var -> TP Type
> inst env x =
>    case mylookup env x of
>      Suc ts -> case ts of
>                 Mono t        -> return t
>                 Poly tvl c t  ->
>                     do s' <- foldl rename (return null_subst) tvl
>                        c' <- apply s' c
>                        t' <- apply s' t
>                        add_constraint c'
>                        return t'
>      Err _  -> do ntv <- newtvar
>                   report_error "undeclared variable" (TV ntv)

> gen :: Typo -> Type -> TP TypeScheme
> gen env t =
>  TP ( \ (n,s,c) ->
>           return (n,s, projection c (fv t c env), Poly (fv t c env) c t) )
>  where fv t1 c1 env1 = without ((tv t1) ++ (tv c1)) (tv_typo env1)
>
```

constraint solver

```
> cs :: (Subst, Constraint) -> TP (Subst, Constraint)

FILL IN: specific constraint solver

incorporating the constraint solver into monad TP

> normalize :: Type -> TP Type
> normalize a = do s <- get_subst
>                  c <- get_cons
>                  (s',c') <- cs (s,c)
>                  t <- apply s' a
>                  return_result s' c' t

> get_subst :: TP Subst
> get_subst = TP ( \ (n,s,c) -> return (n,s,c,s))

> get_cons :: TP Constraint
> get_cons = TP ( \ (n,s,c) -> return (n,s,c,c))

> return_result :: Subst -> Constraint -> Type -> TP Type
> return_result s c t = TP ( \ (n,s',c') -> return (n,s,c,t))

type inference

> tp :: (Typo, Term) -> TP Type
```

```
> tp (env, Id x) =  do a <- inst env x
>                      normalize a

> tp (env, Abs x e) = do a <- newtvar
>                        b <- tp (insert env (x, Mono (TV a)), e)
>                        normalize ((TV a) `Fun` b)

> tp (env, App e e') = do a <- newtvar
>                         t <- tp (env, e)
>                         t' <- tp (env, e')
>                         add_constraint (C [t `Subsume` (t' `Fun` TV a)])
>                         normalize (TV a)

> tp (env, Let x e e') = do a <- tp (env, e)
>                           b <- gen env a
>                           tp ((insert env (x, b)), e')

top-level program

> infer :: Term -> E (TVar, Subst, Constraint, Type)
> infer e = unTP (tp (init_typo, e)) (init_tvar, null_subst, true_cons)

*----------------------------------------
* Category: Hindley/Milner instance
*----------------------------------------
```

```
> init_typo = []
```

Invariant: all constraints are true, hence projection is always trivial

```
> projection _ _ = true_cons
```

Implementation of the Robinson's unification algorithm

```
> unify :: (Subst, Type, Type) -> TP Subst
> unify (s, TV x, TV y) =
>    if x == y then return s
>    else do t <- apply s (TV x)
>            return ((y, t):s)
> unify (s, TV x, t) =
>             do t'' <- apply s t
>                if elem x (tv t'')
>                then report_error "occurs check fails" null_subst
>                 else return ((x, t''):s)
> unify (s, t, TV x) = unify (s, TV x, t)
> unify (s, t1 `Fun` t1', t2 `Fun` t2') = do s' <- unify (s, t1, t2)
>                                           unify (s', t1', t2')

> cs (s, C ((t `Subsume` t'):c)) = do s' <- unify (s,t,t')
>                                     c' <- apply s' (C c)
>                                     cs  (s', c')
> cs (s, C (TRUE:c))         = cs (s, C c)
```

```
> cs (s, C [])                = return (s, true_cons)


test cases


> test = infer (App (Abs 1 (Id 1)) (Abs 2 (Id 2)))
> test1 = infer (Abs 1 (Abs 2 (Id 1)))
> test2 = infer (Abs 1 (Id 1))
> test3 = infer (Abs 1 (App (Id 1) (Id 1)))
> test4 = infer (Let 1 (Abs 2 (Id 2)) (App (Id 1) (Abs 3 (Id 3))))
> test5 = infer (Let 1 (Abs 2 (Id 2)) (Id 1))
> test6 = infer (Let 1 (Abs 2 (Id 2)) (App (Id 1) (Id 1)))



*-----------------------------------------
* Category: Dimension type system
*
* I only mention the necessary changes
* to obtain an instance that deals
* with dimension types
*-----------------------------------------


*-----------------------------------------------
* Category: DATA DECLARATIONS
*-----------------------------------------------


The type language
```

```
> type TVar       = Int
> type Var        = Int
> data Type       = TV TVar
>                 | Type `Fun` Type
>                 | Dim [(Unit,Int)]
>                     deriving Show

> data Unit = UnitTV TVar | UnitMeter | UnitSecond
>                deriving Show

*----------------------------------------
* Category: INSTANCE DECLARATIONS
*----------------------------------------

> instance TypesAndConstraints Predicate where
>   apply s TRUE              = return TRUE
>   apply s (t1 `Subsume` t2) = do t1' <- apply s t1
>                                  t2' <- apply s t2
>                                  return (t1' `Subsume` t2')
>   tv TRUE                   = []
>   tv (t1 `Subsume` t2)      = (tv t1) ++ (tv t2)

> instance TypesAndConstraints c => TypesAndConstraints [c] where
>   apply s  = mapM (apply s)
>   tv  a    = foldl f [] a where
```

```
>                       f z x = z ++ (tv x)

> instance TypesAndConstraints Constraint where
>      apply s (C p)            = do p' <- apply s p
>                                    return (C p')
>      apply s (Proj tvl p)      = do p' <- apply s p
>                                     return (Proj tvl p')
>      tv (C p)                 = tv p
>      tv (Proj tvl p)           = without (tv p)  tvl

> instance TypesAndConstraints Type where
>   apply s (TV tvl)           = case lookup tvl s of
>                                    Just t  -> return t
>                                    Nothing -> return (TV tvl)
>   apply s (t1 `Fun` t2)     = do t1' <- apply s t1
>                                  t2' <- apply s t2
>                                  return (t1' `Fun` t2')
>   apply s (Dim [])          = return (Dim [])
>   apply s (Dim ((UnitTV utv, exp):r)) =
>    case lookup utv s of
>     Just t  ->
>      case t of
>       Dim u ->
>        do t' <- apply s (Dim r)
>           case t' of
>            Dim r' -> return (Dim (normalize_unit
```

```
>                               (map ( \ (ut, ue) -> (ut, ue * exp)) u) ++ r'))
>            _      -> report_error "should be a unit" (Dim [(UnitSecond,0)])
>       _     -> report_error "types are not allowed within units"
>                               (Dim [(UnitSecond,0)])
>     Nothing ->  do t' <- apply s (Dim r)
>                    case t' of
>                     Dim r' -> return (Dim ((UnitTV utv, exp):r'))
>                     _      -> report_error "should be a unit"
>                                            (Dim [(UnitSecond,0)])
>   apply s (Dim ((u,exp):r)) =
>     do t <- apply s (Dim r)
>        case t of
>          Dim r' -> return (Dim ((u,exp):r'))
>          _      -> report_error "should be a unit" (Dim [(UnitSecond,0)])
>   tv (TV tvl)                = [tvl]
>   tv (t1 `Fun` t2)           = (tv t1) ++ (tv t2)
>   tv (Dim [])                    = []
>   tv (Dim ((UnitTV utv, exp):r)) = [utv] ++ (tv (Dim r))
>   tv (Dim ((u,exp):r))           = tv (Dim r)

> instance TypesAndConstraints TypeScheme where
>   apply s (Poly tvl c t)     = do c' <- apply s c
>                                   t' <- apply s t
>                                   return (Poly tvl c' t')
>   apply s (Mono t)           = do t' <- apply s t
>                                   return (Mono t')
```

```
>   tv (Poly tvl c t)          = without ((tv t) ++ (tv c)) tvl
>   tv (Mono t)               = tv t



*----------------------------------------
* Category: Dimension constraint solver
*----------------------------------------


> instance Eq Unit where
>   (UnitTV tv1) == (UnitTV tv2) = tv1 == tv2
>   UnitMeter == UnitMeter = True
>   UnitSecond == UnitSecond = True
>   _ == _ = False


putting a unit type into normal form


> normalize_unit :: [(Unit,Int)] -> [(Unit,Int)]
> normalize_unit [] = []
> normalize_unit ((u,exp):rest) =
>             let (rest', occ) = collect_unit u ((u,exp):rest)
>                 rest'' = normalize_unit rest'
>             in if occ == 0 then sort_unit rest''
>                else sort_unit ((u,occ):rest'')


here comes the 'very' normalized version, units are separated into
constants and variables
```

```
> very_normalize_unit :: [(Unit,Int)] -> [(Unit,Int)]
> very_normalize_unit ul = let (vars, const) = split_unit ul
>                          in (normalize_unit vars) ++ (normalize_unit const)
```

splits a unit into the variable and constant part

```
> split_unit :: [(Unit,Int)] -> ([(Unit,Int)],[(Unit,Int)])
> split_unit [] = ([],[])
> split_unit ((UnitTV tv, exp):rest) = let (vars,const) = split_unit rest
>                                      in  ((UnitTV tv, exp):vars,const)
> split_unit (u:rest) = let (vars,const) = split_unit rest
>                       in  (vars,u:const)
```

sorts a list of units wrt exponent

```
> sort_unit :: [(Unit,Int)] -> [(Unit,Int)]
> sort_unit [] = []
> sort_unit ((u,exp):rest) =
>          let l1 = [(u1,exp1) | (u1,exp1) <- rest, (abs exp1) <= (abs exp)]
>              l2 = [(u2,exp2) | (u2,exp2) <- rest, (abs exp2) > (abs exp)]
>          in (sort_unit l1) ++ [(u,exp)] ++ (sort_unit l2)
```

takes a unit and a unit a list of units,
result is the number of occurenes in that unit, and the list of units where
all occurences have been discarded

```
> collect_unit :: Unit -> [(Unit,Int)] -> ([(Unit,Int)],Int)
> collect_unit u [] = ([],0)
> collect_unit u ((u',exp):rest) = if u == u'
>                                  then let (rest', occ) = collect_unit u rest
>                                       in (rest', occ+exp)
>                                  else let (rest', occ) = collect_unit u rest
>                                       in ((u',exp):rest', occ)
```

Invariant: all constraints are true, hence projection is always trivial

```
> projection _ _ = true_cons
```

Implementation of the Lankford/Butler/Brady unification algorithm for abelian gr

```
> unify :: (Subst, Type, Type) -> TP Subst
> unify (s, TV x, TV y) =
>    if x == y then return s
>    else do t <- apply s (TV x)
>            return ((y, t):s)
> unify (s, TV x, t) =
>      do t'' <- apply s t
>         if elem x (tv t'') then report_error "occurs check fails" null_subst
>          else return ((x, t''):s)
> unify (s, t, TV x) = unify (s, TV x, t)
> unify (s, t1 `Fun` t1', t2 `Fun` t2') = do s' <- unify (s, t1, t2)
```

```
>                                              unify (s', t1', t2')
> unify (s, Dim u, Dim u') = let u'' = map ( \ (ut,ue) -> (ut, -ue)) u'
>                            in unify_unit s (u ++ u'')
> unify (s, _, _) = report_error "not compatible type" null_subst

> unify_unit :: Subst -> [(Unit, Int)] -> TP Subst
> unify_unit s ul =
>  let (vars,const) = group_it (split_unit (normalize_unit ul))
>  in if (vars == []) && (const == []) then return s
>     else
>      if vars == [] then report_error "Error" null_subst
>      else
>       let vars' = normalize_unit vars
>           (UnitTV tv ,ue) = head vars'
>           newlist = map ( \ (u',e') -> (u', -(div e' ue)))
>                                        ((tail vars')++const)
>       in if (foldl ( \ b -> \ (_,e) -> b && (mod e ue == 0)) True
>                    ((tail vars')++const))
>          then return ((tv, Dim newlist):s)
>          else if (tail vars') == [] then report_error "Error" null_subst
>               else do newtv <- newtvar
>                       unify_unit ((tv,Dim ([(UnitTV newtv,1)]++newlist)):s)
>                                  ([(UnitTV newtv, ue)] ++
>                        (map ( \ (u',e') -> (u', mod e' ue))
>                             ((tail vars')++const)))
```

```
ensures that first element of vars list has positive value

> group_it :: ([(Unit,Int)],[(Unit,Int)]) -> ([(Unit,Int)],[(Unit,Int)])
> group_it ([],const) = ([],const)
> group_it ((ut,ue):rest,const) = let f x = map ( \ (ut,ue) -> (ut, -ue)) x
>                                 in if ue >= 0 then ((ut,ue):rest,const)
>                                    else ((ut,-ue): (f rest), f const)
>

> cs (s, C ((t `Subsume` t'):c)) = do s' <- unify (s,t,t')
>                                     c' <- apply s' (C c)
>                                     cs  (s', c')
> cs (s, C (TRUE:c))         = cs (s, C c)
> cs (s, C [])               = return (s, true_cons)

primitive functions

> init_typo = [(1, Poly [300] (C [TRUE]) (Dim [(UnitTV 300, 1)] `Fun`
>                  (Dim [(UnitTV 300, 1)] `Fun` Dim [(UnitTV 300, 1)])))]

   example addition:  plus : forall a. dim(a) -> dim(a) -> dim(a)
```

Appendix E

A Catalogue of Notation

Naming conventions:

$\mathcal{X}$	type system instance
Γ	type environment
α, β	type variables
τ	type
σ	type scheme
C, D	constraints
$fv(\cdot)$	free variables
$\preceq$	subsumption

Judgments:

$C, \Gamma \vdash e : \sigma$	Figure 3.1
$C, \Gamma \vdash^{n} e : \sigma$	Section 3.4.4
$C, \Gamma \vdash^{s} e : \tau$	Figure 3.2
$C, \Gamma \vdash^{\mathcal{X}^c} e : \sigma$	Section 3.6.2
$C, \Gamma \vdash^{\mathcal{X}^t} e : \sigma$	Section 3.6.2
$\phi, C, \Gamma \vdash^{W} e : \tau$	Figure 4.1
$C, \Gamma \vdash^{W_c} e : \tau$	Figure 4.3
$\phi, C, \Gamma \vdash^{W_{term}} e : \tau$	Figure 4.4
$C, \Gamma \vdash^{W_{c2}} e : \alpha$	Figure 4.5
$\phi, C, \Gamma \vdash^{W_{normalize}} e : \tau$	Figure 4.5

Relations:

$C \vdash^{e} D$	Definition 2
$C =^{e} D$	Definition 2
$(\tau_1 = \tau_2)$	Definition 7
$(\tau_1 \preceq \tau_2)$	Section 3.2
$(\tau_1 \lessdot \tau_2)$	Definition 6
$C \vdash^{i} \sigma \preceq \sigma'$	Section 3.4.1
$C \vdash^{i_2} \sigma \preceq \sigma'$	Section 3.4.1
$C \vdash^{i} \sigma \equiv \sigma'$	Section 3.4.1
$C \vdash \sigma \mapsto \sigma'$	Definition 18
$C, \Gamma, \sigma \vdash^{gen} (C', \sigma')$	Sections 3.4, 4.3

Index

closed world approach, 56

comparison relation, *see* instance relation

consistent, 43

constraint, 27
  entailment, 19
  free variables of, 21
  normalization, 80
  problem, 86
    combination of, 86
  satisfiable, 22
  solved, 29
  solving, 80
  subsumption, 79

constraint system, 19, 30
  coherent, 35
  cylindric, 20
  dimension type, 24
  herbrand, 24
  simple, 19
  sound, 33
  strongly sound, 37
  subtyping, 25
  term, 22

dimension types, 24, 31, 80

entailment, *see* constraint

equation–free, 89

equational theory, 85
  non–regular, 84, 97, 104, 109
  regular, 84, 85, 97, 104

full and faithful, 74, 113, 117, 138

ideal semantic, 11, 33

inference, 13
  by need, 96
  completeness of, 15, 98, 103, 106
  eager, 96
  in constraint form, 100
  in term form, 93
  lazy, 55
  soundness of, 14, 97, 103, 105
  strict, 56
  system

$\vdash^{W}$, 14, 94

$\vdash^{W_{byneed}}$, 96

$\vdash^{W_c}$, 101

$\vdash^{W_{normalize}}$, 108

$\vdash^{W_{term}}$, 105

instance relation, 41

$\vdash^{i}$, 41

$\vdash^{i_2}$, 42

lifting property, 91

Lifting Theorem, 93

mgu, *see* most general, unifier

monotype, 33

most general

normalizers, *see* normalization, most general

unifier, 13, 125, 129

non–regular, *see* equational theory

normal form, 86

normalization, 86

by need, 90

eager, 90

lifting of, 91

most general, 88

open world approach, 53, 56

polymorphism

constrained, 8, 50

parametric, 8

predicate, *see* constraint

principal

constraint property, 88

normal form, 86

types, 16, 99

uniqueness, 100

projection

free, 91

in solved form, 53, 58, 59, 62

in syntactic form, 58, 59, *see* projection, in solved form

non–trivial, 58

operator, 21, 50

trivial, *see* projection, in solved form, 58, 138

projection–free, 37

quantifier

elimination rule, 48, 50, 77

existential quantification, *see* projection

introduction rule, 48, 50

nested forall, 47, 60

realizable, 43
records, 114
  concatenation, 122
  extensible, 118
  first–class labels, 126
regular, *see* equational theory

solved forms, 29
subject reduction, 39
substitution, 22, 85
  in constraint form, 73
subsumption, 29, 79
  relation, 30
subtyping, 31

term algebra, 27, 30, 69
Transformation Theorem, 73, 111
type case, 130
type schemes, 10, 27
type soundness, 13, 36
  syntactic, 40
type system, 7
  $\vdash$, 10, 28, 75
  $\vdash^{\mathcal{X}^c}$, 72
  $\vdash^{s}$, 45
  $\vdash^{n}$, 45
  $\vdash^{\mathcal{X}^t}$, 72
  constraint–based, 74
  dimension types, 31
  Hindley/Milner, 9, 30
  in constraint form, 75
  subtyping, 31
  term–based, 74
types
  constrained, 27
  in constraint form, 69
  in term form, 69
  monomorphic, 10
  polymorphic, 10
typing judgment, 10, 29
  valid, 10, 29

unification
  problem, 13
  unitary, 85

weakening, 44